Handwriting Reproduction Robot

Theory Manual for the Software

D.R. Braune
u23539195

University of Pretoria
EPR400 Final Year Project

Contents

1 How to use this manual **13**

2 Glossary **14**

2.1 Arrays and arithmetic 14

2.2 Images and shapes 15

2.3 Networks and training 16

2.4 This project's own terms 17

3 paths.py **19**

3.1 Data 19

3.2 Weights 19

3.3 Profiles 19

3.4 PersonalPages 20

3.5 Cache 20

3.6 DatabaseWriters 20

3.7 PersonalWriters 20

3.8 AllWriters 21

3.9 ValFraction 21

3.10 SplitSeed 21

3.11 PersonalWriterPages 21

4 segment.py **23**

4.1 Integral 23

4.2 BoxSum 25

4.3 BoxCount 26

4.4 BoxMean 26

4.5 Dilate 27

4.6 Erode 28

4.7 Close 29

4.8 AnyWithinRadius 29

4.9 LongRunMask 30

4.10 RowRunLength . . . 31
4.11 Otsu . . . 32
4.12 BinarizeLine . . . 34
4.13 CoreBand . . . 35
4.14 LabelComponents . . . 36
4.15 PieceBounds . . . 37
4.16 Skeletonize . . . 38
4.17 _Weights . . . 39
4.18 Resize . . . 41
4.19 Halve . . . 42
4.20 FitFixedCanvas . . . 43
4.21 _Sample . . . 44
4.22 _Like . . . 45
4.23 Rotate . . . 45
4.24 Warp . . . 46
4.25 Blur . . . 47
4.26 Elastic . . . 48
4.27 DrawPolyline . . . 49
4.28 EstimateSlantDeg . . . 51
4.29 PolylineLength . . . 52
4.30 InkMask . . . 53
4.31 PaperRegionMask . . . 56
4.32 LinePitch . . . 58
4.33 LeastCostSeam . . . 62
4.34 SkewScore . . . 64
4.35 Deskew . . . 66
4.36 RemovePrintedStructure . . . 68
4.37 DropArtifactPieces . . . 69
4.38 LineCentres . . . 72
4.39 SeamCuts . . . 73
4.40 _BandOwnership . . . 75
4.41 _DotsJoinedAbove . . . 78
4.42 _RingsToStem . . . 80
4.43 _FragmentsToNeighbour . . . 82

4.44 AssignOwnership . . . 83
4.45 _BuildLineCrop . . . 84
4.46 SegmentLines . . . 88

5 recognise.py **90**

5.1 CHARSET . . . 90
5.2 CHAR_TO_IDX . . . 91
5.3 IDX_TO_CHAR . . . 92
5.4 WEIGHTS_DIR . . . 92
5.5 INPUT_HEIGHT . . . 92
5.6 INPUT_WIDTH . . . 93
5.7 _TEXT_CANDIDATES . . . 93
5.8 DEFAULT_TEXT_WEIGHTS . . . 94
5.9 _AUTHOR_NPZ . . . 94
5.10 _SHAPE . . . 95
5.11 _W1 . . . 95
5.12 DEFAULT_AUTHOR_WEIGHTS . . . 95
5.13 LoadStateDictAny . . . 96
5.14 Conv2d . . . 97
5.15 BatchNorm2d . . . 100
5.16 Relu . . . 102
5.17 Sigmoid . . . 103
5.18 MaxPool2d . . . 104
5.19 AdaptiveMaxPoolHeight . . . 105
5.20 AdaptiveAvgPoolWidth . . . 106
5.21 Linear . . . 107
5.22 LogSoftmax . . . 108
5.23 Softmax . . . 110
5.24 ConvBlockForward . . . 110
5.25 LstmForward . . . 111
5.26 BidirectionalLstmForward . . . 114
5.27 MultilayerBidirectionalLstm . . . 116
5.28 ResizeLineImageFixed . . . 116
5.29 TensorFromResized . . . 117

5.30 PenWidthXh . . . 118
5.31 StrokeNormalize . . . 118
5.32 DecodeCtc . . . 120
5.33 CtcForcedAlign . . . 122
5.34 FrameXToPixel . . . 125
5.35 PaperCRNNNumpy . . . 126
5.36 ReadText . . . 127
5.37 Levenshtein . . . 128
5.38 CharAcc . . . 129
5.39 WordAcc . . . 130
5.40 AuthorClassifierCNNNumpy . . . 131
5.41 ClassifyImage . . . 132

6 style.py **134**

6.1 DATA_DIR . . . 134
6.2 CACHE_DIR . . . 135
6.3 NOGIT_DIR . . . 135
6.4 TEXT_WEIGHTS . . . 135
6.5 PERSONAL_WEIGHTS . . . 136
6.6 RAW_DIR . . . 136
6.7 MAX_VARIANTS_PER_CHAR . . . 136
6.8 ALIGN_CONF_PCT . . . 137
6.9 FRAGMENT_PEN_FRAC . . . 138
6.10 MIN_PEN_ABS . . . 139
6.11 MIN_LONGEST_STROKE . . . 139
6.12 DOT_MIN_AREA . . . 140
6.13 MAX_GLYPH_W . . . 140
6.14 MAX_GLYPH_W_WIDE . . . 141
6.15 _WIDE_CHARS . . . 141
6.16 _OFFS . . . 142
6.17 TracePolylines . . . 142
6.18 _Dir . . . 144
6.19 _PruneSpurs . . . 145
6.20 _JoinAtJunctions . . . 145

6.21 SimplifyPolyline . . . 147
6.22 _TrimLigatureTails . . . 148
6.23 ExtractLineGlyphs . . . 150
6.24 _ASCEND . . . 154
6.25 _DESCEND . . . 154
6.26 _XONLY . . . 154
6.27 _ClassOk . . . 155
6.28 _PenLength . . . 156
6.29 _LongestStroke . . . 156
6.30 _ShapeGrid . . . 157
6.31 _LETTER_PRIOR . . . 158
6.32 BuildLetterPrior . . . 159
6.33 _PriorScore . . . 160
6.34 PriorFilter . . . 161
6.35 _ConsensusFilter . . . 162
6.36 _ResampleN . . . 163
6.37 _AlignStroke . . . 164
6.38 _DenoisedPrototype . . . 165
6.39 SmoothStroke . . . 166
6.40 RenderRefStats . . . 168
6.41 _MedianWordGap . . . 169
6.42 ExtractAuthorRaw . . . 169
6.43 BuildAuthorProfile . . . 170

7 synthesise.py **174**

7.1 SCRIPT_DIR . . . 174
7.2 CNN_DATA_DIR . . . 175
7.3 _FB . . . 175
7.4 _CURSIVE . . . 176
7.5 _FB_ADV . . . 176
7.6 _SIMILAR . . . 177
7.7 _BaselineGlyph . . . 177
7.8 UNIFORM_INK . . . 179
7.9 UNIFORM_PEN_WIDTH_XH . . . 179

7.10 UNIFORM_SOFT_FRAC . . . 179
7.11 JOIN_PROB_CAP . . . 180
7.12 CORE_SLANT_CAP_DEG . . . 180
7.13 CORE_JITTER_FRAC . . . 181
7.14 COLLISION_MIN_GAP_XH . . . 181
7.15 _ELIGIBLE_VARIANTS . . . 182
7.16 CORE_CURSIVE_CONN . . . 183
7.17 JOIN_LEGIBILITY_COUPLING . . . 183
7.18 IDEAL_ANCHOR_MAX_PRIORD . . . 184
7.19 IDEAL_SOURCE . . . 184
7.20 _PRIOR_REJECT . . . 185
7.21 Trajectory . . . 185
7.22 _GLYPH_CLEAN . . . 186
7.23 _GLYPH_GARBAGE . . . 186
7.24 _COMMIT_LAM . . . 187
7.25 _SwapProb . . . 187
7.26 _AUTO_LAM_GOOD . . . 189
7.27 _AUTO_LAM_BAD . . . 189
7.28 AutoLegibility . . . 190
7.29 _CoreGlyph . . . 191
7.30 _GlyphSource . . . 192
7.31 _Jitter . . . 194
7.32 _Resample . . . 195
7.33 USE_STYLE_CALIBRATION . . . 196
7.34 _STYLE_CAL . . . 196
7.35 StyleCalibration . . . 197
7.36 _ApplyCal . . . 199
7.37 _Ligature . . . 200
7.38 _SplitWords . . . 201
7.39 _WordWidth . . . 202
7.40 SynthesizeText . . . 202
7.41 _Draw . . . 205
7.42 RenderTrajectory . . . 206
7.43 _MatchInkDensity . . . 208

7.44 _LineLogProbs . . . 209
7.45 _WeakChars . . . 210
7.46 REPAIR_ROUNDS . . . 211
7.47 REPAIR_WORST_K . . . 211
7.48 DEFAULT_LEGIBILITY . . . 212
7.49 SynthesizeLegible . . . 212
7.50 SynthesizeJointBestOf . . . 214

8 reproduce.py 216

8.1 GantryConfig . . . 216
8.2 ToMachine . . . 218
8.3 _Decimate . . . 219
8.4 OptimizeStrokeOrder . . . 220
8.5 _Clamp . . . 221
8.6 NewPenState . . . 222
8.7 SetPenLines . . . 223
8.8 WriteGcode . . . 224
8.9 ParseGcode . . . 225
8.10 GantryNotConnectedError . . . 226
8.11 NotCalibratedError . . . 226
8.12 CHIP_PATH . . . 227
8.13 PEN_SWITCH_PIN . . . 227
8.14 PEN_MOTOR_PIN . . . 228
8.15 ENDSTOP_PINS . . . 228
8.16 INVERT_X . . . 229
8.17 INVERT_Y . . . 229
8.18 MICROSTEPS . . . 229
8.19 STEPS_PER_REV . . . 230
8.20 STEPS_PER_MM . . . 230
8.21 CIRCLE_STEP_DELAY_S . . . 231
8.22 DEFAULT_CIRCLE_RADIUS_MM . . . 231
8.23 X_SWITCH_TRAVEL_MM . . . 231
8.24 Y_SWITCH_TRAVEL_MM . . . 232
8.25 EDGE_TOLERANCE_MM . . . 232

8.26 HOMING_STEP_DELAY_S . . . 233
8.27 HOMING_MAX_STEPS . . . 234
8.28 ENDSTOP_DEBOUNCE_S . . . 234
8.29 ENDSTOP_CONFIRM_SAMPLES . . . 234
8.30 RETRACT_MM . . . 235
8.31 PEN_UP_OVERRUN_S . . . 235
8.32 GCODE_MIN_FEED_MM_S . . . 236
8.33 _GCODE_WORD_RE . . . 236
8.34 _RETREAT_SIGN . . . 237
8.35 Calibration . . . 237
8.36 req . . . 238
8.37 CurrentMode . . . 238
8.38 targetRpm . . . 239
8.39 stepDelayS . . . 239
8.40 running . . . 239
8.41 CurrentXSteps . . . 240
8.42 CurrentYSteps . . . 240
8.43 theta . . . 240
8.44 LastCircleTime . . . 241
8.45 CircleIntervalS . . . 241
8.46 CircleRadiusMm . . . 241
8.47 CircleRadiusSteps . . . 241
8.48 estopped . . . 242
8.49 TrippedSwitch . . . 242
8.50 Connect . . . 242
8.51 Disconnect . . . 243
8.52 IsConnected . . . 244
8.53 CalculateDelay . . . 244
8.54 TriggeredEndstop . . . 245
8.55 _AllTriggeredEndstops . . . 246
8.56 AxisDirValue . . . 246
8.57 _RetractOffSwitch . . . 247
8.58 _TriggerEmergencyStop . . . 247
8.59 BresenhamMove . . . 248

8.60 PenIsUp . . . 250
8.61 SetPen . . . 250
8.62 _StepAxis . . . 251
8.63 _CalibrateAxis . . . 252
8.64 Calibrate . . . 253
8.65 ParseGcodeLine . . . 254
8.66 _ClampToUsableArea . . . 254
8.67 _GcodeLinearMove . . . 255
8.68 _GcodeArcMove . . . 256
8.69 RunGcodeFile . . . 258
8.70 WriteGcodeFile . . . 258
8.71 InputListener . . . 259

9 train.py **260**

9.1 Conv2dForward . . . 261
9.2 Conv2dBackward . . . 262
9.3 BatchNorm2dForwardTrain . . . 264
9.4 BatchNorm2dBackward . . . 266
9.5 ReluForward . . . 267
9.6 ReluBackward . . . 268
9.7 MaxPool2dForward . . . 269
9.8 MaxPool2dBackward . . . 269
9.9 AdaptiveMaxPoolHeightForward . . . 270
9.10 AdaptiveMaxPoolHeightBackward . . . 271
9.11 AdaptiveAvgPoolWidthForward . . . 271
9.12 AdaptiveAvgPoolWidthBackward . . . 271
9.13 LinearForward . . . 272
9.14 LinearBackward . . . 273
9.15 DropoutForward . . . 274
9.16 DropoutBackward . . . 275
9.17 ConvBlockForwardTrain . . . 275
9.18 ConvBlockBackwardTrain . . . 276
9.19 LstmForwardTrain . . . 277
9.20 LstmBackward . . . 278

9.21 BidirectionalLstmForwardTrain . . . 280
9.22 BidirectionalLstmBackward . . . 280
9.23 MultilayerBidirectionalLstmForwardTrain . . . 281
9.24 MultilayerBidirectionalLstmBackward . . . 281
9.25 LogSumExp . . . 282
9.26 CtcLoss . . . 283
9.27 CrossEntropyLoss . . . 286
9.28 RMSprop . . . 287
9.29 AdamW . . . 288
9.30 ReduceLrOnPlateau . . . 289
9.31 CosineAnnealingLr . . . 290
9.32 ClipGradNorm . . . 291
9.33 EncodeCrop . . . 292
9.34 DecodeCrop . . . 292
9.35 ReadLabelLines . . . 293
9.36 ClusterRows . . . 293
9.37 FindRuleLineClusters . . . 294
9.38 RemoveStraightFormLines . . . 295
9.39 DetectPageBounds . . . 295
9.40 MergeUndersizedFragments . . . 296
9.41 MergeOrSplitToExpectedLineCount . . . 296
9.42 EstimateLinePitch . . . 297
9.43 SplitLinesByPeriodicity . . . 297
9.44 ExtractLinePatches . . . 298
9.45 EncodeText . . . 299
9.46 FindPeaks . . . 299
9.47 MyAffine . . . 301
9.48 AugmentShear . . . 302
9.49 AugmentRotate . . . 303
9.50 FindPerspectiveCoeffs . . . 303
9.51 AugmentPerspective . . . 304
9.52 PerspectiveJitterPoints . . . 305
9.53 AugmentElastic . . . 305
9.54 AugmentLineImage . . . 306

9.55 AUGMENTATIONS . . . 307
9.56 LineImageCache . . . 307
9.57 IAMLineDatasetRaw . . . 307
9.58 TrainEvalView . . . 308
9.59 TextCollateFn . . . 308
9.60 AuthorLabeledView . . . 309
9.61 AuthorCollateFn . . . 309
9.62 AddPersonalSamples . . . 309
9.63 TekliaLineDataset . . . 310
9.64 MAX_SAFE_LABEL_CHARS . . . 310
9.65 HIDDEN_SIZE . . . 311
9.66 WEIGHTS_DIR . . . 311
9.67 DEFAULT_TEXT_CACHE_DIR . . . 312
9.68 DEFAULT_AUTHOR_CACHE_DIR . . . 312
9.69 DEFAULT_AUTHOR_DATA_DIR . . . 312
9.70 DEFAULT_TEXT_DATA_DIR . . . 312
9.71 InitConv . . . 313
9.72 InitBatchNorm . . . 314
9.73 InitLinear . . . 314
9.74 InitLstmDirection . . . 315
9.75 InitConvBlock . . . 316
9.76 InitBackboneParams . . . 317
9.77 TrainableParams . . . 317
9.78 SaveCheckpoint . . . 318
9.79 LoadCheckpointMeta . . . 318
9.80 LoadParamsInto . . . 319
9.81 AppendLog . . . 319
9.82 BackboneForwardTrain . . . 320
9.83 BackboneBackwardTrain . . . 320
9.84 EvalBackboneForward . . . 321
9.85 IterateBatches . . . 321
9.86 InitTextModelParams . . . 322
9.87 BuildLstmLayerParams . . . 323
9.88 TextModelForward . . . 323

9.89 TextModelBackward 324
9.90 EvalTextForward 324
9.91 TextNumpyBatch 325
9.92 ValidateTextModel 325
9.93 BuildPagesSplit 326
9.94 BuildTekliaSplit 326
9.95 TrainTextModel 327
9.96 InitAuthorModelParams 328
9.97 AuthorModelForward 328
9.98 AuthorModelBackward 329
9.99 EvalAuthorForward 329
9.100AuthorNumpyBatch 330
9.101ValidateAuthorModel 330
9.102TrainAuthorModel 331

1 How to use this manual

This manual explains every function in the handwriting pipeline. For each one it gives the declaration exactly as it appears in the code, what each input means, what comes out, and then the theory: what the function is for, what problem it solves, how it solves it, and why that method was chosen rather than another.

The intention is that nothing is assumed. Where a term from mathematics, image processing or machine learning appears for the first time it is explained in place, and the most common terms are collected in the glossary immediately after this chapter. Where a formula is given, every symbol in it is defined underneath, and if the method comes from a published source that source is named with the formula rather than only in the bibliography.

Conventions used throughout the software. These are worth reading once, because they are assumed by every chapter.

- **Images** are rectangular grids of numbers, stored as `numpy` arrays and indexed `[row, column]`, which is `[y, x]`. The row index increases *downwards*, which is the normal convention for images because that is the order a screen or a printer draws them in.
- **Pen paths** are lists of `(x, y)` points measured in millimetres, with y increasing *upwards*, which is the normal convention for plotters and for ordinary graph paper. The pipeline therefore contains two coordinate systems whose vertical axes point in opposite directions. The conversion happens in exactly two places and both are pointed out where they occur.
- **Letter shapes** are stored in units of the x-height. The x-height is the height of the body of a lower case letter such as `a` or `o`, measured from the line the writing sits on. Storing a letter in these units, rather than in pixels, means the same stored letter can be drawn at any physical size.
- **No libraries.** Apart from `numpy`, which provides arithmetic on arrays of numbers, every algorithm in this manual is implemented in the files described here. There is no image processing library and no machine learning framework.

2 Glossary

Terms used in this manual, in plain language. They are grouped by subject rather than alphabetically, because reading a group in order explains more than reading a definition on its own.

2.1 Arrays and arithmetic

Array
A grid of numbers. A one-dimensional array is a list, a two-dimensional array is a table, and a greyscale image is a two-dimensional array where each number is the brightness of one pixel, usually from 0 for black to 255 for white.

Element-wise
An operation applied to every number in an array independently. Adding 10 to an image, element-wise, makes every pixel ten levels brighter.

Vectorised
Written so that an operation is applied to a whole array at once rather than by looping over the elements one at a time in Python. This matters for speed: the loop runs inside compiled code instead of in the interpreter, which is typically between ten and a hundred times faster.

Cumulative sum, or prefix sum
An array where each entry is the total of everything up to that point. For the input $(2, 3, 1, 4)$ the cumulative sum is $(2, 5, 6, 10)$. Its usefulness is that the total of any stretch of the original can then be found by one subtraction, because the total from i to j equals the entry at j minus the entry at i.

Mask
An array of true and false values, used to mark which pixels belong to something. An "ink mask" is true at every pixel judged to be ink and false elsewhere.

Dot product
For two lists of equal length, the sum of the products of corresponding entries. If both lists are first scaled so that their own length is one, the dot product equals the cosine of the angle between them considered as arrows, which is why it is used in this project to measure how similar two shapes are.

Norm
The length of a list of numbers considered as an arrow from the origin, which is the square root of the sum of the squares of its entries.

Interpolation
Estimating a value between known values. If a transformed image needs the brightness at a position halfway between two pixels, interpolation supplies it as a blend of the neighbours.

Median
The middle value when a set of numbers is sorted. It is used throughout this project in preference to the mean (the ordinary average) because it is insensitive to a small number of extreme values: one badly wrong measurement among eleven good ones moves the mean but not the median.

Percentile
The value below which a given share of the data falls. The 40th percentile is the value that 40 per cent of the data lies below. The median is the 50th percentile.

Variance and standard deviation
Measures of how spread out a set of numbers is. The variance is the average of the squared distances from the mean; the standard deviation is its square root, which has the same units as the data.

2.2 Images and shapes

Threshold
A brightness level used to divide an image into two groups, typically ink and paper. "Thresholding" is applying it.

Histogram
A count of how many pixels fall at each brightness level. It describes an image's distribution of tones without reference to where anything is.

Morphology
A family of operations that change the shapes in a mask rather than the values in an image: growing shapes, shrinking them, filling small holes, removing small specks.

Structuring element, or kernel
The small window a morphological or filtering operation looks through. A square kernel of radius 2 is a five-by-five window centred on each pixel.

Dilation and erosion
Growing and shrinking shapes. Dilation turns a pixel on if anything nearby is on; erosion turns it off unless everything nearby is on.

Closing
A dilation followed by an erosion by the same amount. The net effect is that small gaps are bridged while the overall thickness of a shape is preserved.

Connected component
A group of pixels that touch one another, treated as a single object. A letter, a dot over an `i` and a speck of dust are each a connected component.

Connectivity
Which neighbours count as touching. Four-connectivity counts only the pixels above, below, left and right; eight-connectivity also counts the four diagonals, so two pixels meeting at a corner belong to the same component.

Skeleton, or centreline
A one-pixel-wide line running down the middle of a thick stroke. Thinning a pen stroke to its skeleton recovers approximately the path the pen travelled.

Projection profile
The result of adding up the ink in each row, or in each column, producing a one-dimensional summary of a two-dimensional image. Lines of text appear as peaks in the row profile.

Polyline
A path described by an ordered list of points, understood to be joined by straight segments.

Aspect ratio
The ratio of width to height.

Bounding box
The smallest upright rectangle containing a shape.

2.3 Networks and training

Neural network
A function with a very large number of adjustable numbers, called weights, arranged in layers. Each layer takes the output of the previous one and performs a simple operation; the composition can represent complicated relationships. Training is the process of adjusting the weights so the function gives the right answers on examples.

Weight and bias
The adjustable numbers. A weight multiplies an input; a bias is added afterwards. Together they are the parameters.

Layer
One stage of a network.

Feature map
The output of a convolutional layer: an array showing, for each position in the image, how strongly some learned pattern was detected there.

Channel
One such map. A layer producing 64 channels is detecting 64 different patterns.

Convolution
Sliding a small grid of weights over an image and recording the response at every position, which detects the same pattern wherever it occurs.

Activation function
A simple non-linear function applied after a layer, such as setting negatives to zero. Without one, stacking layers would be pointless, because a chain of purely linear operations is equivalent to a single linear operation.

Pooling
Reducing the size of a feature map by summarising each small window, typically by its largest value, which saves work and makes the result tolerant to small shifts.

Logit
A raw score produced by the final layer, before it is converted into a probability. Logits may be any real number, positive or negative.

Softmax
The conversion from logits to probabilities: exponentiate each score and divide by the total, which forces them to be positive and to sum to one.

Loss function
A single number measuring how wrong the network's output is on a training example. Training consists of reducing it.

Gradient
The collection of derivatives of the loss with respect to every weight: for each weight, the rate at which the loss would change if that weight changed. It is what tells training which direction to move each weight.

Chain rule
The rule of calculus for differentiating a composition of functions, which is what makes it possible to compute the gradient of a loss with respect to a weight many layers away from it.

Backpropagation
Applying the chain rule layer by layer from the loss back to the first layer, so that every weight's gradient is obtained in one backward pass.

Forward pass, backward pass
Computing the output from the input, and computing the gradients from the loss, respectively.

Batch
A group of examples processed together, so that one weight update is based on their average gradient rather than on a single example, which is both faster and less noisy.

Epoch
One complete pass through the training data.

Learning rate
How far each weight is moved per update.

Optimiser
The rule that turns gradients into weight updates.

Overfitting
Learning the training examples rather than the general pattern, so that performance on new data is poor. It is detected by measuring on data the network was not trained on.

Holdout, or validation set
Data deliberately excluded from training and kept for measurement, which is the only way an accuracy figure means anything.

Checkpoint
A saved copy of all the weights, so a trained network can be reloaded later.

Recurrent layer
A layer that processes a sequence one position at a time while carrying information forward, so that a decision at one position can depend on earlier ones.

Inference
Running a trained network forward to get an answer, as opposed to training it.

2.4 This project's own terms

Glyph
One stored example of one character in one writer's hand: a set of pen strokes, in x-height units, with measurements describing how much space it occupies.

Variant
One of several stored glyphs for the same character by the same writer, since nobody writes a letter identically twice.

Prior distance
: A number measuring how far a stored glyph's shape sits from the typical shape of that character pooled across all writers. Low means a well-formed letter; high means the shape was probably cut out wrongly.

Anchor, or fallback letterform
: A clean, readable shape used in place of a stored variant when the stored variants are too damaged to be legible. Preferably the writer's own most typical version of the letter.

Legibility lambda
: A number between 0 and 1 controlling how often the anchor is used in place of the writer's own shapes, and therefore where the output sits between being readable and being recognisably that person's hand.

Connectedness
: The measured fraction of neighbouring letter pairs within a word that the writer joins without lifting the pen.

Ligature
: The joining stroke between two letters written without lifting the pen.

Trajectory
: The pipeline's representation of a drawing: a list of pen strokes in millimetres.

G-code
: The text format understood by the machine, consisting of lines such as `G1 X10 Y20 F900`, meaning "move in a straight line to this position at this feed rate".

Endstop
: A switch at the end of an axis's travel, used to find a known position and to prevent the machine driving into its own frame.

3 paths.py

This file contains no algorithm. It records where the data sits on disk and who the thirteen writers are, and every other module asks it rather than working it out independently.

Why it is a separate file Earlier in the project each module computed its own paths by walking upwards from its own location in the directory tree. That works until a file is moved to a different folder, at which point the chain of parent directories is wrong and the module can no longer find the data. Collecting the knowledge in one place means there is exactly one line to change if the data moves, and a module that moves folder does not break.

It is also deliberately not merged into one of the functional modules. Every module needs to know where the weights and profiles live, and if that knowledge lived inside, say, the recogniser, then the gantry driver would have to import the entire recogniser in order to find a directory name.

3.1 Data

```
Data = Path(__file__).resolve().parent.parent / 'CNN'
```

Output. The folder holding the datasets, the trained weights and the style profiles.

Derived from the location of this file rather than written as an absolute path, so the project can be copied to another machine or another drive without editing anything.

> *This path is one directory level up from where it was before the modules were flattened into a single folder, and the derivation was adjusted accordingly. A path derived by counting parent directories is correct only for the depth it was written for, which is a known fragility of this approach.*

3.2 Weights

```
Weights = Data / 'weights'
```

Output. The sub-folder holding trained checkpoints as `.npz` files.

A checkpoint is a saved copy of every weight in a trained network. The `.npz` format is numpy's own archive of named arrays, which is used here because the networks are built from numpy arrays directly and no framework's format is involved.

3.3 Profiles

```
Profiles = Data / 'NOGIT' / 'StyleProfiles10'
```

Output. The folder holding one JSON style profile per writer.

A style profile is the complete description of one person's handwriting produced by style.py: their stored letterforms plus the measurements describing how they lay writing out. JSON is used because the contents are nested lists of numbers and text, and because a profile can then be inspected in a text editor when something looks wrong.

3.4 PersonalPages

```
PersonalPages = Data / 'PersonalDataset'
```

Output. The folder holding the photographed pages and their transcripts.

The pages written specifically for this project, as opposed to the public database. Each writer has a sub-folder containing the photographs and a matching transcript for each one.

3.5 Cache

```
Cache = Data / 'NOGIT'
```

Output. The folder holding caches and other large generated files.

Extracting line crops from pages, and cutting glyphs out of those crops, are the slowest operations in the pipeline. Their results are cached here so that a change to a later stage can be tested without repeating them. Keeping the caches out of the code repository also keeps the repository small.

3.6 DatabaseWriters

```
DatabaseWriters = ['150', '151', '152', '153', '384', '551', '552',
    '588']
```

Output. The identifiers of the eight writers taken from the public handwriting database.

These are writers from a published corpus of scanned handwriting [1], identified by number. They provide variety that the project could not collect by hand: different ages, different nationalities, and in particular several strongly joined-up hands, which turn out to be the hardest case for every stage of the pipeline.

3.7 PersonalWriters

```
PersonalWriters = ['abhinav', 'dylan', 'owen', 'thiya', 'yeukita']
```

Output. The names of the five writers photographed for this project.

Volunteers who each filled several pages, photographed with a phone camera. These are the writers the project can test end-to-end, because their pages exist as photographs rather than as clean scans and therefore exercise the whole pipeline from uneven lighting onwards.

3.8 AllWriters

```
AllWriters = DatabaseWriters + PersonalWriters
```

Output. All thirteen writers, database then photographed.

Thirteen in total, which is three more than requirement R6 asks for. The order is fixed so that every table in every results document lists writers in the same sequence.

3.9 ValFraction

```
ValFraction = 0.15
```

Output. The share of data held back from training, 0.15.

What a holdout is, and why the fraction lives here Measuring a network on the same data it was trained on says almost nothing, because a network with enough capacity can memorise its training examples. Some data is therefore withheld from training entirely and used only for measurement; that is the holdout, or validation set.

The fraction is defined here, once, rather than in each script that needs a split. If two scripts chose their own fractions, or their own random seeds, the same line could be held out by one and trained on by the other, and every accuracy figure in the project would be quietly wrong.

3.10 SplitSeed

```
SplitSeed = 0
```

Output. The fixed random seed used when splitting data, 0.

Why a random choice is made repeatable Deciding which lines are held out is a random choice, but it must be the *same* random choice every time the program runs. A random number generator started from a fixed seed produces the same sequence every time, so the split is both arbitrary, which is what makes it fair, and reproducible, which is what makes it trustworthy.

This also means a holdout set can be recovered later even if it was not saved: replaying the split with the same seed reproduces it exactly.

3.11 PersonalWriterPages

```
def PersonalWriterPages(writer)
```

Inputs

`writer`
: the lower case name of one photographed writer, for example "owen"

Output. A generator yielding `(imagePath, labelPath)` for each of that writer's pages, in sorted filename order.

What it is for Finds one writer's pages and pairs each with its transcript. Every stage that needs real handwriting with known text starts here: training, profile building, and the measurement of segmentation accuracy.

Why the order matters Several measurement scripts pair the n-th detected line crop on a page with the n-th line of that page's transcript. If the pages were returned in whatever order the file system happened to supply, that pairing would still appear to work but would sometimes be comparing a line against the wrong label, and the resulting accuracy would be wrong in a way that no error message would reveal. Sorting makes the order a property of the filenames rather than of the file system.

Why a generator A generator produces its values one at a time as they are asked for, rather than building the whole list first. Here that means a caller can begin processing the first page immediately, and can stop after one page without the cost of listing them all.

4 segment.py

This chapter covers two things that live in one file: the image toolkit used by the whole pipeline, and the page segmenter that is its heaviest user. They are together because the toolkit has no purpose on its own, and because other modules then import the few functions they need from here rather than from a separate module that would exist only to hold them.

The problem this file solves A photograph of a handwritten page arrives as a single large grid of brightness values. Everything downstream, both networks and the glyph extractor, expects one tidy image per line of writing: straightened, with the printed ruling removed, cropped to the writing, and containing that line and no part of its neighbours. Getting from one to the other is this file's job.

That is harder than it sounds for four reasons, and the structure of the file follows from them.

- The photograph is not evenly lit. One side is brighter than the other, there may be a shadow, and the paper is not pure white. So there is no single brightness that separates ink from paper everywhere on the page.
- The page is never perfectly square to the camera. Even a degree of tilt means a horizontal cut through the image crosses several lines of writing.
- The paper is usually ruled, and the printed rules are darker and more regular than the handwriting. If they are not removed they dominate every measurement.
- Lines of handwriting overlap vertically. A descender from one line hangs into the next, and a straight cut between two lines slices it in half, leaving a fragment attached to the wrong line.

How the toolkit is arranged The first half of the chapter is built strictly from the bottom upwards, because each layer is used by the one above it: sums over rectangles, then shape operations built from those sums, then thresholding, then grouping touching pixels into objects, then thinning a stroke to its centreline, then resizing, then geometric transforms, then measurement.

Why so much is built on rectangle sums Almost every operation in the first half needs a local average or a local count: the average brightness near a pixel, whether any ink lies within a few pixels, how much ink is in a window. Computed directly, a window of radius r costs $(2r+1)$ squared additions per pixel, so a radius of 40 on a page of several million pixels is thousands of additions each and the program becomes unusable. A summed-area table reduces every such query to four lookups, whatever the radius [2]. That single idea is what makes the rest of the file practical, which is why it comes first.

4.1 Integral

```
def Integral(img)
```

Inputs

`img` a two-dimensional array of numbers, normally a greyscale image

Output. An array one row and one column larger than the input, in which each entry holds the total of everything above and to the left of it.

What it is for This is the foundation of the toolkit. On its own it computes nothing useful; what it provides is the ability to ask for the total of any rectangle in the image at a cost that does not depend on the rectangle's size. Every local average, every shape-growing operation and every local ink count in this file uses it.

Background: what a cumulative sum is If a list of numbers is replaced by a list in which each entry is the total so far, then the sum of any stretch of the original list can be found by subtracting two entries of the new list. For the list 2, 3, 1, 4 the running totals are 2, 5, 6, 10, and the sum of the middle two numbers is 6 minus 2, which is 4. No addition is needed at query time, however long the stretch.

A summed-area table, also called an integral image, is the same idea in two dimensions [2]. Each cell holds the total of the whole rectangle from the top-left corner of the image down to that cell:

$$\mathrm{ii}[y, x] \;=\; \sum_{i<y} \sum_{j<x} \mathrm{img}[i, j] \tag{4.1}$$

Source: [2]

where

$\mathrm{ii}[y, x]$
: the value stored at row y, column x of the table

$\mathrm{img}[i, j]$
: the original pixel at row i, column j

y, x the row and column being computed, counted from zero

How a rectangle total is then obtained To get the total inside a rectangle, take the table value at its bottom-right corner, which includes the rectangle but also everything above and to the left of it. Subtract the value at the top-right corner, which removes the band above. Subtract the value at the bottom-left corner, which removes the band to the left. Those two subtractions each removed the top-left block, so it has been taken away twice and must be added back once:

$$S \;=\; \mathrm{ii}[y_2, x_2] \;-\; \mathrm{ii}[y_1, x_2] \;-\; \mathrm{ii}[y_2, x_1] \;+\; \mathrm{ii}[y_1, x_1] \tag{4.2}$$

Source: [2]

where

S the total of all pixels inside the rectangle

y_1, x_1
: the top-left corner of the rectangle

y_2, x_2
: one row and one column past the bottom-right corner, so the rectangle stops just before them

Why the extra row and column of zeros The formula above needs to read the table at the row above the rectangle and the column to its left. For a rectangle touching the top or left edge of the image those positions would be outside the array. Storing the table one larger in each direction, with the first row and column filled with zeros, means those reads land on genuine zeros instead and no special case is needed anywhere in the file.

Why the result is stored as floating point The entry in the bottom-right corner is the sum of every pixel in the image. For a 3000 by 2200 photograph with values up to 255 that is about 1.7 billion, which exceeds the range of a 32-bit signed integer. An overflow would not raise an error; it would wrap around to a negative number and quietly corrupt every rectangle total computed from that part of the table. Using 64-bit floating point avoids this with room to spare.

Cost Two passes over the image to build, then four lookups per query regardless of window size. For the background estimate used by ink detection, with a radius of 40 pixels, the direct method would need 6561 additions per pixel; this needs four.

4.2 BoxSum

```
def BoxSum(img, ry, rx)
```

Inputs

`img` a two-dimensional array

`ry` the window radius in rows, so the window spans 2*ry+1 rows

`rx` the window radius in columns

Output. An array the same shape as the input, where each entry is the total of the input inside the window centred on that position.

What it is for Applies the rectangle query of the summed-area table at every position at once. This is the function that actually gets used; `Integral` exists to make it possible.

How it works The four corner positions of every pixel's window are built as whole arrays of indices rather than being computed one pixel at a time, and the table is then read at all of them simultaneously. The result is the same four-term expression as before, evaluated for the entire image in a handful of array operations.

Doing it this way rather than with a loop matters because the loop would run in the Python interpreter, which is roughly two orders of magnitude slower than the same arithmetic performed inside numpy's compiled code. For a multi-megapixel photograph that is the difference between a fraction of a second and several minutes.

What happens at the edges A window centred near the border of the image hangs off the edge. The corner indices are clipped to stay inside the array, so the window is effectively cut short and the total covers fewer pixels than it would in the middle of the image. That is not corrected here, because the correction depends on what the caller wants: `BoxMean` divides by the number of pixels actually covered, while `Erode` compares against that same count. Both get what they need from `BoxCount`.

4.3 BoxCount

```
def BoxCount(ry, rx, h, w)
```

Inputs

`shape or array`
: used for its dimensions

`ry`
: the window radius in rows

`rx`
: the window radius in columns

Output. An integer array giving, for each position, how many pixels its window actually covered once clipped to the image.

What it is for Window sums near the border of an image cover fewer pixels than those in the middle. This counts how many were covered, which is what makes the border behaviour of the operations above it correct rather than approximately correct.

How it works The count is the clipped height of the window multiplied by its clipped width. Because those two depend only on the row and only on the column respectively, each is computed as a short one-dimensional array and the two are then multiplied together, which is far less work than computing a two-dimensional count directly. A quantity that factorises this way is called separable, and the same idea appears again in `Resize` and `Blur`.

Why it matters: two different uses

- `BoxMean` divides the window sum by this count instead of by the nominal window area. Without that, the average near the border would be computed as though the missing pixels were zero, so the estimated background would fall towards black near every edge and ink detection would misbehave in a band around the whole page.
- `Erode` asks whether every pixel in the window is set, which it tests by comparing the window sum against this count. Comparing against the nominal area instead would make the test impossible to satisfy anywhere near the border, and a band around the page would be eroded away.

4.4 BoxMean

```
def BoxMean(a, r)
```

Inputs

`img` a two-dimensional array

`r` a single radius used for both rows and columns

Output. A floating point array the same shape as the input, giving the average value in the window centred on each position.

What it is for This is how the pipeline estimates the local background brightness of a photograph. Comparing a pixel against the average of its own surroundings, rather than against one number for the whole page, is what allows ink to be found correctly on a page that is brighter on one side, or has a shadow across it, or was photographed with the flash off to one side.

How it works

$$\text{mean}[y, x] \;=\; \frac{\text{BoxSum}[y, x]}{\text{BoxCount}[y, x]} \tag{4.3}$$

where

$\text{mean}[y, x]$
: the local average at this position

$\text{BoxSum}[y, x]$
: the total inside the window, from the summed-area table

$\text{BoxCount}[y, x]$
: how many pixels that window covered

Why a flat window rather than a smooth one A gaussian-weighted average, which weights nearby pixels more than distant ones, would be slightly smoother. A flat window is used because it can be computed from the summed-area table in constant time per pixel whatever the radius, where a gaussian cannot. For a background estimate the difference is not visible: what is wanted is the general level over a region, and the fine detail of how that region is weighted does not affect which pixels end up classified as ink.

4.5 Dilate

```
def Dilate(mask, ky, kx)
```

Inputs

`m` a mask: an array of true and false values

`r` the radius to grow by, in pixels

Output. A mask in which a position is true if any true pixel of the input lay within its window.

Background: what morphology is Morphology is the family of operations that change the shapes in a mask rather than the values in an image. The two basic operations are growing a shape outwards and shrinking it inwards, and almost everything else is built by combining them. The window they look through is called the structuring element; here it is always a square.

What it is for Growing shapes is used in this pipeline to make separate pieces of ink touch, so that they can be treated as one object. The clearest example is a letter such as i or j, which is written as two separate marks: growing both slightly makes them touch, after which the grouping stage treats the letter and its dot as a single piece and cannot assign them to different lines of writing.

How it works A position should be turned on if any true pixel lies within its window. Asking whether any pixel in a region is true is the same as asking whether the number of true pixels is greater than zero, and that number is exactly what a window sum provides:

$$\text{dilate}(m)[p] \;=\; \big(\,\text{BoxSum}(m, r, r)[p] \;>\; 0\,\big) \tag{4.4}$$

where

m the input mask, with true counted as 1 and false as 0

p the position being computed

r the radius of the square structuring element

Expressing it as a window sum means dilation inherits the constant cost per pixel of the summed-area table, so growing by twenty pixels costs the same as growing by two.

4.6 Erode

```
def Erode(mask, ky, kx)
```

Inputs

`m` a mask

`r` the radius to shrink by

Output. A mask in which a position is true only if every pixel its window covered was true in the input.

What it is for Shrinking shapes. On its own it is used to remove small specks and to thin a shape; its main use here is as the second half of a closing, described next.

How it works, and the one subtlety Where dilation asks whether *any* pixel in the window is set, erosion asks whether *every* one is. That is true exactly when the number of set pixels equals the number of pixels the window covered:

$$\text{erode}(m)[p] \;=\; \big(\,\text{BoxSum}(m, r, r)[p] \;=\; \text{BoxCount}(r, r)[p]\,\big) \tag{4.5}$$

where

m the input mask

p the position being computed

BoxCount$(r, r)[p]$
how many pixels the window at p actually covered after clipping

The comparison must be against the covered count and not against the full window area of (2r+1) squared. Near the border the window is clipped, so its sum can never reach the full area, and a test against the area would report false for every position within r pixels of the edge. The visible result would be a band eroded away around the whole page, and since erosion is half of a closing, that band would then be removed from every mask the pipeline closes.

4.7 Close

```
def Close(mask, ky, kx)
```

Inputs

`m` a mask

`r` the radius

Output. A mask with gaps narrower than the structuring element filled in, and the original thickness of the shapes preserved.

What it is for Repairing ink that should be continuous but is not. Thresholding a photograph leaves small holes inside strokes where the paper texture shows through, and a stroke drawn with a drying pen can have genuine breaks in it. Both are repaired by a closing.

How it works, and why the two steps are needed A closing is a dilation followed by an erosion by the same radius. The dilation joins anything separated by less than the window, which fills the gaps, but it also makes every stroke fatter. The erosion then removes a layer of the same thickness, which restores the original width. What does not come back is the gap: once two sides have been joined, eroding the joined shape thins it but does not split it again.

$$\text{close}(m, r) \;=\; \text{erode}\big(\text{dilate}(m, r),\, r\big) \tag{4.6}$$

where

m the input mask

r the radius, which sets the largest gap that will be bridged

So a closing bridges a gap narrower than the kernel while leaving a thin stroke thin, which a plain dilation would not. That property is why it, and not dilation, is used before thinning: thinning is sensitive to both holes and thickness, so the operation applied beforehand must fix the first without changing the second.

4.8 AnyWithinRadius

```
def AnyWithinRadius(m, r, axis)
```

Inputs

`mask`
: a mask

`r` the radius

`axis`
: which axis to work along: 0 for down the rows, 1 for across the columns

Output. A mask that is true wherever any element within r positions along that single axis was true.

What it is for A dilation along one direction only. It is used where the tolerance being applied has a direction to it, and growing in both directions would be wrong.

There are two such places. When a printed rule has been detected, it is widened slightly along its own length, so that a rule which is a fraction of a degree off horizontal is still removed over its whole length rather than in broken pieces; widening it vertically as well would eat into the writing above and below it. And when the paper region has been found, it is shrunk inwards by a margin in each axis separately, to keep the slightly soft edge of the sheet from being read as ink.

4.9 LongRunMask

```
def LongRunMask(m, n, axis)
```

Inputs

`m` a mask

`n` the minimum length, in pixels, that a run must reach

`axis`
the direction along which the run is measured

Output. A mask that is true at every pixel belonging to an unbroken run of at least n set pixels along that axis.

What it is for Telling printed lines apart from handwriting. This is the test that removes the ruling from a lined page and the dark edge of a binder, both of which would otherwise dominate every measurement the segmenter makes.

Why run length is the right test A printed rule is drawn by a machine: it is straight, continuous, and spans the page. A pen stroke changes direction constantly, so the number of pixels it continues in any one direction without a break is small. At the sizes this pipeline works with, the threshold is set to between 1.25 and 1.4 times the spacing between lines of writing, which is roughly 60 to 80 pixels; almost no handwritten stroke runs that far in a straight line, and a rule runs much further.

How it works Two passes of cumulative sums. The first finds where a long run begins. A stretch of n pixels starting at position i is entirely set exactly when the number of set pixels inside it equals n, and that count comes from one subtraction of the cumulative sum:

$$\text{starts}[i] \;=\; \big(\, c[i+n] - c[i] \;=\; n \,\big) \tag{4.7}$$

where

starts$[i]$
true when a run of length n begins at position i

c the cumulative sum of the mask along the chosen axis, with a leading zero

n the minimum run length

i the candidate starting position

The second pass turns starting positions into coverage. A run of length n that covers position j must begin somewhere between n-1 places before j and j itself, and it cannot begin so late that it would run off the end. A second cumulative sum over the starts then answers "is there any start in that range" with another single subtraction:

$$\text{covered}[j] \;=\; \big(\, cs[h_j + 1] - cs[l_j] > 0 \,\big), \qquad l_j = \max(0,\, j - n + 1),\; h_j = \min(j,\, L - n) \tag{4.8}$$

where

covered[j]
: true when position j lies inside some long run

cs the cumulative sum of the starts array

l_j, h_j
: the first and last positions at which a run covering j could begin, clamped to the valid range

L the length of the axis being measured

Why it is done with cumulative sums The cost is proportional to the number of pixels and does not depend on n. Scanning each window directly would cost n times as much, and n here is around 80.

How one function handles both directions The axis of interest is moved to the front of the array, the fixed code runs down it, and the axis is moved back afterwards. Moving an axis in numpy only changes how the array is interpreted, not the data in memory, so this costs nothing and avoids writing the same logic twice.

> *A simpler equivalent exists.* `RowRunLength`*, described next, already gives every pixel the length of its own run, which makes this function the same as asking whether that length is at least n, with a transpose for the other axis. That form was checked against this one on 4800 randomly generated masks of every shape, with n from 1 to 40 and both axes, and never disagreed; on a page-sized array it also ran about twice as fast, because it makes one labelling pass and one count instead of building two cumulative-sum arrays and several arrays of clamped indices.*

4.10 RowRunLength

```
def RowRunLength(mask)
```

Inputs

`mask`
: a two-dimensional mask; runs are measured across the rows

Output. An integer array in which every set pixel holds the length of the unbroken horizontal run it belongs to, and every clear pixel holds zero.

What it is for Measuring how far ink continues horizontally through each pixel. It is used to find printed rules, and in `BinarizeLine` to delete them from a line crop.

How it works, in three vectorised steps First, the places where a run begins are found as the positions where the row steps from clear to set, which is a comparison between the row and a copy of itself shifted by one.

Second, a cumulative sum over those starting positions numbers the runs consecutively: the first run becomes 1, the second 2, and so on. The numbering is then cleared wherever there is no ink, so that only ink carries a run number.

Third, counting how many pixels carry each number gives the length of each run, and indexing those counts by the run number image hands every pixel the length of its own run.

Why the numbering does not leak between rows The cumulative sum is taken over the whole array flattened into one long sequence, which would be wrong if a run could continue from the end of one row into the start of the next. It cannot: the comparison that finds starts is padded at both ends of every row, so each row begins and ends with a boundary and the last run of a row is always closed before the next row's first run begins.

Background: what counting occurrences means here The counting step uses a standard operation that, given a list of non-negative whole numbers, returns how many times each value occurs. Applied to an image of run numbers it gives the size of every run in one pass, which is why no loop over runs is needed.

4.11 Otsu

```
def Otsu(v, hi=None)
```

Inputs

`v` an array of pixel values, either whole numbers or fractions

`hi` when the input is fractional, the largest value worth considering; defaults to 1.2, which suits the ratio image produced by `InkMask` because nothing above that carries ink

Output. A threshold. For whole-number input it is a brightness level from 0 to 255; for fractional input it is a value in the input's own units. Pixels below the threshold are ink.

What it is for Deciding, for a given image, which brightness separates ink from paper. The point of computing it rather than fixing it is that no single number works for two photographs: one taken in daylight and one under a lamp have completely different brightness ranges, and a fixed threshold would classify the whole of one of them as paper.

Background: what a histogram is A histogram counts how many pixels fall at each brightness level, ignoring where they are. For a page of handwriting it usually has two humps: a large one at the bright end for the paper and a small one at the dark end for the ink, with a valley between them. The threshold should sit in that valley.

The idea: measure the quality of every candidate threshold Otsu's method [3] turns the choice into something measurable. Any candidate threshold divides the pixels into two groups. A good threshold is one where the two groups are as different from each other as possible, which is measured by the variance *between* the groups:

$$\sigma_B^2(t) \;=\; \omega_B(t)\,\omega_F(t)\,\big[\mu_B(t) - \mu_F(t)\big]^2 \tag{4.9}$$

Source: [3]

where

$\sigma_B^2(t)$
: the between-class variance, the quantity being maximised

t
: the candidate threshold

$\omega_B(t)$
: the fraction of all pixels at or below t, that is the share in the darker group

$\omega_F(t)$
: the fraction above t, so the two fractions add to 1

$\mu_B(t)$
: the mean brightness of the darker group

$\mu_F(t)$
: the mean brightness of the brighter group

Reading the formula: the squared bracket is how far apart the two group means are, so it rewards a threshold that genuinely separates dark from bright. The product of the two fractions in front is largest when the groups are of comparable size and falls to zero when either is empty, which prevents the degenerate answer of putting the threshold at one extreme and calling almost everything one group.

Why maximising this is the right objective The total variance of the image is a fixed property of the image and does not depend on the threshold. It can be split exactly into the variance within the two groups plus the variance between them. Since the total is fixed, making the between-group part as large as possible is the same as making the within-group part as small as possible, that is, choosing the threshold that makes each group internally as uniform as it can be. The between-group form is used because it needs only running totals, where the within-group form would need a second pass over the data for every candidate.

How all 256 candidates are evaluated at once Writing p for the histogram scaled so that its entries sum to one, the two quantities needed are themselves cumulative sums, and the criterion rearranges into a form built entirely from them:

$$\sigma_B^2(t) \;=\; \frac{\big[\mu_T\,\omega(t) - \mu(t)\big]^2}{\omega(t)\,\big(1-\omega(t)\big)} \tag{4.10}$$

Source: [3]

where

$\omega(t)$ the cumulative sum of p up to t, the share of pixels at or below t

$\mu(t)$ the cumulative sum of p weighted by brightness, the contribution of those pixels to the mean

μ_T the mean brightness of the whole image, which is the final value of the weighted cumulative sum

Both cumulative sums are computed once, the expression is evaluated for every t simultaneously as array arithmetic, and the largest entry is the answer. No loop over thresholds is needed.

The guard in the denominator At the two extreme thresholds one group is empty, so the product in the denominator is exactly zero. Dividing by it would give infinity, and since the method picks the largest value those positions would always win. The denominator is therefore floored at a very small positive number, which leaves every genuine candidate unchanged and makes the degenerate ones lose.

Why whole numbers and fractions are handled separately A whole-number image has exactly 256 possible values, so its histogram can be counted exactly and the answer returned as a brightness level. A fractional image, such as the ratio of each pixel to its local background, has no natural set of levels, so its values are clipped at `hi` and sorted into 256 bins, and the answer is returned as the value at the centre of the chosen bin rather than as a bin number.

4.12 BinarizeLine

```
def BinarizeLine(gray, stripRules=True, closeGaps=True)
```

Inputs

`gray`
: a greyscale image of one line of writing

Output. A mask of the ink in that crop, with small holes closed, specks removed and printed rules deleted.

What it is for The standard way of turning a grey line crop into ink. The glyph extractor uses it before tracing strokes, and the synthesiser uses it to measure its own rendered output with the same code that measured the writer's real pages, which is what makes the two comparable.

The four steps and why each is needed **Threshold.** `Otsu` chooses the level, so nothing about the brightness of this particular photograph is assumed.

Close small holes. Thresholding a photograph leaves single-pixel holes inside strokes where the paper texture shows through. These matter more than their size suggests, because the next stage after binarising is usually thinning, and thinning preserves the topology of the shape it is given: a hole in a stroke forces the resulting centreline to split and run round both sides of it, producing a small loop that was never written.

Remove specks. Pixels are grouped into connected components and any group too small to be a pen mark is deleted as dust or camera noise. The decision is made for a whole group at once, so a letter is never partly removed.

Delete printed rules. Horizontal runs longer than a limit are removed, on the reasoning given under `LongRunMask`: a pen stroke does not run level for that distance, and a printed rule does nothing else.

4.13 CoreBand

```
def CoreBand(ink)
```

Inputs

`ink` a mask of one line of writing

Output. A pair of row indices marking the top and the baseline of the x-height band, or nothing at all if there is no ink or the band is under three rows tall.

Background: what the x-height is In any piece of writing, most lower case letters have a body of roughly the same height, such as a, c, e, m and o. The height of that body, measured up from the line the writing sits on, is called the x-height, after the letter x. Some letters reach above it, such as b, d and l, and these are called ascenders; some reach below the line, such as g, p and y, and these are called descenders.

What it is for, and why it is so important The x-height is the unit of measurement for the entire pipeline. Every stored letter shape is divided by it, so that a letter extracted from a photograph at one resolution can be drawn at any physical size. Every proportion in a style profile is expressed as a multiple of it: how far ascenders reach, how far descenders drop, how wide the gaps between words are, how thick the pen is. Without a reliable x-height none of those numbers would be comparable between two writers or between two photographs of the same writer.

How it works Counting the ink in each row of the crop gives a one-dimensional summary called a projection profile. Rows that cross the bodies of the letters contain ink from nearly every letter on the line; rows that contain only ascenders or only descenders contain ink from the few letters that have them. The band is therefore the set of rows whose ink count is a large fraction of the maximum:

$$\mathcal{B} \;=\; \Big\{\, y \;:\; \mathrm{prof}[y] \;\geq\; 0.45 \max_{y'} \mathrm{prof}[y'] \,\Big\} \tag{4.11}$$

where

$\mathcal{B}$ the set of rows making up the band; its first and last members are returned

$\mathrm{prof}[y]$
: the number of ink pixels in row y

0.45 the fraction of the peak that a row must reach to count as part of the band

Where the 0.45 comes from It is the one tuned constant, and the reasoning behind it is that the quantity being thresholded has a wide gap in it. A row through the letter bodies collects ink from most letters on the line, while a row above the band collects ink only from the few ascenders, which in ordinary English text is a small minority of letters. Any threshold within that gap gives the same answer, so the exact figure is not critical; what would fail is a value low enough to admit the ascender rows, which would inflate the measured x-height, or high enough to admit only the single densest row, which would collapse the band to nothing.

The rejection case A band under three rows tall is reported as a failure rather than returned. That is what happens when a crop contains a stray mark rather than a line of writing, and returning a nonsensical x-height would propagate into every measurement taken from that crop.

4.14 LabelComponents

```
def LabelComponents(mask)
```

Inputs

`mask`
: a two-dimensional mask of ink

Output. A pair: an array in which every ink pixel carries the number of the object it belongs to, counting from 1, with 0 for background; and the number of objects found.

Background: what a connected component is Two ink pixels that touch belong to the same physical mark. Following that relation outwards from any pixel gives a group of pixels all connected to one another, called a connected component. On a page of handwriting the components are the individual pen marks: a letter, the dot of an i, a speck of dust, or several joined letters where the writer did not lift the pen.

What counts as touching has to be decided. Counting only the four pixels above, below, left and right is called four-connectivity; also counting the four diagonals is eight-connectivity. This implementation uses eight, because a thin pen stroke drawn at an angle advances diagonally, and under four-connectivity such a stroke would break into a chain of separate dots.

What it is for This is what lets the pipeline reason about objects rather than pixels, and that distinction runs through the whole segmenter. Whether a mark is dust is a question about a whole object; which line of writing a letter belongs to is a question about a whole object. Tests made pixel by pixel would remove half a letter, or split one letter across two lines.

How it works: runs instead of pixels Each row is first reduced to horizontal runs of ink, found from the places where the row steps from clear to set and back again. Every pixel in a run necessarily belongs to the same component, so the algorithm reasons about runs, of which there are far fewer. This is the run-based two-scan approach [4].

Each run is compared against the runs of the row above. Because the runs in both rows are already in left-to-right order, a single index walks forward through the previous row and never goes back, so the total work is proportional to the number of runs rather than to their number squared. A run above counts as touching when its span overlaps, and the comparison is written so that a touch at a single corner counts, which is what produces eight-connectivity.

The problem that makes this non-trivial Labels must be handed out while scanning downwards, before it is known how a shape continues. A letter U is the clearest example: its two arms are separate for many rows and receive different labels, and only at the bottom does a run join them, revealing that the two labels were always one object.

Union-find, and why it is fast Rather than going back and relabelling, which would mean rescanning everything above, the fact that two labels are the same is recorded. Each label stores a pointer to another label, and following the pointers reaches a representative for the group; joining two groups is then one assignment, pointing the larger representative at the smaller. This structure is called union-find [5].

Chains of pointers could grow long, so each lookup shortens the chain it walks by pointing every label it passes at its grandparent:

```
while parent[a] != a:
    parent[a] = parent[parent[a]]   # halve the chain
    a = parent[a]
```

With that shortening the cost of a lookup is so close to constant that for any practical image it can be treated as constant [5].

The final renumbering, and why callers depend on it Because labels were handed out before the merges were known, the surviving labels have gaps: label 7 may have been absorbed into label 3, so no pixel carries 7 any more. A closing pass resolves every label to its representative and renumbers the survivors consecutively from 1. Callers rely on this, because it means the components are numbered 1 to n with nothing missing, so their sizes can be counted in a single operation and used directly as array indices.

4.15 PieceBounds

```
def PieceBounds(labelAt, ys, xs, n, H, W)
```

Inputs

`pl` the component number of every ink pixel

`ys` the row coordinate of every ink pixel

`xs` the column coordinate of every ink pixel

`nc` how many components there are

`H` the image height

`W` the image width

Output. Five arrays indexed by component number: each object's topmost row, bottommost row, leftmost column, rightmost column and pixel count.

What it is for Every ownership rule in the segmenter asks geometric questions about objects: how tall is this piece, how wide is it, where does it sit, how much ink does it contain. This answers all of them for every object at once.

Why it is written as a scatter rather than a loop The obvious implementation loops over components and, for each one, selects the pixels belonging to it and takes their minimum and maximum. That reads the whole image once per component, so on a page with eight hundred pen marks it reads the image eight hundred times.

Instead the coordinates are scattered into per-component accumulators in a single pass, using operations that take a list of destinations and a list of values and apply a minimum or a maximum at each destination. The pixel count comes from the same occurrence-counting operation used by `RowRunLength`. The result is one pass over the ink rather than one pass per object.

4.16 Skeletonize

```
def Skeletonize(mask)
```

Inputs

`mask`
: a mask of ink

Output. A mask of the one-pixel-wide centreline of that ink.

What it is for A photograph records a pen stroke as a thick band of dark pixels. What the project needs is the path the pen travelled down the middle of that band, because a path can be stored as a short list of points, compared against another writing of the same letter, and above all redrawn by a machine holding a pen. Thinning recovers it.

Background: what thinning must preserve Deleting pixels from the edge of a shape repeatedly will eventually make it thin, but done naively it destroys the shape: an o can be broken open into a c, and a crossing can fall apart into four disconnected arms. A thinning algorithm must therefore delete only pixels whose removal cannot change how the shape is connected. The algorithm here is the standard parallel method of Zhang and Suen [6].

The two quantities the test is built from Consider a pixel and its eight neighbours, numbered clockwise from the one directly above. Two numbers describe the local situation:

$$B(P_1) \;=\; \sum_{k=2}^{9} P_k, \qquad A(P_1) \;=\; \#\big\{\, k \;:\; P_k = 0 \;\wedge\; P_{k+1} = 1 \,\big\} \tag{4.12}$$

Source: [6]

where

P_1 the pixel being considered

$P_2 \ldots P_9$
: its eight neighbours, clockwise from the north, each 1 where there is ink and 0 where there is not

$B(P_1)$
: how many of the eight neighbours are ink

$A(P_1)$
: how many times the neighbours change from empty to ink while walking once round the ring, the ring being treated as closed so that the ninth neighbour is the second again

The second quantity deserves an explanation, because it is the heart of the method. Walking once round the eight neighbours and counting the steps from empty into ink tells you how many separate groups of ink surround the pixel. If the answer is one, all the neighbouring ink forms a single unbroken arc, so the pixel lies on the edge of the stroke and deleting it leaves that arc joined. If the answer is two or more, the pixel sits between separate groups and is the only thing holding them together, so deleting it would break the stroke apart or open a hole.

The deletion test

$$\begin{aligned} &\text{(a)} && 2 \le B(P_1) \le 6 \\ &\text{(b)} && A(P_1) = 1 \\ &\text{(c)} && P_2 \cdot P_4 \cdot P_6 = 0 \\ &\text{(d)} && P_4 \cdot P_6 \cdot P_8 = 0 \end{aligned} \tag{4.13}$$

Source: [6]

where

(a) at least two neighbours, so the tip of a stroke is kept and the line is not eaten back from its end; at most six, so a pixel deep inside a thick region is not removed before the layers outside it

(b) the neighbouring ink forms a single arc, so removal cannot disconnect anything

(c), (d)
: the corner conditions, which restrict this sub-iteration to thinning from particular sides; in the second sub-iteration they become P2$P4$P8 = 0 and P2$P6$P8 = 0

Why there must be two alternating sub-iterations If deletion were allowed from every side at once, a line two pixels wide would vanish completely: both halves satisfy the test simultaneously and both are deleted in the same pass, leaving nothing behind. Alternating the corner conditions means each pass thins from opposite sides, so one side is always held while the other is removed. The result stays connected and the centreline does not drift to one side of the original stroke.

Termination and cost The pair of sub-iterations repeats until a full pass deletes nothing, at which point no pixel can be removed without changing the connectivity, and the shape is one pixel wide everywhere. The number of passes is proportional to the thickest stroke, since one layer comes off each side per pass, and each pass is a fixed number of whole-array operations.

> *This is also why* `BinarizeLine` *closes small holes before thinning. Thinning preserves connectivity faithfully, including the connectivity of a hole that was never written: a pinhole inside a stroke forces the centreline to divide and pass round both sides of it, leaving a small loop in the recorded pen path that the writer never drew.*

4.17 _Weights

```
def _Weights(inSize, outSize, kernel)
```

Inputs

`inSize`
: the length of the axis before resizing

`outSize`
: the length wanted afterwards

`kernel`
: which filter to use: "linear" for a triangle, "box" for a flat window

Output. A matrix with one row per output sample and one column per input sample; each row holds the weight of every input sample in that output sample, and each row sums to one.

Background: what resampling actually is Changing the size of an image means asking for brightness values at positions where there were none. In an output image half the width, output pixel 1 sits between input pixels 2 and 3, so its value has to be built from the inputs around it. Choosing how to build it is choosing a filter, and a filter is described by a weight for each nearby input sample.

What it is for Every size change in the pipeline goes through one resampler, so there is exactly one piece of code to get right: reducing a page for the skew search, fitting a crop to the networks' fixed input size, and rendering glyphs. This function builds the weights and `Resize` applies them.

Where an output sample sits

$$s \;=\; \frac{\text{inSize}}{\text{outSize}}, \qquad c_i \;=\; \left(i + \tfrac{1}{2}\right) s \tag{4.14}$$

Source: [7]

where

s the scale factor; greater than one means the image is being shrunk

c_i the centre of output sample i, expressed in input coordinates

i the index of the output sample, counting from zero

The halves matter. A pixel is not a point at an integer coordinate; it is a small square whose centre lies at an integer plus a half. Treating pixels as points displaces the whole image by half a pixel, which shows as a slight softening and as a systematic shift that accumulates if an image is resized more than once.

Why the filter widens when shrinking This is the part that is easy to get wrong and expensive to get wrong. The width of the filter is multiplied by the scale factor whenever that factor exceeds one:

$$f \;=\; \max(1,\, s), \qquad \text{support} \;=\; k_0\, f \tag{4.15}$$

Source: [7]

where

f the widening factor: one when magnifying, equal to the scale factor when minifying

s the scale factor defined above

k_0 the base support of the chosen kernel: one for the triangle, one half for the box

support
: how far the filter reaches from its centre, measured in input samples

If the filter did not widen, shrinking by a factor of four would consult one input sample in every four and ignore the rest. Thin strokes falling between the consulted samples would disappear entirely, and fine texture would turn into coarse false patterns, an effect called aliasing. Widening the filter in step with the reduction means every input sample contributes to some output sample [7]. The page reduction used by the skew search shrinks by exactly such a factor, so this is load bearing rather than theoretical.

The two kernels

$$w_{\text{linear}}(t) = \begin{cases} 1 - |t| & |t| < 1 \\ 0 & \text{otherwise} \end{cases} \qquad w_{\text{box}}(t) = \begin{cases} 1 & |t| \le \frac{1}{2} \\ 0 & \text{otherwise} \end{cases} \tag{4.16}$$

Source: [7]

where

t the distance from the centre of the output sample's window to an input sample, measured in filter widths

$w(t)$ the weight that input sample receives

The triangle weights nearby samples more than distant ones and falls smoothly to zero, which gives a smooth result. The box weights everything inside the window equally, which is exactly a local average and is the right choice when reducing by a whole number, because then each output pixel covers a whole number of input pixels.

Edges, and why weights are accumulated rather than assigned A window near the border reaches past the end of the axis. Those out-of-range positions are folded back onto the first or last sample, so several positions in one window can refer to the same input sample. Their weights must therefore be added together rather than written one over another: assigning would keep only the last and discard the rest, leaving the edge of the image too dark.

Each row is finally divided by its own total so that it sums to one. That is what guarantees a uniform area keeps its brightness after resizing, and in particular that the white padding around a fitted crop does not come back grey.

4.18 Resize

```
def Resize(arr, newW, newH, kernel='linear')
```

Inputs

`img` the image to resize

`newW`
: the width wanted

`newH`
: the height wanted

`kernel`
: which filter to use

Output. The resampled image at the requested size, in the same number format as the input.

What it is for The single resampling routine used by the whole pipeline.

Background: what separable means, and why it is worth it Resizing in two dimensions could be done with a two-dimensional filter, where each output pixel is a weighted sum over a square of input pixels. For a filter k samples across that is k squared multiplications per output pixel.

A filter is called separable when its two-dimensional weight is the product of two one-dimensional weights, one per axis. Both kernels used here are separable, so the same result is obtained by resizing along the rows and then along the columns, at a cost of 2k multiplications per pixel instead of k squared [7]. At the filter widths used when reducing a page that is roughly an order of magnitude less work.

How it works `_Weights` is called once per axis and the two matrices are applied in turn as matrix products, so the arithmetic happens inside numpy's compiled routines rather than in interpreted loops.

4.19 Halve

```
def Halve(a)
```

Inputs

`img` the image to reduce

Output. The image at half the width and half the height.

What it is for A fast reduction for stages that do not need full resolution: finding the sheet of paper inside the photograph, and the coarse stage of the skew search.

Why it is a separate function At the exact ratio of two to one a box filter reduces to averaging each two-by-two block, which can be done by reshaping the array and taking a mean, with no weights built at all. It is both the cheapest correct reduction at that ratio and exactly what the general resampler would compute, so nothing is lost by using it.

Why reduce at all: every operation in the toolkit costs time proportional to the number of pixels, so halving each axis quarters the work. For a decision that only needs to locate something to within a few pixels, the reduction costs nothing in accuracy.

4.20 FitFixedCanvas

```
def FitFixedCanvas(gray, height, width, padValue=255)
```

Inputs

`gray`
: a line crop

`height`
: the canvas height the networks expect, 64

`width`
: the canvas width the networks expect, 640

`padValue`
: the value filling the unused area; 255 means white paper

Output. An image of exactly the requested size containing the crop, scaled to fit, centred vertically and flush to the left, on a paper-coloured background.

What it is for Both trained networks accept one exact input size and nothing else, because the number of weights in their final layers depends on it. Every crop therefore has to be brought to that size before it can be read, and how that is done affects the answer.

The central rule: one scale factor, never two

$$\sigma \;=\; \min\!\left(\frac{H_{\text{canvas}}}{H_{\text{crop}}},\; \frac{W_{\text{canvas}}}{W_{\text{crop}}}\right) \tag{4.17}$$

where

σ
: the single scale factor applied to both axes

H_{canvas}, W_{canvas}
: the fixed canvas size the networks were trained on

H_{crop}, W_{crop}
: the size of the crop being fitted

Taking the smaller of the two ratios means the crop is scaled until whichever axis runs out of room first exactly fills the canvas, leaving the other axis short; the leftover space is filled with the paper colour.

The alternative, scaling each axis independently so the crop fills the canvas exactly, would change the ratio of width to height. That ratio is precisely what both networks read: the reader distinguishes letters partly by their proportions, and the writer namer treats proportion as a characteristic of a hand. A stretched crop is, to these networks, a different person's handwriting.

Why centred vertically but flush to the left It matches how the training crops were produced. Writing starts at the left margin, so the left edge carries meaning and is preserved. The vertical position of a line of writing inside its crop carries none, so centring removes a difference the network would otherwise have to learn to ignore.

A small implementation guard The scaled size is rounded to whole pixels, and rounding can produce a result one pixel larger than the canvas. The final copy therefore limits both the source and the destination, so an overshoot is trimmed instead of raising an error.

4.21 _Sample

```
def _Sample(a, sx, sy, fill, edge='area')
```

Inputs

`img` the source image

`sy` for each output pixel, the row to read from, which may fall between rows

`sx` the corresponding column, which may fall between columns

Output. The sampled values, one per requested position.

What it is for Rotating or bending an image asks for brightness at positions falling between pixels. This supplies them, and is the shared sampler behind every geometric transform in the file.

Background: bilinear interpolation The value at a position between four pixels is their weighted average, each weight given by how close the position is to that pixel. It is computed one axis at a time: blend the upper pair horizontally, blend the lower pair horizontally, then blend those two results vertically.

$$\begin{aligned} v_{\text{top}} &= p_{00}\,(1 - f_x) + p_{01}\,f_x \\ v_{\text{bot}} &= p_{10}\,(1 - f_x) + p_{11}\,f_x \\ v &= v_{\text{top}}\,(1 - f_y) + v_{\text{bot}}\,f_y \end{aligned} \tag{4.18}$$

Source: [7]

where

p_{00}, p_{01}
: the two source pixels above the position, left and right

p_{10}, p_{11}
: the two below it

f_x how far the position lies between the left and right columns: 0 at the left pixel, 1 at the right

f_y the same fraction between the upper and lower rows

v the interpolated value returned

Why interpolate at all The cheaper alternative is to take whichever single pixel the position is nearest. On a rotated image the result is a visible staircase along every edge, because neighbouring output pixels jump between different source pixels. Blending removes it, and that matters here beyond appearance: the ink masks and skeletons computed downstream are sensitive to jagged edges, which produce spurious spurs when thinned.

4.22 _Like

```
def _Like(out, like)
```

Inputs

`out` a floating point result

`ref` the array whose number format should be matched

Output. The result converted back to the reference's format, rounded and clipped to its range.

What it is for Interpolation must be done in floating point, but the pipeline passes images around as whole numbers from 0 to 255, because that is what a photograph is and what the networks were trained on. This performs the conversion back.

The two things it gets right It rounds rather than truncating. Truncation would bias every transformed image slightly darker, since 254.7 would become 254 rather than 255, and the bias would accumulate through successive transforms.

It clips to the valid range. A blend can overshoot slightly past 255 or below 0, and converting such a value to an 8-bit whole number without clipping makes it wrap around to the opposite extreme, turning a bright pixel black. A handful of such pixels is enough to disturb a threshold or a component count.

4.23 Rotate

```
def Rotate(gray, deg, fill=255)
```

Inputs

`img` the image to rotate

`deg` the angle in degrees

`fill`
: the value used where no source pixel exists

Output. The rotated image, on a canvas the same size as the input.

What it is for Straightening a photographed page. The page is never square to the camera, and a tilt of even one degree means a horizontal cut through the image crosses more than one line of writing, which makes every later stage harder.

Background: forward and inverse mapping There are two ways to transform an image. The obvious one, called forward mapping, takes each input pixel, works out where it should land, and writes it there. The problem is that the destinations are fractional and unevenly spread, so some output pixels receive two input pixels and others receive none, leaving a scatter of holes.

The method used here is inverse mapping: for each *output* pixel, work out where it came from in the input and read that position. Every output pixel is then visited exactly once, so holes are impossible [7]. The source position is fractional, which is what `_Sample` is for.

The mapping

$$\begin{aligned} s_x &= \cos\theta\,(x - c_x) + \sin\theta\,(y - c_y) + c_x \\ s_y &= -\sin\theta\,(x - c_x) + \cos\theta\,(y - c_y) + c_y \end{aligned} \tag{4.19}$$

Source: [7]

where

x, y the output pixel being filled

s_x, s_y
: the position in the input it is read from

c_x, c_y
: the centre of the image, which the rotation is about

θ the rotation angle in radians

Note that the signs are those of a rotation by minus theta: because the mapping runs backwards from output to input, it must be the inverse of the rotation being applied.

Filling rather than clamping A source position outside the image is filled with the paper value rather than clamped to the nearest edge pixel. Clamping would smear whatever lies on the boundary across the whole corner, producing streaks that the ink detector would then have to cope with.

The edge convention, and why it was determined experimentally Deciding whether a source position lies inside the image is less obvious than it looks. If a pixel is a point at an integer coordinate, the image spans 0 to size minus 1. If a pixel is a unit square centred on its index, the image spans minus a half to size minus a half. This implementation uses the second, area, convention.

> *The convention was not guessed. Replacing a library rotation with this one was verified by comparing the two on a test page: with the area convention the two agreed to a maximum difference of one grey level out of 255, while the other plausible convention differed by up to 245 along the edges. A difference of that size is not a rounding detail, it is a row of pixels in the wrong place.*

4.24 Warp

```
def Warp(gray, coeffs, fill=255)
```

Inputs

`img` the source image

`coeffs`
: six numbers for an affine transform, or eight for a perspective one

`fill`
: the background value

Output. The transformed image.

What it is for One function covering every straight-line-preserving transform the pipeline needs: scaling, shearing, rotation, and the keystone effect of a page photographed from an angle. It is used mainly to generate training variations of a crop.

Background: affine and perspective An affine transform is any combination of scaling, rotation, shear and translation. It keeps parallel lines parallel, so a rectangle becomes a parallelogram. A perspective transform, also called a homography, additionally allows parallel lines to converge, so a rectangle becomes any four-sided shape. That is what happens when a flat page is photographed obliquely: the far edge appears shorter than the near one.

The mapping

$$s_x = \frac{a\,x + b\,y + c}{g\,x + h\,y + 1}, \qquad s_y = \frac{d\,x + e\,y + f}{g\,x + h\,y + 1} \tag{4.20}$$

Source: [7]

where

x, y the output pixel being filled

s_x, s_y
: the source position it is read from

a, b, d, e
: the linear part: scaling, rotation and shear

c, f the translation

g, h the perspective part; both zero gives a purely affine transform, since the divisor is then 1 everywhere

The divisor is what distinguishes the two cases. When g and h are zero it equals one at every pixel and the mapping is linear; otherwise it varies across the image, so equal steps in the output correspond to unequal steps in the input, which is exactly the foreshortening that perspective produces.

The half-pixel convention The coefficients act on the *centre* of the output pixel, so a half is added to the output coordinate before the mapping is applied and subtracted from the result afterwards. This follows from the same pixels-are-squares reasoning as in `_Weights`.

> *Like the rotation convention, this was settled by measurement rather than argument: with the half-pixel offsets in place the output matched the library implementation it replaced on zero pixels of difference across a test page, and without them the entire image was displaced by half a pixel.*

4.25 Blur

```
def Blur(gray, sigma, mode='edge')
```

Inputs

`img` the image to blur

`sigma`
: the standard deviation of the gaussian, in pixels, which sets how far the blurring reaches

Output. The blurred image, in the input's number format.

What it is for Two uses. It smooths the random displacement fields used for training augmentation, which is what turns random noise into a coherent bending of the strokes. And it softens the edge of synthesised ink, so that a drawn stroke has the gradual edge of a real pen rather than a hard step, which matters because the networks judging that output were trained on photographs of real ink.

Background: what a gaussian blur is Each output pixel is a weighted average of its neighbourhood, with weights following the bell-shaped gaussian curve: strongest at the centre and falling away smoothly with distance. Compared with a flat average it produces no visible edge at the limit of the window, because the weights reach zero gradually.

Why it is done as two one-dimensional passes A two-dimensional gaussian factorises exactly into the product of two one-dimensional gaussians:

$$G(x,y) \;=\; \exp\!\left(-\frac{x^2+y^2}{2\sigma^2}\right) \;=\; \exp\!\left(-\frac{x^2}{2\sigma^2}\right)\cdot\exp\!\left(-\frac{y^2}{2\sigma^2}\right) \;=\; G(x)\,G(y) \tag{4.21}$$

where

$G(x,y)$
: the two-dimensional weight at an offset of x columns and y rows from the centre

σ the standard deviation, which sets the width of the bell

$G(x)$, $G(y)$
: the one-dimensional weights along each axis

So blurring down the rows and then across the columns gives exactly the same answer as a single two-dimensional pass, at a cost of 2k multiplications per pixel instead of k squared. This is the same separability argument as for `Resize`.

Truncation and normalisation The gaussian never reaches zero, so the kernel is cut off at about three standard deviations, where under one per cent of the total weight remains. The remaining weights are then divided by their sum so they total one, which keeps the overall brightness of the image unchanged; without that, blurring would darken the image by the fraction of weight that was discarded.

4.26 Elastic

```
def Elastic(gray, sigma, alpha, rng)
```

Inputs

`img` the image to distort

`rng` a seeded random number generator, so a distortion can be reproduced

`alpha`
: how far pixels are moved, in pixels

`sigma`
: how smoothly the displacement varies, in pixels

Output. A locally bent version of the image.

What it is for Training augmentation. A network trained on a fixed set of examples can memorise them; showing it plausible variations of each example forces it to learn the general pattern instead. For handwriting the most useful variation is the one that looks like the same person writing the same words again, which is not a rotation or a scaling but a local bending of the strokes [8].

How it works

$$\begin{aligned} d_x &= \alpha \cdot \mathrm{Blur}\big(U(-1,1),\, \sigma\big) \\ d_y &= \alpha \cdot \mathrm{Blur}\big(U(-1,1),\, \sigma\big) \\ \mathrm{out}[y,x] &= \mathrm{sample}\big(\mathrm{img},\; y + d_y[y,x],\; x + d_x[y,x]\big) \end{aligned} \tag{4.22}$$

Source: [8]

where

d_x, d_y
: the displacement fields: how far the pixel at each position moves horizontally and vertically

$U(-1,1)$
: an array of independent random numbers drawn uniformly between minus one and one, one per pixel

α the displacement magnitude in pixels

σ the blur width, which controls how smoothly the displacement varies from pixel to pixel

sample
: bilinear interpolation, since the displaced positions are fractional

Why the smoothing is the essential step Without the blur, each pixel would move independently of its neighbours and the image would be scrambled into noise: strokes would break into disconnected dots. Blurring the displacement field means nearby pixels move by nearly the same amount, so a stroke moves as a stroke, bending and stretching while staying continuous. The width of the blur sets the scale of the bending: small gives wobbly strokes, large gives a gentle warp of the whole crop.

The two parameters interact. A large displacement with a small blur tears the writing apart; a large displacement with a large blur is nearly a rigid shift. Useful augmentation lives between those extremes.

4.27 DrawPolyline

```
def DrawPolyline(img, pts, value, width)
```

Inputs

`img` the array to draw into, which is modified in place

`pts` the path as a list of (x, y) points

`value`
: the ink level to write, 0 being black

`width`
: the pen width in pixels

Output. The same array, returned for convenience when chaining calls.

What it is for Drawing a pen stroke into an image. It is used to render synthesised handwriting so that the networks can judge it, to redraw ink at a uniform width inside stroke normalisation, and in the measurement harness to compare two sets of geometry. Because all three use the same function, they all see ink laid down the same way.

The idea: fill by distance rather than by stepping along the line The familiar way to draw a line is to step along it filling pixels. That is fast but makes a round pen awkward, because the ends and the corners between segments have to be handled separately or they come out square and notched.

Instead, every pixel near the segment is tested by how far it lies from the nearest point *on* the segment, and filled if that distance is within half the pen width. The nearest point is found by projecting onto the segment and clamping to its ends:

$$t \;=\; \mathrm{clamp}\!\left(\frac{(P-A)\cdot(B-A)}{\|B-A\|^2},\,0,\,1\right), \qquad Q \;=\; A + t\,(B-A) \tag{4.23}$$

where

A, B
: the two ends of the segment being drawn

P the pixel being tested

t how far along the segment the nearest point lies, 0 at A and 1 at B

Q the nearest point on the segment to P

$(P-A)\cdot(B-A)$
: the dot product, which measures how far along the segment's direction P lies

A pixel is then filled when the distance from P to Q is at most half the pen width.

Why the clamp gives round ends and joins for free Without the clamp, t could fall outside the range 0 to 1 and the distance would be measured to the infinite line through A and B, so ink would extend beyond the ends of the segment. Clamping means that for a pixel beyond an end, the nearest point is the end itself, so the filled region there is a disc of radius half the pen width. That is a round cap. At a corner between two segments both discs overlap, which is a round join. No separate handling is needed for either.

Why it is affordable Testing every pixel in the image against every segment would be hopeless. Work is restricted to the bounding box of each segment, expanded by the pen radius, since no pixel outside that box can possibly be within half a pen width of it. The cost is therefore proportional to the amount of ink drawn rather than to the size of the canvas.

A segment of zero length, which happens when a path contains a repeated point or consists of a single point, falls back to plain distance from that point, which draws a dot.

4.28 EstimateSlantDeg

```
def EstimateSlantDeg(ink)
```

Inputs

`ink` a mask of one line of writing

Output. The slant in degrees, positive for the usual forward lean. Returns zero when there are fewer than fifty ink pixels.

What it is for Two things. The glyph extractor removes the slant before storing a letter, so that stored letters are upright and can be compared with one another; and the synthesiser re-applies the writer's own slant when drawing, because how far somebody leans their writing is one of the most visible features of their hand.

Background: what shearing is Shearing slides each row of an image sideways by an amount proportional to its height, which tilts upright strokes without changing the height of anything. It is the operation that turns upright writing into slanted writing, so undoing it is how slanted writing is made upright.

The method: try angles and score the result For each candidate angle the ink is sheared by that amount and the ink in each column is counted, giving a column profile. The angle whose profile is the *sharpest* is the answer [9]:

$$x' \;=\; x - \tan(\theta)\,(y - \bar{y}), \qquad \text{score}(\theta) \;=\; \sum_{j} \text{prof}_{\theta}[j]^2 \tag{4.24}$$

Source: [9]

where

x, y the coordinates of an ink pixel

x' its column after shearing by the trial angle

$\bar{y}$ the mean row of all the ink, which is the height the shear pivots about

θ the trial angle

$\text{prof}_{\theta}[j]$
: how many ink pixels land in column j after that shear

$\text{score}(\theta)$
: the sharpness being maximised

Why squaring the profile measures sharpness This is the key step and it is worth being precise about. The total amount of ink does not change when the image is sheared, so the sum of the profile is the same for every angle; only its *distribution* changes. For a fixed total, the sum of squares is largest when the quantity is concentrated in a few entries and smallest when it is spread evenly. Ten pixels in one column score a hundred; one pixel in each of ten columns scores ten.

So maximising the sum of squares finds the angle at which the ink stacks into the fewest, tallest columns, which is the angle at which the vertical strokes of the writing are truly vertical. The same reasoning underlies `SkewScore`, applied to rows instead of columns.

Why coordinates are sheared rather than the image The score needs only the column each ink pixel lands in, not an image. Shearing the coordinates and counting them is therefore enough, and it avoids interpolating a whole image once per candidate angle. With sixty candidate angles that is the difference between a measurement that is free and one that dominates the runtime.

Resolution and the sign The search runs from minus 45 to plus 45 degrees in steps of 1.5 degrees, which bounds how precisely the answer can be reported. The result is negated before being returned so that a positive number means the ordinary forward lean, which makes the deslanting step read naturally as subtracting the tangent of the slant times the height.

Fewer than fifty ink pixels is treated as too little to measure and zero is returned, since on that much ink the profile is dominated by where the individual pixels happen to fall.

4.29 PolylineLength

```
def PolylineLength(p)
```

Inputs

`p` a path as a list or array of (x, y) points

Output. The total distance walked along the path.

What it is for A size measure for pen strokes. Its main use is rejecting fragments: a piece cut out of a line of writing that contains far less pen path than a real example of that letter is a fragment rather than a letter, whatever its bounding box suggests.

How it works

$$L = \sum_{i} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} \tag{4.25}$$

where

L the total path length, also called the arc length

x_i, y_i
the coordinates of the i-th point on the path

Each term is the straight-line distance between two consecutive points, by Pythagoras, and the total is their sum. It is computed as whole-array arithmetic: the differences between consecutive points, then their lengths, then one sum.

Why path length and not bounding box A fragment can have a deceptively large bounding box. Half of a letter o spans nearly the full width and height of the whole letter while containing only half the ink, so a box-based test would accept it. Path length measures how much pen movement the shape actually represents, which is what distinguishes half a letter from a whole one.

4.30 InkMask

```
def InkMask(gray)
```

Inputs

`gray`
: the page as a 2-D array of grey values, 0 for black and 255 for white, held as floating point numbers so the division below does not round

Output. a pair. The first item is a boolean array the same shape as the page, true wherever the pixel is judged to be ink. The second is the **ratio image**, the page divided by its own local background, which later stages reuse to measure how dark each piece of ink is.

What it is for This is the first real decision in the whole pipeline: for every pixel of the photograph, is this pen or is this paper? Everything downstream, from finding the lines to cutting out letters, works on the answer. Getting it wrong in one direction loses strokes; getting it wrong in the other fills the page with shadows and paper texture that then get mistaken for writing.

Background: why a single threshold is not enough The obvious method is a global threshold: pick one grey value, call everything darker than it ink. That works for a scanner, where the lamp lights the page evenly. It fails badly for a photograph taken with a phone, because the illumination is not even. One corner of the page sits under a lamp and the opposite corner sits in shadow, and the page is often slightly curved so the middle is brighter than the edges. The result is that paper in the bright corner can be lighter than 220 while paper in the shadowed corner is darker than 150, and real pen ink in the bright corner is around 90. There is no single number that separates ink from paper everywhere at once: any threshold low enough to reject the shadowed paper also rejects the pale pen strokes, and any threshold high enough to catch the pale strokes also catches the shadow.

The idea: divide out the illumination The fix is to stop asking "is this pixel dark?" and ask instead "is this pixel darker than the paper immediately around it?". The illumination varies smoothly across the page, so over a small neighbourhood it is almost constant. If the brightness of a pixel is modelled as the true reflectance of the page at that point multiplied by the local illumination, then dividing by an estimate of that local illumination cancels it and leaves the reflectance, which is the thing that actually distinguishes pen from paper.

$$\text{ratio}[y,x] \;=\; \frac{\text{gray}[y,x]}{\max\big(\text{BoxMean}(\text{gray},r)[y,x],\,1\big)} \tag{4.26}$$

where

ratio$[y,x]$
: the illumination-corrected value at this pixel; near 1 for paper, well below 1 for ink

gray$[y,x]$
: the measured grey value at this pixel

BoxMean(gray,r)
: the average grey value over a square window of radius r centred on each pixel, which serves as the estimate of the local illumination; see the entry for BoxMean

r
: the radius of that window, in pixels

max$(\cdot,\,1)$
: a guard so that a window which happens to average to zero cannot cause a division by zero

A ratio of 1 means the pixel is exactly as bright as its surroundings, which is what paper looks like. A ratio of 0.4 means the pixel is well under half as bright as the paper around it, which is what a pen stroke looks like. The key property is that this is now true in the bright corner and the shadowed corner alike, because the shadow divides out of both the numerator and the denominator.

Choosing the window size, and the trap in it The radius has to satisfy two opposing requirements. It must be large enough that the window is mostly paper even when it is centred on a stroke, because the denominator is supposed to estimate the paper brightness, not the ink. If the radius were the width of a pen stroke, then a window centred inside a thick stroke would be entirely ink, the estimated background would be as dark as the ink itself, the ratio would come out near 1, and the stroke would be declared paper. That failure has a name in this file: it is the "solid ink" case handled below. The radius must also be small enough that the illumination really is roughly constant across the window, which rules out making it enormous.

$$r \;=\; \text{clip}\!\left(\text{round}\!\left(\frac{H}{41}\right),\;8,\;40\right) \tag{4.27}$$

where

r
: the background window radius used above

H
: the height of the page image in pixels

clip(v,a,b)
: v forced into the range from a to b: the smaller of b and the larger of a and v

41
: a divisor chosen so that the window comes out a few times the thickness of a pen stroke at the resolutions this pipeline works at

8, 40
: hard floor and ceiling, so a very small or very large input image still gets a sensible window

Scaling the radius with the image height rather than fixing it is what lets the same code handle a 1000-pixel-tall photograph and a 4000-pixel-tall one. A stroke in the larger photograph is four times as many pixels wide, so the window that must exceed it has to grow in step.

Turning the ratio into a yes or no The ratio image still has continuous values and a decision is needed. Otsu's method supplies the threshold, and it is unusually well suited here. Otsu's method assumes the values fall into two groups and picks the cut that separates them best in a precise statistical sense; see the entry for Otsu for the derivation. After the division the ratio image genuinely does have two groups, because the illumination that was smearing them together has been removed: a tall narrow spike just below 1 for the paper, and a much smaller spread of low values for the ink. A pixel is ink when its ratio is below the threshold Otsu returns.

$$\text{mask}[y, x] \;=\; \big[\, \text{ratio}[y, x] \;<\; t_{\text{Otsu}}(\text{ratio}) \,\big] \tag{4.28}$$

where

$\text{mask}[y, x]$
: true when this pixel is taken to be ink

$t_{\text{Otsu}}(\text{ratio})$
: the threshold Otsu's method returns for the ratio image, that is, the value that best splits its two groups

$[\,\cdot\,]$ square brackets around a condition mean 1 or true when the condition holds and 0 or false when it does not

> *Note that the threshold is computed from the ratio image, not from the original grey image. This is the whole point of the two-step design: a global method like Otsu's, which cannot adapt to position, is made to work on a photograph by first removing the position-dependent part of the signal.*

The solid-ink correction One class of object defeats the division: a region of darkness much larger than the background window. The spine of a ring binder, a deep shadow along a fold, or the black edge of the desk behind the page are all far wider than 40 pixels. Inside such a region the window used for the denominator is entirely dark, so the estimated background is dark too, the ratio comes back near 1, and the interior is correctly rejected. The problem is the boundary, where the window straddles light and dark and the ratio dips, marking a band of false ink around every large dark object.

These regions are found by their defining property: they are completely filled. Writing is not. A pen stroke is thin, so any small window placed on the page contains some paper even when it contains a stroke; a window inside a binder shadow contains nothing but shadow. Averaging the mask itself measures exactly this fill fraction.

$$\text{solid}[y, x] \;=\; \big[\, \text{BoxMean}(\text{mask},\, 4)[y, x] \;>\; 0.999 \,\big] \tag{4.29}$$

where

$\text{solid}[y, x]$
: true when this pixel lies in a completely filled dark region

$\text{BoxMean}(\text{mask}, 4)$
: the fraction of pixels marked as ink within a 9 by 9 window, obtained by averaging the mask treated as ones and zeros

4 the window radius, giving a 9 by 9 window: small enough that no ordinary pen stroke can fill it, large enough that noise does not

0.999
: effectively a demand that every pixel in the window be ink; it is not written as 1.0 only to be safe against floating-point rounding in the average

The flagged cores are then grown slightly, with a second averaging test, and subtracted from the mask. Growing them is necessary because the false-ink band lies just outside the solid core, not inside it, so removing only the core would leave the band behind, which is the artefact being targeted.

Why the ratio image is returned as well Later stages need to ask how dark a piece of ink is, in order to tell a real pen stroke from a faint printed rule or a crease. The original grey values cannot answer that, because they still contain the illumination. The ratio image can, and it is already computed, so it is passed out rather than recomputed. DropArtifactPieces uses it for precisely this test.

4.31 PaperRegionMask

```
def PaperRegionMask(g, P)
```

Inputs

`g` the page as a 2-D grey array in floating point

`P` the line pitch in pixels, that is, the typical distance between consecutive lines of writing; see LinePitch

Output. a boolean array the size of the page, true inside the sheet of paper and false outside it, shrunk slightly inward from the true paper edge.

What it is for A photograph of a page is rarely just the page. Around it there is desk, keyboard, hand, shadow and the dark gap at the edge of the sheet. None of that is writing, but much of it is darker than the paper and therefore survives into the ink mask. This function finds the outline of the sheet so that everything outside it can be discarded in one move, before any effort is spent trying to interpret it.

The assumption being made The method rests on one observation: of everything in the frame, the sheet of paper is both bright and large. The desk may be bright too, but it surrounds the page rather than forming one solid region with it, because a visible dark rim separates them, which is the shadow the sheet casts and the contrast at its cut edge. So the paper is the largest **connected** bright region, where connected means one can walk from any pixel of it to any other without leaving it.

Step one: work small The search is done on a copy of the page reduced by a factor of four in each direction, by averaging each 4 by 4 block of pixels into one. Two reasons. Speed: a sixteenth of the pixels means a sixteenth of the work, and the paper outline does not need pixel accuracy to be useful. Robustness: averaging suppresses noise and small features, so the result is dominated by the large-scale structure, which is what is being looked for. The reduction uses a block average rather than taking every fourth pixel, because taking every fourth pixel can land entirely in a gap and miss a thin dark rim completely, whereas an average over the block always registers it.

Step two: find the dark surround

$$\text{dark}[y,x] \;=\; \left[\; c[y,x] \;<\; 0.45 \cdot \text{median}(c) \;\right] \tag{4.30}$$

where

$\text{dark}[y,x]$
: true when this block of the shrunken page is part of the dark surround

c
: the shrunken, locally averaged copy of the page

$\text{median}(c)$
: the middle value of all the block brightnesses when sorted; it stands in for the brightness of the paper

0.45
: how much darker than the paper a block must be before it counts as surround rather than page

The median is used rather than the mean because it is **robust**, meaning that a minority of extreme values barely moves it. In a photograph where a third of the frame is black desk, the mean brightness is dragged a long way down by the desk and no longer describes the paper at all, while the median still sits in the paper, because the paper is the single largest group of similar values. The threshold is a fraction of that median rather than an absolute grey level, which is again what makes it work under different lighting: in a dim photograph everything scales down together and the fraction is unchanged.

The no-surround case If less than 0.2 percent of the frame comes out dark, the conclusion is that there is no surround in the picture, because the page fills the whole frame or the image is a scan rather than a photograph. In that case the entire image is returned as paper. This matters because the "largest bright region" logic has nothing to find when there is no dark rim to separate regions in the first place, and forcing it to choose would pick an arbitrary piece of the page.

Step three: the largest bright component Connected-component labelling, described in the entry for LabelComponents, is run on the inverse of the dark mask, that is, on the bright blocks. It returns an array in which every bright block carries an integer label, with all the blocks of one connected region sharing a label. Counting how many blocks carry each label and taking the largest count identifies the sheet.

$$\ell^{*} \;=\; \arg\max_{\ell \geq 1} \;\bigl|\{\,(y,x) : \text{lab}[y,x] = \ell\,\}\bigr| \tag{4.31}$$

where

ℓ^{*}
: the label belonging to the sheet of paper

$\text{lab}[y,x]$
: the component label of this block, with 0 meaning it was not bright

$\arg\max$
: the value of the index at which the quantity is largest, as opposed to the largest value itself

$|\{\cdot\}|$
: the number of elements in the set, here the number of blocks carrying that label

$\ell \geq 1$
: labels start at 1, so starting the search at 1 excludes the background

Step four: back to full size The chosen region is a mask over blocks, and what is needed is a mask over pixels. Each block stands for a 4 by 4 square of the original, so the block mask is expanded by replacing every entry with a 4 by 4 square of copies of itself. This is a **Kronecker product** with a 4 by 4 block of ones, which is the standard name for exactly that operation, and in practice it is nearest-neighbour upsampling: a hard-edged enlargement with no smoothing, which is the correct choice because the value being enlarged is a yes-or-no flag and a smoothed version of it would be meaningless.

Step five: shrink inward Finally the mask is eroded, that is, every pixel within a radius of the paper boundary is removed from it, using a radius of 0.4 line pitches. The reason is that the boundary itself is the single least trustworthy part of the estimate. It was found on a coarse grid, the sheet's edge is darkened by shadow over a band several pixels wide, and the page may curl. Pulling the accepted region inward by a fraction of a line's height costs nothing, because people do not write right up to the cut edge of a sheet, and it reliably excludes the edge shadow that would otherwise be interpreted as a long horizontal stroke.

> *Expressing the shrink in line pitches rather than pixels is a habit worth noticing, and it recurs throughout the segmenter. The pitch is the natural unit of the problem: it is set by the size of the writing, so a quantity measured in pitches means the same thing on a photograph taken close up as on one taken from further away.*

4.32 LinePitch

```
def LinePitch(mask)
```

Inputs

`mask`
: the ink mask of the page, a boolean 2-D array

Output. a pair. The first item is the line pitch, an integer number of pixels giving the typical vertical distance from one line of writing to the next. The second is the smoothed row profile, an array with one value per image row, which LineCentres reuses to locate the lines.

What it is for, and why it comes first Almost every decision later in the segmenter is a judgement of size: is this blob too tall to be a letter, is this gap wide enough to be a word break, is this dot close enough to that stem to belong to it. Expressing such rules in pixels would tie them to one camera at one distance. Expressing them as fractions of the line pitch makes them scale-free, because the pitch grows and shrinks with the writing. So the pitch is measured once, early, and becomes the ruler for everything else. The symbol P is used for it throughout the file.

Background: what a projection profile is A **projection profile** is what results from collapsing a 2-D image onto one axis by summing along the other. The horizontal projection profile used here has one number per image row: how many ink pixels that row contains.

$$\text{prof}[y] \;=\; \sum_{x} \text{mask}[y, x] \tag{4.32}$$

where

$\text{prof}[y]$
: the profile value for row y, that is, the count of ink pixels in that row

$\text{mask}[y, x]$
: 1 where the pixel is ink, 0 where it is not

$\sum_x$ a sum over every column x of that row

The profile of a page of handwriting has a characteristic shape. Rows passing through the bodies of letters cross nearly every letter on the line and so collect a great deal of ink. Rows in the white space between two lines cross almost nothing and collect very little. The profile is therefore a run of peaks separated by troughs, one peak per line of writing, and the distance between consecutive peaks is the pitch. Using row projections to find text lines is a long-standing technique in document analysis [10].

Columns are trimmed before summing: the leftmost 5 percent and everything past 97 percent of the width are ignored. The margins are where page edges, binder holes and the shadow at the spine live, and a vertical artefact there contributes to every row equally, which raises the whole profile and flattens the contrast between peak and trough that the method depends on.

Smoothing, and why it is necessary The raw profile is jagged. Within one line of writing the count jumps about from row to row according to which letters happen to have a horizontal stroke at that exact height, so a single line of text produces not one clean peak but a cluster of small ones. Counting those as separate lines would be wrong. Smoothing replaces each value with a weighted average of its neighbours, which merges the cluster into one broad peak while leaving the large gaps between lines intact.

$$k[j] \;=\; \exp\!\left(-\tfrac{1}{2}\left(\frac{j}{\sigma}\right)^2\right), \qquad j = -12, \ldots, 12, \qquad \sigma = 4 \tag{4.33}$$

where

$k[j]$ the unnormalised weight given to a neighbour j rows away

j the offset from the row being smoothed, running from -12 to +12

σ the standard deviation of the Gaussian in rows; it sets how far the smoothing reaches, here 4 rows

$\exp(\cdot)$
: the exponential function, which here makes the weight fall away smoothly and symmetrically with distance

$$\text{sm}[y] \;=\; \frac{\sum_j k[j]\,\text{prof}[y+j]}{\sum_j k[j]} \tag{4.34}$$

where

$\text{sm}[y]$
: the smoothed profile at row y

$\text{prof}[y+j]$
: the raw profile j rows away from y

$\sum_j k[j]$
: the sum of the weights, dividing by which keeps the smoothed profile on the same scale as the raw one, so that a flat input comes back unchanged instead of multiplied by the weight total

This operation is a **convolution**: slide a small set of weights along a signal, and at each position output the weighted sum of the values under it. The particular weights here form a Gaussian, or bell curve, which is the usual choice for smoothing because it is smooth itself and so does not introduce ripples of its own. A width of 4 rows is chosen to be comparable to the thickness of a pen stroke, which is the scale of the jaggedness being removed, and much smaller than the distance between lines, which is the structure being kept.

Method one: the gaps between peaks A row is a **local maximum**, or peak, when it is at least as large as the row above and strictly larger than the row below, and in addition exceeds 6 percent of the largest value anywhere in the profile. The last condition rejects the small bumps that noise produces in the white space between lines. The asymmetry, at least as large on one side and strictly larger on the other, is a detail that matters: if both tests were "at least as large", a flat-topped peak several rows wide would report every one of those rows as a separate peak.

With two or more peaks found, the pitch is the **median** of the differences between consecutive peak positions.

$$P \;=\; \operatorname{median}\big(\, y_{i+1} - y_i \;:\; i = 1, \ldots, m-1 \,\big) \tag{4.35}$$

where

P the line pitch in pixels

y_i the row position of the i-th peak, in increasing order

m how many peaks were found

median
: the middle value of the differences once sorted

The median rather than the mean, again for robustness. If the page contains a paragraph break, one of the gaps is two or three times the others. The mean would be pulled upward by that one value and would overstate the pitch for the whole page; the median ignores it entirely, since it only cares which value sits in the middle.

Method two: autocorrelation When fewer than two peaks are found there are no gaps to measure, and a different line of attack is needed. **Autocorrelation** asks how much a signal resembles a shifted copy of itself. For a repeating signal the resemblance is high when the shift equals the period of the repetition, because peaks then land on peaks.

$$\mathrm{ac}[\tau] \;=\; \sum_{y} a[y]\, a[y+\tau], \qquad a[y] \;=\; \mathrm{sm}[y] - \overline{\mathrm{sm}} \tag{4.36}$$

where

ac[τ]
: the autocorrelation at shift tau: large when the profile lines up with itself after being moved tau rows

τ the shift, called the lag, measured in rows

$a[y]$ the profile with its average removed, so that the measurement responds to the pattern of variation rather than to the overall level

$\overline{\text{sm}}$ the mean of the smoothed profile

Subtracting the mean is essential. Without it every product in the sum is positive, the autocorrelation is dominated by the constant part of the signal, and it decreases steadily with lag simply because fewer terms overlap, so no peak corresponding to the period is visible. After subtraction the values straddle zero, and the sum is large only when peaks genuinely align with peaks and troughs with troughs.

The first clear peak at a lag of at least 15 rows, reaching at least 70 percent of the maximum in the searched range, is taken as the pitch. The lower bound of 15 skips the trivial peak at zero lag, where any signal matches itself perfectly, along with the shoulder beside it caused by the smoothing. The search is also limited to a third of the signal length, because at large lags very few rows overlap and the estimate becomes noise.

Method three: letter height If even that fails, as it does for a crop containing a single line, the pitch is inferred from the size of the letters themselves. Connected components are labelled, each one's height is measured, pieces smaller than 8 pixels in area or under 3 pixels tall are discarded as noise, and the median height of what remains is multiplied by 2.2.

$$P \;=\; \max\big(15,\; \operatorname{round}\left(2.2 \cdot \operatorname{median}(h)\right)\big) \tag{4.37}$$

where

P the line pitch in pixels

h the heights of the accepted connected components

2.2 the ratio of line pitch to typical component height observed for ordinary handwriting

15 a floor, below which the estimate is treated as unusable

The factor 2.2 comes from the structure of writing. A typical connected piece of ink is roughly one x-height tall, the x-height being the height of a letter like x or o with no ascender or descender. A line of writing occupies the x-height plus the ascender zone above it plus the descender zone below, and lines are separated by a further gap, which together come to a little over twice the x-height. The estimate is coarse, but it is only reached when the two better methods have already failed.

The final fallback With no usable ink at all, 48 pixels is returned. This is an arbitrary number and is not expected to be right. Its purpose is to keep the caller running on a blank page instead of failing on a division by zero, and a blank page produces no crops regardless of what pitch is assumed, so the value cannot do any harm.

> *The cascade of three methods, each a fallback for the last, is worth reading as a design pattern rather than as clutter. The first method is the most accurate but needs at least two lines; the second needs only a repeating pattern; the third needs only some ink. Each is tried in order of preference, so the accurate method is used whenever it applies and the crude ones only when nothing better is possible.*

4.33 LeastCostSeam

```
def LeastCostSeam(cost, step=3)
```

Inputs

`cost`
: a 2-D array in which every entry is the price of passing through that pixel; here, large where there is ink and small where there is not

`step`
: the largest number of rows the path may move up or down when advancing one column, default 3

Output. an array with one entry per column, giving the row the path occupies in that column: the cheapest left-to-right route across the array.

What it is for Two lines of handwriting have to be separated from one another. A straight horizontal cut will not do it, because the descender of a g on the upper line hangs below the ascender of an l on the lower one, and any single row that clears the g also slices the l. What is needed is a wavy cut that dodges around the ink: it should dip below a descender and rise above an ascender, as long as it keeps moving left to right. That wavy cut is called a **seam**, and finding the cheapest one is what this function does. The technique is seam carving, introduced for resizing images [11] and adapted to the separation of handwritten text lines [12].

The problem stated precisely A seam is a choice of one row for each column, with the restriction that consecutive choices differ by no more than `step` rows, which is what makes the result a connected curve rather than a scatter of disconnected points. Its cost is the sum of the cost array along it. The task is to find the seam of least total cost.

$$\text{Cost}(y_1, \ldots, y_W) \;=\; \sum_{x=1}^{W} \text{cost}[y_x,\, x], \qquad |y_{x+1} - y_x| \le \text{step} \tag{4.38}$$

where

y_x the row the seam occupies in column x

W the number of columns

cost$[y, x]$
: the price of the pixel at row y, column x

step the maximum vertical move allowed between neighbouring columns

Why brute force is impossible Counting the candidates shows why a direct search is out of the question. With h rows to choose from in the first column and up to 2·step+1 choices at each of the remaining W−1 columns, the number of seams is about h(2·step+1)^(W−1). For a modest page half a size down, with 1200 columns and step 7, that is 15 raised to the power 1199, a number with over a thousand digits. No computer will enumerate them.

The idea that makes it easy: dynamic programming **Dynamic programming** is a method for problems that can be solved by building up the answer one step at a time, where the best way to finish does not depend on how the start was reached [13]. That condition holds here. Suppose the cheapest way to reach row y of column x has been computed, for every row y. The cheapest way to reach a particular row of column x+1 must arrive from one of the rows of column x that is within `step` of it, and once it is known how cheaply each of those rows can be reached, the best arrival is just the smallest of those costs plus the price of the new pixel. Nothing about the earlier part of the route matters beyond its total cost. So the whole column can be solved from the previous column alone.

$$A[y,x] \;=\; \mathrm{cost}[y,x] \;+\; \min_{|d|\le \mathrm{step}} \Big(A[y-d,\,x-1] \;+\; \lambda\,|d| \Big) \tag{4.39}$$

where

$A[y,x]$
: the accumulated cost: the price of the cheapest route that starts at the left edge and ends at row y of column x

$\mathrm{cost}[y,x]$
: the price of the current pixel

d
: the vertical move made to arrive here, so the route came from row y−d of the previous column

step
: the limit on that move

λ
: a small penalty per row of vertical movement, 0.02 in this implementation

min
: the smallest value over all the choices of d

The first column needs no recursion, since a route that ends there has visited only that one pixel, so A in column 1 is just the cost in column 1. Every later column follows from its predecessor by the formula above. The work is one pass over the array, with 2·step+1 comparisons per pixel, instead of the astronomical enumeration. That is the entire gain of dynamic programming: it replaces an exponential count of paths with a linear count of pixels.

The movement penalty The term $\lambda|d|$ charges a small amount for each row of vertical movement. Without it, any two routes through equally empty space would cost the same, and the seam would wander about aimlessly in white space, picking an arbitrary one of the many equally cheap options. The penalty breaks those ties in favour of going straight, so that the seam moves only when moving buys it something, namely avoiding ink. At 0.02 per row it is far smaller than the cost of crossing a stroke, so it never persuades the seam to cut through a letter in order to stay level; it only decides between routes that are otherwise equivalent. This is a **regularisation** term, which is the general name for a small extra cost added to express a preference among solutions that the main objective cannot distinguish.

Recovering the path The accumulated cost gives the price of the best seam but not its route, so as each cell is filled the winning move is recorded in a second array. When the pass is complete, the lowest value in the final column is the end of the best seam, and following the stored moves backwards from there reconstructs the whole path, column by column, right to left. This is **backtracking**, and it is the standard companion to dynamic programming: the forward pass computes the value of the best solution, the backward pass reads off what that solution actually was.

How it is made fast in practice Written as a loop over rows inside a loop over columns this would be slow in Python. Instead, for each column, the candidate costs from all 2·step+1 possible moves are formed as whole shifted copies of the previous column, stacked into a small array, and the minimum is taken down the stack in one operation. Shifting a column by d rows and adding the penalty handles every row's d-th candidate at once. Only the loop over columns remains in Python, because columns are genuinely sequential: column x cannot be computed before column x−1.

> *The shifted copies are filled with infinity where the shift runs off the end of the array. Infinity is the natural value for an impossible move, because taking a minimum automatically never selects it, with no need for a special case in the code.*

4.34 SkewScore

```
def SkewScore(mask, angleDeg)
```

Inputs

`mask`
: the ink mask of the page

`angleDeg`
: the trial rotation angle in degrees

Output. a single number: how sharply the ink piles into rows when the page is considered at this angle. Larger is better aligned.

What it is for A photographed page is never perfectly square to the camera, so the lines of writing run slightly uphill or downhill. Every later stage assumes they are horizontal: the row profile is used to find lines, horizontal seams are used to separate them, and a crop is a horizontal strip. A tilt of even two degrees smears the profile enough to merge neighbouring lines. This function provides the measurement that lets the tilt be found: given a candidate angle, it scores how level the page would look at that angle.

The principle: concentration Consider the row profile again, the count of ink pixels per row. When the page is level, every letter on a line contributes to the same few rows, so those rows have very large counts and the rows between lines have almost none. When the page is tilted, the left end of a line sits higher than the right end, so the same ink is spread over many more rows, each with a smaller count. The total amount of ink is identical either way, since rotating the page does not create or destroy pixels. What changes is how **concentrated** the profile is. So the correct angle is the one that concentrates the profile most, and all that is needed is a number that measures concentration.

Why the sum of squares measures concentration Squaring, then summing, does it. Squaring rewards large values disproportionately: doubling a count quadruples its contribution. So for a fixed total, the sum of squares is largest when the total is piled into as few entries as possible and smallest when it is spread evenly.

$$S(\theta) \;=\; \sum_{k} \mathrm{prof}_{\theta}[k]^2 \tag{4.40}$$

where

$S(\theta)$
: the skew score at trial angle theta

$\text{prof}_\theta[k]$
: the number of ink pixels that fall into row index k once the page is considered rotated by theta

k
: the row index in the rotated frame

A numerical illustration makes the behaviour concrete. Suppose 600 ink pixels belong to one line. Level, they might fall into 4 rows of 150, giving a score of $4{\cdot}150^2 = 90\{,\}000$. Tilted, the same 600 might spread over 20 rows of 30, giving $20{\cdot}30^2 = 18\{,\}000$. The totals match at 600 but the scores differ fivefold, and the larger score correctly identifies the level case. The sum of squares is used in the same spirit elsewhere in the project, where EstimateSlantDeg uses it to find the slant of letters rather than the skew of lines.

Scoring an angle without rotating the image Rotating the whole image for each candidate angle would mean interpolating every pixel, seventeen times over for the seventeen angles tried. That is unnecessary. Only the row index each ink pixel would land in is needed, not an image, and that index follows from one line of trigonometry. The standard rotation of a point about a centre gives the rotated row as

$$k \;=\; \text{round}\Big((y - c_y)\cos\theta \;-\; (x - c_x)\sin\theta \;+\; c_y\Big) \tag{4.41}$$

where

k
: the row the pixel lands in after rotation

y, x
: the pixel's original row and column

c_y, c_x
: the centre of rotation, taken as the centre of the image

θ
: the rotation angle in radians, converted from the degrees supplied

cos, sin
: the cosine and sine functions, which express how much of each original axis contributes to the new vertical position

round
: rounding to the nearest whole number, since rows are counted in whole pixels

Only the row is computed; the column is never needed, because the profile sums over columns anyway. Counting how many pixels land in each row is then a single tally of the k values, so the cost of scoring one angle is proportional to the number of ink pixels rather than to the number of pixels in the image. Since ink is a small fraction of a page, that is a large saving on top of avoiding the interpolation.

A subtle trap: never clip the row index Rotation moves some pixels outside the original image rectangle, so k can come out negative or beyond the image height. The obvious response, forcing k into the valid range, is badly wrong, and the code carries a comment saying so. Clipping

piles every out-of-range pixel into the first or last row. That creates an enormous artificial count in one row, and because the score squares the counts, this fake spike scores higher than any genuine line of text. The effect grows with the angle, since larger angles push more pixels out of range, so the search would reliably select the most extreme angle offered. The code instead shifts the indices so the smallest becomes zero, which keeps every pixel in its own distinct row and lets the profile grow as tall as it needs to be.

> *This is a good example of why a plausible-looking defensive measure can be worse than none. Clipping an index is usually the safe thing to do; here it silently converts a correct measurement into one that is guaranteed to give the wrong answer.*

4.35 Deskew

```
def Deskew(page)
```

Inputs

`page`
: the photograph, as a 2-D grey array or a 3-D colour array of 8-bit values

Output. five items: the straightened full-resolution page, the straightened half-resolution copy, the ink mask of that copy, the ratio image, and the angle in degrees that was applied.

What it is for This is the first stage of the segmenter, and its job is to hand the rest of the pipeline a page whose lines of writing run horizontally, together with the derived images every later stage needs. It combines three things: reducing the page to grey, searching for the tilt angle, and applying the rotation.

Step one: colour to grey A colour photograph arrives as three values per pixel, for red, green and blue. The whole segmenter works on a single brightness value, so the three channels are averaged. A simple mean is used rather than one of the perceptual weightings that weight green more heavily, because the distinction being drawn is dark pen against light paper, which shows up in all three channels, and a plain mean has the advantage of treating blue and black ink alike.

Step two: work at half size Every decision in the segmenter is taken on a copy of the page reduced by a factor of two in each direction, which is a quarter of the pixels, so everything runs about four times faster. The decisions involved concern whole lines and whole letters, which are many pixels across, so halving the resolution does not blur the evidence they rest on. The full resolution page is kept because the final crops are cut from it, so the recogniser still sees every pixel the camera captured.

The reduction averages each 2 by 2 block rather than taking every second pixel. Point sampling would risk a thin pale stroke falling between the samples and disappearing entirely, whereas any ink inside a block darkens that block's average and survives. This is the same argument that applies to the widening of a resampling filter when minifying, discussed in the entry for _Weights.

Step three: search for the angle The angle is found by a **grid search**, which means simply trying every candidate in a fixed list and keeping the best. The candidates run from -4 to $+4$ degrees in steps of 0.5, which is seventeen angles, each scored by SkewScore, with the highest score winning.

$$\theta^* \;=\; \arg\max_{\theta \in \{-4,\,-3.5,\,\ldots,\,4\}} S(\theta) \tag{4.42}$$

where

θ^* the chosen rotation angle in degrees

$S(\theta)$
: the concentration score at that angle, from SkewScore

arg max
: the angle at which the score is largest

Two choices in that range deserve justification. The limit of 4 degrees reflects how a page is actually photographed: someone aiming a phone at a sheet on a desk is square to it within a few degrees, and a search that allowed 45 degrees would mostly offer opportunities to pick a wrong answer. The step of 0.5 degrees is chosen against the consequence of residual error. Over a page 1500 pixels wide, half a degree of tilt lifts one end relative to the other by $1500 \cdot \tan(0.5^\circ) \approx 13$ pixels, which is well under the height of a line of writing, so the remaining tilt cannot merge neighbouring lines. A finer step would cost more scoring for no benefit the later stages could use.

The search runs on a quarter-size mask, halved again from the working copy, giving sixteen times fewer pixels to tally per angle. The angle of a whole page is a very coarse property, determined by the global arrangement of its lines, so it survives aggressive reduction intact.

Step four: apply the rotation, and the fill values The chosen angle is applied to both the full-size and half-size pages with Rotate. Rotating a rectangle leaves wedges at the corners that correspond to no original pixel, and these have to be filled with something. The fill is 255, white, because what lies just outside the frame of a photograph of a page is more page, and declaring it white means the later stages see paper rather than a black triangle that would read as a huge ink blob. Where a mask rather than a grey image is rotated the fill is 0 instead, for the same underlying reason: in a mask, 0 means no ink, so filling with 0 adds nothing that could be mistaken for writing.

Step five: derive the mask Finally InkMask is run on the straightened half-size page to produce the ink mask and the ratio image. Note the ordering: the mask is computed after rotation, not before. Rotating a mask would require interpolating yes-or-no values, which produces fractional ink along every stroke edge and forces an arbitrary second threshold to clean up. Rotating the grey image and then thresholding avoids that entirely, since interpolating grey values is exactly what interpolation is for.

> *The ink mask is computed twice in this function, once before the rotation to drive the angle search and once after it for the output. The first is thrown away. That is deliberate: the search needs a mask to score, and the output must describe the rotated page, and no single mask can do both.*

4.36 RemovePrintedStructure

```
def RemovePrintedStructure(mask, g)
```

Inputs

`mask`
: the ink mask of the page

`g`
: the grey page at the same scale, used to locate the sheet

Output. five items: the cleaned mask, the paper-region mask, the line pitch, how many ink pixels lay outside the paper, and how many were removed as printed structure.

What it is for A sheet of lined or gridded paper carries printing that was never written by hand: the ruled lines, the red margin line, the holes and the darkened spine where the page meets the binding. All of it is ink as far as the mask is concerned, and all of it is poison for the stages that follow. A ruled line runs the full width of the page, so it joins every letter on a line into one connected blob, which destroys the per-letter analysis that the cutter depends on, and it adds a huge horizontal stroke to every row profile. This function removes that printing while leaving the writing intact.

The distinguishing property: length The property that separates printing from writing is not darkness, since a ruled line can be as dark as a pencil stroke, and not thickness, since both are a pen's width. It is **length**. A ruled line crosses the whole page in one unbroken straight run. Handwriting contains straight pieces too, but short ones: the longest horizontal stroke in ordinary writing is the crossbar of a t or the join between two letters, neither more than a fraction of a line's height. So a threshold on run length, expressed in line pitches, separates the two reliably.

$$\text{flat} \;=\; \text{LongRunMask}\big(\text{mask},\; \lfloor 1.25\,P \rfloor,\; \text{horizontal}\big) \tag{4.43}$$

where

flat
: the mask of pixels belonging to a long horizontal run

LongRunMask
: marks every pixel that lies in an unbroken run of set pixels at least the given length; see its own entry

P
: the line pitch in pixels

1.25
: the length threshold as a multiple of the pitch

$\lfloor \cdot \rfloor$
: rounding down to a whole number of pixels

The threshold of 1.25 pitches sits in a wide gap. A ruled line is tens of pitches long, and the longest horizontal stroke in handwriting is well under one pitch, since a letter is about half a pitch wide. Anything between about 1 and several pitches would give the same result, so the exact figure is not delicate. The vertical direction gets 1.4 pitches, slightly more, because a tall letter like a capital L or an f with a long descender genuinely can run more than a pitch vertically, and the extra margin keeps such letters out of the deletion.

Tolerating a tilt A ruled line on a page that is still a fraction of a degree off level is not a single row of pixels. It drifts downward as it crosses the page, so any one row contains only the piece of it that happens to lie in that row, and a run-length test applied row by row sees a series of short runs rather than one long one, and deletes nothing. The fix is to dilate the mask by one pixel vertically before the test, so that each row contains the line's pixels from the rows immediately above and below as well and the run becomes unbroken again.

But dilating also thickens the writing, and a test applied to a thickened mask would mark pixels that were never ink. So the result is dilated back and then intersected with the original mask, which restricts the deletion to pixels that really were ink to begin with. In morphological terms the sequence dilate, test, dilate, intersect is a closing used as a detector rather than as a filter: the dilation makes the feature visible to the test, and the intersection confines the consequences to the original shape.

The vertical pass uses a radius of 3 rather than 1 because the structures it targets, the shadow at the spine of a notebook and the edge of the sheet, are not straight. They wobble by several pixels across the height of the page, so a tolerance of one pixel is not enough to make them read as one continuous run.

Masking off the surround first Before any of this, PaperRegionMask locates the sheet and everything outside it is deleted from the mask in one intersection. This is done first because it is the cheapest and most decisive filter available: the desk, the hand and the shadow beyond the page account for a large share of the false ink on a photograph, and removing them wholesale means the subtler tests that follow are not distracted by them.

What is returned, and why the counts Besides the cleaned mask, the function returns the pitch and the paper mask, both already computed here and needed later, and two counts: how many ink pixels lay outside the paper and how many were deleted as printed structure. The counts feed the diagnostics dictionary that SegmentLines returns. They exist because a failure of this stage is otherwise invisible: if the paper mask is wrong, the ink-outside-paper count is enormous, and if the run-length thresholds are wrong for a given page, the removed-pixel count is far larger or smaller than expected. Reporting them makes a silent failure a visible one.

4.37 DropArtifactPieces

```
def DropArtifactPieces(mask, ratio, P, H, W)
```

Inputs

`mask`
: the ink mask after the printed structure has been removed

`ratio`
: the ratio image from InkMask, used to judge how dark each piece is

`P`
: the line pitch in pixels

`H`
: the mask height

`W`
: the mask width

Output. five items: the mask with the artefacts deleted, the component label array, the number of components, the area of each, and how many pieces were dropped.

What it is for After the printing has been removed, what remains is mostly handwriting but not entirely. Creases cast faint shadows. The fold at the centre of a spread leaves a dark line. The cut edge of the sheet leaves a sliver. Fragments of the ruling survive where a letter crossed it and broke the run. This function examines the leftovers one piece at a time and deletes those that are not writing.

Why judge whole pieces, not pixels This is the central design decision and it is worth stating plainly. Every test in this function is applied to a connected component as a unit, and a component is either kept whole or deleted whole. The alternative, testing each pixel on its own merits, produces half-erased letters, and a half-erased letter is worse than either outcome: it is still there, so it still has to be interpreted, but it no longer looks like the letter it is. Deciding per piece means the worst case is a missing letter, which the recogniser handles as a gap, rather than a corrupted one, which it confidently misreads.

Closing before labelling The components are found not on the mask itself but on a closed copy, dilated by about a tenth of a pitch and then eroded by the same amount. The purpose is to make a letter that the threshold broke into two pieces, where the pen lifted or the ink ran thin, count as one piece for the purposes of these tests. Otherwise the two halves are judged separately and each half, being smaller and thinner than a letter, is more likely to be mistaken for an artefact. The tests then read their statistics from the original mask but group pixels by the labels from the closed one, so the closing influences the grouping without inflating the measurements.

The geometric tests Several tests follow from the simple fact that a letter has a characteristic size, and the pitch gives the scale for it.

- **Too tall:** a piece more than 2.2 pitches high cannot be a letter, because that is taller than a whole line of writing including its ascender and descender zones.
- **Too thin:** a piece half a pitch wide or less, combined with another suspicious property, is a sliver rather than a letter.
- **A vertical bar:** a piece under 0.12 pitches wide and over 1.2 pitches tall is deleted outright. No letter has that aspect ratio; a printed margin rule does, and so does the fold at the centre of a page.
- **Touching an edge:** a piece reaching within half a pitch of the paper boundary is suspect, because writing does not start at the cut edge of a sheet. Touching the left edge is enough on its own, since the binding is there; touching the right or the top or bottom is only grounds for deletion in combination with being thin.
- **Far left of the text:** the fifth percentile of the left edges of all the letter-sized pieces estimates where the writing begins, and a piece lying more than 1.2 pitches to the left of that is in the margin, not in the text.

The fifth percentile, rather than the minimum, estimates the left margin of the text. A **percentile** is the value below which a given fraction of the data falls, so the fifth percentile is the value exceeded by 95 percent of the left edges. Using the minimum would let a single surviving artefact define the margin and so protect every other artefact to its right; using a percentile means one stray piece cannot shift the estimate.

The darkness test Geometry alone cannot separate a faint printed rule from a pen stroke of the same shape, so the ratio image is brought in. The mean ratio over a piece measures how dark it is relative to the paper around it, and the comparison is made against the same Otsu threshold that decided what counted as ink in the first place.

$$\overline{\text{ratio}}_\ell \;=\; \frac{1}{A_\ell} \sum_{(y,x)\,\in\,\ell} \text{ratio}[y,x], \qquad \text{faint}_\ell \;=\; \left[\, \overline{\text{ratio}}_\ell > 0.85\, t_{\text{Otsu}} \,\right] \tag{4.44}$$

where

$\overline{\text{ratio}}_\ell$
: the mean ratio value over the pixels of piece l, that is, its average darkness relative to the local paper

A_ℓ : the number of pixels in piece l

$\text{ratio}[y,x]$
: the illumination-corrected value at that pixel, from InkMask

t_{Otsu}
: the ink threshold on the ratio image

0.85 : how close to the threshold a piece's average may come before it is called faint

faint_ℓ
: true when piece l only barely qualified as ink at all

The reasoning is that a real pen stroke is not marginal. It is far darker than the threshold, not just past it, so its mean ratio sits well below the threshold value. A piece whose average only just scrapes under is made of pixels that barely qualified, which is the signature of a crease, a shadow or a printed rule. Being faint is not grounds for deletion by itself, since a pencil stroke is faint too; it is combined with a shape test, so that a faint thin vertical thing goes but a faint letter-shaped thing stays.

The per-piece mean is computed in a single pass by a weighted tally. Tallying the labels with the ratio values as weights sums the ratio over each piece, and dividing by the pixel count gives the mean, with no loop over pieces. The same trick appears throughout the segmenter, and the general pattern is worth recognising: a per-group sum is a weighted count indexed by group.

The edge-shadow test One composite test targets a specific and common artefact: the shadow of a fold or the curl at the top or bottom of the sheet. It is identified by five properties at once: it touches the top or bottom edge, it is no taller than 1.2 pitches, it is wider than a pitch, it is faint, and its **density** is below 0.15.

$$\text{density}_\ell \;=\; \frac{A_\ell}{(\,\text{bot}_\ell - \text{top}_\ell + 1\,)\cdot(\,\text{xmax}_\ell - \text{xmin}_\ell + 1\,)} \tag{4.45}$$

where

density_ℓ
: the fraction of this piece's bounding box that the piece actually fills

A_ℓ : the piece's pixel count

top_ℓ, bot_ℓ
: the first and last rows the piece occupies

xmin_ℓ, xmax_ℓ
: the first and last columns it occupies

Density distinguishes a sparse scatter from a compact shape. A word fills a reasonable share of its bounding box; a broken shadow stretching across the page fills almost none of it, because its box is wide and tall while its ink is a thin broken line. Requiring all five properties together is what makes the test safe: each one individually also describes some piece of real writing, but nothing in handwriting is simultaneously edge-touching, wide, short, faint and sparse.

> *The background label is explicitly protected from deletion. Label 0 means "not part of any piece", and deleting it would erase everything that was not assigned to a component, which is a one-line guard against a catastrophic error.*

4.38 LineCentres

```
def LineCentres(mask, H)
```

Inputs

`mask`
: the cleaned ink mask

`H`
: the mask height in pixels

Output. a pair: the line pitch, and a sorted list of the row positions of the lines of writing.

What it is for The pitch says how far apart the lines are; this function says where they actually are. Each entry of the returned list is the row at the heart of one line of writing, and the rest of the segmenter uses them as anchors: cuts are placed between consecutive centres, and a piece of ink is judged by how far above or below a centre it sits.

Finding candidates The smoothed row profile is reused from LinePitch, and the candidates are its local maxima, found by the same test: at least as large as the row above, strictly larger than the row below, and above 6 percent of the profile's peak. The rows where the ink piles up are the rows through the bodies of the letters, which is the right place to call the centre of a line.

The real problem: suppressing duplicates Smoothing does not remove every spurious peak. A line containing several tall ascenders can produce a secondary maximum a few rows above its main one, and a line of writing that drifts slightly can produce two. Accepting both would create two lines where there is one, and the consequence is severe: a cut would be placed between them, straight through the middle of the writing, and the line would be delivered to the recogniser as two half-height crops. So the candidates must be thinned.

Greedy non-maximum suppression The thinning is done by a method known as **non-maximum suppression**: sort the candidates by strength, take the strongest, discard anything too close to it, and repeat with what is left. It is called **greedy** because it commits to the best available option at each step and never revisits that choice.

$$\mathcal{C} \leftarrow \mathcal{C} \cup \{c\} \quad \text{if} \quad |c - o| \geq 0.6\,P \quad \text{for every } o \in \mathcal{C} \tag{4.46}$$

where

$\mathcal{C}$ the set of centres accepted so far, considered in order of decreasing profile value

c the candidate peak currently being considered

o a centre already accepted

P the line pitch in pixels

0.6 the exclusion radius as a fraction of the pitch

Processing the candidates strongest first is what makes the greedy rule correct rather than arbitrary. The strongest peak in a cluster is the one through the letter bodies, so it is accepted and the weaker peaks of the same line are then excluded by it. Had the candidates been processed top to bottom instead, a weak secondary peak slightly above a line would be accepted first and would then exclude the true centre below it, placing the line in the wrong row.

Why 0.6 pitches The exclusion radius has to exceed the width of a spurious cluster and fall short of the true spacing between lines. By definition the true spacing is one pitch, and the duplicate peaks of a single line lie within a fraction of a pitch of each other, since they are all within one line's x-height. A radius of 0.6 pitches is comfortably inside that gap: it cannot suppress a genuine neighbour, which is a full pitch away, and it does suppress a duplicate, which is much closer. Choosing slightly more than half a pitch, rather than exactly half, leaves margin for the pitch estimate itself being a little low.

> *Accepted centres are collected in order of strength and only sorted into row order at the end. The sort matters because every later stage assumes the centres run down the page in sequence, pairing each with the next to place a cut between them.*

4.39 SeamCuts

```
def SeamCuts(mask, centres, P, H, W)
```

Inputs

`mask`
: the cleaned ink mask

`centres`
: the row positions of the lines, from LineCentres

`P` the line pitch in pixels

`H` the mask height

`W` the mask width

Output. three items: the cuts as a list of one row-per-column array each, the same cuts stacked into a single 2-D array, and L, the number of bands the cuts divide the page into.

What it is for With the lines located, boundaries have to be drawn between them. This function produces one boundary curve per gap, each given as the row it occupies in every column, so that the page is divided into L horizontal bands with one line of writing in each. The curves are seams rather than straight rows, for the reason given in the entry for LeastCostSeam: a straight cut between two lines of handwriting cannot avoid slicing through the descenders of the upper line or the ascenders of the lower one, because at some column they overlap vertically.

The cost landscape A seam needs a cost array, and the art of this function is in building one whose cheapest path is the cut that a person would draw. It has two terms.

$$\text{cost}[y, x] \;=\; 4.0 \cdot m[y, x] \;+\; 0.012 \left(\frac{y - y_{\text{mid}}}{g} \right)^2 \tag{4.47}$$

where

$\text{cost}[y, x]$
: the price of routing the cut through this pixel

$m[y, x]$
: 1 where there is ink in the closed mask, 0 where there is not

4.0
: the price of crossing a single ink pixel

y_{mid}
: the row midway between the two line centres this cut separates

g
: half the distance between those two centres, used to express the displacement in units of the gap rather than in pixels

0.012
: the strength of the pull back toward the midline

The first term is the one that does the work: ink is expensive, so the cheapest route goes around letters. The second is a weak **quadratic penalty** on straying from the middle of the gap, quadratic meaning it grows with the square of the displacement, so small excursions are nearly free while large ones become progressively more costly. Its purpose is to settle what happens in genuinely empty space. Between two words there is no ink at all and every route is equally cheap on the first term alone, so without the second term the cut would wander to an arbitrary height and arrive at the next word from a silly position. The pull keeps it near the middle of the gap, ready to deflect in either direction when the next letter arrives.

The ratio between the two coefficients is what matters, not either value alone. At 4.0 against 0.012, crossing one ink pixel costs as much as sitting about eighteen normalised units off the midline, which is far outside the gap. So the cut will always prefer a long detour through white space to cutting a stroke, and the midline term only ever acts as a tie-breaker. Expressing the displacement in units of the half-gap rather than in pixels keeps that balance the same for widely and narrowly spaced writing.

Closing the mask first The mask is closed, dilated then eroded by about 0.07 pitches, before being used as cost. The reason is a specific failure mode: handwriting has narrow gaps inside it, between the two strokes of an n or where a pen lifted, and a seam is happy to thread through such a gap, because threading it is free while going round costs ink. The cut then passes through the middle of a letter. Closing fills gaps narrower than the structuring element while leaving the outer shape of the stroke nearly unchanged, so the threading routes disappear but the cut still traces the true outline of the letter.

Closing rather than plain dilation is the point. Dilation alone would also close the gaps, but it would leave every stroke thickened by the full radius, and the cut would then trace a bloated blob standing well clear of the real ink, wasting vertical space in both crops. The erosion that follows the dilation undoes the thickening in the places where it was not needed, which is exactly the property that distinguishes a closing from a dilation.

The step limit The seam here is allowed 7 rows of vertical movement per column, against the default of 3. A tall narrow descender, such as the tail of a y, demands a sharp deflection: the cut must drop below the tail and climb back within the few columns the tail occupies. With a small step limit the cut simply cannot turn fast enough and is forced through the tail. The cost of the larger limit is a little more work per pixel and a little more freedom to wander, the latter being what the midline penalty is there to restrain.

The outer boundaries Cuts exist only between consecutive centres, so the first line has no cut above it and the last none below. Two straight boundaries are added at 0.75 pitches beyond the outermost centres, clipped to the page. They are straight because there is no second line on the far side to avoid, so there is nothing for a seam to dodge, and 0.75 pitches is enough to clear the ascenders of the first line and the descenders of the last. The number of bands is then one less than the number of boundaries, which is why L is computed that way.

4.40 _BandOwnership

```
def _BandOwnership(mask, lab, nc, cuts, cutsArr, L, P, H, W, centres)
```

Inputs

`mask`
: the cleaned ink mask

`lab` the component labels from DropArtifactPieces

`nc` the number of components

`cuts`
: the cuts as a list of per-column row arrays

`cutsArr`
: the same cuts as one 2-D array

`L` the number of bands

`P` the line pitch

`H` the mask height

`W` the mask width

`centres`
: the row positions of the lines

Output. six items: the ownership array assigning each ink pixel to a line, a copy of it taken before the later corrections, the band index of every pixel, the area of each component, and the row and column coordinates of the ink pixels.

What it is for The cuts divide the page into bands, but a band is not yet an answer. The question that must be settled for every ink pixel is which line of writing it belongs to, and the band it falls in is only the first guess at that. This function makes that guess and then applies several corrections to it, producing the ownership map that the crops are cut from.

Computing the band index without a loop Each cut is a curve, so asking which band a pixel is in means asking how many of the curves lie above it. That count is the band index directly, and it can be formed by adding up one comparison per cut over the whole image at once.

$$\text{region}[y, x] \;=\; \sum_{n=1}^{L-1} \left[\; y \;\geq\; \text{cuts}_n[x] \;\right] \tag{4.48}$$

where

$\text{region}[y, x]$
: the band index of this pixel, counting from 0 at the top

$\text{cuts}_n[x]$
: the row occupied by cut n in column x

L : the number of bands, so there are L−1 interior cuts

$[\,\cdot\,]$: 1 when the condition holds, 0 otherwise

The expression is read as: a pixel is below cut n, so it lies past that boundary, and summing those indicators over all the interior cuts counts the boundaries it has passed. The outer two boundaries are excluded from the sum, since nothing lies beyond them that needs numbering; instead they are used separately to discard ink above the first line and below the last.

Why the band is only a guess If every piece of ink lay wholly inside one band, the band index would be the final answer. Two things prevent that. First, a connected piece of ink can genuinely cross a cut: the descender of a g belongs to the upper line but hangs into the lower band, and the cut goes around the ink, so the piece ends up with pixels in both. Taking the band pixel by pixel would split that g in half, delivering its bowl to one crop and its tail to another, and neither fragment looks like a g. Second, small marks like the dot of an i are separate pieces that float well above their own line's body and can easily fall on the wrong side of a cut.

The piece-by-band table Both problems need the same information: how much of each piece lies in each band. That is a two-dimensional table, pieces against bands, and it is built with one tally by packing the two indices into a single number.

$$\text{key} \;=\; \ell \cdot L \;+\; b, \qquad \text{cnt}[\ell,\, b] \;=\; \left|\{\,(y, x) : \text{lab}[y, x] = \ell,\; \text{region}[y, x] = b\,\}\right| \tag{4.49}$$

where

key : a single integer encoding the pair of a piece and a band

ℓ : the piece's component label

b : the band index

L : the number of bands, used as the multiplier so that no two distinct pairs can collide

$\mathrm{cnt}[\ell, b]$
: how many pixels of piece l fall in band b

This is **index packing**, and the reason it works is place value, the same principle as writing a two-digit number: multiplying the first index by the number of possible second values guarantees that each pair maps to its own integer. A single tally over the packed keys then produces the whole table, and reshaping the result into an array of pieces by bands unpacks it. The alternative, looping over pieces and counting each one's pixels, would be far slower in Python for no gain.

Assigning each piece: the topmost substantial band A band holds a **substantial** share of a piece when it contains at least 15 percent of that piece's ink. Among the substantial bands, the piece is assigned to the topmost.

$$\mathrm{major}(\ell) \;=\; \min\,\left\{\, b \;:\; \mathrm{cnt}[\ell, b] \;\geq\; 0.15 \sum_{b'} \mathrm{cnt}[\ell, b'] \,\right\} \tag{4.50}$$

where

$\mathrm{major}(\ell)$
: the band piece l is assigned to

$\mathrm{cnt}[\ell, b]$
: the pixels of piece l in band b

$\sum_{b'} \mathrm{cnt}[\ell, b']$
: the piece's total ink

0.15 the share of its own ink a band must hold to count as substantial

min the smallest such band index, that is, the highest on the page

The 15 percent test exists to tell a genuine crossing from a trivial one. A piece whose ink is 95 percent in one band and 5 percent in the next merely touches the boundary, and treating it as spanning two lines would be wrong. A piece split 60 to 40 really does belong to both, and something must be decided.

Choosing the topmost, rather than the band with the most ink, is a deliberate asymmetry. When a piece spans two lines it is usually because a descender from the upper line reaches down, or because two lines of writing touch. In either case the upper line is the one that is incomplete without the piece, since the shape originates there, while the lower line is merely intruded upon. Giving it to the upper line keeps the crop that would otherwise be damaged intact. If no band reaches 15 percent, which happens when a piece is spread thinly over three or more bands, the fallback is the band with the most ink.

Small pieces: three graded rules Small pieces are handled separately, because for them the band is unreliable and their size tells us what kind of thing they are. The thresholds are areas measured in P^2, the square of the pitch, which is the natural unit of area here: a letter occupies a region roughly a third of a pitch by a half, so letter-sized is a few hundredths of P^2.

- **Under 0.06 $\mathbf{P^2}$, a stray mark.** It is compared with the letter-sized pieces assigned to its own band. If none sits within six pitches horizontally, there is no writing anywhere near it and it is discarded. Otherwise the median vertical centre of the nearby writing says where its line really is, and if the mark sits more than half a pitch below that it is moved down

a band, or more than 0.7 pitches above it, up a band. A speck under 0.01 P^2 is discarded in that situation rather than moved, on the grounds that something that small and that far out of place is noise.

- **Between 0.06 and 0.15 P^2, a small but real piece.** Its own band and the two neighbours are considered, and it goes to whichever has writing whose median vertical centre is closest to its own. This is a nearest-centre rule, and it is used here rather than the band because a piece this size can easily fall on the wrong side of a cut while clearly belonging, by position, to one line.
- **A minor part of a larger piece.** A band holding a small, narrow part of a piece, under 0.12 P^2 and under 35 percent of the piece, no wider than 0.55 pitches, in a band adjacent to the piece's assigned one, is treated as a cut-off tail and follows the rest of the piece. The width condition is what distinguishes a tail from a whole word: a tail is narrow, and a wide part of a piece in another band is that band's own writing.

The median is used for the position of nearby writing rather than the mean, for the usual reason: one misplaced piece among the neighbours would drag a mean, while the median reports where the bulk of the line actually is.

The dot rule The last and most specific correction handles the dots of i and j. A dot floats clear of its own letter, often nearer to the line above than to its own line's body, so neither the band nor a nearest-centre test places it reliably. What does place it is the letter underneath it.

A piece is a dot candidate when it is small, under 0.1 P^2, compact, no more than 0.45 pitches in either direction, and roughly round, its longer side no more than 2.2 times its shorter. The roundness test is what excludes a comma or an accent from being treated as a dot. The rows below the candidate are then scanned, up to 0.4 pitches down, for the first substantial piece appearing beneath it, and the dot is given to that piece's line, provided the geometry agrees: the line's writing must sit between 0.25 and 0.7 pitches below the dot, which is where a dot floats above its own line's body. That final check is what stops the rule firing when the piece found below happens to belong to a line further away.

Writing the result All of this produces, for each piece, either a band index or a marker meaning discard. The ownership array is then filled in one operation: a pixel takes its piece's assigned band if the piece was one of those the rules relocated, and its own band index otherwise. A copy is kept before the later corrections run, because those corrections need to know which pixels were already moved here, in order not to undo that work.

4.41 _DotsJoinedAbove

```
def _DotsJoinedAbove(own, mask, lab, area, region, centres, L, P, H, W,
    ys, xs)
```

Inputs

`own` the ownership array, modified in place

`mask`
the cleaned ink mask

`lab` the closed-mask component labels

`area`
: the area of each closed-mask component

`region`
: the band index of every pixel

`centres`
: the line centres

`L` the number of bands

`P` the line pitch

`H` the height

`W` the width

`ys` the row coordinates of the ink pixels

`xs` their column coordinates

Output. four items describing the components of the unclosed mask: the label array, the count, the per-pixel labels and the areas. The ownership array is corrected in place.

The problem it solves The closing applied before labelling in DropArtifactPieces is useful but it has a side effect. If the dot of an i on one line happens to sit just below a descender from the line above, the closing can bridge the small gap between them and fuse the two into a single component. The dot then inherits whatever the fused piece was assigned, which is the upper line, and the lower line's crop loses its dot while the upper line's crop gains a stray mark.

Why a second labelling is needed The fix requires seeing the dot as its own object, which the closed labelling can no longer do. So the mask is labelled again, this time without closing. In the unclosed labelling the dot is separate from the descender, because the gap between them is real ink-free space; it was only the closing that joined them. The function therefore works with two labellings at once, the unclosed one to identify the dot and the closed one to recognise when a piece is in fact part of a larger structure.

Identifying the dot The same three-part test as before: small area, between 0.015 and 0.1 P^2, compact, no more than half a pitch across, and roughly round. The lower area bound is new and matters here, since it excludes single-pixel noise that would otherwise pass every other condition.

Finding the stem and its line The rows immediately below the candidate are scanned for the first other substantial piece, and the band of the pixel where the two meet is read from the region array. That gives the line the stem belongs to, which is the line the dot should belong to. The dot's own current assignment is read as the most common ownership value over its pixels.

The three conditions for acting

- **The stem must be exactly one band below.** A fusion of this kind can only displace a mark by one line, since the closing bridges a gap of a few pixels. A discrepancy of two bands means something else is wrong, and acting on it would make things worse.

- **The fused piece must not be a whole word.** If the closed component containing the dot has an area of 0.2 P^2 or more, it is a word, not a dot welded to a descender. A genuine floating dot stays a small component even after closing, so a large closed area is evidence that the candidate is really part of a letter, such as the bowl of an a that the threshold happened to detach.
- **The dot must sit well below the upper line's centre.** At least 0.45 pitches below it. A mark that close to the upper line's body really is part of the upper line, and the rule declines to move it.

Only the pixels currently assigned to the upper line are moved, not every pixel of the unclosed piece. This matters when a letter has broken apart in the unclosed mask, so that the candidate shares its unclosed component with ink that was already correctly assigned: moving the whole component would drag that correct ink along with it.

> *The pattern of conditions here is characteristic of the whole ownership stage. Each correction rule is narrow on purpose, requiring several independent pieces of evidence to agree before it fires. A broad rule would fix more cases and break more, and a crop that is wrong is more costly than a crop that is merely imperfect, because the recogniser will read it confidently either way.*

4.42 _RingsToStem

```
def _RingsToStem(own, raw, nr, pr, Ar, lab, area, region, centres, P, H,
    W, ys, xs)
```

Inputs

`own` the ownership array, modified in place

`raw` the unclosed component labels

`nr` the number of unclosed components

`pr` the unclosed label of each ink pixel

`Ar` the area of each unclosed component

`lab` the closed labels

`area`
: the closed areas

`region`
: the band index of every pixel

`centres`
: the line centres

`P` the line pitch

`H` the height

`W` the width

`ys` the row coordinates of the ink pixels

`xs` their column coordinates

Output. nothing is returned; the ownership array is corrected in place.

The problem it solves This is the mirror image of the previous correction. There, a dot on the lower line was welded to a descender from above and wrongly given to the upper line. Here the welding runs the other way: a small ring-shaped mark belonging to the lower line, such as the dot of an i written as a small circle, or the loop at the top of a letter, is fused to the stem it sits on, and the stem's line is the one it should follow. The two corrections exist separately because the evidence needed differs: finding a ring requires detecting a hole, which the dot rule does not do.

Finding a candidate: walking up from the bottom The candidate is not a whole component here but a part of one, so it cannot be found by labelling. It is found geometrically. For each component, the lowest ink pixel in each column is located, and those bottom points are examined in order from the lowest upward, since the ring sits at the top of a stem and the stem's foot is the lowest point of the structure.

From each bottom point the algorithm walks upward, measuring at every row how wide the component is within a window around the column, and tracking the widest it has been. The walk stops when the width drops below 45 percent of that maximum. That point is the junction between the blob and the stem: a ring is wide, the stem below it is narrow, so the width collapses exactly where one ends and the other begins. This is a one-dimensional version of a shape-decomposition idea that recurs in image analysis: a narrow neck is where an object naturally divides.

Validating the shape Three conditions must hold before the candidate is accepted as a ring.

- **Its width is between 0.2 and 0.45 pitches.** Narrower is a stroke, wider is a letter body.
- **It is roughly as tall as it is wide,** with a height-to-width ratio between 0.6 and 1.6. A ring is round; a stroke is not.
- **It encloses a hole.** This is the decisive test, and it is carried out by labelling the components of the inverse of the blob after padding it with a border. If the blob encloses a hole then the background splits into two components, the outside and the hole, so a count of fewer than two components means there is no hole and the candidate is a hook or a curl rather than a ring.

The padding before that test is not optional. Without a border of background all round, a blob touching the edge of the extracted window would let the outside connect to the hole along that edge, merging them into one component and hiding the hole. Padding guarantees the outside is a single region that surrounds the blob completely. Counting holes by labelling the background is a standard technique, and the number it yields is related to the **Euler number** of the shape, the classical topological measure of components minus holes.

The decision As before, the first other piece appearing below the ring identifies the line to move to, the two bands must be adjacent, the closed component must not be large enough to be a whole word, and the junction must lie no more than 0.45 pitches above the target line's centre. Only the ring's own pixels are reassigned, confined to the rows between the junction and the bottom point and to the columns the blob occupies, and only where they currently belong to the band being moved from.

Why the examination is limited At most 60 bottom points are examined per component, and accepting one point suppresses the columns within 0.35 pitches of it. Both are cost controls. A

long word has hundreds of columns and the upward walk at each is not free, while the suppression prevents the same ring being found again from every column it spans. The cap is a deliberate compromise: a word with more than 60 distinct bottom points may have a ring missed, which costs one misplaced dot, whereas examining every column would slow the segmenter measurably on every page.

4.43 _FragmentsToNeighbour

```
def _FragmentsToNeighbour(own, ownBeforeEF, mask, centres, L, P, H, W)
```

Inputs

`own` the ownership array, modified in place

`ownBeforeEF`
: ownership as it stood before the two dot corrections, used to tell which pixels they already moved

`mask`
: the cleaned ink mask

`centres`
: the line centres

`L` the number of bands

`P` the line pitch

`H` the height

`W` the width

Output. nothing is returned; the ownership array is corrected in place.

The principle: a line's writing lies in its x-height band This is the most general of the corrections, and it rests on a single observation. A line of handwriting is defined by the band of rows its letter bodies occupy, around its centre. Anything genuinely belonging to that line must have some ink there, because a letter has a body. A piece assigned to a line that has no ink at all within a quarter pitch of its centre is therefore not part of that line, whatever the cut said. It is a visitor: a descender hanging down from above, or an ascender reaching up from below.

How the test is made For each band in turn, the ink currently owned by that band is labelled into connected pieces, and for each piece the number of its pixels lying within 0.28 pitches of the band's centre is counted.

$$\text{inband}(c) \;=\; \left|\{\,(y,x) \in c \;:\; |y - \text{centre}_n| \;\le\; 0.28\,P\,\}\right| \tag{4.51}$$

where

inband(c)
: how many pixels of piece c lie in the band's x-height zone

c a connected piece of the ink currently owned by band n

centre_n
: the row position of line n

P : the line pitch

0.28 : the half-height of the x-height zone as a fraction of the pitch

A piece with a count of exactly zero, and an area of at least 0.006 P^2 so that single-pixel noise is ignored, is a candidate for rehoming. The 0.28 figure is about half the x-height expressed in pitches, which follows from the factor of roughly 2.2 between pitch and x-height noted in LinePitch: half an x-height is about 0.23 pitches, and 0.28 allows a little margin for the centre estimate being slightly off.

Deciding where it belongs: two routes Having established that a fragment does not belong where it is, two independent kinds of evidence are tried for where it does.

- **Adjacency.** The fragment is dilated by two pixels and the ownership of the surrounding ink is examined. If at least three of those neighbouring pixels belong to one other band, the fragment is physically touching that line's writing, which is strong evidence that it is part of it. The move is only made if that band is adjacent and lies on the correct side: a fragment below this line's centre may only move down, one above it only up. That consistency check is what prevents a chain of bad moves.
- **Position.** If no neighbouring band reaches three pixels, the fragment is isolated and adjacency says nothing. The fallback is to ask whether the fragment's own vertical centre falls inside a neighbouring line's x-height band, within 0.2 pitches of its centre, and if so to move it there. This is weaker evidence, which is why it is used only as a fallback and with a tighter tolerance.

Not undoing the previous corrections The ownership array as it stood before the two dot corrections is passed in so that the pixels they moved can be identified, by comparing the two arrays. A small compact fragment that sits on pixels those rules already placed is skipped. The reason is direct: a dot deliberately placed above its line by the dot rule has no ink in its line's x-height band, by the very nature of being a dot, so it looks exactly like a fragment to this test and would be moved straight back. The exception is deliberately narrow, applying only to pieces under 0.04 P^2 and no more than 0.4 pitches across, so that a genuine tail loop that happens to overlap a placed dot is still free to move.

> *Every move is recorded and applied only after the scan over all the bands is complete. Applying them as they are found would mean later tests read ownership that earlier moves had already changed, so the outcome would depend on the order the bands happened to be visited. Collecting first and applying afterwards makes the result independent of that order.*

4.44 AssignOwnership

```
def AssignOwnership(mask, lab, nc, cuts, cutsArr, L, centres, P, H, W)
```

Inputs

`mask`
: the cleaned ink mask

`lab` the component labels

`nc` the component count

`cuts`
: the cuts as a list

`cutsArr`
: the cuts as an array

`L` the number of bands

`centres`
: the line centres

`P` the line pitch

`H` the height

`W` the width

Output. the ownership array: for every pixel, the index of the line that owns it, or -1 for background and for ink that was discarded.

What it is for This function contains no logic of its own. It runs the band guess and then the three corrections, in order, and returns the result. Its value is organisational, and the organisation is the point worth explaining.

Why the rules are separate functions The ownership problem is where the segmenter is least principled and most empirical. The band guess follows from the cuts, but each correction after it exists because a specific failure was observed on real pages and diagnosed. Keeping each as its own function with its own name has three practical consequences. A rule can be read on its own, with its conditions in one place rather than interleaved with others. A rule can be disabled by commenting out one line here, which is how its contribution was measured. And a rule can be tested against the cases it was written for without running the whole page.

Why the order is fixed The sequence is not arbitrary. The band guess must come first because everything else corrects it. The two welding rules come next, and they are the most specific: each requires several independent conditions and fires rarely. The fragment rule comes last and is the most general, which is the right place for it, because a general rule run first would act on cases the specific rules would have handled better. The ordering principle, specific before general, is the same one that governs a cascade of fallbacks in LinePitch: apply the best-informed rule wherever it applies, and the broad rule only to what is left.

The dependency between them is explicit in the arguments. The fragment rule receives the ownership array as it stood before the welding rules ran, precisely so it can recognise and respect their decisions. That is why the band guess returns that copy, rather than each rule working from whatever it finds.

4.45 _BuildLineCrop

```
def _BuildLineCrop(n, own, norm, P, f, H, W, minInk)
```

Inputs

`n` which line to build, as a band index

`own` the ownership array

`norm`
the illumination-corrected full-resolution page

`P` the line pitch, in half-size pixels

`f` the scale factor from half-size to full-size coordinates

`H` the half-size height

`W` the half-size width

`minInk`
the least ink a band must own to be a real line

Output. None if this band is not a usable line; otherwise a dictionary with the finished grey crop, a fixed-size copy for the recogniser, and the bounding box the crop came from.

What it is for All the analysis so far has been at half resolution and in terms of masks. This function turns one band's worth of ownership into the actual image the recogniser will read: a grey picture of one line of writing, cut from the full-resolution page, contrast-corrected, with the neighbouring lines removed.

Rejecting a band that is not a line Two tests reject a band outright. It must own at least minInk pixels, a threshold the caller derives from the median ink per line so that it adapts to the page, and its ink must span at least 0.2 pitches vertically. The height test is the more interesting one: a band containing a horizontal scatter of specks has plenty of ink but no vertical extent, and is a surviving artefact rather than writing. The floor of six pixels keeps the test meaningful on a very small image. The height demanded is deliberately well under a full x-height, because a short line containing only lowercase letters is still a real line.

Trimming a distant stray A band can own a legitimate line plus one mark far away, perhaps a page number the artefact rules did not catch. Cropping to all of it would produce an image mostly of blank paper with writing at one end, which is poor input for a recogniser. So the rows containing ink are examined, the gaps between them found, and the largest contiguous run of inked rows kept.

The trim is only performed if it removes more than 0.8 pitches of height. That guard is what distinguishes a paragraph-sized gap from the ordinary small gaps inside a line of writing, which occur between the body of the line and a stray ascender. Without the guard, a line with an ordinary internal gap would have its ascenders trimmed away.

From half-size decisions to full-size pixels The bounding box of the kept ink is padded by 0.15 pitches, so that the writing does not touch the edge of the crop, and then multiplied by the scale factor to give the corresponding rectangle of the full-resolution page. This is where the two resolutions meet: every decision was made cheaply at half size, and the pixels are taken at full size.

Building the alpha channel The crop must show this line's ink and not the neighbouring lines' intrusions, so a per-pixel weight is needed saying how much of each pixel to show. That weight is called **alpha**, borrowing the standard graphics term for opacity, where 1 means fully shown and 0 fully replaced by white.

It starts as the ownership mask for this line, enlarged to the crop size bilinearly. Bilinear rather than nearest-neighbour is a deliberate choice here, and for an unusual reason: the soft edge that interpolation produces is what makes the stroke boundary in the final crop look like a real pen edge rather than a staircase of hard pixels. The slight blur is the feature, not an artefact to be tolerated.

Recovering ink the mask missed The half-size mask inevitably misses some faint ink, and at full resolution that ink is visible. So a second, local threshold is computed, on the crop alone rather than the whole page, using the same divide-by-local-background method as InkMask but with a radius scaled to the crop's own height. Because the crop is one line of writing rather than a whole page, the contrast within it is far more uniform, so this local threshold can pick up strokes that the page-wide one could not.

Those recovered pixels cannot simply be added, or the neighbouring lines' ink would come back with them. Two gates restrict them.

- **The halo.** The line's own ink is dilated by about 0.2 pitches and the recovery is confined to that neighbourhood. The reasoning is that a stroke the mask missed lies next to strokes it caught, since it is part of the same writing, whereas ink from the line above or below is further away.
- **The ownership veto.** Any pixel the page-level ownership already gave to a different line is excluded, unconditionally. This gate is enlarged with nearest-neighbour rather than bilinear interpolation, precisely the opposite choice from the alpha channel, and for the opposite reason: ownership is a yes-or-no fact about which line a pixel belongs to, and a blurred version would half-assign a pixel to two lines at once.

The final alpha is the larger of the ownership alpha and the gated recovery, taken pixel by pixel, so recovery can only ever add ink and never remove it.

Contrast stretching The crop's grey values still span whatever range the photograph happened to produce, and the recogniser was trained on images with ink near black and paper near white. A **contrast stretch** maps the observed range onto the full range linearly.

$$\text{linear}[y,x] \;=\; \text{clip}\Big(\big(\text{region}[y,x]-b\big)\cdot\frac{255}{\max(w-b,\,1)},\;0,\;255\Big) \tag{4.52}$$

where

$\text{linear}[y,x]$
: the stretched grey value

$\text{region}[y,x]$
: the illumination-corrected value at this pixel

b
: the black point: the value that should become 0, taken as the 40th percentile of the values under the ink

w the white point: the value that should become 255, taken as the 75th percentile of the whole crop

$\max(w - b,\, 1)$
a guard against a degenerate crop in which the two points coincide

$\mathrm{clip}(\cdot, 0, 255)$
values forced into the valid grey range

The two reference points are percentiles rather than the minimum and maximum, and the choice is what makes the stretch stable. One unusually dark pixel, from a blob of ink or a speck of dust, would set a minimum-based black point and leave all the real ink grey. The 40th percentile of the ink pixels describes the typical darkness of a stroke, which is the value that should become black. Likewise the 75th percentile of the crop describes the paper, since paper is the majority of the area.

The gamma correction A linear stretch alone leaves faint strokes faint, because a stroke at 70 percent of the way from ink to paper maps to 70 percent of the way from black to white. A **gamma correction** applies a power to the darkness, which bends the mapping.

$$d'[y,x] \;=\; \Big(\, d[y,x] \,\Big)^{\gamma}, \qquad d[y,x] \;=\; \frac{255 - \mathrm{linear}[y,x]}{255}, \qquad \gamma = 0.55 \tag{4.53}$$

where

$d[y,x]$
the darkness at this pixel, 0 for white and 1 for black: the stretched value inverted and scaled

$d'[y,x]$
the corrected darkness

γ the gamma exponent, here 0.55

With an exponent below 1 the power function lies above the straight line on the interval from 0 to 1, so every darkness is increased, and increased most where it starts small. A faint stroke at darkness 0.3 becomes 0.3 raised to the power 0.55, which is about 0.51, nearly doubling, while ink already at 0.9 becomes 0.94, barely changing. That is precisely the desired behaviour: pull the faint strokes up to where the recogniser can see them, and leave the strong ink alone. The corrected darkness is then converted back to grey and composited with white through the alpha channel.

Aligning the left margin Finally the crop is shifted horizontally so that the first column containing ink sits the same distance from the left edge in every crop. The reason is that the recogniser's convolutional layers respond to position, so a line whose writing starts 20 pixels in and one whose writing starts 80 pixels in present measurably different inputs for the same text. Standardising the left offset removes a variation that carries no information.

> *The returned dictionary contains the crop twice: once at its natural size for display and for the letter cutter, and once squeezed onto the recogniser's fixed input canvas by FitFixedCanvas. Keeping both avoids any later stage having to guess which geometry it is looking at.*

4.46 SegmentLines

```
def SegmentLines(page)
```

Inputs

`page`
: the photograph of the page, grey or colour

Output. three items: the list of line crops in top-to-bottom order, a dictionary of raw intermediates for drawing diagnostic overlays, and a dictionary of summary statistics about the segmentation.

What it is for This is the entry point to the whole segmenter, and it is deliberately short. Every stage of the method is a named function above, so this function is a list of those calls in order, which means the sequence of operations can be read off directly instead of being reconstructed from a long body of code.

The pipeline in order

- **Deskew.** Reduce to grey, make a half-size copy, find the tilt and rotate, and derive the ink mask and ratio image.
- **RemovePrintedStructure.** Locate the sheet, discard everything outside it, and delete the long straight runs that are ruling and page edges.
- **DropArtifactPieces.** Judge what remains piece by piece and delete the creases, slivers and shadows that are not writing.
- **LineCentres.** Locate each line of writing as a peak of the row profile, with duplicates suppressed.
- **SeamCuts.** Draw a least-ink seam between each pair of neighbouring lines, dividing the page into bands.
- **AssignOwnership.** Decide which line owns every ink pixel, starting from the band and applying the three corrections.
- **_BuildLineCrop,** once per band: cut the crop from the full-resolution page, mask it to this line's ink, correct its contrast and align it.

Why the crops come from a separate normalised image The crops are cut not from the raw rotated page but from a full-resolution illumination-corrected version of it, built here. The background is estimated at half size, where the box average is cheap, then enlarged bilinearly to full size and divided out, with the same scale-aware radius that InkMask uses. Estimating the background at full resolution would cost four times as much for no benefit, because the illumination varies smoothly by definition, so a half-resolution estimate of it enlarged by interpolation is accurate to well within the precision that matters.

Why the ink threshold adapts to the page The minimum ink a band must own before it is accepted as a line is computed from the page itself: 12 percent of the median ink per band, with a floor of 60 pixels.

$$\text{minInk} \;=\; \max\big(60,\; 0.12 \cdot \text{median}\{\, I_n \;:\; I_n > 0 \,\}\big) \tag{4.54}$$

where

minInk
: the ink threshold for accepting a band as a line

I_n
: the number of ink pixels owned by band n

$\{\, I_n : I_n > 0 \,\}$
: the counts for the bands that own any ink at all, so that empty bands do not drag the median to zero

0.12
: the fraction of a typical line's ink that a band must own

60
: an absolute floor in pixels

An absolute threshold would be wrong in both directions: too high for small writing, where a real line has few pixels, and too low for a large photograph, where an artefact has many. Comparing against the median line on the same page sidesteps that, because it measures what a line looks like in this particular image. The fraction is set low, at 12 percent, because a short line containing one or two words is perfectly legitimate and must not be discarded; the threshold is aimed at bands containing a speck, not at short lines.

Why three return values The crops are the product. The other two exist because a segmenter that only returns its output is very hard to debug. The second dictionary carries the raw cuts, the ratio image and the paper mask, which the test code overlays on the page so that a bad cut can be seen rather than inferred. The third carries summary numbers: the angle applied, the pitch, what fraction of the frame was paper, how much ink lay outside it, how many straight pixels and artefact pieces were removed, and how many lines came out. Those numbers are how a failure gets localised: an angle at the edge of the search range, a pitch far from the usual, or a paper coverage near zero each point at a different stage, without any need to look at the image.

> *The diagnostics deliberately leave the project's own evaluation results out of the pipeline code. They report what this page did, not what the method scores in general. That separation keeps the core code free of anything that would have to be revised every time the method is measured again.*

5 recognise.py

Two trained neural networks live in this file. The first, called the reader throughout, takes a crop of one line of handwriting and returns the text written on it. The second, the namer, takes the same kind of crop and returns which of the enrolled writers produced it. Both run forward only; the code that trained them is in train.py.

Background: what a neural network is, in the sense used here A **neural network** is a function built by stacking simple operations, most of them a multiplication by a table of numbers followed by a simple non-linear squashing. The tables are called **weights** or **parameters**, and they are not written by hand: they are found by training, which means repeatedly adjusting them so that the network's output on examples moves closer to the known correct answer. A **layer** is one of those operations. Running the layers in order from input to output is the **forward pass**, which is all this file does; computing how each weight should change is the **backward pass**, which lives in train.py.

Nothing in this file uses a machine-learning framework. Every layer is a few lines of array arithmetic, which is possible because a forward pass really is nothing more than that. The point of a framework is mostly to supply the backward pass automatically, and that was derived by hand for this project.

The shape convention Arrays flowing through the networks carry four axes in a fixed order, written throughout as (B, C, H, W).

- **B, the batch:** how many separate images are being processed at once. It is 1 everywhere in this file, since crops are read one at a time, but the axis is kept because the training code uses many.
- **C, the channels:** how many separate images the layer is carrying per input. The input crop has one channel, its greyness. After the first convolution it has as many channels as that layer had filters, each one a map of where some particular stroke feature was found.
- **H and W, height and width:** the spatial axes, in pixels.

The recurrent half of the reader uses a different convention, (T, B, F), where T is a position along the line, B is still the batch and F is a vector of features at that position. The transition between the two conventions happens inside the reader's forward method, and it is the single most important structural step in the network; it is explained there.

5.1 CHARSET

```
CHARSET = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
    .,;:\'"!?()-'
```

Output. a string listing every character the reader is able to output, in the order of their slot numbers.

What it is for A network cannot emit a letter. It emits numbers, and something must fix which number means which letter. This string is that agreement: the character at position i of the string is what output slot i+1 of the network stands for. It is a constant rather than a setting because it was fixed when the network was trained; changing it now would silently relabel every output.

What is in it, and what is deliberately not It holds the 26 lowercase letters, the 26 uppercase letters, the ten digits, the space, and eleven punctuation marks, giving 74 characters and so 75 output slots once the blank is counted. Upper and lower case are separate slots rather than being folded together, because the reader is required to reproduce capitalisation, and a letterform really is different between the two cases, so merging them would ask one slot to cover two unrelated shapes.

What is absent is as deliberate. There are no accented letters and no currency or mathematical symbols, because none appear in the training data, and an output slot that is never trained is worse than no slot at all: it occupies capacity and can still win a column by accident.

Why the space is an ordinary character The space is in the set like any letter. This is worth noticing because it is not obvious: a space is the absence of ink, so it might seem to need no symbol. But the reader must distinguish a gap between words from a gap between the strokes of one letter, and that distinction is a judgement about the writing, not a measurement of white space. Making the space a trainable character lets the network learn where word boundaries fall from examples, rather than forcing a threshold on gap width to be chosen by hand.

> *Slot 0 does not appear in this string. It is reserved for the CTC blank, which is not a character at all but a signal meaning "nothing is being emitted here". Its role is explained in the entry for DecodeCtc, and it is the reason every index in the two lookups below is shifted by one.*

5.2 CHAR_TO_IDX

```
CHAR_TO_IDX = {c: i + 1 for i, c in enumerate(CHARSET)}
```

Output. a dictionary mapping each character of the charset to its output slot number, starting at 1.

What it is for Training and alignment both need the opposite direction from reading. Given the known text of a line, each character must become the slot number the network is supposed to favour, so that the network's output can be compared with it. This dictionary performs that conversion.

Why a dictionary and not a search The same answer could be had by searching the charset string for the character each time, but a dictionary is a **hash table**, meaning it computes a position directly from the key rather than scanning, so a lookup costs the same whether the set has ten entries or ten thousand. Since every character of every training label passes through this lookup, the difference is worth having for free.

The offset by one Slot numbers start at 1 rather than 0 because slot 0 belongs to the blank. Every use of this dictionary therefore inherits the offset, which is why the alignment code builds its label sequence from these values directly and never adds one of its own.

> *A character absent from the charset has no entry, and the callers rely on that: alignment and training both filter the label text through a membership test and silently drop anything unknown. The alternative, substituting a placeholder, would teach the network to emit that placeholder for an unfamiliar mark instead of leaving it out.*

5.3 IDX_TO_CHAR

```
IDX_TO_CHAR = {i: c for c, i in CHAR_TO_IDX.items()}
```

Output. a dictionary mapping an output slot number back to its character.

What it is for This is the reverse lookup, used when the network's output is turned back into readable text. The decoder picks a winning slot per column and this dictionary says what letter that slot stands for.

Why it is derived rather than written out It is constructed by reversing the forward dictionary, not by listing the pairs again. Writing both by hand would create two places that must agree, and a disagreement between them would not raise an error; it would quietly produce text in which some letters are substituted for others, which is exactly the kind of fault that is hard to notice and harder to trace. Deriving one from the other makes the agreement structural.

> *Slot 0 has no entry here either, which is correct: the blank is not a character and must never appear in decoded text. The decoder removes blanks before consulting this dictionary, and also passes a default of the empty string, so that an out-of-range slot contributes nothing rather than raising an error in the middle of reading a page.*

5.4 WEIGHTS_DIR

```
WEIGHTS_DIR = paths.Weights
```

Output. the directory holding every trained checkpoint file.

What it is for It names one place where trained weights live, taken from the paths module so that the location is defined once for the whole project. The value matters less than the fact that it is a single named constant: every checkpoint below is built from it, so moving the weights folder is one edit in paths.py rather than a search through several files.

5.5 INPUT_HEIGHT

```
INPUT_HEIGHT = 64
```

Output. 64, the crop height in pixels that both networks were trained on.

Why the height is fixed and the width is too A convolution can be applied to an image of any size, so it may seem that a network should accept any crop. The constraint comes from what the network learned. During training every crop was 64 pixels tall, so the filters learned the size a letter appears at when a line of writing is 64 pixels high, and the layer that collapses the height assumes that much to collapse. Presenting a 128-pixel crop would show the network letters at twice the scale its filters were tuned for, and the features they detect would simply not be found.

Why 64 specifically It is a compromise between detail and cost. A line of handwriting has to be tall enough that the difference between an a and an o survives, which needs a few tens of pixels across the x-height, and the crop must also hold the ascender and descender zones above and below it. Against that, every extra row multiplies the work in all the convolutional layers. A power of two is convenient because the network halves the height twice with pooling and then collapses what is left, so 64 divides cleanly where 60 or 70 would leave a remainder to be discarded.

5.6 INPUT_WIDTH

```
INPUT_WIDTH = 640
```

Output. 640, the crop width in pixels that both networks were trained on.

Why the width is fixed as well The width constraint is weaker in principle than the height one, because the reader treats width as the axis along which it reads and could in principle accept any length. It is fixed because training in batches requires every crop in a batch to be the same shape, and because a fixed width makes the mapping from an output column back to a pixel position a single scale factor, which is what FrameXToPixel relies on.

The ten-to-one ratio 640 is ten times the height, which reflects the shape of a line of writing: a line is long and thin. After two halvings the width is 160 columns, and since the reader emits one set of character scores per remaining column, that sets the maximum length of text it can represent. The CTC method requires at least one column per character and in practice rather more, so 160 columns comfortably covers the sixty or seventy characters a line of handwriting holds.

5.7 _TEXT_CANDIDATES

```
_TEXT_CANDIDATES = ['paper_cnn_bilstm_ctc_scratch_best.npz',
        'paper_cnn_bilstm_ctc_joint_best.npz','paper_cnn_bilstm_ctc_hf_
           best.npz']
```

Output. a list of reader checkpoint filenames, searched in order.

What it is for Several reader checkpoints exist, produced by different training runs. This list names them in the order they are to be preferred, and the first one present on disk is the one that gets used. The arrangement means the module runs on whatever subset of the checkpoints a given copy of the project happens to have, instead of failing when one specific file is absent.

What the three names mean

- **scratch:** trained from randomly initialised weights on the public database alone.
- **joint:** trained on the public database and this project's own photographed pages together.
- **hf:** trained from a checkpoint obtained elsewhere and then adapted, so not trained from scratch within this project.

Why the order of this list is a substantive decision Because the first existing file wins, this list alone decides which network actually reads a page, with no further input from the caller. That makes the ordering a functional choice rather than a cosmetic one, and it is worth being explicit that the checkpoints are not interchangeable: measured on the same 669 photographed lines, the scratch and joint checkpoints differ by a very large margin in character accuracy, the joint one being far stronger. Whichever name stands first in this list is therefore the one that determines the accuracy a user sees.

> *A list searched in preference order is a reasonable pattern, but it hides which choice was made behind the contents of a directory. Anything depending on the reader's accuracy should confirm which file was actually loaded, which is why the reader records its checkpoint path as an attribute.*

5.8 DEFAULT_TEXT_WEIGHTS

```
DEFAULT_TEXT_WEIGHTS = next((WEIGHTS_DIR / n for n in _TEXT_CANDIDATES if
    (WEIGHTS_DIR / n).exists()),
        WEIGHTS_DIR / _TEXT_CANDIDATES[0])
```

Output. the path of the reader checkpoint chosen from the candidate list, or the first candidate if none exists.

How the choice is made The candidate names are tested in order and the first that exists on disk is taken. If none exists, the first name is returned anyway rather than nothing at all.

Why a missing file is reported late rather than early Returning a path that does not exist looks like an oversight but is deliberate. If this constant were allowed to be empty, every later use would need to check for that, and the eventual error would complain about an empty value rather than a missing file. Returning the expected path means the failure surfaces when a network is actually constructed, as an error naming the exact file that could not be found, which is the message someone can act on.

> *Because this is evaluated once when the module is first imported, a checkpoint added to the directory afterwards is not picked up until the program restarts. Any code wanting a specific checkpoint should pass its path explicitly rather than relying on this constant.*

5.9 _AUTHOR_NPZ

```
_AUTHOR_NPZ = WEIGHTS_DIR / 'author_classifier_10new_scratch_weights.npz'
```

Output. the path of the preferred writer-naming checkpoint.

What it is for The first of three candidate checkpoints for the namer, trained from scratch on the writers enrolled in this project. It is preferred when present because it was trained within the project on the current set of writers, so its output slots match the writer list the rest of the pipeline uses.

5.10 _SHAPE

```
_SHAPE = WEIGHTS_DIR / 'author_shape_10new_weights.npz'
```

Output. the path of the shape-based writer-naming checkpoint.

What makes this one different It was trained on crops that had been stroke-normalised, meaning redrawn at a single constant pen width. The reason that matters is explained in full under StrokeNormalize: a namer trained on raw crops can identify a writer by how dark or thick their pen is, which is a property of the pen and not of the handwriting. Removing pen width forces the network to judge letter shapes instead.

This is the checkpoint used to judge the robot's own output, and that use is why it exists. Drawn output has a single uniform line width by construction, since a plotter draws every stroke with the same pen. A namer that had learned to use ink thickness would therefore be judging the plotter rather than the style, and would answer the wrong question.

5.11 _W1

```
_W1 = WEIGHTS_DIR / 'author_classifier_10new_weights.npz'
```

Output. the path of the ink-based writer-naming checkpoint.

What makes this one different It was trained on raw grey crops, with pen width and darkness left in. That makes it the right choice for identifying the writer of a real photographed page, where the pen is part of the evidence: a person's usual pen, and the pressure they write with, genuinely are characteristic of them. It is the checkpoint the live server uses on a photograph.

> *The project therefore keeps two namers that answer the same question from different evidence, and which to use depends on what the input is. Measured on the same crops the ink-based one scores higher, which is expected, because it is allowed to use information the shape-based one has been deliberately denied. The higher score is not grounds for preferring it on drawn output, where that extra information does not exist.*

5.12 DEFAULT_AUTHOR_WEIGHTS

```
DEFAULT_AUTHOR_WEIGHTS = _AUTHOR_NPZ if _AUTHOR_NPZ.exists() else _SHAPE
    if _SHAPE.exists() else _W1
```

Output. whichever of the three namer checkpoints was found, in order of preference.

How the choice is made The scratch-trained checkpoint is taken if it exists, otherwise the shape-based one, otherwise the ink-based one. The same preference-order idea as the reader's list, written as nested conditions because there are only three.

Why the fallback order ends with the ink model The ink-based checkpoint is last, which makes it the one reached when nothing else is present. That is the safe end of the order for a general-purpose default: the commonest use of the namer is on a photographed page, which is precisely the input the ink model is correct for. Code that needs the shape-based model, such as the evaluation of drawn output, names its checkpoint explicitly rather than trusting this default.

5.13 LoadStateDictAny

```
def LoadStateDictAny(checkpointPath)
```

Inputs

`checkpointPath`
: path to a saved .npz checkpoint file

Output. a pair: a dictionary mapping parameter names to arrays, and the writer name-to-slot mapping if the file carries one, otherwise nothing.

What it is for A trained network is just a collection of named arrays, and this function reads that collection off disk. Its value is that it is the only place in the file that knows how a checkpoint is stored, so every network below receives a plain dictionary and none of them contains any file handling.

Background: what a state dictionary is A **state dictionary** is the standard way to store a trained network: a mapping from the name of each parameter to its values. A name like `stage1.0.weight` encodes a position in the network, here the weight array of layer 0 of the block called stage1. The network's structure is not stored at all; it is written in code, and loading consists of looking up the names that code expects. That is why the forward methods below are full of string keys: those strings are the contract between the training script that wrote the file and the code that reads it.

The .npz format An `.npz` file is a zip archive of individual arrays, each stored under a name. That suits a state dictionary exactly, and it needs no library beyond numpy, which is the constraint this project works under.

Why pickling is refused The file is opened with object pickling disabled. **Pickling** is a mechanism for storing arbitrary Python objects, and restoring one executes code from the file, so a checkpoint able to contain pickled objects is able to run anything when loaded. Refusing it restricts the file to plain numeric arrays, which is all a set of weights needs to be. It also acts as a check on the files themselves: a checkpoint that will not load without pickling is storing something other than arrays, and that is worth discovering rather than tolerating.

Keys beginning with a double underscore Names starting with a double underscore are skipped when building the parameter dictionary, because they hold metadata rather than weights. One is read specially: a list of writer names, which is converted into a mapping from each name to its output slot.

Carrying that mapping inside the checkpoint matters more than it appears. The namer outputs one score per slot, and a slot means nothing without knowing which writer it stands for. If the correspondence were kept separately, in a configuration file or in code, then a checkpoint trained on a different set of writers, or on the same writers in a different order, would load without complaint and attribute every page to the wrong person. Storing the names with the weights makes that mismatch impossible.

5.14 Conv2d

```
def Conv2d(x, weight, bias, stride=1, padding=1)
```

Inputs

`x` the input, shaped (batch, channels, height, width)

`weight`
: the filters, shaped (out channels, in channels, filter height, filter width)

`bias`
: one number added per output channel, or nothing

`stride`
: how many pixels the filter advances between positions, default 1

`padding`
: how many rows and columns of zeros to add round the input before filtering, default 1

Output. the filtered result, shaped (batch, out channels, output height, output width).

What a convolution is, from scratch A **convolution** takes a small grid of numbers, called a **filter** or **kernel**, places it over a patch of the image, multiplies each filter number by the image value beneath it, and adds the products into one output number. Then it moves along and does it again, covering every position. The result is a new image, one value per position, recording how strongly the pattern in the filter was present there.

$$y[o,\,p,\,q] \;=\; b[o] \;+\; \sum_{c=0}^{C-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} w[o,\,c,\,u,\,v] \cdot x[c,\;p\,s+u,\;q\,s+v] \tag{5.1}$$

where

$y[o, p, q]$
: the output value for filter o at row p, column q

$b[o]$ the bias, one learned number added to every position of output channel o

c which input channel, running over all C of them

u, v the position inside the filter

k_h, k_w
: the filter's height and width, 3 by 3 everywhere in this project

$w[o, c, u, v]$
: the learned filter weight at that position

$x[c, y, x]$
: the input value in channel c

s : the stride, how far the filter moves between output positions

Read the triple sum as: for one output position, visit every cell of the filter in every input channel, multiply by the input underneath, and total it. The sum over channels is the part that is easy to overlook and important: a filter is not a flat grid but a stack as deep as the input has channels, so it combines evidence from every feature map below it rather than looking at one.

Why this is the right operation for handwriting Two properties make convolution suited to images. It is **local**: each output depends only on a small patch, which matches the fact that a stroke, a corner or a loop is a local arrangement of ink. And it is **translation-equivariant**, meaning that moving the input moves the output by the same amount without changing it: the same filter detects a vertical stroke wherever on the line it occurs. A plain fully connected layer has neither property; it would have to learn what a vertical stroke looks like separately at every position, needing enormously more parameters and far more examples.

What padding is for A 3 by 3 filter cannot be centred on a pixel in the outermost row, because part of it would hang off the edge, so without intervention each convolution would shrink the image by one pixel on every side. **Padding** adds a border of zeros so the filter can reach the edge, and a padding of 1 with a 3 by 3 filter keeps the output exactly the same size as the input. With twelve convolutions stacked in this network, the unpadded alternative would eat twelve pixels off the height, which is a fifth of a 64-pixel crop. Zero is the correct pad value because the input has been arranged so that zero means no ink; see TensorFromResized.

$$H_{\text{out}} \;=\; \left\lfloor \frac{H + 2\,\text{pad} - k_h}{s} \right\rfloor + 1 \tag{5.2}$$

where

H_{out}
: the output height

H : the input height

pad : the padding added on each side

k_h : the filter height

s : the stride

$\lfloor \cdot \rfloor$: rounding down, because a filter position that does not fit entirely inside the padded input is not used

The im2col trick, and why it is used Written literally the definition is a loop over output positions inside a loop over filters, which in Python would be unusably slow. The standard remedy is **im2col**, short for image to columns [14]: rearrange the input so that every patch the

filter will see becomes one column of a large matrix. The convolution then becomes a single matrix multiplication, because that is exactly what a matrix product computes, a dot product of every row of one matrix with every column of the other.

$$Y \;=\; W_{\text{flat}}\, X_{\text{cols}}, \qquad W_{\text{flat}} \in \mathbb{R}^{O\times(Ck_hk_w)}, \qquad X_{\text{cols}} \in \mathbb{R}^{(Ck_hk_w)\times(H_{\text{out}}W_{\text{out}})} \tag{5.3}$$

where

Y the output, as a matrix of one row per filter and one column per output position

W_{flat}
: the filters flattened so each becomes one row of length C times the filter area

X_{cols}
: the patches, each flattened into one column in the same order

O the number of filters, that is, output channels

$\mathbb{R}^{m\times n}$
: the set of real-valued matrices with m rows and n columns; this notation just states each array's shape

The gain is not from doing less arithmetic, since the number of multiplications is unchanged. It is that matrix multiplication is handed to **BLAS**, a long-standing library of linear algebra routines written to use the processor's vector instructions and its cache well. A matrix product runs perhaps two orders of magnitude faster than the equivalent Python loops, and reformulating the problem to reach it is worth more than any amount of tuning the loops.

Building the columns without copying The patches are not assembled by copying. Neighbouring patches overlap heavily, a 3 by 3 filter at stride 1 sharing six of nine pixels with its neighbour, so a literal im2col would duplicate most of the input nine times over. Instead the columns are produced as a **strided view**, which needs some explanation.

A numpy array is a flat block of memory plus a set of **strides**, one per axis, each saying how many bytes to step to advance by one along that axis. Indexing is then just arithmetic on those strides. Supplying a different shape and a different set of strides therefore produces an array that reads the same memory in a different pattern, with no data copied. Here a six-axis view is constructed whose axes are batch, channel, filter row, filter column, output row and output column, so that the patches appear laid out ready to be reshaped into columns. The strides for the two output axes are the input's own row and column strides multiplied by the stride, which is what makes stepping to the next output position step the right distance through the image.

> *A strided view of overlapping patches reports a shape far larger than the memory behind it, because the same bytes appear at many positions. That is safe to read and would be meaningless to write to. The reshape that follows does force one copy, since the pattern cannot be expressed as strides alone, but it is one copy instead of nine.*

The bias One learned number per output channel is added to every position of that channel. It lets a filter shift its whole response up or down, which matters because the squashing that follows treats zero specially: without a bias, a filter could only ever decide between zero and its raw response, whereas with one it can set the level at which it starts responding at all. Adding one number to a whole channel is written as a reshape to four axes and a plain addition, relying on **broadcasting**, numpy's rule that an axis of length 1 is repeated as needed to match the other operand.

5.15 BatchNorm2d

```
def BatchNorm2d(x, weight, bias, runningMean, runningVar, eps=1e-05)
```

Inputs

`x` the input, shaped (batch, channels, height, width)

`weight`
: the learned scale, one per channel

`bias`
: the learned shift, one per channel

`runningMean`
: the mean of each channel recorded during training

`runningVar`
: the variance of each channel recorded during training

`eps` a small constant guarding the division, default 1e-5

Output. the input with each channel standardised and then rescaled, same shape as the input.

The problem it solves Stack a dozen layers and the scale of the numbers flowing through them drifts. Each layer multiplies by its weights, so if those weights are on average slightly larger than the scale that preserves magnitude, the values grow layer by layer; slightly smaller, and they shrink toward nothing. Worse, the scale a layer sees changes during training, because the layers beneath it are themselves being changed. A layer is therefore chasing a moving target, which forces a small learning rate and slow training. **Batch normalisation** [15] fixes the scale explicitly: it standardises each channel so that it arrives at the next layer with a known mean and spread.

Background: mean, variance and standardising The **mean** is the average value. The **variance** is the average of the squared distances from the mean, a measure of how spread out the values are, and its square root is the **standard deviation**, which is in the same units as the values themselves. To **standardise** is to subtract the mean and divide by the standard deviation, which leaves a quantity with mean 0 and standard deviation 1 regardless of what scale it started on.

$$\hat{x}[c] \;=\; \frac{x[c] - \mu[c]}{\sqrt{\sigma^2[c] + \epsilon}} \tag{5.4}$$

where

$\hat{x}[c]$ the standardised value in channel c

$x[c]$ the incoming value

$\mu[c]$ the mean of channel c, recorded during training

$\sigma^2[c]$
: the variance of channel c, also recorded during training

ϵ a small constant, 0.00001, added so that a channel with no variation cannot cause a division by zero

Why the scale and shift are put back Forcing every channel to mean 0 and spread 1 would be too strong a constraint, because for some features the useful information is precisely that they are usually large or usually near zero. So two learned numbers per channel are applied afterwards, a scale and a shift, letting the network restore any mean and spread it finds useful, including the original one. The point is not that the statistics are fixed at 0 and 1; it is that they are now set by two parameters the network controls directly, instead of emerging from the accumulated behaviour of every layer below.

$$y[c] \;=\; \gamma[c]\,\hat{x}[c] \;+\; \beta[c] \tag{5.5}$$

where

$y[c]$ the output value in channel c

$\gamma[c]$ the learned scale for that channel, held in the argument called weight

$\beta[c]$ the learned shift, held in the argument called bias

$\hat{x}[c]$ the standardised value from above

Training-time and inference-time behaviour differ The method takes its name from how the statistics are obtained during training: from the current **batch**, the group of examples being processed together. That cannot be done when reading one crop. A single example's own mean and variance are not a useful description of anything, and worse, the answer for one image would depend on which other images happened to accompany it, which is unacceptable at inference time.

The resolution is that training also keeps a running average of the statistics across all batches, which estimates the mean and variance over the whole dataset. Those stored values are what this function uses. So the layer behaves differently in training and in use, and that difference is intended: in training it standardises by the batch, in use by fixed numbers read from the checkpoint. This is why `runningMean` and `runningVar` are arguments here while a training implementation computes them.

Folding four operations into two Written directly the layer is a subtraction, a division, a multiplication and an addition at every one of the hundreds of thousands of values in the array. But at inference time all four constants are fixed, so the whole thing is a linear function and can be rearranged into one multiplication and one addition.

$$y \;=\; \underbrace{\frac{\gamma}{\sqrt{\sigma^2+\epsilon}}}_{a}\; x \;+\; \underbrace{\left(\beta - \frac{\gamma\,\mu}{\sqrt{\sigma^2+\epsilon}}\right)}_{b} \tag{5.6}$$

where

a the combined scale, computed once per channel

b the combined shift, computed once per channel

γ, β the learned scale and shift

μ, σ^2 the stored mean and variance

ϵ the guard constant

The two combined constants are computed once per channel, over an array of a few hundred numbers, and then applied to the full array in one multiply and one add. The square root, in particular, happens once per channel rather than once per pixel. This rearrangement is exact, not an approximation, and it is the whole reason the implementation is two lines.

Why it is per channel and not per pixel There is one mean and one variance for each channel, shared by every pixel in it. That follows from what a channel is: the map of where one particular filter responded. A filter's typical response size is a property of that filter, the same wherever on the image it fires, so one statistic describes the whole map. Normalising per pixel would instead remove the differences between positions, which is the information the map exists to carry.

5.16 Relu

```
def Relu(x)
```

Inputs

`x` any array

Output. the array with every negative value replaced by zero.

Why a network needs this at all Convolution and the linear layer are both **linear** operations, meaning the output is a weighted sum of the inputs. Composing linear operations gives another linear operation: two matrix multiplications are equivalent to one by the product of the matrices. So a stack of a hundred linear layers computes exactly what a single one could, and depth buys nothing. Something non-linear must sit between them, and that is what an **activation function** is for.

$$\mathrm{ReLU}(x) \;=\; \max(x,\,0) \;=\; \begin{cases} x & x > 0 \\ 0 & x \le 0 \end{cases} \tag{5.7}$$

where

$\mathrm{ReLU}(x)$
: the output, named for rectified linear unit

x : the input value

$\max(x, 0)$
: the larger of x and zero

$\{$: case notation: the two lines give the value in each of the two situations listed on the right

Why this particular function ReLU is nearly the simplest non-linear function there is, and that is much of its appeal, but it has three concrete advantages over the smooth S-shaped functions that preceded it. Its derivative is 1 for every positive input, so a gradient passing back through many layers is not repeatedly multiplied by a number smaller than one, which is what made deep networks with sigmoid activations so hard to train. It produces exact zeros, so a layer's output is **sparse**, with many units silent on any given input, which tends to give cleaner and more separable features. And it costs one comparison, against an exponential for the alternatives.

What it means in this network After a convolution, a positive value means the filter's pattern was found and a negative one means something like its opposite was. ReLU discards the negatives entirely, which is to say the layer reports where each feature *was* present and stays silent about where it was not. The next layer therefore builds on evidence of presence rather than on a signed score, which is the behaviour that makes stacked filters compose into detectors for larger shapes: strokes into corners, corners into loops, loops into letters.

> *A unit whose input is always negative outputs zero always, and since the derivative of ReLU is zero there, nothing ever pushes it back. This is known as a dying unit, and it is ReLU's one real drawback. It is a training-time concern, handled in train.py by careful initialisation, and has no effect on the forward pass here.*

5.17 Sigmoid

```
def Sigmoid(x)
```

Inputs

`x` any array

Output. the array squashed into the range 0 to 1.

What it is for Where ReLU is used to introduce non-linearity, the sigmoid is used when a number has to be interpreted as a **proportion**: how much of something to let through, on a scale from none to all. In this file its only use is in the LSTM, whose gates each need exactly that.

$$\sigma(x) \;=\; \frac{1}{1 + e^{-x}} \tag{5.8}$$

where

$\sigma(x)$
: the sigmoid of x, a value strictly between 0 and 1

x
: the input, any real number

e
: the base of the natural logarithm, about 2.71828

e^{-x}
: the exponential of minus x: large when x is very negative, near zero when x is large and positive

Why the shape is right for a gate Follow the formula at three points. For a large positive x the exponential is near zero, so the result approaches 1, fully open. For a large negative x the exponential is huge, so the result approaches 0, fully closed. At zero the exponential is exactly 1 and the result is exactly one half. The curve is smooth and increases everywhere, so the gate can take any intermediate position, and because it is smooth it has a derivative everywhere, which training requires. The S shape also means it saturates: once a gate is firmly open or firmly closed, moderate changes in the input barely move it, which is the stability a memory mechanism needs.

> *For a very large negative input the exponential overflows and numpy issues a warning while returning infinity, whose reciprocal is correctly zero. The answer is right; only the warning is spurious. The implementation is left in its direct form because the alternative formulation adds a branch to every element of every gate at every time step to suppress a warning that misrepresents nothing.*

5.18 MaxPool2d

```
def MaxPool2d(x, kernel=2, stride=2)
```

Inputs

`x` the input, shaped (batch, channels, height, width)

`kernel`
: the window size, default 2

`stride`
: how far the window moves, default 2

Output. the input reduced by the stride in both directions, each output value being the largest in its window.

What it is for **Pooling** shrinks the spatial size of a feature map by summarising each small window as a single number. With a 2 by 2 window and a stride of 2, the windows do not overlap and the map comes out half as tall and half as wide, a quarter of the values.

$$y[c,\, p,\, q] \;=\; \max_{0 \le u,v < k} \; x[c,\; p\,s + u,\; q\,s + v] \tag{5.9}$$

where

$y[c, p, q]$
: the pooled value in channel c at row p, column q

k the window size

s the stride

u, v the position within the window

max the largest of the values in the window

Three reasons it is done

- **Cost.** A quarter of the values means a quarter of the work in every layer that follows. Since this network pools twice, the deepest and widest convolutional block runs on a sixteenth of the original area.
- **Receptive field.** The patch of the original image that influences one value, called its **receptive field**, grows slowly if only 3 by 3 convolutions are stacked, by two pixels per layer. Halving the resolution doubles the area each later filter covers in original pixels, which is how the network gets from detecting strokes to detecting whole letters without needing very many layers.
- **Small-shift tolerance.** Taking the largest value in a window gives the same answer wherever in that window the feature occurred, so the output is unchanged by a shift of a pixel. For handwriting this is exactly right: the same letter written a pixel higher is the same letter.

Why the maximum rather than the average After ReLU, a value is evidence that a feature is present, and zero is silence. The maximum answers "was this feature found anywhere in this window", which is the useful question about evidence. An average would dilute a strong local response by the surrounding silence, so a thin stroke correctly detected in one position would be reported as a weak response over a wider area. Averaging has its place, and this file uses it later where the question genuinely is about a typical level rather than a presence.

Implementation, and the discarded remainder The windows are formed as a strided view, by the same mechanism described under Conv2d, with two extra axes for position within the window; taking the maximum over those two axes then pools everything at once. An odd final row or column, which cannot fill a window, is dropped before the view is built. Discarding input is acceptable here because the loss is a single row or column at the edge of a padded crop, where there is no ink, while the alternative of padding would introduce a window containing invented values whose maximum could be reported as a detected feature.

5.19 AdaptiveMaxPoolHeight

```
def AdaptiveMaxPoolHeight(x)
```

Inputs

`x` the input, shaped (batch, channels, height, width)

Output. the same array with its height collapsed to 1, keeping the largest value in each column of each channel.

What it is for, and why it is the pivot of the whole network This one line is where the reader stops being an image network and becomes a sequence network. Before it, the data is a stack of two-dimensional feature maps. After it, each column of the image is a single vector of features, and the line is a sequence of such vectors, left to right. That is the form the LSTM and CTC both require, and this is the step that produces it.

$$y[c,\,0,\,q] \;=\; \max_{0 \le p < H} \; x[c,\,p,\,q] \tag{5.10}$$

where

$y[c, 0, q]$
: the collapsed value for channel c at column q, the height axis now being of length 1

$x[c, p, q]$
: the feature value at row p of that column

H : the feature map height at this point in the network, which is 16 after two poolings of a 64-pixel crop

$\max_{0 \le p < H}$
: the largest value down the whole column

Why collapsing the height is legitimate It looks like throwing away a great deal, and it would be for a general image. For a line of handwriting it is justified by the geometry of the problem. The crop contains exactly one line of text, so vertical position within it carries no information about *which* character is present, only about which part of a character a pixel belongs to. What identifies the character at a given horizontal position is the set of features found anywhere in that column. Taking the maximum down each column asks precisely that: was this feature present anywhere at this horizontal position.

The word **adaptive** means the output size is specified rather than the window: here the output height is 1 whatever the input height, so the window is as tall as the input. That is why the function needs no parameters, and why it would behave identically on a crop of a different height, unlike the fixed pooling above.

> *A maximum, not an average, for the same reason as before: after ReLU these values are evidence of presence, and a feature found in two rows out of sixteen should be reported as found, not diluted by the fourteen silent rows. This is the choice that lets an ascender and the body of a letter in the same column both register.*

5.20 AdaptiveAvgPoolWidth

```
def AdaptiveAvgPoolWidth(x)
```

Inputs

`x` the input, shaped (batch, channels, height, width)

Output. the array averaged down axis 2, with that axis kept at length 1.

What it is for The namer, unlike the reader, has to produce one answer for the whole crop rather than one per column. The identity of a writer is a property of the line as a whole, not of a position along it. So at the end of the namer's backbone the remaining axis is collapsed by averaging, turning a sequence of feature vectors into a single feature vector describing the whole line.

$$y[c] \;=\; \frac{1}{N}\sum_{q=0}^{N-1} x[c,\,q] \tag{5.11}$$

where

$y[c]$ the pooled value for channel c

$x[c, q]$
the feature value at position q

N the number of positions being averaged over

Why averaging is right here and maximum was right before The question has changed, and so the summary must. The reader asks whether a feature occurs at a given position, which is a question about presence, answered by a maximum. The namer asks how characteristic a feature is of this writing, which is a question about prevalence, answered by an average. A writer who

makes a particular join in most of their letters should score differently from one who makes it once; a maximum would report both identically, since both have it somewhere. Averaging is what makes the feature vector describe a habit rather than an occurrence.

This construction is known as **global average pooling**, and beyond matching the question it has a practical benefit: the output size depends only on the number of channels, not on the length of the line. The classifier head that follows therefore has a fixed input size regardless of how long the crop was.

> *The function name says width and the code averages axis 2, which is the height axis in the four-axis convention. There is no inconsistency in effect, because by the time it is called the height has already been collapsed and the array has been reduced to three axes, so axis 2 is the sequence position. The name describes the intent and the index describes the array as it actually arrives.*

5.21 Linear

```
def Linear(x, weight, bias)
```

Inputs

`x` the input, whose last axis is the feature vector

`weight`
: the weight matrix, shaped (outputs, inputs)

`bias`
: one number added per output, or nothing

Output. the input multiplied by the transposed weight matrix, with the bias added; the last axis becomes the number of outputs.

What it is The **fully connected** or **linear** layer, the most basic layer there is: every output is a weighted sum of every input, plus a constant. Where a convolution looks at a local patch and reuses one filter everywhere, a linear layer looks at the whole input vector and has an independent weight for each input-output pair.

$$y[j] \;=\; b[j] \;+\; \sum_{i=0}^{n-1} W[j,\, i]\, x[i] \tag{5.12}$$

where

$y[j]$ the j-th output

$b[j]$ the bias for that output

$W[j, i]$
: the weight connecting input i to output j

$x[i]$ the i-th input

n the number of inputs

Why the matrix is transposed The weight array is stored with one row per output, which is the convention the training framework used, while the input has its features along its last axis. Multiplying the input by the transpose pairs each input vector with each row of weights, giving one output per row. Transposing is free in numpy, being only a reordering of the strides, so this costs nothing and keeps the stored weights in the layout the checkpoint was written with.

Where it is used in each network In the reader it is the final step, turning the LSTM's output at each column into one score per character. The weighted sum is the right operation there because any feature of the column may bear on any character. In the namer it appears twice, as a hidden layer with a ReLU between and then a final layer producing one score per writer, which is enough to combine the pooled features non-linearly before the decision.

> *The multiplication applies to the last axis and leaves every earlier axis alone, so the same call works on a single vector, on a batch of them, or on the reader's three-axis sequence of columns. That is broadcasting again, and it is why no reshaping surrounds either use.*

5.22 LogSoftmax

```
def LogSoftmax(x, axis)
```

Inputs

`x` an array of raw scores

`axis`
: which axis the scores to be compared lie along

Output. the log of the softmax along that axis: values at most zero, whose exponentials sum to one.

Background: turning scores into probabilities A network's final layer produces one unbounded number per class, called a **logit**. These are not probabilities: they can be negative, and they need not sum to anything in particular. The **softmax** converts them into probabilities by exponentiating each, which makes them positive, and dividing by the total, which makes them sum to one.

$$p_k = \frac{e^{x_k}}{\sum_j e^{x_j}} \tag{5.13}$$

where

p_k the probability assigned to class k

x_k the raw score, or logit, for class k

$\sum_j e^{x_j}$
: the sum of the exponentials over every class, which makes the outputs total 1

Exponentiating is what makes the conversion monotone and amplifying: a score two units higher becomes a probability about 7.4 times larger, so softmax preserves the ranking while sharpening the differences.

Why the logarithm is taken CTC multiplies many probabilities together, one per column of a line, and a line has well over a hundred columns. A product of a hundred numbers each around 0.1 is around ten to the power minus one hundred, far below the smallest number a double can represent, so it becomes exactly zero and all information is lost. This is **underflow**. Working with logarithms removes it entirely, because the logarithm of a product is the sum of the logarithms, and a sum of a hundred moderate negative numbers is an ordinary number.

$$\log\Big(\prod_t p_t\Big) \;=\; \sum_t \log p_t \tag{5.14}$$

where

$\prod_t p_t$
: the product of the per-column probabilities along a path

$\sum_t \log p_t$
: the equivalent sum of their logarithms

p_t : the probability at column t

The log-sum-exp trick Computing the logarithm of the softmax naively would exponentiate first, and a logit of 800 overflows to infinity before any logarithm is taken. The remedy uses the fact that softmax is unchanged by subtracting a constant from every score, since the constant's exponential cancels between numerator and denominator. Subtracting the largest score makes the biggest exponential exactly 1 and every other smaller, so nothing can overflow.

$$\log p_k \;=\; (x_k - m) \;-\; \log \sum_j e^{\,x_j - m}, \qquad m \;=\; \max_j x_j \tag{5.15}$$

where

$\log p_k$
: the log probability of class k, the function's output

x_k : the raw score for class k

m : the largest score, subtracted from every score before any exponential is taken

$\log \sum_j e^{x_j - m}$
: the log-sum-exp term, which is the normalisation expressed in log space

Underflow in the other direction is also harmless here. A score far below the maximum exponentiates to zero, contributing nothing to the sum, which is the correct contribution for a class of negligible probability. The maximum term is always present and equals 1, so the sum can never be zero and its logarithm is always defined. This rearrangement is known as the **log-sum-exp trick**, and it is standard wherever probabilities are handled in log space.

> *Every output of this function is at most zero, because a probability is at most one and the logarithm of a number at most one is at most zero. Seeing negative numbers in the reader's output is therefore expected, not a fault, and a value near zero means near-certainty.*

5.23 Softmax

```
def Softmax(x, axis)
```

Inputs

`x` an array of raw scores

`axis`
: which axis the scores to be compared lie along

Output. probabilities along that axis: non-negative and summing to one.

What it is for The same conversion as above without the logarithm, used where an actual probability is wanted for a person to read rather than a log probability for further arithmetic. In this file its only use is in the namer, where the output is a confidence per writer that gets reported.

The same stabilisation, for the same reason The maximum is subtracted before exponentiating here too. The result is mathematically identical, since the subtraction cancels, but it cannot overflow: the largest exponential is 1 rather than potentially enormous. The cost is one pass to find the maximum and one subtraction, which is negligible against the exponentials themselves.

Why the namer uses this and the reader uses the log version The distinction follows from what happens next. The namer's output is compared and reported directly, so a probability is what is wanted. The reader's output is summed along thousands of paths by CTC, so log probabilities are what is wanted. Converting between them afterwards would be possible but would reintroduce exactly the underflow that working in log space exists to avoid.

> *A confidence from this function is a probability over the enrolled writers only, and it necessarily sums to one across them. It therefore cannot express "none of these": a crop from somebody not enrolled is still assigned to whichever enrolled writer resembles it most, with a confidence that may be high. This is a property of the output layer, not a fault in the training, and any use of the namer on an unknown writer has to account for it.*

5.24 ConvBlockForward

```
def ConvBlockForward(x, sd, prefix, nLayers)
```

Inputs

`x` the input feature maps

`sd` the loaded state dictionary of parameters

`prefix`
: the name prefix identifying this block in the checkpoint, such as stage1

`nLayers`
: how many convolution, normalise, activate groups this block contains

Output. the feature maps after that many groups have been applied.

What it is for Both networks are built from the same repeated unit: a convolution, then batch normalisation, then ReLU. This function applies that unit a given number of times, so the unit is written once rather than twelve times in each network.

Why that particular order The order is not arbitrary and each adjacency has a reason. Normalisation comes after the convolution because its purpose is to fix the scale of the convolution's output, which it cannot do before that output exists. ReLU comes after normalisation because the normalisation needs to see the signed values to standardise them meaningfully; applied to an already rectified signal, with its mass piled at zero, the mean and variance would describe mostly how many units were silent. And ReLU must come before the next convolution, since the whole point of an activation is to break the chain of linear operations.

A further consequence of this order is that the convolution's bias is redundant: the normalisation immediately subtracts a mean, which removes any constant the bias added. It is kept because the checkpoint stores it and the trained values must be applied as they were learned, but it is worth knowing that it has no effect on the result.

How a layer is located in the checkpoint Within a block the layers are numbered in threes, so group i occupies positions 3i for the convolution and 3i+1 for the normalisation, the activation having no parameters and so no position of its own. The names are built by joining the prefix and those numbers, which is how a checkpoint saved by the training framework names its layers. The pattern is effectively a contract, and the next entry describes what happens when it is relied on across different files.

The three stages, and why they get deeper

- **stage1, two groups,** on the full-resolution crop, where a filter sees a 3 by 3 patch of actual pixels and so can only detect the smallest features: an edge, a stroke, the direction of a line.
- **stage2, four groups,** after one pooling, where features combine into corners, junctions and curves.
- **stage3, six groups,** after a second pooling, where a filter's receptive field covers a large part of a letter and the features become letter-like.

The number of groups increases with depth because the number of distinguishable patterns does. There are few ways to arrange ink in a 3 by 3 patch and a great many ways to arrange strokes into a letter, so the later stages need more capacity. Placing the deepest stage after two poolings is also what makes this affordable: it runs on a sixteenth of the original area, so its six groups cost less than stage1's two.

5.25 LstmForward

```
def LstmForward(x, weightIh, weightHh, biasIh, biasHh, hiddenSize)
```

Inputs

`x` the input sequence, shaped (time, batch, features)

`weightIh`
: the weights applied to the input, holding all four gates stacked

`weightHh`
: the weights applied to the previous output, also all four gates stacked

`biasIh`
: the bias on the input side

`biasHh`
: the bias on the recurrent side

`hiddenSize`
: the width of the memory and output vectors

Output. a pair: the output at every time step, shaped (time, batch, hidden), and the final hidden and cell state.

The problem: a column is not enough After the convolutional backbone the line has become a sequence of feature vectors, one per column. Classifying each column on its own would be hopeless. A single narrow column of a handwritten line is often ambiguous even to a person: the left half of an a and the left half of a d are the same stroke, and which it is depends on what follows. A smudge is identifiable only from the letters around it. What is needed is a layer whose output at each column depends on the whole line, and that is what a **recurrent** layer provides.

Background: what recurrence means A **recurrent neural network** processes a sequence one step at a time, carrying a vector of numbers from each step to the next. That carried vector is the **hidden state**, and it is the network's summary of everything seen so far. At each step the layer combines the new input with the incoming hidden state to produce a new hidden state, which is also its output for that step. The same weights are used at every step, so the layer has a fixed number of parameters however long the sequence is.

Why a plain recurrent layer is not used In a plain recurrent layer the hidden state is overwritten at every step, as a weighted combination of the old state and the new input passed through a squashing function. Information therefore survives only by being repeatedly multiplied by the recurrent weights. Over a hundred and sixty steps that product either shrinks to nothing or grows without bound, depending on whether the weights are on average below or above the scale that preserves magnitude. These are the **vanishing** and **exploding gradient** problems, and the practical consequence is that a plain recurrent layer cannot learn to connect events more than a few steps apart. For reading a line of writing that is useless, since the evidence resolving an ambiguous letter can be a whole word away.

The LSTM idea: a cell you add to rather than overwrite The **long short-term memory** layer [16] solves this by keeping a second vector, the **cell state**, whose default behaviour is to persist unchanged. It is not recomputed each step; it is modified, by having some of its contents erased and some new contents added. Because the default is to carry forward, information can survive arbitrarily many steps without being multiplied by anything, which is precisely what the plain layer could not do.

What to erase, what to add and what to reveal are decided by three **gates**, each a vector of numbers between 0 and 1 produced by a sigmoid. A gate value of 0 blocks completely, 1 lets everything through, and anything between is a proportion. A fourth quantity, the candidate, supplies the contents that might be added.

$$
\begin{aligned}
i_t &= \sigma\big(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}\big) \\
f_t &= \sigma\big(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}\big) \\
g_t &= \tanh\big(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}\big) \\
o_t &= \sigma\big(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}\big)
\end{aligned}
\tag{5.16}
$$

Source: [16]

where

i_t the **input gate** at step t: how much of the candidate to write into the cell

f_t the **forget gate**: how much of the existing cell to keep

g_t the **candidate**, the new contents that might be written; it uses tanh rather than a sigmoid because it is a value to be stored, which may usefully be negative, not a proportion

o_t the **output gate**: how much of the cell to reveal as this step's output

x_t the input vector at step t

h_{t-1}
: the hidden state carried in from the previous step

W_{i*}, W_{h*}
: the learned weights applied to the input and to the previous hidden state respectively

b_{i*}, b_{h*}
: the corresponding learned biases

σ the sigmoid, squashing to between 0 and 1; see its own entry

tanh
: the hyperbolic tangent, squashing to between -1 and 1

Every one of the four lines has the same form: a weighted sum of the new input, plus a weighted sum of the previous output, plus biases. They differ only in their weights and in the squashing applied. That uniformity is what lets the implementation compute all four in a single matrix multiplication, described below.

The two state updates

$$
c_t \;=\; f_t \odot c_{t-1} \;+\; i_t \odot g_t, \qquad h_t \;=\; o_t \odot \tanh(c_t)
\tag{5.17}
$$

Source: [16]

where

c_t the cell state after step t: the long-term memory

c_{t-1} the cell state carried in from the previous step

h_t the hidden state after step t, which is also this step's output

$\odot$ elementwise multiplication: each position of one vector multiplied by the matching position of the other, as opposed to a matrix product

f_t, i_t, g_t, o_t
: the gates and candidate from above

The cell update is the heart of the method and is worth reading slowly. It is an addition, not a replacement. If the forget gate is 1 and the input gate is 0 at some position, that position of the cell passes through completely untouched, for as many steps as those gate values persist. No weight matrix is applied to it along the way, so it neither shrinks nor grows. That is the path along which information travels a long distance, and it is what the name long short-term memory refers to: a short-term memory that can be made to last.

The hidden state is then a filtered view of the cell. The tanh keeps it in a bounded range, and the output gate decides how much of each position to expose. So the layer can hold something in the cell for many steps while revealing nothing about it, and then reveal it when it becomes relevant. For reading handwriting that is directly useful: the shape of an ambiguous stroke can be held until the letter that disambiguates it arrives.

How all four gates are computed at once The four weight matrices are stored stacked into one array, four times the hidden size tall. One matrix multiplication therefore produces all four pre-activation vectors end to end, and the implementation simply slices the result into four equal blocks in the order input, forget, candidate, output. That ordering is not a choice made here; it is the convention of the framework the weights were trained with, and slicing in a different order would silently use the forget gate's weights as the input gate's. One large matrix multiplication is also considerably faster than four small ones, for the BLAS reasons given under Conv2d.

Why there are two biases Mathematically a single bias would do, since adding two constants is adding one constant. Both are kept because the framework that trained the weights stores both, and the trained values must be applied as they were learned. Their sum is what has any effect.

Why this loop cannot be vectorised Every other layer in this file processes its whole input at once. This one cannot, and the reason is structural rather than a shortcoming of the implementation: step t needs the hidden and cell state from step t−1, so the steps are genuinely sequential. Within a step everything is already whole-array arithmetic across the batch and the hidden positions. This is the same situation as the column loop in LeastCostSeam, and the same conclusion holds: the loop is over the one axis that is irreducibly ordered.

> *Both states start as vectors of zeros. Zero is the natural choice for "nothing seen yet", and it has the useful property that an all-zero cell with a forget gate near 1 stays empty until something is deliberately written into it.*

5.26 BidirectionalLstmForward

```
def BidirectionalLstmForward(x, paramsFwd, paramsBwd, hiddenSize)
```

Inputs

`x` the input sequence, shaped (time, batch, features)

`paramsFwd`
: the weights and biases for the forward pass

`paramsBwd`
: the weights and biases for the backward pass

`hiddenSize`
: the width of each direction's output

Output. the two directions' outputs joined along the feature axis, so twice as wide as one direction.

Why one direction is not enough An LSTM reading left to right has, at each column, seen everything to the left and nothing to the right. For handwriting that is half the available evidence and often the less useful half. Consider the stroke that begins both an a and a d: what distinguishes them is the ascender that comes afterwards. A left-to-right reader at the start of that letter cannot know. It will often get the answer right in the end, because the output at the later column can be correct, but its representation of the earlier columns is impoverished, and the earlier columns are where CTC may want to place the character.

The construction The remedy is to run a second, completely separate LSTM over the sequence reversed, then put its outputs back into forward order and join the two side by side. Each position then carries a summary of everything to its left, from the forward pass, and a summary of everything to its right, from the backward pass. The two have independent weights throughout; nothing is shared.

$$y_t \;=\; \left[\, \overrightarrow{h}_t \,;\, \overleftarrow{h}_t \,\right] \tag{5.18}$$

where

y_t the combined output at position t, of width twice the hidden size

$\overrightarrow{h}_t$ the forward LSTM's hidden state at position t, summarising positions 1 to t

$\overleftarrow{h}_t$ the backward LSTM's hidden state at the same position, summarising positions t to the end

$[\,\cdot\,;\,\cdot\,]$
: concatenation: the two vectors laid end to end, not added or combined

Concatenation rather than addition is the right choice because the two summaries answer different questions and the layer above should be free to weigh them differently. Adding them would force every feature to mean the same thing in both directions and would discard which direction it came from.

Why this is legitimate here and not always Reading the future is only possible because the whole line is available before any output is needed. A system transcribing speech as it is spoken could not do this, since the future has not happened yet. Handwriting recognition on a captured image has no such constraint: the entire line was photographed at once. Bidirectional recurrence is standard in handwriting recognition for exactly this reason, and the architecture this network follows is built on it [17].

> *Reversing a numpy array along its first axis is a view, not a copy, so running the backward pass costs nothing extra in memory. The output of the backward pass is reversed again so that position t of both halves refers to the same column; forgetting that second reversal would align each position with its mirror image and produce fluent-looking nonsense.*

5.27 MultilayerBidirectionalLstm

```
def MultilayerBidirectionalLstm(x, layerParams, hiddenSize)
```

Inputs

`x` the input sequence

`layerParams`
: a list of parameter sets, one per layer, each holding a forward and a backward set

`hiddenSize`
: the width of each direction in each layer

Output. the sequence after passing through every layer in turn.

What stacking buys Each layer's output sequence becomes the next layer's input sequence. The reader uses two such layers. The gain is the same as from depth anywhere in a network: the first layer works on the convolutional features directly and can form short-range relationships between neighbouring columns, such as grouping the strokes that make up one letter; the second works on the first's output, where each position already summarises a neighbourhood, and can form relationships between those summaries, such as relating a letter to the word around it.

Why the second layer's input is twice as wide Because the first layer concatenates two directions, its output has twice the hidden size, and the second layer's input weights are shaped to accept that. This is handled implicitly here: the weights come from the checkpoint and already have the right shape, so the function needs no knowledge of the widths and the same loop would work for any number of layers.

> *Two layers is the depth the architecture specifies [17]. More would add parameters and sequential work for a task where the first layer already covers a letter and the second a word, which is the range the evidence actually spans.*

5.28 ResizeLineImageFixed

```
def ResizeLineImageFixed(img, height=INPUT_HEIGHT, width=INPUT_WIDTH,
    padValue=255)
```

Inputs

`img` the line crop as a grey image

`height`
: the target height, default 64

`width`
: the target width, default 640

`padValue`
: the value to fill unused canvas with, default 255 for white

Output. the crop fitted onto a canvas of exactly the trained size.

What it is for The networks accept exactly one input size, for the reasons given under INPUT_HEIGHT. A crop cut from a page is whatever size that line of writing happened to be. This function reconciles the two, by delegating to FitFixedCanvas in segment.py.

Why the aspect ratio is preserved The important property, explained in full in the entry for FitFixedCanvas, is that a single scale factor is applied to both axes, chosen so the crop fits inside the canvas, and the remainder is padded with white rather than stretched. Stretching each axis independently to fill the canvas would change the proportions of the letters, and proportion is precisely what distinguishes some letters from one another and one writer's hand from another's. A tall narrow l stretched to fill a wide canvas becomes something no letter looks like, and the filters trained on correctly proportioned letters would not find their features in it.

> *White, not black, is the pad value, because at this point the image is still in its natural form of dark ink on light paper. The inversion happens in the next function, and padding after inversion would require black instead. Getting this the wrong way round surrounds every crop with a solid block of apparent ink.*

5.29 TensorFromResized

```
def TensorFromResized(img)
```

Inputs

`img` the fitted grey crop

Output. a floating-point array shaped (1, 1, height, width), scaled and inverted into the range the networks were trained on.

What it is for An image is 8-bit integers from 0 to 255 with white high. A network expects floating-point numbers of a particular scale and sign convention. This function performs that conversion, in three steps that must match exactly what was done during training.

$$x \;=\; \frac{\left(1 - \frac{g}{255}\right) - 0.5}{0.5} \;=\; 1 - \frac{2g}{255} \tag{5.19}$$

where

x the value handed to the network, running from -1 for paper to $+1$ for black ink

g the original grey value, 0 for black and 255 for white

$\frac{g}{255}$ the grey value scaled to the range 0 to 1

$1 - \cdot$ the inversion, making ink high and paper low

$\frac{\cdot - 0.5}{0.5}$ the standardisation, mapping the range 0 to 1 onto -1 to $+1$

Why invert After inversion, ink is a large number and blank paper is zero before standardising. This matters because of padding: every convolution pads with zeros, and zero must therefore mean nothing rather than something. With the original convention zero means black, so the padding round the edge of every crop would read as a border of solid ink and the first filters would spend their response on an artefact of the padding.

Why standardise to a range centred on zero Networks train better when their inputs are centred near zero and of a magnitude near one, for the same reasons batch normalisation exists. Inputs in the hundreds would make the first layer's outputs correspondingly large and would demand very small initial weights to compensate. Mapping to -1 to $+1$ puts the input on the scale the initialisation assumed.

Why the two extra axes The array is given the shape (1, 1, height, width) to match the four-axis convention described in the chapter introduction: one image in the batch, one channel in that image. The axes are necessary even though both lengths are 1, because every layer indexes by position and would otherwise misinterpret which axis is which.

> *These three steps are not a design decision that can be revisited independently; they are a reproduction of what the training code did. A network is only correct on inputs prepared the way its training inputs were, and a mismatch here, such as a forgotten inversion, produces a network that loads without error and returns confident nonsense.*

5.30 PenWidthXh

```
PenWidthXh = 0.14
```

Output. 0.14, the pen width that stroke normalisation redraws at, expressed as a fraction of the x-height.

Why the width is given in x-heights and not in pixels A width in pixels would mean something different on every crop, since crops differ in resolution and writers differ in size. Expressed as a fraction of the x-height it means the same thing everywhere: a stroke whose thickness is about a seventh of the height of a letter body. The actual pixel width is computed per crop from its measured x-height, which is what makes the normalisation resolution-free.

Why this value It has to be thick enough that the redrawn writing looks like writing rather than a wire diagram, since the network judging it was trained on strokes of a realistic thickness, and thin enough that the counters, the enclosed white spaces inside letters like a, e and o, survive. At about a seventh of the x-height a counter remains open, while a noticeably thicker pen would fill the loop of an e and make it indistinguishable from a c with a closed top. That failure is not hypothetical for this project; distinguishing those two shapes has been a real source of error in the glyph library.

5.31 StrokeNormalize

```
def StrokeNormalize(gray, penWidthXh=PenWidthXh)
```

Inputs

`gray`
: a line crop as a grey image

`penWidthXh`
: the pen width to redraw at, in x-heights, defaulting to the constant above

Output. the writing redrawn at one constant pen width as ink on white, or nothing if the crop has no usable ink.

The problem it solves A network asked to identify a writer from a raw crop will use whatever is most reliably distinctive, and that is not necessarily the handwriting. Pen width and ink darkness are extremely easy to measure and happen to be consistent within each writer's pages, because each writer used one pen. A network can reach high accuracy on such a dataset while having learned nothing about letterforms at all. This is a **confound**: a variable correlated with the answer that is not the thing being studied.

The confound becomes a real failure precisely where the project needs the namer most. The robot draws with one pen, so every writer's reproduced output has identical stroke width. A namer relying on width therefore has nothing to go on and is being asked a question it was never really trained for. Removing pen width from the input forces the network to use shape, which is the property that does survive reproduction.

The method: thin, then re-ink The writing is reduced to its centre line and then redrawn at a chosen width. Whatever the original thickness, the output thickness is the same, so the information is genuinely destroyed rather than merely de-emphasised.

- **Binarise.** The crop is thresholded into ink and paper by BinarizeLine.
- **Measure the x-height.** CoreBand finds the band of rows holding the letter bodies, and its height is the x-height, which sets the scale for the pen width.
- **Thin.** Skeletonize reduces the ink to a one-pixel-wide centre line by the Zhang-Suen method [6], which removes boundary pixels repeatedly while preserving connectivity, so the skeleton has the same topology as the original writing.
- **Re-ink.** The skeleton is dilated by a square of the chosen width, which draws a stroke of that width centred on the centre line, and the result is inverted back to dark ink on white paper.

Why the pen width is forced to be odd

$$k \;=\; 2\left\lfloor \frac{w}{2} \right\rfloor + 1, \qquad w \;=\; \mathrm{clip}\big(\mathrm{round}(0.14 \cdot \mathrm{xh}),\; 2,\; 9\big) \tag{5.20}$$

where

k the structuring element size actually used, always odd

w the pen width in pixels before being made odd

xh the measured x-height in pixels

0.14 the pen width as a fraction of the x-height

clip$(\cdot, 2, 9)$
: the width held between 2 and 9 pixels, so a tiny crop still gets a visible stroke and a large one does not get a blob

$\lfloor \cdot \rfloor$ rounding down

A dilation by an even-sized square has no centre cell, so the redrawn stroke is offset by half a pixel from the skeleton it was drawn around. Over a whole line that shifts the writing slightly and, worse, shifts it in a direction that depends on the rounding, so two crops of the same writing could be offset differently. Forcing the size odd puts the pen exactly on the centre line.

The x-height fallback If no usable band is found, or the band is under four pixels tall, the x-height is taken as a third of the crop height instead, with a floor of eight pixels. A crop is built around one line of writing whose body occupies roughly the middle third, so that is a reasonable estimate when measurement fails, and it keeps the function returning something usable rather than propagating a nonsensical width.

> *Two cases return nothing at all: a crop with no ink, and one whose skeleton is empty. Returning nothing rather than a blank image is deliberate, because the caller must be able to tell "normalisation failed" from "this line is blank" and fall back to the raw crop. ClassifyImage does exactly that.*

5.32 DecodeCtc

```
def DecodeCtc(logProbs)
```

Inputs

`logProbs`
: the reader's output, shaped (time, batch, classes), holding a log probability per character per column

Output. a list of strings, one per item in the batch.

The problem CTC exists to solve The reader produces one set of character scores per column, around 160 of them, while the line contains perhaps 40 characters. Nothing says which columns correspond to which character, and the correspondence differs for every line and every writer: a wide m covers many columns, a narrow i covers few. Training a network to produce a short sequence from a long one would ordinarily require knowing that correspondence, which would mean marking by hand where every letter begins and ends on every training line. For tens of thousands of lines that is not feasible.

Connectionist temporal classification [18] removes the requirement. It lets the network be trained on the text alone, with no indication of where the characters sit, by defining a rule that maps any per-column output onto a shorter string, and then training the network so that the total probability of all column-outputs mapping to the correct string is high. The alignment is never specified; it is whatever the network finds convenient.

The collapsing rule The rule has two steps, applied in this order: merge runs of the same character into one, then delete the blanks.

$$\mathcal{B}(\pi) \ : \text{ merge repeats, then remove blanks} \tag{5.21}$$

Source: [18]

where

$\mathcal{B}$ the collapsing map, which takes a sequence of one label per column and returns a string

π a **path**: one label chosen for each column, including possibly the blank

`blank`
: the extra label, slot 0, which stands for "emitting nothing here"

Why the blank is necessary The blank is the part that makes the scheme work, and its purpose is easiest to see from what goes wrong without it. If the rule were only "merge repeats", the word `letter` could never be produced, because its two consecutive t's would merge into one. The blank provides a separator: a path reading t, t, blank, t, t collapses first to t, blank, t and then, with the blank removed, to tt. So a blank between two runs of the same character means a genuine doubled letter, while its absence means one letter held across several columns.

- **a, a, a, b, b** collapses to **ab**, one a and one b.
- **a, blank, a** collapses to **aa**, a doubled letter.
- **a, a, blank, b** collapses to **ab**, the blank being harmless between different letters.

The blank also gives the network somewhere to put the columns between letters, the gaps and joins where no character is being written. In practice a trained CTC network emits blank for the large majority of columns and spikes briefly on a character where it is confident, which is why its output looks sparse.

Greedy decoding, and what it approximates What is wanted is the single most probable *string*, which requires summing over every path that collapses to it, since many paths give the same string. That is expensive. This function instead takes the most probable *path*, by picking the best-scoring character independently in each column, and collapses that. This is **greedy** or **best-path** decoding.

$$\pi_t^* \ = \ \arg\max_k \ \log p_t(k), \qquad \hat{y} \ = \ \mathcal{B}\big(\pi^*\big) \tag{5.22}$$

Source: [18]

where

π_t^* the chosen label for column t

$\log p_t(k)$
: the reader's log probability for character k at column t

$\arg\max_k$
: the character index with the highest score

$\hat{y}$ the decoded string

$\mathcal{B}$ the collapsing map defined above

The approximation is not exact: the most probable path need not collapse to the most probable string, because a string whose probability is spread over many moderately good paths can beat one concentrated in a single excellent path. In practice the difference is small for a well-trained network, whose output is confident enough that one path dominates, and the measured accuracy of 96 to 99 percent on real crops is obtained with this decoder. A beam search would recover the difference at several times the cost.

> *Because the comparison within a column is only between scores in that same column, the logarithm makes no difference to which character wins. The log probabilities could be replaced by raw logits here and the output would be identical. They are log probabilities because the same array is used by the alignment, which does need them.*

5.33 CtcForcedAlign

```
def CtcForcedAlign(logProbs, text)
```

Inputs

`logProbs`
: the reader's output for one line, shaped (time, classes)

`text`
: the known text of that line

Output. a list pairing each character with the span of columns it occupies, together with a confidence; or nothing if the text cannot be aligned.

What it is for, and why the project needs it Decoding answers "what does this line say". This answers a different question: given that the line says this, *where* is each character. That is what the glyph extractor needs. To build a library of a writer's letterforms, the pipeline must cut individual letters out of their line crops, and to do that it must know which columns of the crop hold which letter. No separate letter-finding step is needed, because the reader already contains that information implicitly in its per-column scores; forced alignment extracts it.

Why it is called forced The text is not being recognised; it is given. The algorithm is **forced** to produce an alignment consistent with that text, and among all such alignments it finds the most probable. So it cannot return the wrong letters, only a wrong idea of where they are. This is a much easier problem than recognition and correspondingly more reliable, which is why the project uses known transcripts for glyph extraction rather than the reader's own output.

The extended label sequence The first step is to write out the target as a sequence that allows blanks everywhere they are permitted: a blank, then each character followed by a blank.

$$\text{ext} \;=\; \big(\varnothing,\, \ell_1,\, \varnothing,\, \ell_2,\, \varnothing,\, \ldots,\, \ell_n,\, \varnothing\big), \qquad S \;=\; 2n+1 \tag{5.23}$$

Source: [18]

where

ext the extended sequence of states the alignment moves through

$\varnothing$ the blank label, slot 0

ℓ_i the slot number of the i-th character of the text

n the number of characters

S the length of the extended sequence, hence the number of states

Every valid path through the columns can now be described as a non-decreasing walk through these states: at each column the path is in some state, and as columns advance the state index only ever stays put or increases. Characters sit at odd positions and the optional blanks at even ones. A path that skips a blank writes two characters back to back; a path that lingers in a blank puts a gap between them.

The allowed moves From state s at the previous column, three moves are considered: stay at s, advance to s from s−1, or advance to s from s−2. The third, the skip, is what allows the optional blanks to be passed over, and it carries one restriction.

- **Stay.** The same label continues across another column: a wide letter, or a long gap.
- **Advance by one.** Move to the next state, either from a character into its following blank or from a blank into the next character.
- **Skip by two.** Jump over a blank, placing two characters in consecutive columns. Permitted only when the target state is a character and that character differs from the one two states back.

The restriction on the skip is the whole reason the blanks were inserted. If the two characters either side of a blank are the same letter, skipping the blank would make a doubled letter collapse back into one, by the merge rule. So for a doubled letter the blank between them is compulsory, and the skip is forbidden. A skip onto a blank is likewise meaningless, since it would be jumping over a character. Both cases are enforced by setting the candidate score to a very large negative number, which the maximisation then never selects, which is the same device as the infinities in LeastCostSeam.

The recurrence This is dynamic programming again, with the same structure as the seam, but maximising a log probability instead of minimising a cost. Working in log space means the multiplication of probabilities along a path becomes an addition, as established under LogSoftmax.

$$\alpha[t,s] \;=\; \log p_t\big(\mathrm{ext}[s]\big) \;+\; \max\Big(\alpha[t-1,s],\; \alpha[t-1,s-1],\; \alpha[t-1,s-2]\Big) \tag{5.24}$$

Source: [18]

where

$\alpha[t,s]$
: the score of the best path that reaches state s at column t

$\log p_t(\mathrm{ext}[s])$
: the reader's log probability, at column t, of the label belonging to state s

$\alpha[t-1,\,\cdot\,]$
: the best scores at the previous column, for the three states a path may arrive from

max the best of those three arrivals; the third is excluded when the skip is not allowed

The initial column may only be in state 0, the leading blank, or state 1, the first character, since a path cannot have progressed further than one state in one column. At the final column the path must have reached either the last state, the trailing blank, or the state before it, the last character; whichever of those two scores higher is the end of the best path.

Why maximum and not sum Training CTC sums over all paths, because the quantity being maximised is the total probability of the target string. Alignment takes the maximum instead, because the quantity wanted is one specific path: the single most probable way the text could be laid out across the columns. Taking the maximum of a sequence of choices like this is the **Viterbi** algorithm [19], long used for exactly this purpose of recovering the most likely state sequence behind an observed signal.

From the path to column spans Backtracking the recorded moves gives the state occupied at every column. The i-th character of the text lives in state 1+2i, so the columns whose state equals that number are the columns that character occupies, and the first and last of them are its span. A character whose state is never occupied gets no span, which happens when the aligner had to pass through it in a single column shared with its neighbours; the caller handles that by skipping the glyph rather than cutting a zero-width one.

The confidence

$$\text{conf} \;=\; \frac{1}{T}\sum_{t=1}^{T} \log p_t\big(\text{ext}[\text{path}[t]]\big) \tag{5.25}$$

where

conf the returned confidence: the mean log probability per column along the chosen path

T the number of columns

path[t]
: the state the best path occupies at column t

$\log p_t(\cdot)$
: the reader's log probability for that state's label at that column

Dividing by the number of columns is what makes the figure comparable between lines. A total would favour short lines simply because fewer negative terms are added, so the mean is the per-column average, with zero meaning perfect certainty and more negative meaning less. The glyph extractor uses it to reject lines whose alignment is not trustworthy enough to cut letters from, which matters because a confidently wrong alignment would poison the library with mislabelled glyphs.

> *Two guards return nothing rather than a bad answer: a text with no recognisable characters, and a line with fewer columns than characters. The second is a genuine impossibility rather than a difficulty, since every character needs at least one column of its own, so no alignment exists at all.*

5.34 FrameXToPixel

```
def FrameXToPixel(canvasX, rawW, origH)
```

Inputs

`canvasX`
: a horizontal position on the network's input canvas

`rawW`
: the width of the original crop in pixels

`origH`
: the height of the original crop in pixels

Output. the corresponding position in the original crop, clamped to lie within it.

What it is for An alignment span is measured in network columns, but a glyph has to be cut from the original crop in original pixels. This function reverses the geometric transformation that got the crop onto the canvas, so a column index becomes a pixel position.

Reversing the fit The fit applied one scale factor to both axes, chosen as the largest that lets the crop fit inside the canvas, which is the smaller of the two ratios. Undoing it is division by that same factor.

$$s \;=\; \min\!\left(\frac{H_{\text{in}}}{H_{\text{orig}}},\, \frac{W_{\text{in}}}{W_{\text{raw}}}\right), \qquad x_{\text{orig}} \;=\; \text{clip}\!\left(\frac{x_{\text{canvas}}}{s},\, 0,\, W_{\text{raw}}\right) \tag{5.26}$$

where

s
: the scale factor that was applied when fitting the crop

H_{in}, W_{in}
: the network's canvas height and width, 64 and 640

H_{orig}, W_{raw}
: the original crop's height and width

x_{canvas}
: the position on the canvas

x_{orig}
: the corresponding position in the original crop

$\text{clip}(\cdot, 0, W_{\text{raw}})$
: the result held inside the crop

Why the smaller ratio, and why no offset Taking the smaller of the two ratios is what guarantees the crop fits on both axes; the larger would overflow one of them. There is no offset term because the fit places the crop at the top left of the canvas and pads to the right and below, so the origins coincide. Had the crop been centred, this conversion would need to subtract the left margin first.

Why the result is clamped A column index can map slightly beyond the crop, because the network's columns are a coarser grid than the original pixels and the last one may extend past the real content into the padding. Clamping keeps the returned position usable as an index without the caller checking. The value is returned as a floating-point number rather than an integer because the caller may want to interpolate between pixels, and rounding here would discard precision that was correctly computed.

5.35 PaperCRNNNumpy

```
class PaperCRNNNumpy
```

Inputs

`checkpointPath`
: which checkpoint to load, defaulting to the one chosen above

`hiddenSize`
: the LSTM width, default 256

Output. an object holding the loaded weights, whose forward method maps a crop to log probabilities per character per column.

What a CRNN is A **convolutional recurrent neural network** [17] is the standard architecture for reading a line of text, and it is a direct expression of the structure of the problem. Reading a line requires two different kinds of reasoning: recognising a shape, which is local and spatial, and assembling shapes into words, which is sequential and long-range. Convolutions do the first, recurrence does the second, and CTC joins them to text without requiring the letters to be located first. Each of the three parts is doing the job it is suited to.

The layer sequence, and what happens to the data

- **stage1, two convolution groups,** on a (1, 64, 640) input, finding edges and stroke directions.
- **Pool by two,** halving to 32 by 320.
- **stage2, four groups,** finding corners, junctions and curves.
- **Pool by two,** halving again to 16 by 160.
- **stage3, six groups,** finding letter-scale structure, ending with 128 channels.
- **Collapse the height,** taking the maximum down each column, giving 128 features per column.
- **Transpose to (160, 1, 128),** making width the sequence axis.
- **Two bidirectional LSTM layers,** each position gaining a summary of the whole line to its left and right.
- **A linear layer,** turning each position's features into 75 character scores.
- **Log softmax,** making those scores log probabilities.

The transpose is the key structural step Moving the width axis to the front is what converts an image into a sequence. Before it, the array is an image with a width; after it, the array is a sequence of 160 feature vectors, and the word width no longer appears. Everything downstream, the recurrence and CTC alike, is defined on sequences and would be meaningless on an image. The 160 columns become CTC's time axis, and the fact that they derive from horizontal position is what makes CTC's monotonic left-to-right assumption correct for this problem: writing proceeds left to right, so the alignment between columns and characters never needs to go backwards.

Why the weights are unpacked in the constructor The LSTM parameters are gathered into a structured list once, when the object is built, rather than looked up by name on every call. Reading a page means running the forward pass once per line, so repeating a few dozen dictionary lookups per line would be harmless in cost, but gathering them once means the key names appear in exactly one place. A checkpoint missing a parameter then fails at construction, naming the key, rather than part way through reading a page.

> *The reverse-direction parameters are stored under keys ending in `_reverse`, which is the naming convention of the framework the weights were trained with. The forward direction's keys are built with a helper while the reverse ones are written out, which is a small asymmetry of style in otherwise identical work.*

5.36 ReadText

```
def ReadText(image, model)
```

Inputs

`image`
: a line crop as a grey image

`model`
: a loaded reader

Output. the text read from that crop, as a string.

What it is for The three steps of reading in one call: fit the crop to the canvas, convert it to the network's input convention, run the forward pass, and decode. It exists so that no caller has to remember the order, and so that there is a single place where the preparation and the decoding are paired with each other.

Why that matters The preparation steps are not independently meaningful. Fitting without inverting, or inverting without standardising, produces an array the network will accept and misread. Keeping them together in one function makes the complete sequence the only thing a caller can invoke, which removes a class of error that would otherwise be silent.

> *Only the first element of the decoded batch is returned, since the batch axis is always 1 here. The batch machinery is retained throughout the layers because the training code uses it, and because removing it would mean two versions of every layer.*

5.37 Levenshtein

```
def Levenshtein(a, b)
```

Inputs

`a` one string

`b` the other string

Output. the smallest number of single-character insertions, deletions and substitutions that turn one string into the other.

What it is for Scoring a reader requires a way to say how wrong an answer is. Counting matching positions does not work, because a single missing letter shifts everything after it and would be scored as if every later letter were also wrong. What is needed is a measure that understands insertion and deletion, and that is the **edit distance**, or **Levenshtein distance** [20]: the least number of single-character edits needed to turn one string into the other.

- **Insertion:** add a character. `cat` to `cart` is one edit.
- **Deletion:** remove a character. `cart` to `cat` is one edit.
- **Substitution:** replace a character. `cat` to `cot` is one edit.

The recurrence This is dynamic programming once more. Let the problem be solved for every pair of prefixes: how many edits turn the first i characters of one string into the first j of the other. The answer for a larger pair follows from three smaller ones, because the last edit must be one of the three kinds.

$$D[i,j] \;=\; \min \begin{cases} D[i-1,\,j]+1 & \text{delete } a_i \\ D[i,\,j-1]+1 & \text{insert } b_j \\ D[i-1,\,j-1]+\big[\,a_i \neq b_j\,\big] & \text{match or substitute} \end{cases} \tag{5.27}$$

Source: [20]

where

$D[i,j]$
: the edit distance between the first i characters of a and the first j characters of b

a_i, b_j
: the i-th and j-th characters of the two strings

$[\,a_i \neq b_j\,]$
: 1 when the two characters differ and 0 when they match, so a match is free and a substitution costs one

min
: the cheapest of the three ways to arrive

The boundary cases are immediate: turning an empty string into a prefix of length j takes j insertions, and the reverse takes i deletions. From there the table fills row by row, and the bottom right corner is the answer for the whole strings.

Two implementation economies Only the previous row is kept, not the whole table. Each row depends only on the one before it, so storing all of them would use memory proportional to the product of the lengths for no purpose; this version uses memory proportional to the shorter string alone. That is possible because the distance itself is wanted and not the sequence of edits, which would require the full table to backtrack through, exactly as the seam does.

The function also swaps its arguments so the shorter string controls the row length. The distance is symmetric, since every edit has an inverse of the same cost, so swapping cannot change the answer, and it makes the retained row as short as possible.

5.38 CharAcc

```
def CharAcc(pred, truth)
```

Inputs

`pred`
: the predicted text

`truth`
: the correct text

Output. the fraction of characters correct, between 0 and 1.

How it is defined The edit distance counts errors; accuracy expresses them as a proportion of the length of the correct text.

$$\text{CharAcc} \;=\; \max\!\left(0,\; 1 - \frac{D(\text{pred},\, \text{truth})}{|\text{truth}|}\right) \tag{5.28}$$

where

CharAcc
: the character accuracy

D
: the edit distance between the two strings

|truth|
: the number of characters in the correct text

max(0, ·)
: the floor at zero, explained below

Why it is divided by the truth length The denominator is the length of the correct text, not of the prediction and not the longer of the two. That choice makes the figure answer "what fraction of what should have been written came out right", which is the question of interest, and it makes the measure comparable across lines of different lengths. Dividing by the prediction length instead would let a system improve its score by predicting less.

Why the floor at zero The edit distance can exceed the length of the correct text, when the prediction is much longer than the truth: turning a two-character truth into a fifty-character prediction takes around fifty edits, giving a raw score around -24. Negative accuracy is meaningless and, more practically, averaging it across a page would let one runaway line dominate the mean and produce a negative overall accuracy. The floor caps the damage at zero, which is the honest statement that nothing on that line was right.

> *The empty-truth case is handled separately: an empty prediction scores 1, anything else scores 0. Without it the division would be by zero. This is the right convention, since correctly reading a blank line as blank is a success and reading text off a blank line is a complete failure.*

5.39 WordAcc

```
def WordAcc(pred, truth)
```

Inputs

`pred`
: the predicted text

`truth`
: the correct text

Output. the fraction of the correct text's words that appear, in order, in the prediction.

Why a second measure Character accuracy can be high while the text is still hard to read, because one wrong character per word leaves every word wrong. Word accuracy is the stricter measure and is closer to whether a person could use the output: a word counts only if every character in it is right. The gap between the two figures is informative, since a large gap means errors are spread thinly across many words rather than concentrated.

Longest common subsequence The implementation counts how many of the correct words appear in the prediction in the same order, which is the **longest common subsequence** of the two word lists. A **subsequence** takes items in order but allows gaps, so it tolerates both missing and spurious words without penalising the words around them.

$$L[i,j] \;=\; \begin{cases} L[i-1,\,j-1]+1 & t_i = p_j \\ \max\big(L[i-1,\,j],\; L[i,\,j-1]\big) & t_i \neq p_j \end{cases} \tag{5.29}$$

where

$L[i,j]$
: the length of the longest common subsequence of the first i true words and the first j predicted words

t_i the i-th word of the correct text

p_j the j-th predicted word

max the better of skipping a word on either side when the two do not match

$$\text{WordAcc} \;=\; \frac{L[m,n]}{m} \tag{5.30}$$

where

WordAcc
: the word accuracy

$L[m,n]$
: the length of the longest common subsequence of the full word lists

m
: the number of words in the correct text

n
: the number of predicted words

Why case is ignored Words are compared with case folded, so `The` matches `the`. That is a deliberate relaxation: a capitalisation error is a real error and is counted in full by the character measure, but counting it again as a whole wrong word would double-count one mistake in the measure meant to indicate readability. A reader is not impeded by a wrong capital.

> *Words are split on whitespace, so punctuation stays attached to the word it touches and a word followed by a comma must have that comma right to count. That is strict, and it is part of why the measured word accuracies sit well below the character accuracies.*

5.40 AuthorClassifierCNNNumpy

```
class AuthorClassifierCNNNumpy
```

Inputs

`checkpointPath`
: which checkpoint to load, defaulting to the namer chosen above

Output. an object holding the weights and the writer name mapping, whose forward method maps a crop to one raw score per writer.

What it is for The namer answers a **classification** question: given this crop, which of the enrolled writers produced it. Unlike the reader it produces one answer for the whole crop, not a sequence, and so it needs no recurrence and no CTC.

What it shares with the reader, and why The convolutional backbone is identical: the same three stages with the same number of groups, the same two poolings, the same collapse of the height. That is not laziness. The features useful for recognising a letter and the features useful for recognising a hand are largely the same features: stroke direction, curvature, how strokes join, the proportions of a letter. The two tasks diverge only in what is done with those features, and that is exactly where the two networks differ.

Where it differs

- **No recurrence.** Writer identity is not a sequential property. There is no analogue of using a later letter to disambiguate an earlier one, because the answer is a single global label.
- **Average pooling over position.** The remaining sequence axis is averaged away, turning the line into one feature vector. Averaging rather than maximising, for the reason given in that entry: the question is how prevalent a trait is, not whether it occurred.
- **A two-layer head.** A linear layer, a ReLU, then a second linear layer producing one score per writer. The non-linearity between them lets the decision depend on combinations of pooled features rather than on each independently, which a single layer could not express.
- **Raw scores out.** The forward pass returns logits, leaving the softmax to the caller, so that the same output can be used for a decision, which needs only the ranking, or for a confidence, which needs the normalisation.

The dropout layer that is not there The head's parameters are numbered 0 and 3, with 1 and 2 missing. Position 1 is the ReLU, which has no parameters, and position 2 is **dropout**, a training-time technique that randomly sets a fraction of the values to zero at each step [21]. Its purpose is to prevent the network relying on any one feature, since that feature may be absent, which forces redundancy and reduces overfitting. At inference time it is disabled entirely, so the forward pass steps from 0 to 3 and the gap in the numbering is the trace of a layer that exists only during training.

> *The writer name mapping is read from the checkpoint alongside the weights and kept on the object. This is what lets an output slot be reported as a person's name, and keeping the two together is what prevents a checkpoint trained on a different writer list from silently misattributing every page.*

5.41 ClassifyImage

```
def ClassifyImage(image, model, authorMapping=None, strokeNormalise=True)
```

Inputs

`image`
: a line crop as a grey image

`model`
: a loaded namer

`authorMapping`
: an optional name-to-slot mapping, unused by this function

`strokeNormalise`
: whether to redraw at a constant pen width first, default true

Output. a pair: the index of the most likely writer, and the full array of probabilities across all writers.

What it is for The namer's equivalent of ReadText: the whole identification in one call, from a crop to a writer and a set of confidences.

The normalisation switch The one real decision in this function is whether to stroke-normalise first, and it must match the checkpoint being used. A network trained on normalised crops must be given normalised crops, and one trained on raw crops must be given raw crops. Passing the wrong kind does not fail; it produces confident answers from a network looking at a distribution of inputs it has never seen. The default is to normalise, which suits the shape-based checkpoint used for judging drawn output, and the ink-based checkpoint is used with the switch turned off.

If normalisation returns nothing, because the crop has no usable ink or no skeleton, the raw crop is used instead. That is a compromise: the network is then seeing the wrong kind of input. It is preferred to failing outright because the alternative on a marginal crop is no answer at all, and the caller can weigh a low-confidence answer. The condition is rare, since a crop that produces no skeleton has essentially no writing in it.

From scores to an answer The logits are turned into probabilities by softmax and the largest is taken as the prediction. Both are returned, because they answer different questions: the index says who, and the full array says how sure and who the runners-up were. The second is what the synthesis search uses, since it needs a continuous score to optimise rather than a discrete decision.

> *As noted under Softmax, these probabilities are normalised across the enrolled writers and so cannot express that the crop belongs to none of them. A page from an unenrolled writer still receives a confident-looking answer. The* `authorMapping` *argument is accepted and not used, the mapping being carried on the model itself; it remains in the signature for callers that pass it positionally.*

6 style.py

This file answers one question: given some pages a person wrote, what is their handwriting? The answer it produces is a **style profile**, which holds a library of that writer's own letterforms together with measurements of how they lay letters out. synthesise.py then uses that profile to write new text in the same hand.

The four steps

- **Trace.** The ink is thinned to a one-pixel centre line and then followed, turning a picture of a stroke into an ordered list of points: a path the pen could have taken.
- **Cut.** Each line is divided into individual characters, using the reader from recognise.py to say where each character sits, so no separate letter-finding method is needed.
- **Filter.** Most cuts are imperfect, so a series of gates rejects anything that is not a plausible letter. A gate only ever rejects; nothing is invented or repaired.
- **Measure.** Slant, x-height, how far ascenders reach, the gap left between words, and how joined-up the hand is.

The one convention that makes the rest possible Every stored coordinate is measured in **x-heights**, not pixels. The x-height is the height of a letter body such as an o or an x, with no ascender or descender. A glyph is stored with its origin at the left edge of its cell and the baseline at y = 0, with y increasing upward, and both axes divided by the x-height.

Three consequences follow, and they are why the convention is worth the trouble. The same stored glyph can be drawn at any size, because scaling is a single multiplication. Glyphs from pages photographed at different distances become directly comparable, so they can be averaged and compared. And the measurements of the hand describe the writing rather than the photograph, so a profile built from a close-up page and one built from a distant page agree.

> *Stored y increases upward, which is the opposite of image coordinates, where row 0 is the top. The conversion happens once, inside ExtractLineGlyphs, and everything downstream works in the upward convention because that is how handwriting is naturally described: an ascender has a positive height and a descender a negative one.*

6.1 DATA_DIR

```
DATA_DIR = paths.Data.parents[1] / 'Data' / 'Datasets' / 'IAMpages10'
```

Output. the folder holding the public handwriting database pages.

What it is for Names where the dataset pages live. The path is built from the central paths module rather than written out, so the project has one definition of where data sits. The dataset itself is the IAM database of handwritten English text [1], a standard corpus of scanned pages with transcripts, which is what makes it usable here: the transcripts are what the forced alignment needs in order to place characters.

6.2 CACHE_DIR

```
CACHE_DIR = paths.Cache / 'line_cache_authors10'
```

Output. the folder holding cached per-line extractions.

Why anything is cached Cutting a page into letters is the slow part of building a profile, because every line has to be read by the network, thinned, traced and divided. The filtering and measuring that follow are fast. Caching the result of the slow part means that adjusting a filter threshold, which is the thing most often adjusted, costs seconds instead of re-running the whole extraction.

This is the ordinary reason for a cache, but the division of labour here is worth noticing as a design choice: the expensive stage was deliberately made to produce an unfiltered result, so that the cheap stage can be re-run against it without invalidating it. A cache that stored filtered glyphs would have to be thrown away whenever a filter changed, which is exactly when its value is wanted.

6.3 NOGIT_DIR

```
NOGIT_DIR = paths.Cache
```

Output. the folder for large generated files that are kept out of version control.

Why it exists Extraction caches and glyph libraries run to hundreds of megabytes. Version control stores a full copy of every revision of every file it tracks, so a large file that changes often makes the repository grow without bound, and nothing is gained, because the files can be regenerated from the code and the data. Keeping them under one named directory means a single ignore rule covers all of them, rather than a rule per file pattern that someone must remember to extend.

6.4 TEXT_WEIGHTS

```
TEXT_WEIGHTS = paths.Weights / 'paper_cnn_bilstm_ctc_best.npz'
```

Output. the path of the reader checkpoint used to place character boundaries.

What it is for Glyph extraction needs a reader, because the character positions come from forced alignment of that reader's output. This constant names which checkpoint to use for that purpose.

Why the choice of checkpoint matters here The quality of the alignment sets the quality of every glyph cut from it. A reader that is unsure where a character sits produces boundaries in the wrong place, and a boundary in the wrong place cuts a letter in half or sweeps part of its neighbour into it. Those are precisely the two defects the filters later spend most of their effort rejecting. So a stronger reader does not merely improve recognition; it reduces the amount of work the filters have to do and raises the proportion of cuts that survive them.

The filename this constant points at is not present in the weights directory of this project as it stands. Code that depends on it therefore has to pass an explicit checkpoint path, which the callers do. It is recorded here because a constant naming a file that does not exist will mislead anyone reading the module to believe it is the checkpoint actually in use.

6.5 PERSONAL_WEIGHTS

```
PERSONAL_WEIGHTS = paths.Weights / 'paper_cnn_bilstm_ctc_personal_best.
    npz'
```

Output. the path of the reader checkpoint adapted to the photographed writers.

What makes it different A reader trained on the public database has seen thousands of writers but none of the five photographed for this project, and it has seen scans rather than phone photographs. A checkpoint further trained on this project's own pages is better matched to them on both counts. For alignment, where the text is already known and only its position is in question, that closer match is directly useful.

The trade-off being made Adapting a network to a small set of writers risks making it worse on everyone else, a phenomenon called **catastrophic forgetting**: training on new data overwrites what was learned from the old. That is acceptable here precisely because this checkpoint has one narrow job. It is used to align pages from the writers it was adapted to, and the database writers are aligned with a checkpoint that has seen them.

6.6 RAW_DIR

```
RAW_DIR = paths.Cache / 'GlyphCache10'
```

Output. the folder holding the uncut, unfiltered extraction per writer.

What it is for Where the output of ExtractAuthorRaw is kept, one entry per writer. The word raw is the important part: what is stored is every glyph the cutter produced, including the fragments and the over-wide cuts, before any gate has rejected anything. That is what allows a filter to be changed and the effect measured without re-extracting, and it is also what allowed the rejection rates quoted in this project's results to be computed at all, since a filtered cache could not report what it had discarded.

6.7 MAX_VARIANTS_PER_CHAR

```
MAX_VARIANTS_PER_CHAR = 12
```

Output. 12, the number of different shapes of one letter kept per writer.

Why more than one shape is kept A person does not write a letter identically twice. Storing one shape per letter would make the output visibly mechanical, because every a on the page would be the same a, which no handwriting ever is. Keeping several genuine variants lets synthesis choose among them and gives the output the irregularity real writing has.

Why not keep all of them A writer's pages yield far more than twelve examples of a common letter, and the extras are not free. They enlarge the profile, they slow the search that synthesis runs over candidate glyphs, and, most importantly, the ones beyond the first dozen are the least typical: the variants are sorted by how close they sit to the writer's own usual size and shape before this limit is applied, so the ones discarded are the outliers. Keeping a bad variant is worse than keeping no extra variant, because synthesis may select it.

> *This cap interacts with a separate limit applied at synthesis time, which considers only a few of the stored variants per character when choosing. The consequence is that a substantial share of stored variants can never actually be drawn. That is a property of the two limits together rather than of either alone, and it is worth knowing before the cap here is raised in the hope of more variety.*

6.8 ALIGN_CONF_PCT

```
ALIGN_CONF_PCT = 25
```

Output. 25, the percentile of alignment confidence below which a line is not used.

What it is for Forced alignment returns a confidence, the mean log probability per column along the chosen path, as described in the entry for CtcForcedAlign. A low value means the reader could not comfortably fit the known text to what it saw, so the character positions it produced are not to be trusted. Lines in the bottom quarter by this measure are dropped before any glyph from them is used.

Why a percentile rather than a fixed threshold This is the important design point. A fixed confidence cut would mean something different for every writer, because confidence depends on how closely a hand resembles what the reader was trained on. A clearly written upright hand produces high confidences throughout, and a cut placed to suit it would discard almost every line of a heavily cursive writer, leaving them with no profile at all. Conversely a cut placed low enough to admit the cursive writer would admit everything from the clear one, including its genuinely bad lines.

A percentile is **relative**: it asks which lines are the worst quarter *for this writer*, whatever the absolute numbers. Every writer therefore contributes their best three quarters, which is the intended behaviour. The cost is that a writer whose every line aligned badly still contributes their best three quarters, since the measure cannot express that all of them are poor in absolute terms.

Why a quarter The figure trades the number of glyphs against their quality. Discarding nothing admits the lines where alignment failed outright; discarding half would roughly halve the glyph count, and since a letter needs several examples before a prototype can be formed from it, that is a real loss for the rarer letters. A quarter removes the clear failures while leaving enough examples of an uncommon letter such as a q or a z to be usable.

> *The percentile is only applied when at least eight lines are available. Below that the ranking is meaningless, since a quarter of five lines is one line chosen on very little evidence, so the cut is disabled and every line is kept.*

6.9 FRAGMENT_PEN_FRAC

```
FRAGMENT_PEN_FRAC = 0.5
```

Output. 0.5, the fraction of a letter's usual pen path that a cut must have to count as a whole letter.

The defect it targets The commonest bad cut is a **fragment**: a boundary placed inside a letter, so that what is stored is half an a or the first stroke of an n. A fragment is dangerous in a way that other bad cuts are not, because it looks like a legitimate small glyph. Its bounding box is plausible, its stroke count is plausible, and it will be drawn without complaint, producing a letter that is simply missing half of itself.

Why pen path length is the right measure The bounding box cannot detect a fragment. Half of an o spans nearly the full width and the full height of the whole letter while containing half the ink, so any test on extent accepts it. **Pen path length**, the total distance the pen travels, measures how much writing the shape actually represents. Half an o has half the path of a whole one, and no amount of clever positioning changes that.

Why the threshold is relative to the letter itself The comparison is against the median pen length of that writer's own examples of that same character, not a universal figure. This matters because letters differ enormously in how much pen path they contain: an i is a short stroke and a dot, while an m is three arches, perhaps four times the path. A single absolute threshold would either accept half an m or reject every whole i. Taking each character's own median gives each letter its own floor, derived from how that writer actually writes it.

$$\text{penFloor}(c) \;=\; \max\Big(\text{MIN_PEN_ABS},\;\; 0.5 \cdot \text{median}\big\{\, L(g) : g \in \text{instances}(c) \,\big\}\Big) \tag{6.1}$$

where

penFloor(c)
: the least pen path a glyph of character c may have

$L(g)$
: the total pen path length of glyph g, in x-heights

instances(c)
: every cut of character c this writer produced, before filtering

0.5 this constant: the fraction of the typical path required

MIN_PEN_ABS
: an absolute floor, described in its own entry

Why one half The threshold has to sit between the shortest legitimate variant of a letter and the longest fragment. Legitimate variation in how much pen path one writer puts into one letter is modest, perhaps 20 or 30 percent, because they are writing the same shape each time. A fragment, by definition, is a part of a letter, so it has well under all of it. One half sits in the gap: generous enough that an unusually compact but complete letter passes, strict enough that a genuine half rarely does.

The median, not the mean, for the reason it is used throughout this project: the pool being summarised still contains the fragments at the point the floor is computed, and a mean would be dragged down by them, lowering the very threshold meant to exclude them. The median is unmoved as long as fragments are a minority.

6.10 MIN_PEN_ABS

```
MIN_PEN_ABS = 0.3
```

Output. 0.30, the least pen path any glyph may have, in x-heights.

Why a relative threshold needs an absolute floor beneath it The relative test above compares a glyph with the median of its own character's instances, which fails in one specific situation: when most of the instances are themselves bad. If two thirds of the cuts of some rare letter are fragments, the median is a fragment's length, half of that is a quarter of a fragment, and almost anything passes. The relative test has no way to detect this, because it has no external reference for what the letter should be.

An absolute floor supplies that reference. Whatever the median says, a glyph with less than 0.3 x-heights of pen path is rejected. Three tenths of an x-height is shorter than the shortest meaningful mark in handwriting: the stem of an i alone is a full x-height, and a comma or an apostrophe is a short stroke but still around a third. So the floor cannot reject a real letter while it does catch the degenerate case.

The pairing of a relative test with an absolute floor appears repeatedly in this file, and the reason is always the same. A relative test adapts to the writer, which is what is wanted, but it assumes the population it measures is mostly sound. The floor is what holds when that assumption fails.

6.11 MIN_LONGEST_STROKE

```
MIN_LONGEST_STROKE = 0.55
```

Output. 0.55, the least length the longest single stroke of a glyph may have, in x-heights.

The defect it targets Total pen path can be reached by accumulation: a glyph made of eight short disconnected scraps has the same total path as one made of a single proper stroke. The total test therefore accepts a scatter of debris, which is what a cut through a crowded cursive region produces. This test adds the requirement that at least one of the strokes be substantial on its own.

Why the longest stroke is the right thing to measure Every letter contains one dominant stroke. An l is a single long stroke, the body of an a is one continuous curve, an n has two arches of which either is substantial. The dominant stroke is the letter's skeleton in the ordinary sense of the word, and if it is absent then whatever was cut is not a letter however much ink it contains. Asking for the longest stroke rather than the average also means a legitimate letter with a small extra mark, such as the dot of an i or the bar of a t, is not penalised for having one short stroke among its strokes.

Why 0.55 A little over half an x-height. The shortest dominant stroke in ordinary writing belongs to a letter like a c or an r, which spans roughly the x-height as a curve and so has a path at least comparable to it. Setting the bar at just over half gives that case clear room while excluding the scraps, which are typically a fraction of an x-height each.

6.12 DOT_MIN_AREA

```
DOT_MIN_AREA = 0.004
```

Output. 0.004, the least area a component may have to be treated as a dot rather than dust, in squared x-heights.

The problem it solves A dot, such as the one over an i, is nearly round and only a few pixels across. Thinning reduces it to a single pixel, and a single pixel is not a path, so the polyline tracer produces nothing for it and the dot is silently lost. An i stored without its dot is a wrong letterform that will be drawn as a bare stem.

The recovery is to find connected components of ink that the tracer produced no path for, and add each as a short horizontal dash across its own width. This threshold separates the dots worth recovering from the specks that are not.

Why the units are squared x-heights

$$\text{minDotPx} \;=\; \max\bigl(2,\ \text{round}(0.004 \cdot \text{xh}^2)\bigr) \tag{6.2}$$

where

minDotPx
: the least number of pixels a component must contain to be recovered

xh
: the measured x-height of this line, in pixels

0.004
: this constant, an area expressed as a fraction of the square of the x-height

$\max(2, \cdot)$
: a floor of two pixels, so a single isolated pixel is never recovered

Area scales with the square of a length, so an area threshold that is to mean the same thing at every resolution must be proportional to the square of the x-height. A dot of 0.004 squared x-heights is about one sixteenth of an x-height across, which is the right order for the dot of an i. Expressing it in pixels instead would make it correct at one resolution only: at twice the resolution, a dot has four times the pixels.

6.13 MAX_GLYPH_W

```
MAX_GLYPH_W = 2.1
```

Output. 2.1, the greatest width a glyph may have, in x-heights.

The defect it targets The mirror image of the fragment. When a boundary is placed too far to one side, the cut takes its own letter plus part or all of a neighbour. The stored glyph is then not one letter but one and a bit, and drawing it produces a doubled letter in the output. This is the defect that makes reproduced text contain letters with pieces of other letters attached.

Why width detects it A letter has a bounded width relative to its own height. In ordinary handwriting a lowercase letter is roughly as wide as the x-height, perhaps a little more for a cursive hand that stretches its letters. Two letters together are about twice that. A limit of 2.1 x-heights therefore sits above any single letter and below most pairs, which is exactly where a cut-off between the two belongs.

> *It is a blunt instrument, and deliberately so. It cannot catch a cut that took only a small part of a neighbour, because that stays within the width limit. That case is left to the shape-based filters further on, which compare the glyph with what the letter is supposed to look like. The ordering is intentional: the cheap test runs first and removes the gross failures, so the expensive test has fewer glyphs to consider.*

6.14 MAX_GLYPH_W_WIDE

```
MAX_GLYPH_W_WIDE = 2.9
```

Output. 2.9, the same limit for the letters that genuinely are wide.

Why a second limit is needed A limit chosen to suit most letters is wrong for m and w. An m really is about twice as wide as an a, so the general limit would reject every legitimate m while still permitting an a that had swallowed its neighbour. The two cases cannot be separated by width alone, so the limit is made to depend on which letter is being judged.

Why not simply raise the general limit Because that would weaken the test everywhere to accommodate two letters. The point of a per-character limit is that each character is judged against what that character should be, which is more information than a single threshold can use. The same reasoning produced the per-character pen floor described above, and it is the general pattern of the filters in this file: compare a glyph with its own letter's norm wherever the norm is available.

6.15 _WIDE_CHARS

```
_WIDE_CHARS = set('mwMW')
```

Output. the set of characters that receive the wider width allowance.

Which letters, and why only these Lowercase m and w and their capitals. These are the letters built from repeated arches or strokes, so they occupy two letter widths by construction rather than by accident. Every other letter in the charset fits the general limit, including the ones that feel wide: a capital W is in the set, while a capital M is too, and letters such as O or D are tall rather than wide and so stay within the ordinary bound.

A set, not a list, because the only operation performed on it is a membership test, and a set answers that in constant time by hashing rather than by scanning. With four members the difference is immaterial; the habit is worth keeping because the same test inside a loop over every glyph of every line is not immaterial.

6.16 _OFFS

```
_OFFS = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0),
    (1, 1)]
```

Output. the eight coordinate offsets from a pixel to its neighbours.

What 8-connectivity means Two notions of neighbour exist for a pixel grid. **4-connectivity** counts only the four pixels sharing an edge, up, down, left and right. **8-connectivity** also counts the four sharing only a corner, so a pixel has eight neighbours. This list is the eight offsets of the second kind.

Why 8 and not 4 for following a stroke A diagonal line drawn on a pixel grid is a staircase of pixels that touch only at their corners. Under 4-connectivity such a line is not connected at all: every pixel is isolated from the next. A tracer using 4-connectivity would therefore break every diagonal stroke into a sequence of single pixels, which for handwriting, where most strokes are slanted, would break practically everything. 8-connectivity is the only workable choice for following a thinned stroke.

The choice is not free of consequences. Under 8-connectivity two strokes that merely pass corner to corner count as joined, so the tracer can walk from one stroke into another that it only touches. That is part of why the tracer is followed by the spur pruning and junction rejoining described below, rather than being trusted on its own.

6.17 TracePolylines

```
def TracePolylines(skel)
```

Inputs

`skel`
: a boolean array holding the thinned ink, one pixel wide

Output. a list of polylines, each an ordered list of (x, y) points following one stroke.

What it is for Thinning turns a stroke into a one-pixel-wide curve, but the result is still a *set* of pixels: a picture, with no notion of order. A pen path is an ordered sequence. This function supplies the order, by walking along connected pixels, so that a stroke becomes something a plotter could draw and something that can be compared, resampled and averaged.

Background: degree, and the three kinds of pixel The **degree** of a pixel is how many of its eight neighbours are also ink. Degree classifies every pixel of a skeleton into one of three kinds, and the whole algorithm is organised around the classification.

- **Degree 1, an endpoint.** The loose end of a stroke. A stroke has two of these, one at each end.
- **Degree 2, an interior point.** The middle of a stroke, with one neighbour behind and one ahead. Walking is possible here because the continuation is unambiguous.
- **Degree 3 or more, a junction.** Where strokes meet, as in the crossing of a t or the point where a cursive join meets a letter. Walking must stop, because there is more than one way to continue and nothing locally distinguishes them.

The walk Starting from a pixel that is not of degree 2, that is an endpoint or a junction, the walk steps to a neighbour and then continues as long as the current pixel has degree exactly 2, each time moving to the neighbour that is not the one just left. It halts on reaching anything that is not of degree 2, which is either the far end of the stroke or a junction. So each walk traces one unambiguous run of stroke between two decision points.

Starting from the non-degree-2 pixels rather than anywhere is what makes the decomposition well defined. Beginning in the middle of a stroke would produce two half-strokes that would then have to be matched up again.

Why edges are marked as used, rather than pixels The walk records that it has traversed the connection between two pixels, in both directions, rather than recording that it has visited a pixel. The distinction matters at a junction: the junction pixel belongs to every stroke meeting there, so marking it as visited would prevent all but the first stroke from using it, and the remaining strokes would be traced as starting one pixel away from where they really start. Marking connections instead lets each stroke pass through the junction while still guaranteeing no connection is traced twice.

The pure-loop case A closed loop, such as a well-formed o, consists entirely of degree-2 pixels. It therefore has no endpoint and no junction, so the seeding rule finds nothing to start from and the loop would be missed. A second pass handles it: any pixel not yet covered by a traced polyline is used as an arbitrary starting point, which for a loop gives the whole loop traced from a place chosen arbitrarily. That is acceptable, because a loop has no natural starting point to find.

Suppressing redundant diagonal steps When a diagonal neighbour is available and so is a path to it through an orthogonal neighbour, the diagonal step is not offered. Without this, a stroke two pixels thick in places, which thinning can leave, allows the walk to cut a corner diagonally and then find itself able to step back, producing a short spurious loop. Preferring the orthogonal route keeps the traced path on the pixels the skeleton actually laid out.

The two tidying passes The raw traces are then cleaned by _PruneSpurs, which removes the short whiskers thinning leaves at thick junctions, and _JoinAtJunctions, which rejoins traces that the junction split but which were one pen stroke. Both have their own entries. The important point here is that the tracer is not expected to be correct by itself: it produces a decomposition that is correct as graph theory, and the two passes then adjust it toward what a pen actually did, which is a different question.

Points come out as (x, y) floating-point pairs, with x the column and y the row, swapped from the (row, column) order the pixels were found in. The swap is to the convention the rest of the geometry uses, where the first coordinate is horizontal, and it happens once, here, rather than being remembered at every later use.

6.18 _Dir

```
def _Dir(pts, fromEnd, n=4)
```

Inputs

`pts` a polyline

`fromEnd`
: true for the direction at the last point, false for the first

`n` how many points back to look, default 4

Output. a unit vector giving the direction the path is heading at that end.

What it is for Deciding whether two traced paths were one pen stroke requires knowing which way each is heading where they meet. This measures that direction at either end of a path.

Why it looks several points back The direction could be taken from the last two points, but those are adjacent pixels, so the step between them is one of only eight possible directions, each 45 degrees apart. That resolution is far too coarse to decide whether two strokes continue smoothly. Measuring from a point four steps back gives a chord over several pixels, whose angle is continuous and which averages out the staircase the pixel grid imposes.

$$\mathbf{d} = \frac{\mathbf{p}_{\text{end}} - \mathbf{p}_{\text{end}-n}}{\left\| \mathbf{p}_{\text{end}} - \mathbf{p}_{\text{end}-n} \right\|} \tag{6.3}$$

where

$\mathbf{d}$ the returned direction, a vector of length 1

$\mathbf{p}_{\text{end}}$
: the point at the end being measured

$\mathbf{p}_{\text{end}-n}$
: the point n steps in from that end

n how far back to look, 4 by default

$\| \cdot \|$ the length of the vector, by Pythagoras

Why it is normalised to unit length Dividing by the length leaves only the direction, discarding how far apart the two points were. That is what makes two directions comparable by a dot product, as the next function does: for unit vectors the dot product is the cosine of the angle between them, with no dependence on the lengths. Without normalisation the same comparison would confound the angle with how long the paths happened to be.

A degenerate path whose endpoints coincide has no direction, and a default pointing along the positive x axis is returned. The value is arbitrary; what matters is that it is a unit vector, so the caller's angle arithmetic stays well defined instead of producing a division by zero.

6.19 _PruneSpurs

```
def _PruneSpurs(chains, minLen=2.5)
```

Inputs

`chains`
: the traced polylines

`minLen`
: the shortest a dangling path may be before it is removed, default 2.5 pixels

Output. the polylines with the short dangling ones removed.

The artefact it removes Thinning a shape where strokes cross leaves a characteristic defect. Where the ink is thick, as at the junction of a t or where a cursive join meets a letter body, the thinning does not produce a clean crossing but a small cluster from which short stubs project, going nowhere. These are **spurs**, and they are an artefact of the thinning rather than anything the pen did.

They matter because they are traced as polylines like any other, so a letter that should have two strokes acquires five, three of which are two-pixel whiskers. That inflates the stroke count, which several filters test, it disturbs the stroke ordering, and it confuses the averaging that forms a prototype, which matches strokes between variants by position.

How a spur is recognised Two conditions together. It is short, under 2.5 pixels of path length. And it is **dangling**, meaning at least one of its two endpoints is not shared with any other path, so it leads nowhere. Requiring both is what makes the test safe: a short path joined at both ends is a genuine short connection between two strokes and must be kept, while a long dangling path is a real stroke with a free end, such as the stem of a t.

The sharing is determined by counting how many paths have an endpoint at each coordinate. A count of one means that endpoint belongs to this path alone.

Why the threshold is in pixels and not x-heights Almost every threshold in this file is in x-heights, for the scale-independence reasons given in the introduction. This one is deliberately not, because the artefact it targets is not a property of the writing at all: it is a property of the thinning algorithm, whose spurs are a few pixels long whatever the resolution, because thinning works pixel by pixel. A threshold in x-heights would grow with resolution and start removing real strokes from high-resolution crops.

> *If the pruning would remove everything, the original paths are returned instead. A glyph consisting entirely of short dangling paths is most likely debris, but returning nothing would hide it from the filters that are designed to judge it; returning it unchanged lets them do so.*

6.20 _JoinAtJunctions

```
def _JoinAtJunctions(chains, maxAngleDeg=75.0)
```

Inputs

`chains`
: the traced polylines

`maxAngleDeg`
: how much the direction may change at a join and still count as one stroke, default 75 degrees

Output. the polylines with those that continue through a junction merged into one.

The problem it solves The tracer stops at every junction, because locally there is no way to know which way to continue. But a pen usually does continue. Where a cursive join meets the body of a letter, or where one stroke crosses another, the tracer produces several short paths where the pen drew one long one. Leaving them separate misrepresents the writing: the glyph appears to have more strokes than it does, and a plotter drawing it would lift the pen where the writer did not.

The criterion: continuation of direction What distinguishes a continuation from a genuine corner is direction. If one path arrives at a junction heading one way and another leaves heading the same way, they were one stroke. If the second leaves at a sharp angle, the junction is a real corner or a crossing of two different strokes.

$$\cos\theta \;=\; -\,\mathbf{d}_i \cdot \mathbf{d}_j, \qquad \theta \;=\; \arccos\big(\mathrm{clip}(\cos\theta,\,-1,\,1)\big) \tag{6.4}$$

where

θ
: the turn angle: 0 for a path continuing perfectly straight through the junction

$\mathbf{d}_i$, $\mathbf{d}_j$
: the unit directions at the two ends being considered, each from _Dir

$\cdot$
: the dot product, which for unit vectors equals the cosine of the angle between them

$-$
: the negation, explained below

arccos
: the inverse cosine, converting a cosine back to an angle

$\mathrm{clip}(\cdot, -1, 1)$
: the cosine forced into its valid range before the inverse cosine is taken

Why the dot product is negated This is the step most easily got wrong. Both directions are measured *outward* from their own path's end. Two paths that continue through a junction therefore point in opposite directions: one arrives pointing into the junction, the other leaves pointing away, and as outward measurements those are roughly antiparallel. So perfect continuation gives a dot product of -1, not $+1$, and negating it turns continuation into a cosine of $+1$, which is what the angle formula expects.

Why the cosine is clipped The inverse cosine is defined only on the range -1 to 1. Two unit vectors cannot mathematically produce a dot product outside it, but floating-point rounding can: a vector whose length is stored as 1.0000000001 yields a dot product a fraction beyond 1, and the inverse cosine of that is not a number. The result would then propagate silently through every comparison, since comparisons involving it are all false. Clipping first costs nothing and removes the possibility.

Why 75 degrees is generous A threshold of 75 degrees permits a substantial change of direction, far more than a straight continuation. That is deliberate, because handwriting genuinely turns: the pen rounds a corner at the bottom of a u or the top of an n without lifting, so insisting on near-straightness would refuse to rejoin strokes that really were continuous. The errors are asymmetric, which is what justifies the generosity. Failing to join two halves of one stroke leaves a glyph that a filter will then reject as a fragment, losing a usable letter. Joining two strokes that were separate merely merges two strokes of the same letter, which affects the stroke count and little else.

Why it repeats until nothing changes One merge can create a new opportunity. Three paths meeting in a line are joined by first merging two, after which the merged path has a new free end that may continue into the third. So the search repeats until a pass finds nothing to merge. Each pass also takes the single best candidate, the pair with the smallest turn angle, rather than merging everything that qualifies. That matters at a junction where three or more paths meet and more than one pairing is permissible: taking the best first means the most convincing continuation is committed to before the others are considered, which is the greedy strategy used for the line centres in segment.py, and it is correct for the same reason.

> *The search compares every pair of ends on every pass, which is quadratic in the number of paths and runs inside a loop. That would matter for hundreds of paths; here it runs on the handful belonging to one letter, where the cost is irrelevant and the clarity of an exhaustive search is worth having.*

6.21 SimplifyPolyline

```
def SimplifyPolyline(poly, eps=0.6)
```

Inputs

`poly`
: a polyline

`eps` how far a point may sit from the simplified path before it must be kept, default 0.6 pixels

Output. the polyline with the points that add nothing removed.

What it is for A traced path has one point per pixel, so a straight stroke fifty pixels long is stored as fifty points when two would describe it exactly. Removing the redundant points shrinks the stored profile, speeds up everything that iterates over points, and removes the pixel staircase, leaving a path described by where it actually changes direction.

The Douglas-Peucker method The algorithm [22] is recursive and rests on one idea: a path can be replaced by the straight line between its endpoints if no point on it strays far from that line. So measure the furthest stray; if it is within tolerance, the whole path becomes two points, and if not, the path is split at the furthest point and each half is simplified the same way.

$$d_k \;=\; \frac{\left|\,(x_b - x_a)(y_k - y_a) - (y_b - y_a)(x_k - x_a)\,\right|}{\sqrt{(x_b - x_a)^2 + (y_b - y_a)^2}} \tag{6.5}$$

Source: [22]

where

d_k the perpendicular distance from point k to the straight line joining the two endpoints

(x_a, y_a)
: the first point of the path

(x_b, y_b)
: the last point of the path

(x_k, y_k)
: the point whose distance is being measured

$|\cdot|$ the absolute value, since a distance has no sign

$\sqrt{\cdot}$ the length of the line segment, which turns the numerator into a true distance

The numerator is the **cross product** of the vector along the line and the vector to the point. Its magnitude equals the area of the parallelogram the two vectors span, and that area also equals the base times the height. The base is the length of the line segment, so dividing by it leaves the height, which is the perpendicular distance. This is why the formula computes a distance without any trigonometry.

Why the tolerance is 0.6 pixels Just over half a pixel. The reasoning is that the staircase to be removed deviates from a true straight line by up to half a pixel, that being the most a rounded position can differ from the real one. A tolerance just above that removes the staircase and nothing else: any deviation larger than half a pixel is a real change of direction that the writer put there, and it is kept.

The closed-path special case A loop, such as the body of an o, defeats the basic method. Its endpoints coincide or nearly so, which makes the reference line a point rather than a line, and every distance measured from it is meaningless. The function detects this by checking whether the far end of the path returns close to the start, and if so splits the path first at the point furthest from the start, simplifying the two halves separately. Each half is then an ordinary open path with distinct endpoints, and the method applies.

> *The two halves are rejoined by dropping the last point of the first before concatenating, since the split point belongs to both and would otherwise be stored twice. A duplicated point is a zero-length segment, which is harmless to draw but divides by zero in any calculation that normalises by segment length.*

6.22 _TrimLigatureTails

```
def _TrimLigatureTails(strokes,connL=True,connR=True,lowY=0.42,minKeep
    =0.18,
         climbY=1.6,climbSlope=2.2)
```

Inputs

`strokes`
: the glyph's strokes, normalised and ordered left to right

`connL`
: whether ink genuinely crossed the left boundary

`connR`
: whether ink genuinely crossed the right boundary

`lowY`
: how high a carrier stroke may run, in x-heights, default 0.42

`minKeep`
: the least fraction of the glyph's width that must survive, default 0.18

`climbY`
: the higher ceiling used by the second, steeper test, default 1.6

`climbSlope`
: the steepest slope that second test tolerates, default 2.2

Output. three things: the trimmed strokes, the point where the pen entered the glyph, and the point where it left.

The problem it solves In joined-up writing the letters are connected by **ligatures**, the carrying strokes that run from the end of one letter to the start of the next. A cut placed between two letters therefore leaves part of a ligature attached to each: the stored glyph begins with a stroke running in from the previous letter and ends with one running out toward the next.

Storing those tails is wrong for a specific reason. The glyph will later be placed next to some other letter, chosen at synthesis time, and the tail it carries was shaped to reach the letter that followed it originally. Keeping it produces a join that points at nothing, which is visible in the output as a stray lead-in stroke, or a doubled connector where the next glyph brings its own. What should be stored is the letter proper, together with a note of where the pen entered and left it, so that synthesis can draw a join appropriate to whatever pair of letters it actually places.

How a tail is recognised A ligature has a consistent geometry, because of what it is for: it carries the pen from the end of one letter across to the start of the next, along the baseline. So it runs low, it runs outward, and it runs fairly flat. Three conditions follow, and the leading points of a stroke are consumed as long as all three hold.

- **Low.** The point sits below a ceiling, by default 0.42 x-heights, which is under half the height of a letter body. A ligature travels near the baseline; the letter itself rises above it.
- **Outward.** The step moves in the positive x direction. A tail entering from the left runs rightward into the letter; a stroke that doubles back is part of the letter.
- **Flat.** The vertical step is small relative to the horizontal one, so the stroke is travelling across rather than climbing. The slope limit is 0.7 for the low test.

Why there is a second, steeper test Not every ligature stays low. In some hands the join into a letter such as an l or a b climbs steeply from the baseline almost to the ascender height before the letter proper begins. The low test cannot see such a tail, because it exits at the first point above the ceiling, which is only a short way in. So a second test is tried when the first finds nothing, with a much higher ceiling of 1.6 x-heights and a correspondingly looser slope limit of 2.2.

The two are tried in order rather than combined, and the order is the safeguard. The low test is specific and fires on the common case; the climbing test is permissive and could consume a real stroke, so it is only consulted when the specific test found nothing at all. This is the same specific-before-general ordering used among the ownership rules in segment.py.

The safeguard on how much may be removed Whatever the tests say, a trim is only applied if what remains spans more than 18 percent of the glyph's original width. Without that condition a permissive test could consume almost the whole glyph, leaving a stub, and a stub is exactly the fragment that the later filters are built to catch. It is better to leave a tail attached, which costs a slightly wrong join, than to destroy a letter, which costs the letter.

Why the right side is handled by mirroring The exit connector leaves to the right, so it is the reverse of the entry case in both senses: it runs outward in the opposite direction and it is at the end of the stroke rather than the start. Rather than writing a second version of the test with the comparisons reversed, the stroke is reversed and its x coordinates negated, which makes an outgoing rightward tail look exactly like an incoming leftward one, and the same test is reused. Two tests that must agree are a place for them to disagree; one test applied to a transformed input cannot.

Why the connection flags are required Trimming only happens on a side where ink genuinely crossed the boundary, which the caller establishes by observing whether a traced centre line actually passed through the cut. That is the honest test for whether the pen was lifted there. Without it, the trimmer would attack the legitimate opening stroke of a letter at the start of a word, where there is no preceding letter and therefore no ligature, and the opening stroke of many letters is low, outward and flat.

> *The entry and exit points returned are the ends of the trimmed strokes, not of the original ones, so they mark where the letter proper begins and ends. Those are the points synthesis attaches its own connector to, which is what makes a join between two glyphs meet the ink rather than the air beside it.*

6.23 ExtractLineGlyphs

```
def ExtractLineGlyphs(gray, text, model)
```

Inputs

`gray`
: one line crop as a grey image

`text`
: the known text of that line

`model`
: a loaded reader, used for alignment

Output. a pair: a list holding one glyph dictionary per character of the text, with nothing where a character could not be cut, and a dictionary of per-line statistics.

What it is for This is the central function of the file and the hardest step in the project. It takes one line of handwriting and its known text, and returns one stored letterform per character. Everything else in this file either prepares its input or judges its output.

The problem: where does one letter end and the next begin In print this question is trivial, because letters are separated by white space. In handwriting it often has no clean answer at all. In cursive the pen does not lift between letters, so there is no gap anywhere, and the boundary is a place on a continuous stroke that the writer never marked. Any cut will leave some of one letter with the other. The aim is therefore not a perfect cut, which does not exist, but the least damaging one.

Step one: ask the reader where the characters are No separate letter-finding method is used. The reader already knows, implicitly, where each character sits, because it produces character scores per column. Forced alignment extracts that, as described under CtcForcedAlign, returning for each character the span of network columns it occupies. Those spans are converted back to pixel positions in the original crop by FrameXToPixel, undoing the scaling that fitted the crop to the network's canvas.

Using the recogniser for this is the design decision that makes the rest tractable. A geometric letter-finder would have to solve segmentation and recognition together, which is the problem CTC was adopted to avoid. Here the recognition is already done and the positions come free with it.

Step two: the cut cost, and why the reader's boundary is moved The aligner's boundary is approximately right but not precisely placed, because the network's columns are coarse, eight original pixels each, and because the network was never trained to put boundaries anywhere in particular. Cutting exactly there often cuts through the thick part of a letter. So the boundary is moved to the best nearby column, where best is defined by a cost that encodes what an honest cut looks like.

$$\text{cutCost}[x] \;=\; \text{thin}[x] \;+\; 1.15 \cdot \text{topAbove}[x] \tag{6.6}$$

where

cutCost[x]
: the price of cutting at column x

thin[x]
: how much ink the column holds, relative to the 60th percentile of the inked columns, so 1 is a typical column and less is thinner

topAbove[x]
: how high the highest ink in that column reaches above the baseline, in x-heights, capped at 3

1.15 how much the height term counts relative to the thickness term

The two terms express the two properties of a good cut. **Thin** matters because a column crossing one stroke damages less than one crossing three. **Low** matters because the place where two cursive letters genuinely meet is the ligature near the baseline, so a cut that stays low passes through the join rather than through a letter body. A column with no ink at all is given a cost of zero outright, which guarantees that a real gap, where one exists, always wins.

The column profile is smoothed before use, over a window of about 0.12 x-heights, so that a single-pixel gap inside a stroke does not read as a cheap cut. Normalising the thickness against the 60th percentile rather than the maximum makes the scale robust: the maximum is set by the single thickest column on the line, often a crossing, while a percentile describes the typical column.

Step three: snapping, with a pull toward the original Within a search radius of half an x-height either side, the column of least cost is chosen, with a small penalty added for distance from the aligner's own boundary.

$$x^* \;=\; \arg\min_{|x-x_0|\le R} \Big(\mathrm{cutCost}[x] \;+\; 0.6 \cdot \frac{|x-x_0|}{R} \Big) \tag{6.7}$$

where

x^* the chosen cut column

x_0 the aligner's boundary, the midpoint between two character centres

R the search radius, half an x-height

0.6 the strength of the pull back toward the original boundary

arg min
: the column at which the total is smallest

The pull is a tie-breaker of the same kind as the midline penalty in SeamCuts: where several columns are about equally good, it prefers the one the recogniser nominated, because that carries information the geometry does not. The radius is limited to half an x-height for the same reason that the penalty exists at all: the aligner's boundary is approximately right, and a search allowed to wander a whole letter away could find a cheap column belonging to a different letter entirely.

Step four: making the boundaries consistent Snapping each boundary independently can produce an inconsistency: two neighbouring boundaries can land on the same column, giving a character a cell of zero width, or they can cross, giving a negative one. So the boundaries are built as a single increasing sequence, each constrained to leave at least a minimum cell width, 0.08 x-heights, between itself and its predecessor, and to leave enough room for all the cells still to come before the right-hand end.

This is a small instance of a general point about pipelines of independent local decisions: each decision can be locally optimal and the set of them still be jointly invalid. The remedy here is to impose the joint constraint explicitly as the sequence is built, rather than to detect and repair violations afterwards.

Step five: thin once, then cut the paths The whole line is thinned once, as a whole, and the resulting centre lines are traced and only then divided at the boundaries. The alternative, slicing the image into cells and thinning each, is wrong, and the reason is worth stating. Thinning is a boundary-driven process: it erodes a shape from its edges inward. A cut edge is a boundary as far as the thinning is concerned, so thinning a cell produces a spurious spine running along the cut, which becomes a vertical stroke in the stored glyph that the writer never drew.

Dividing the traced paths instead means a path that crosses a boundary is split into two pieces at that point, each assigned to its own character. The crossings are recorded as they happen,

and they serve a second purpose: a centre line crossing a boundary means the pen never lifted there, which is the honest definition of two letters being joined, and it is what the connectedness measurement is built from.

Step six: recovering the dots A dot thins to a single pixel, which is not a path, so the tracer produces nothing for it. Any connected component of ink for which the tracer produced no path at all is therefore recovered, provided it is larger than the dust threshold, and added as a short horizontal dash across its own width. The representation is crude but correct in the respects that matter: it is in the right place, it has the right extent, and it will be drawn as a mark rather than omitted.

Step seven: normalising into the stored frame Each surviving path is simplified and then converted into the stored convention, in one expression per point.

$$y_n \;=\; \frac{y_{\text{base}} - y}{\text{xh}}, \qquad x_n \;=\; \frac{x - x_{\text{left}}}{\text{xh}} \;-\; \tan(\text{slant}) \cdot y_n \tag{6.8}$$

where

x_n, y_n
: the stored coordinates, in x-heights

x, y
: the pixel coordinates in the crop

y_{base}
: the baseline row, so subtracting it puts the baseline at zero and makes y increase upward

x_{left}
: the left boundary of this character's cell, so each glyph starts at its own origin

xh
: the measured x-height in pixels, which is what makes the result scale-free

tan(slant)
: the shear that removes the writer's slant, described next

Why the slant is removed and then put back The last term is a **shear**: each point is shifted horizontally in proportion to its height, which tilts the glyph upright. The reason for doing it is comparison. Two writings of the same letter at different slants are different shapes as numbers, so any averaging or distance between them is dominated by the slant rather than by the letterform. De-slanting separates the two, so that shape can be compared as shape and slant is carried once, as a single number in the profile.

But the glyph will be drawn re-slanted, and its width and left edge must describe how it will look then, not how it looks upright. So after trimming, the extent is measured in the re-slanted frame by applying the shear back, and the glyph is shifted so its leftmost re-slanted point is at zero. Measuring the width upright would misstate it, because shearing a tall shape changes how much horizontal room it needs.

What is stored per glyph

- **strokes:** the normalised paths, ordered left to right and each oriented left to right, so that two variants can be matched stroke by stroke.
- **width:** how much horizontal room the ink occupies, re-slanted.

- **advance:** how far the pen should move on to the next character, taken from the cell width. This is what preserves the writer's own letter spacing.
- **lead:** the gap from the left edge of the cell to where the ink starts. Without it, synthesis would butt every glyph against its neighbour and lose the writer's internal spacing.
- **top and bot:** the highest and lowest points, which is what the height-class filter tests.
- **entry and exit:** where the pen came in and went out, for drawing joins.
- **connL and connR:** whether ink actually crossed each boundary, the evidence for connectedness.

The returned list has one slot per character of the text, with nothing in the slots that could not be cut, rather than being a shorter list of the successes. Preserving the positions is what allows the caller to ask whether two adjacent *characters were joined, which the connectedness measure needs and which a compacted list could not answer.*

6.24 _ASCEND

```
_ASCEND = set('bdfhklt') | set('ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789')
```

Output. the set of characters expected to reach above the x-height band.

What it encodes An **ascender** is the part of a letter rising above the x-height band, as in b, d, f, h, k, l and t. Every capital and every digit also reaches up, because capitals are built to the ascender height and digits in most hands match them. Collecting them into one set lets a single test ask whether a glyph reaches where its character ought to.

Lowercase i and j are deliberately excluded, although their dots sit high. The dot is a separate mark and is often lost or recovered crudely, so requiring height of an i would reject every correctly cut i whose dot the tracer dropped. The letter t is included even though in many hands it is shorter than a full ascender, because it still clearly exceeds the x-height band, which is all the test asks.

6.25 _DESCEND

```
_DESCEND = set('gjpqy')
```

Output. the set of characters expected to drop below the baseline.

What it encodes A **descender** is the part of a letter below the baseline, as in g, j, p, q and y. These five are the only lowercase letters in English with one. A glyph of one of these that never goes below the baseline is missing its tail, which means the cut took only the body, and that is a fragment.

6.26 _XONLY

```
_XONLY = set('acemnorsuvwxz')
```

Output. the set of characters expected to stay inside the x-height band.

What it encodes The lowercase letters with neither ascender nor descender: a, c, e, m, n, o, r, s, u, v, w, x and z. These are the letters for which the test runs the other way. Rather than requiring reach, it forbids it: a glyph of one of these that extends well above or below the band has collected ink from a neighbour, most often the ascender of the letter after it or the descender of the one before.

> *The three sets are not exhaustive and are not meant to be. Punctuation, the space and a few letters belong to none of them, and for those the test simply passes. A filter that cannot judge a case should abstain rather than guess, because every one of these gates can only reject, and a wrong rejection is a permanently lost letterform.*

6.27 _ClassOk

```
def _ClassOk(ch, top, bot)
```

Inputs

`ch` the character this glyph is supposed to be

`top` its highest point, in x-heights above the baseline

`bot` its lowest point, negative below the baseline

Output. true if the glyph reaches where that character should reach.

Why this is the cheapest and most reliable gate The height class of a letter is one of the few facts about a letterform that is true for every writer. Individual hands vary enormously in slant, width, roundness and how joined they are, but an l reaches above the x-height band in all of them, because otherwise it would not be an l. A test on reach is therefore almost free of false rejections while catching a large class of bad cuts, which is why it runs first.

The three tests

$$\begin{aligned} \text{x-only:} &\quad \text{top} \le 1.42 \ \text{ and } \ \text{bot} \ge -0.42 \\ \text{ascender:} &\quad \text{top} \ge 1.25 \\ \text{descender:} &\quad \text{bot} \le -0.20 \end{aligned} \tag{6.9}$$

where

top the glyph's highest point, in x-heights, where 1.0 is the top of the x-height band

bot its lowest point, negative below the baseline

1.42 how far above the band an x-only letter may stray before it is judged to have collected a neighbour's ascender

1.25 how far above the band an ascender letter must reach

−0.20
how far below the baseline a descender letter must drop

−0.42
how far below an x-only letter may stray

Why the thresholds are not at 1.0 The band is nominally 0 to 1, so it might seem that an x-only letter should be required to stay below 1.0 and an ascender to exceed it. The margins exist because the band itself is measured, not known. CoreBand estimates the x-height from a row profile, and that estimate carries error, so a letter sitting exactly at the top of the true band can measure as 1.1. The limits are placed outside the range of that measurement error.

The asymmetry between 1.42 and 1.25 is deliberate and follows from the direction each test guards. An x-only letter is being checked for contamination, which when it happens is gross: a swallowed ascender reaches 1.7 or more, so the limit can be placed generously at 1.42 and still catch it. An ascender letter is being checked for truncation, and must be given the benefit of the doubt only as far as is safe, so 1.25 sits just above where a complete x-only letter could reach. Between 1.25 and 1.42 both tests would pass, which is intentional: that band is where the measurement is genuinely ambiguous, and neither test should fire there.

6.28 _PenLength

```
def _PenLength(g)
```

Inputs

`g` a glyph dictionary

Output. the total length of all its strokes, in x-heights.

What it measures How far the pen travelled to draw this glyph, obtained by summing the lengths of every stroke, each of which is the sum of the straight-line distances between its consecutive points. It is the quantity the fragment test compares against a per-character floor, for the reasons set out under FRAGMENT_PEN_FRAC.

Why total rather than per stroke A letter may legitimately be drawn in one stroke or several, depending on the hand and on whether the pen lifted. The same letter from the same writer can appear with two strokes on one line and one on the next, where a join happened to survive the thinning. The total is invariant to that, so it measures the letter rather than the accident of how it came out.

6.29 _LongestStroke

```
def _LongestStroke(g)
```

Inputs

`g` a glyph dictionary

Output. the length of its single longest stroke, in x-heights.

What it measures, and why it is a separate test The longest single continuous stroke. Where the total pen length asks how much writing is present, this asks whether any of it is a real stroke. The two are independent: a scatter of eight short scraps passes the total test and fails this one, while a single clean stroke that is slightly short passes this and fails the total. Keeping them separate means each can be set to catch its own defect, rather than one compromise threshold catching neither well.

> *Both measures ignore strokes of fewer than two points, which have no length to contribute. A single isolated point can survive into a glyph from the dot recovery, and including it as a zero-length stroke would be harmless here but would divide by zero in the resampling that forms prototypes.*

6.30 _ShapeGrid

```
def _ShapeGrid(g, n=12)
```

Inputs

`g` a glyph dictionary

`n` the grid resolution, default 12

Output. a vector of 144 numbers describing the glyph's shape, scaled to unit length; or nothing if the glyph has no points.

What it is for Several filters need to ask how similar two glyphs are, and one needs to ask how similar a glyph is to the general idea of a letter. Comparing two lists of strokes directly is not workable: they may have different numbers of strokes, different numbers of points, and the points need not correspond. What is needed is a **descriptor**, a fixed-length vector computed from a shape such that similar shapes give similar vectors. Then comparing shapes becomes comparing vectors, which is arithmetic.

Step one: turn strokes into points Each segment of each stroke is filled in with evenly spaced points along it, at a density of about 24 per unit length. This is necessary because the stored strokes have been simplified, so a long straight run is two points, and counting points directly would weight a simplified stroke far less than a wiggly one of the same length. Filling in restores the proportionality between a stroke's length and the number of points representing it.

Step two: scale the shape into the grid The points are rescaled so the shape exactly fills the grid in both directions, and each point is assigned to a cell. This discards absolute size and aspect ratio deliberately, so that the same letter written large and small gives the same descriptor. Size is already tested separately by the width and height gates, so there is nothing lost by removing it here, and a great deal gained: the descriptor then responds to shape alone.

Step three: count, then damp

$$G[i,j] \;=\; \sqrt{\left|\{\,\text{points falling in cell } (i,j)\,\}\right|} \tag{6.10}$$

where

$G[i, j]$
: the value stored for grid cell i, j

$|\{\cdot\}|$ the number of points that landed in that cell

The square root is a **damping** transformation. Without it, a cell where the pen dwelt, at a sharp corner or where two strokes overlap, accumulates many times the count of an ordinary cell and dominates the whole descriptor, so two glyphs would be judged similar or not chiefly by where their pen dwelt. Taking the square root compresses large counts toward small ones while keeping the ordering, so a cell with ink still counts for more than an empty one but not overwhelmingly more.

Step four: blur The grid is blurred with a small 3 by 3 kernel, weight 0.25 in the centre and 0.09375 at each of the eight neighbours. The problem it solves is that a grid is a hard partition: two writings of the same letter that differ by a hair can put corresponding ink in adjacent cells, which as vectors makes them share nothing at all in those positions. Blurring spreads each cell's value into its neighbours, so near-misses contribute to the similarity instead of being lost. This is the same reasoning that puts tolerance into any grid-based descriptor, and without it the measure would be far too brittle to be useful.

Step five: normalise to unit length

$$\mathbf{v} \;=\; \frac{\mathbf{g}}{\|\mathbf{g}\|}, \qquad \|\mathbf{g}\| \;=\; \sqrt{\textstyle\sum_k g_k^2} \tag{6.11}$$

where

v the returned descriptor, of length 1

g the blurred grid flattened into a 144-element vector

$\|\mathbf{g}\|$ its Euclidean length, the square root of the sum of the squares of its entries

Scaling to unit length removes the total amount of ink from the descriptor, leaving only how it is distributed. The decisive benefit is what it does to the comparison: for two unit vectors, the dot product is exactly the cosine of the angle between them, which is 1 for identical shapes and falls toward 0 as they diverge. So similarity becomes a single multiplication and sum, with a value on a fixed and interpretable scale. This is **cosine similarity**, and it is the standard way to compare descriptors precisely because normalisation makes magnitude irrelevant.

> *The resolution of 12 by 12 is a compromise. Coarser would stop distinguishing letters that differ in a small region, such as a and o, which agree everywhere but the stem. Finer would make the descriptor sensitive to exactly the variation between two writings of one letter that it is meant to tolerate, and the blur would have to widen to compensate, which gives back the resolution that was bought.*

6.31 _LETTER_PRIOR

```
_LETTER_PRIOR = {}
```

Output. a cache holding the reference shape for each letter, empty until filled.

What it is for A module-level dictionary that holds the cross-writer reference shapes once they have been computed, so that building them, which requires descriptors for every variant of every writer, happens once rather than once per writer processed.

> *A mutable value shared across a whole module is a pattern worth being cautious with, because anything that writes to it affects every later caller invisibly. It is acceptable here because what it holds is derived from fixed data and so is the same whenever it is computed, which means a stale entry cannot be wrong, only old.*

6.32 BuildLetterPrior

```
def BuildLetterPrior(libsByAuthor, minSamples=12)
```

Inputs

`libsByAuthor`
: a mapping from writer to that writer's glyph library

`minSamples`
: the least number of examples a letter needs before a reference is formed, default 12

Output. a mapping from character to a reference descriptor, including only the letters with enough examples.

What it is for, and the one thing it must not do Several of this project's gates reject a glyph for not looking enough like the letter it claims to be, which requires knowing what that letter looks like. This function supplies that: for each character it pools every variant from every writer and forms a single reference descriptor, the **prior**, meaning what is expected before any particular writer is considered.

The restriction on its use matters as much as its construction. It is used only to reject a writer's own bad cuts. It is never used to supply a shape, so no part of another person's handwriting ever enters a writer's profile. The distinction is the difference between using the pool as a quality check and using it as a source, and only the first is legitimate for a system whose purpose is to reproduce an individual's hand.

Why the median across writers

$$\mathbf{p}_c \;=\; \frac{\mathbf{m}_c}{\|\mathbf{m}_c\|}, \qquad \mathbf{m}_c[k] \;=\; \operatorname{median}\!\left\{\, \mathbf{v}^{(1)}[k],\, \ldots,\, \mathbf{v}^{(N)}[k] \,\right\} \tag{6.12}$$

where

$\mathbf{p}_c$ the reference descriptor for character c

$\mathbf{m}_c$ the elementwise median across every pooled example

$\mathbf{v}^{(i)}$ the descriptor of the i-th pooled example of that character

k which of the 144 descriptor positions

N how many examples were pooled

$\|\cdot\|$ the length, so the reference is itself a unit vector and can be compared by dot product

The median is taken independently in each of the 144 positions, and it is chosen over the mean for a reason that is central to whether this works at all. The pool is built from unfiltered glyph libraries, so it contains fragments and over-wide cuts: the very defects the prior is meant to detect. A mean would absorb them, shifting the reference toward the typical defect, and the gate built on it would then fail to reject what it was built to reject. The median ignores them entirely so long as they are a minority, which they are.

Pooling across writers is what makes the reference describe the letter rather than any one hand. Individual idiosyncrasies differ between writers and so disagree in the pool, where the median removes them; what every writer's a has in common is the thing that makes it an a, and that is what survives.

Why twelve examples are required A median over very few samples is not robust: with four examples, two of them fragments, the median sits between a fragment and a letter. Twelve is enough that a minority of defects cannot reach the middle. Characters with fewer are simply omitted, and the gates then abstain on them rather than testing against an unreliable reference, which is the correct behaviour for a filter that can only reject.

6.33 _PriorScore

```
def _PriorScore(g, ch, prior)
```

Inputs

`g` a glyph dictionary

`ch` the character it claims to be

`prior`
: the mapping of reference descriptors

Output. a distance from 0, identical to the reference, upward; 0 when no comparison can be made.

What it measures How far this glyph's shape sits from the cross-writer idea of its letter. It is the quantity stored on each glyph as `priorD`, and synthesis later uses it to judge how far a writer's own stored shape can be trusted.

$$D = 1 - \mathbf{v} \cdot \mathbf{p}_c \tag{6.13}$$

where

D the returned distance: 0 when the glyph matches the reference exactly, larger as it diverges

$\mathbf{v}$ the glyph's own descriptor, a unit vector

$\mathbf{p}_c$ the reference descriptor for that character, also a unit vector

$\cdot$ the dot product, which for unit vectors is the cosine of the angle between them

Subtracting the cosine similarity from 1 converts a similarity into a distance, so that small means good. This is the **cosine distance**. Since both descriptors have non-negative entries, their dot product cannot be negative, so the result lies between 0 and 1.

Why a missing reference scores zero rather than failing When no reference exists for a character, or the glyph yields no descriptor, the score returned is 0, which is the value meaning a perfect match. That is deliberate: it makes the gate abstain. Every filter in this file can only reject, so the safe value in the absence of evidence is the one that rejects nothing. Returning a large value instead would quietly delete every glyph of every rare character, since those are exactly the characters without enough examples to have a reference.

6.34 PriorFilter

```
def PriorFilter(variants, ch, prior, keepFrac=0.7, minKeep=3)
```

Inputs

`variants`
: this writer's glyphs for one character

`ch`
: that character

`prior`
: the mapping of reference descriptors

`keepFrac`
: the fraction to keep, default 0.7

`minKeep`
: the least number to keep, default 3

Output. the kept variants, sorted best first.

What it does Sorts this writer's variants of one character by their distance from the cross-writer reference and keeps the best 70 percent. The worst 30 percent are the ones least like the letter is supposed to look, which is where the mis-cuts that survived the geometric gates are concentrated.

Why a proportion rather than a threshold A fixed distance threshold would be wrong per writer, for exactly the reason given for the alignment confidence percentile. A writer whose hand is unusual sits further from the cross-writer reference in every variant, so a threshold tuned on a conventional hand would reject that writer's letters wholesale. Keeping a proportion asks which of this writer's own variants are the least typical, which is a question about them rather than about the population, and every writer therefore keeps their best seven tenths.

The floor, and the abstention At least three variants are kept whatever the proportion says, and the filter does nothing at all when fewer than three exist or when no reference is available for the character. Three is the smallest number from which the consensus filter downstream can work and from which a prototype can be formed, so dropping below it costs more than the bad variant does. For a rare character, having three imperfect examples is better than having one perfect one, because synthesis needs something to choose among and a prototype needs several shapes to average.

The variants come back sorted best first, which is not incidental. Later stages take the first few, and the ranking established here is part of what determines which of a writer's shapes actually get drawn.

6.35 _ConsensusFilter

```
def _ConsensusFilter(variants, keepFrac=0.75)
```

Inputs

`variants`
: this writer's glyphs for one character

`keepFrac`
: the fraction to keep, default 0.75

Output. the kept variants, ordered by how typical they are.

What it adds that the prior filter cannot The prior filter compares a writer against everyone else, which catches a shape that is not really the letter. This one compares a writer against themselves, which catches a shape that is not really *their* letter. The two find different faults. A glyph carrying half of its neighbour may still resemble the general idea of the letter closely enough to pass the prior, while differing clearly from the writer's own twenty other examples, because those twenty agree with each other and it does not.

Background: the medoid The most typical member must be identified before the outliers can be found. A mean cannot serve, because the mean of several descriptors is not itself any real glyph and, being an average, is pulled by the outliers. Instead the **medoid** is used: the member of the set whose total distance to all the others is least. It is an actual glyph, and it is the multidimensional analogue of the median, with the same robustness.

$$m \;=\; \arg\min_{i} \sum_{j} \left\| \mathbf{v}^{(i)} - \mathbf{v}^{(j)} \right\|_1, \qquad \left\| \mathbf{a} - \mathbf{b} \right\|_1 \;=\; \sum_{k} \left| a_k - b_k \right| \tag{6.14}$$

where

m the index of the medoid variant

$\mathbf{v}^{(i)}$ the descriptor of variant i

$\| \cdot \|_1$
: the **L1 distance**, or Manhattan distance: the sum of the absolute differences, as opposed to the square root of the sum of squares

k which descriptor position

$\arg\min_i$
: the variant for which that total is smallest

The L1 distance is used rather than the ordinary straight-line distance because squaring amplifies large differences, so a single badly disagreeing cell would dominate the comparison. Summing absolute differences treats every cell's disagreement in proportion to its size, which is the behaviour wanted when the question is overall typicality rather than worst-case mismatch. It is the same robustness argument that favours the median over the mean.

The filter itself With the medoid identified, the variants are ordered by distance from it and the furthest quarter is dropped. Ordering by distance from the most typical member means the kept variants are the ones that agree with the writer's own consensus, and the ordering also survives into the output, so later stages taking the first few get the most typical ones.

Why both filters are run, in this order The prior filter runs first and the consensus filter second. The order matters because the consensus filter trusts the set it is given: it finds the most typical member of whatever it receives. If most of the variants were mis-cuts, the medoid would be a mis-cut and the filter would discard the correct letters as outliers. Running the prior filter first removes the glyphs that are not the letter at all, using an external reference that cannot be corrupted by this writer's cuts, so the consensus filter operates on a set in which the majority really is correct, which is the condition it needs.

> *The distance matrix is computed between every pair, which is quadratic in the number of variants. With at most a few dozen variants per character that is trivial, and the whole matrix is wanted anyway, since the medoid is defined by row sums of exactly that matrix.*

6.36 _ResampleN

```
def _ResampleN(poly, n)
```

Inputs

`poly`
: a polyline

`n`
: how many points to produce

Output. an array of n points evenly spaced along the path.

What it is for Averaging several writings of a letter requires adding corresponding points together, which requires that they correspond. Traced strokes have no correspondence at all: one may have 40 points and another 55, placed wherever the pixel grid and the simplification happened to put them. Resampling puts every stroke on the same footing, n points spaced equally along its length, so that the k-th point of one stroke and the k-th of another both sit at the same fraction of the way along.

How it works: arc-length parameterisation The cumulative distance along the path is computed, giving for each original point how far along the path it sits. That turns the path into a function of distance, and the new points are then read off at n equally spaced distances by linear interpolation between the original points.

$$s_i \;=\; \sum_{j<i} \left\| \mathbf{p}_{j+1} - \mathbf{p}_j \right\|, \qquad t_k \;=\; \frac{k}{n-1}\, s_{\text{last}}, \qquad k = 0, \ldots, n-1 \tag{6.15}$$

where

s_i
: the cumulative distance from the start of the path to its i-th point

$\mathbf{p}_j$ the j-th original point

$\|\cdot\|$ the straight-line distance between two consecutive points

t_k the distance at which the k-th new point is placed

s_{last} the total path length

n how many points are wanted

This is called **arc-length parameterisation**, and the alternative makes clear why it is necessary. Spacing the new points evenly by index rather than by distance would place them densely wherever the original happened to be dense, so two strokes resampled that way would still not correspond. Distance is the only parameterisation that both strokes share.

> *A path of zero length, which a dot recovery can produce, returns n copies of its single point. That is the correct degenerate answer and it keeps the array shape uniform, so the averaging that follows needs no special case.*

6.37 _AlignStroke

```
def _AlignStroke(s, N)
```

Inputs

`s` one stroke

`N` the number of points to resample to

Output. the stroke resampled to N points with a consistent starting point and direction, or nothing if it is too short.

The problem beyond resampling Resampling makes two strokes have the same number of points, but corresponding points still need to be at corresponding places, and two things can spoil that. A stroke may be traced in either direction, so one writing of a letter may run left to right and another right to left, in which case point 0 of one corresponds to point N−1 of the other. And a closed loop has no inherent starting point at all, so two tracings of the same o may begin anywhere on the circle.

The two fixes

- **Open strokes are oriented left to right.** If the stroke ends left of where it began, it is reversed. Left to right is chosen because it is the direction of writing, so it is the orientation most strokes already have.
- **Closed strokes are rotated to begin at their leftmost point.** A stroke is recognised as closed when its two ends come within 0.3 x-heights of each other. The points are then rotated so the leftmost becomes first, and the original start is appended to close the loop. The leftmost point is used because it is well defined for any loop and is found identically in every tracing of it.

Both fixes work by choosing a canonical form: a rule that picks, from among the several equivalent ways of representing the same stroke, one particular way that depends only on the shape. Any such rule works provided every stroke obeys the same one, which is the real requirement.

Without these fixes the averaging would not merely be inaccurate; it would be nonsense. Averaging a stroke with a reversed copy of itself yields a shape collapsing toward its own midpoint, since the first point of one is averaged with the last of the other. The prototype would be a smear through the middle of the letter.

6.38 _DenoisedPrototype

```
def _DenoisedPrototype(variants, minN=3, N=44)
```

Inputs

`variants`
: this writer's kept glyphs for one character

`minN`
: the least variants needed, default 3

`N`
: the number of points per stroke, default 44

Output. one clean letterform as a list of strokes, or nothing if no reliable prototype can be formed.

What it is for Every individual variant carries noise: the staircase left by the pixel grid, a boundary placed a little wrongly, a tremor in the hand. Those errors differ between variants while the letter does not, so averaging several writings cancels the noise and leaves the shape the writer meant. The result is the **prototype**, the cleanest available statement of how this writer forms this letter.

Step one: group by stroke count Only variants with the same number of strokes can be averaged, because the averaging matches strokes one to one. The variants are grouped by stroke count and the largest group is used. Taking the largest rather than any particular count lets the data decide how the writer usually forms the letter: if most of their a's are one stroke, the prototype is a one-stroke a.

A group with fewer than three members, or with more than five strokes, is refused outright. Three is the least from which an average means anything. The five-stroke limit guards against averaging glyphs whose stroke counts are high because the tracing fragmented them, where the stroke-to-stroke matching would be matching pieces that do not correspond.

Step two: match strokes between variants Within the group, each variant's strokes are sorted by their leftmost point and matched by position in that order, then aligned and resampled to a common 44 points. Sorting by position is a weak way to establish correspondence, but a sound one for this case: the strokes of one letter written by one person occur in a consistent left-to-right arrangement, so the leftmost stroke of one a is the same stroke as the leftmost of another.

Step three: iterative alignment to the median This is the step that distinguishes the method from a plain average. The variants are not yet aligned horizontally, because each was cut at a slightly different boundary, so one may sit a tenth of an x-height right of another. Averaging them in that state blurs the prototype horizontally. So each variant is shifted along

x to whichever offset, among nine candidates spanning plus and minus 0.22 x-heights, brings it closest to the current median, and the median is then recomputed from the shifted variants. That is repeated twice.

$$\delta_c^* \;=\; \arg\min_{\delta} \; \frac{1}{N} \sum_{k=1}^{N} \Big\| \big(\mathbf{c}_k + \delta\,\hat{\mathbf{x}}\big) - \mathbf{m}_k \Big\| \tag{6.16}$$

where

δ_c^* the best horizontal shift for variant c

$\mathbf{c}_k$ the k-th point of that variant's stroke

$\mathbf{m}_k$ the k-th point of the current median stroke

$\hat{\mathbf{x}}$ the unit vector along the horizontal axis, so the shift moves x only

δ a candidate shift, one of nine values from -0.22 to $+0.22$ x-heights

N the number of points per stroke, 44

The alternation of aligning to an estimate and then re-forming the estimate is the general shape of an **iterative refinement**, the same pattern as the classical algorithms for registering point sets. It is not guaranteed to find the best possible joint alignment, but each step cannot make the agreement worse, and two rounds are enough here because the misalignment being corrected is small and the first round removes most of it.

Step four: reject the worst fifth, then take the median After alignment, each variant's mean deviation from the median is computed, and the worst 20 percent are dropped before the final median is taken. This is a second line of defence: a variant that survived every earlier gate but is still wrong will disagree with the aligned consensus, and here it is identified by that disagreement alone, with no reference to what kind of defect it has.

The median, not the mean, at every stage. Its robustness is exactly what is wanted: a prototype must not be bent toward a single bad variant, and with the median one outlier among several has no influence at all on the result rather than a proportional one.

> *Returning nothing is common and is not a failure. A character with few examples, or whose examples disagree about stroke count, has no reliable prototype, and inventing one from two variants would produce a shape less trustworthy than either. The caller falls back on the writer's actual variants, which is the right behaviour: a real if imperfect letterform beats a confidently wrong average.*

6.39 SmoothStroke

```
def SmoothStroke(pts, step=0.055, passes=2)
```

Inputs

`pts` a stroke

`step`
: the spacing to resample to, in x-heights, default 0.055

`passes`
: how many smoothing passes, default 2

Output. the stroke resampled evenly and smoothed, as rounded coordinates.

What it is for A traced stroke still carries the pixel staircase: it was assembled from whole-pixel positions, so it zigzags at the scale of one pixel even where the writer's stroke was smooth. Drawn by a plotter that staircase is visible, and it also misrepresents the writing, since the writer drew a curve. This function removes it.

Step one: resample evenly The stroke is first resampled to points about 0.055 x-heights apart. Even spacing is required before smoothing, because the smoothing averages each point with its neighbours by index, and if the points are unevenly spaced then that averaging reaches different distances in different places, smoothing the sparse parts far more than the dense ones.

Step two: the smoothing pass

$$q'_k \;=\; 0.25\,q_{k-1} \;+\; 0.5\,q_k \;+\; 0.25\,q_{k+1} \tag{6.17}$$

where

q'_k the new position of the k-th point

q_k its current position

q_{k-1}, q_{k+1}
: its neighbours along the stroke

0.25, 0.5, 0.25
: the weights, which sum to 1 so the stroke is not scaled or shifted

This is a convolution along the stroke with a three-point kernel, the same operation as the profile smoothing in LinePitch but applied to a path rather than a signal. Weights summing to 1 are what guarantee the stroke keeps its position and size; weights that summed to less would shrink it toward the origin a little with every pass.

Two passes are applied. Applying the same small kernel repeatedly is equivalent to one wider kernel, and it is the cheaper way to get a gentle wide smoothing, since each pass costs three multiplications per point. Two passes of this kernel remove the one-pixel zigzag while leaving the curvature of a letter, which acts over many points, essentially untouched.

Why the endpoints are held fixed Only the interior points are smoothed; the first and last are kept exactly. The endpoints carry meaning that the interior does not: they are where the pen entered and left, they are what a join is drawn to, and they define the glyph's extent. Smoothing them would pull them inward, shortening every stroke slightly with each pass and, worse, moving the attachment points that synthesis relies on.

> *Coordinates are rounded to three decimal places on output. At a scale where 1 is the x-height, three decimals is a thousandth of a letter's height, far finer than any plotter can draw or any descriptor can distinguish. The rounding roughly halves the size of the stored profile, which runs to thousands of strokes, at no cost in fidelity.*

6.40 RenderRefStats

```
def RenderRefStats(gray)
```

Inputs

`gray`
: one real line crop from this writer

Output. a dictionary of measurements describing how that line looks, both to the networks and geometrically.

What it is for, and the idea behind it Reproducing a hand is not only a matter of letter shapes. The output must also *look* like a photograph of handwriting to the networks that judge it, and that involves properties no glyph library captures: how much of the image is ink, how dark the ink is, how bright the paper is, the proportions of the line. This function measures those on a real line, so that synthesis has a target to match rather than having to guess.

Background: domain shift **Domain shift** is the name for the situation where a model is used on data drawn from a different distribution than it was trained on. A network trained on photographs of handwriting has learned the statistics of photographs: paper that is not pure white, ink that is not pure black, and soft grey edges where the two meet. Rendering a synthetic line as pure black on pure white produces an image that is correct as geometry but statistically unlike anything in training, and the network's response to it is unreliable, which would corrupt the judgement of the output.

This is why the paper and ink levels are measured as percentiles of the real crop rather than assumed: the median of the bright pixels is what this writer's paper actually is, and the median of the dark pixels is what their ink actually is. The fraction of intermediate greys is measured too, since that describes the softness of the stroke edges.

Why the word gap is measured from the ink, not the recogniser The width of the space character as the aligner reports it is not a reliable measure of how widely this writer spaces their words. CTC concentrates its output and tends to assign a space a narrow territory, well inside the actual gap. So the gap is measured directly instead: the columns with no ink are found, the runs between inked columns are collected, those under half an x-height are discarded as being gaps within a word, and the median of the rest is taken.

The half-x-height threshold is what separates the two kinds of gap. Within a word the pen may lift, leaving a small gap, but a word boundary is a deliberate space and is substantially wider. The median of the wide gaps therefore describes word spacing, and the median again rather than the mean, so that one unusually wide gap at a paragraph end does not inflate it.

Why slant, ascender and descender are measured here too These are also measured in the profile from the glyphs themselves, so measuring them again on the rendered line looks redundant. It is not, and the comment in the code gives the reason: these are measured *the same way* they will be measured on the synthesised output. A quantity measured by two different methods is not comparable with itself, so to check whether the synthesis reproduced the writer's slant, both sides must be measured by the same procedure. These values exist to be the reference half of that comparison.

Fields that cannot be measured are filled with a not-a-number value rather than a plausible default. That is deliberate: the caller takes a median across lines with those values excluded, so a line that could not be measured contributes nothing, whereas a default would contribute a fabricated measurement that is indistinguishable from a real one.

6.41 _MedianWordGap

```
def _MedianWordGap(refs, spaceAdvs)
```

Inputs

`refs`
: the per-line reference statistics

`spaceAdvs`
: the widths the aligner gave to space characters

Output. the typical gap this writer leaves between words, in x-heights.

The preference order Three sources are tried in order. The measured ink gaps from RenderRefStats are preferred, because they measure the real thing. If none is available, the aligner's space widths are used, which are biased narrow for the reason given above but are at least derived from this writer. If there are no spaces at all, 1.2 x-heights is returned as a generic value.

Why the fallback order is this way round It descends from the most direct evidence to the least. The same cascade pattern appears in LinePitch and in the checkpoint selection: try the best-informed method, fall back only when it cannot apply. The consequence worth noting is that the fallback is known to be biased, so a profile built from a writer with no measurable gaps will have word spacing that is too narrow, and that is a better failure than having none at all.

Not-a-number values are dropped by testing whether each value equals itself, which is false only for not-a-number. That is the standard idiom and it works without importing anything, which is why it is used in a file that avoids dependencies.

6.42 ExtractAuthorRaw

```
def ExtractAuthorRaw(authorId, lineItems, model)
```

Inputs

`authorId`
: which writer these lines belong to

`lineItems`
: pairs of a line crop and its known text

`model`
: a loaded reader for alignment

Output. a list of per-line results, each a pair of the glyphs cut from that line and its statistics.

What it is for Runs the cutter over every line of one writer and collects the results. It is the expensive half of profile building, and the half that gets cached.

Why the output is deliberately unfiltered Nothing is rejected here. Every glyph the cutter produced is kept, including the fragments and the over-wide cuts. This is the design decision that makes the filters tunable: the filtering happens in BuildAuthorProfile, which runs on this output, so a threshold can be changed and the effect measured in seconds against a cache that is still valid. Had filtering happened here, every change to a filter would require re-reading and re-cutting every page.

It is also what made the project's rejection statistics possible. Reporting what fraction of cuts were discarded, and why, requires having kept the discarded ones; a pipeline that filtered as it extracted could report only what survived.

> *A line whose extraction returned nothing is skipped rather than recorded as empty. Those are the lines where alignment failed outright or no x-height band could be found, which carry no glyphs and no usable statistics, so an entry for them would only have to be filtered out again by every consumer.*

6.43 BuildAuthorProfile

```
def BuildAuthorProfile(authorId, parsed, refs=None, prior=None, verbose=
    True)
```

Inputs

`authorId`
: which writer this profile is for

`parsed`
: the raw per-line extraction from ExtractAuthorRaw

`refs`
: the per-line reference statistics, taken from the extraction if not supplied

`prior`
: the cross-writer reference descriptors

`verbose`
: whether to print a one-line summary, default true

Output. the finished style profile as a dictionary, or nothing if the writer has too little usable data.

What it is for The second half of profile building, and the cheap one. It takes everything the cutter produced, discards what is not a letter, keeps the most typical examples of what remains, builds fallback letterforms, and measures the hand. Its output is the profile that synthesise.py consumes.

Stage one: discard the badly aligned lines Lines in the worst quarter by alignment confidence are dropped before anything else, for the reasons set out under ALIGN_CONF_PCT. This is first because it is the only filter that acts on whole lines: a line whose alignment failed has every boundary wrong, so its glyphs are unreliable as a set, and judging them individually afterwards would be more work for a worse result.

Stage two: the gates, cheapest first Every surviving glyph passes through a series of tests, each of which can only reject. The order is by cost, so that the expensive shape comparisons run on the fewest glyphs.

- **Extent.** Width within bounds, which depends on the character; height over 0.25 x-heights; no more than six strokes. Three comparisons on numbers already stored.
- **Height class.** Does the ink reach where this letter should reach, from _ClassOk.
- **Cell overflow.** The ink must not be much wider than the cell it was cut from: lead plus width at most 1.45 times the advance plus 0.25. Ink spilling well outside its own cell means the cut took part of a neighbour, which is a different and more sensitive test than the absolute width limit.
- **Pen path.** Total path above the per-character floor, and longest single stroke above 0.55 x-heights.
- **Size consistency.** Width between 0.6 and 1.5 times the 40th percentile of the surviving widths, and height between 0.55 and 1.7 times the median height. Applied only if at least three variants survive it.
- **Cross-writer prior,** then **internal consensus.** The two descriptor-based filters, in that order, for the reason given under _ConsensusFilter.

Why the 40th percentile and not the median The reference width for the consistency test is taken at the 40th percentile rather than the middle. The reason is specific: a doubled letter, where the cut took two copies, is wide, and if such cuts are common for a character they push the median up and the test then accepts them while rejecting the correct narrow ones. Anchoring a little below the middle biases the reference toward the narrower, more likely correct, population.

Stage three: the two fallback letterforms Two single letterforms are built per character, and they are different things built for different purposes.

- **idealMedoid:** the one real variant most typical of this writer's own set, found as the medoid of the descriptors. It is genuine writing, with all the irregularity that implies.
- **idealGlyphs:** the denoised prototype, the per-point median of several aligned variants. It is cleaner than any real variant but is an average, so it is not something the writer actually wrote.

Keeping both lets the choice be made according to what is needed: the medoid when authenticity matters, the prototype when legibility does. That trade-off between resembling the writer and being readable is the central tension of the whole project, and having both forms available is what lets synthesis navigate it.

When the prototype is promoted to a variant

$$\text{promote} \iff D_{\text{proto}} < D_{\text{raw}} - 0.02 \quad \text{and} \quad D_{\text{proto}} < 0.32 \tag{6.18}$$

where

D_{proto}
: the prior distance of the denoised prototype

D_{raw}
: the median prior distance of this character's kept variants

0.02
: the margin by which the prototype must beat them, so a tie does not count

0.32
: an absolute ceiling, so a prototype that is merely better than a bad set is still not accepted

$\iff$
: if and only if: the promotion happens exactly when both conditions hold

The prototype is placed at the front of the variant list, so that it is drawn in preference, only when it is measurably closer to the cross-writer idea of the letter than the writer's own variants are. Both conditions matter. The margin prevents promotion on a difference within noise. The absolute ceiling prevents the comparison being won by default when every variant is poor, which is exactly the situation where an averaged shape is least trustworthy, since averaging several bad variants gives a clean rendering of a bad shape.

Stage four: measuring the hand

- **slantDeg,** the median per-line slant: how far the writing leans.
- **xHeightPx,** the median x-height in pixels, which records the scale the profile was built at.
- **strokeWidthXh,** ink area divided by skeleton length, which is the average stroke thickness, expressed in x-heights.
- **ascender and descender,** how far above and below the band this writer's letters actually reach, measured only from glyphs that clearly do reach, so that a mis-cut flat l cannot pull the ascender figure down.
- **wordSpaceXh,** the gap left between words.
- **connectedness,** the fraction of adjacent in-word letter pairs where ink genuinely crossed the boundary.
- **letterAdvance,** the median advance per character, which preserves this writer's own letter spacing per letter rather than one global figure.

Why connectedness is measured this way

$$\text{connectedness} = \frac{\left|\{\text{adjacent in-word pairs with ink crossing the cut}\}\right|}{\left|\{\text{adjacent in-word pairs}\}\right|} \tag{6.19}$$

where

connectedness
: a number from 0 for fully separated printing to 1 for fully joined cursive

`adjacent in-word pairs`
: pairs of neighbouring characters where both were cut successfully and neither is a space

The numerator counts the pairs where a traced centre line actually passed through the boundary, which means the pen was never lifted there. That is the honest definition, as opposed to inferring joining from the gap between letters, which confuses tight spacing with genuine connection. Spaces are excluded because letters either side of a space are not expected to join and including them would make the measure depend on how many words the sample happened to contain.

This single number turns out to be the most useful descriptor of a hand in the whole profile. Synthesis uses it to decide how to join letters, and it is the figure this project's results use to separate print-like hands from cursive ones, since reproduction accuracy falls as it rises.

The two ways the function declines to produce a profile Nothing is returned when there are no parsed lines at all, and again when no line survived the confidence cut with a measurable x-height. Returning nothing rather than a profile with empty fields is the right behaviour, because every consumer of a profile assumes its measurements are real, and a profile full of defaults would be used confidently and produce output in nobody's hand.

> *The verbose summary printed at the end reports the line count, character count, variant count, slant, connectedness and reach. Those six numbers are enough to recognise a failed build at a glance: a writer with far fewer variants than usual has had most of their cuts rejected, and a connectedness of 0 or 1 exactly is almost always a sign that the crossing detection has gone wrong rather than a real hand.*

7 synthesise.py

This file does the reverse of style.py. Given a style profile and some text, it produces a pen path: an ordered list of strokes in millimetres that a plotter can draw, written in the hand the profile describes.

The central tension of the whole project Two goals pull against each other here, and most of the design is a response to that. The output should **look like the writer**, which argues for using their own stored letterforms exactly as extracted. And the output should **be readable**, which argues for clean, well-formed letters. These conflict because the stored letterforms are not clean: they were cut from joined-up writing at boundaries that often do not exist, so a proportion of them are fragments or carry part of a neighbour. Drawing those faithfully reproduces the writer's hand and some of the extraction's mistakes along with it.

The file resolves this with a single number, called the **legibility lambda** and written as lam, running from 0 to 1. At 0 the writer's own shapes are always used. At 1 they are abandoned entirely in favour of a fallback letterform. In between, each glyph is swapped with a probability that depends both on lam and on how far that particular shape sits from what the letter should look like. The value is derived automatically per writer, so a hand whose library extracted cleanly keeps its own shapes and one whose library is mostly fragments does not.

The three sources a letter can come from

- **own:** a variant cut from this writer's own pages. The most faithful and the least reliable.
- **core, the anchor:** a single clean letterform. Preferably the writer's own most typical variant of that letter, falling back to a built-in alphabet when even that is too rough.
- **fallback:** the built-in alphabet directly, used when the writer has no example of a character at all.

Every rendered glyph records which source it came from, and the counts are returned in the trajectory's metadata. That is what makes the trade-off measurable rather than a matter of impression: a page drawn mostly from the anchor is readable but is not really the writer's hand, and the counts say which happened.

Units Glyphs arrive in x-heights, as stored. Layout converts them to millimetres using the requested x-height size, and the rasteriser converts millimetres to pixels. So three coordinate systems appear in this file, in that order, and the conversions happen at exactly two points.

7.1 SCRIPT_DIR

```
SCRIPT_DIR = Path(__file__).resolve().parent
```

Output. the directory this file sits in.

What it is for Records the module's own location, from which a path to data beside the code could be built. The project's paths are otherwise defined centrally in paths.py, which is the better practice, since one definition serves the whole project.

> *This constant is assigned and never read anywhere in the module. It is a remnant of an earlier layout, before the paths module existed. It is harmless, but a reader should not take its presence as evidence that the file locates its data this way.*

7.2 CNN_DATA_DIR

```
CNN_DATA_DIR = Path(__file__).resolve().parent.parent / 'CNN'
```

Output. a path to a sibling directory of network data.

What it is for Points at a folder beside the pipeline holding network-related data, built by stepping up from this file's own location.

> *Like the constant above, this is assigned and never read. Both are left in place rather than removed because removing a name is a change that has to be checked against every caller, and neither does any harm; but neither describes how the module actually finds anything.*

7.3 _FB

```
_FB = {'a': [[(0.75,0.75),(0.45,0.95),(0.15,0.75),(0.1,0.4),(0.35,0.05)
    ,(0.65,0.1),
         (0.78,0.35),(0.78,0.0)],[(0.78,0.3),(0.95,0.0)]],'b': [[(0.1,1.8)
             ,(0.1,
         0.0)],[(0.1,0.55),(0.35,0.85),(0.7,0.7),(0.75,0.3),(0.5,0.0)
             ,(0.15,0.1)]],
         'c': [[(0.8,0.75),(0.5,0.95),...}
```

Output. a dictionary mapping a character to a built-in printed letterform, as a list of strokes in x-heights.

What it is and why it exists A hand-written alphabet of 47 characters: the 26 lowercase letters, the ten digits, and eleven punctuation marks, each given as one or more strokes of points in the same x-height convention the extracted glyphs use. These are the letterforms of last resort.

Three situations require them. A writer may have no example of a character at all, which is common: a page of ordinary English contains no q, z or semicolon. A writer's examples of some character may all have been rejected by the filters. And the legibility mechanism may decide a stored shape is too far from a letter to be drawn, in which case something must take its place.

Why they are written out by hand They could not be obtained any other way. A font is not usable: a font describes letters as filled outlines, and what is needed here is the path a pen travels along the centre of a stroke, which is a different object. Deriving centre lines from a font by thinning would reintroduce every artefact that the extraction pipeline exists to deal with, for letters that are supposed to be the clean reference. So the coordinates were laid out directly, in the same coordinate system as the stored glyphs, so that a fallback letter can be substituted for an extracted one with no conversion at all.

The coordinate convention A point of (0.4, 0.7) means four tenths of an x-height to the right of the glyph's origin and seven tenths of an x-height above the baseline. A y above 1.0 is in the ascender zone and a negative y is below the baseline, in the descender zone. Being in the same convention is what allows _BaselineGlyph to stretch these shapes to a particular writer's reach by rescaling only the parts outside the band.

> *There are no uppercase letters in this dictionary. A capital is handled instead by taking the lowercase form and scaling it up to the writer's ascender height, which is what _GlyphSource does under the label* `case`*. That is correct for letters whose capital is a larger version of the lowercase, such as C, O, S and W, and is a visible approximation for the ones where it is not, such as A, E and R.*

7.4 _CURSIVE

```
_CURSIVE = {'a': [[(0.0,0.3),(0.2,0.62),(0.46,0.72),(0.62,0.52)
    ,(0.6,0.18),(0.42,
         0.02),(0.2,0.1),(0.16,0.34),(0.3,0.52),(0.52,0.46),(0.62,0.2)
             ,(0.64,0.02),
         (0.86,0.3)]],'b': [[(0.0,0.3),(0.14,1.1),(0.24,1.66),(0.3,1.2)
             ,(0.26,0.44),
         (0.34,0.1),(0.56,0.04),(0.7,...}
```

Output. a dictionary mapping each lowercase letter to a joined-up letterform.

Why a second alphabet is needed The printed alphabet above is wrong for a cursive hand, and not merely in appearance. A printed letterform begins and ends wherever is convenient, often at the top, and has no provision for a stroke arriving from the previous letter or leaving toward the next. Substituting one into joined writing produces a letter that cannot be connected: the join from the preceding letter has nowhere to attach, so either it is omitted, leaving a visible break in the middle of a word, or it is drawn to a point on the letter that no writer would join to.

These forms are drawn to enter near the baseline on the left and leave near the baseline on the right, which is what a cursive letter does, so a ligature can be drawn to and from them naturally.

Which alphabet is chosen The choice is made per writer, from their measured connectedness: a hand at or above 0.5 gets the cursive forms and one below gets the printed ones. Using a single number measured from their own writing, rather than a setting, means the decision follows the evidence; see CORE_CURSIVE_CONN.

> *This alphabet holds only the 26 lowercase letters, where the printed one also holds digits and punctuation. That is sufficient because joining only ever applies between two adjacent alphabetic characters, so a digit or a comma never needs a joinable form. Where a cursive writer needs one of those, the lookup falls through to the printed alphabet.*

7.5 _FB_ADV

```
_FB_ADV = 1.0
```

Output. 1.0, the advance after a fallback letter, in x-heights.

What an advance is The **advance** is how far the pen origin moves to the right after drawing a character, before the next one is placed. It is not the same as the character's width: it includes the space that belongs between this letter and the next, so advancing by the width alone would butt every letter against its neighbour.

For extracted glyphs the advance is measured from the cell the character was cut from, which is what preserves the writer's own spacing. A built-in letterform has no such measurement, so this constant supplies one: one x-height, which is approximately the advance of a lowercase letter in ordinary handwriting.

> *It is used only when the writer's profile has no recorded advance for the character either. Where the profile does have one, even though the shape is a fallback, the writer's own figure is preferred, so the spacing stays theirs even when the letterform is not.*

7.6 _SIMILAR

```
_SIMILAR = {'I': 'l','O': '0','l': 'I','0': 'O','o': '0',';': ':','"':
    "'",'!': 'l'}
```

Output. a mapping from a character to another whose letterform can stand in for it.

What it is for Some characters are near-identical in handwriting, so one writer's example of the one can serve for the other. A capital I and a lowercase l are the same vertical stroke in most hands. A capital O and a zero differ by convention rather than by shape. A semicolon is a colon with a tail, an opening quote is a pair of apostrophes, and an exclamation mark is a vertical stroke with a dot.

Where it sits in the order of preference It is consulted after the writer's own examples of the character itself and after the opposite case of it, and before resorting to the anchor or the built-in alphabet. The ordering reflects how much of the writer's own hand each option preserves: their own l really is their own stroke, so borrowing it for an I is closer to their writing than substituting a designed letterform would be.

> *The mapping is deliberately not symmetric in every case. An exclamation mark may borrow an l, but an l may not borrow an exclamation mark, because the dot would come with it. Substitution is only valid in the direction where nothing spurious is inherited.*

7.7 _BaselineGlyph

```
def _BaselineGlyph(ch, profile, printOnly=False)
```

Inputs

`ch` the character wanted

`profile`
: the writer's style profile

`printOnly`
: force the printed alphabet even for a cursive hand

Output. a pair of the strokes and the advance, scaled to this writer's reach; or nothing if the character is in neither alphabet.

What it is for Fetches a built-in letterform and adapts it to the writer. Without the adaptation a substituted letter would be visibly foreign: a generic l reaching 1.75 x-heights dropped into a hand whose own ascenders reach 2.4 looks short, and in a hand that reaches only 1.4 it towers over its neighbours. The letterform itself is generic; its proportions should not be.

Why only the zones outside the band are rescaled

$$y' = \begin{cases} 1 + (y-1)\,\max\!\left(0.35,\ \frac{\text{asc}-1}{0.75}\right) & y > 1 \\ y \cdot \max\!\left(0.35,\ \frac{|\text{desc}|}{0.55}\right) & y < 0 \\ y & 0 \le y \le 1 \end{cases} \tag{7.1}$$

where

y the built-in letterform's height at this point, in x-heights

y' the adapted height

asc this writer's measured ascender reach, in x-heights above the baseline

desc their measured descender reach, negative

0.75 the ascender reach above the band that the built-in forms were drawn with, so dividing by it gives a ratio of 1 for a writer matching them

0.55 the same for the descender zone

$\max(0.35, \cdot)$
a floor, so a writer with very little measured reach does not collapse the zone to nothing

The x-height band itself, from 0 to 1, is left untouched, and that is the point of the piecewise form. The band is the reference against which everything is measured, so it is 1 by definition and rescaling it would be meaningless. What varies between writers is how far they reach *beyond* the band, and those are exactly the two zones the formula adjusts. A letter like an o, entirely inside the band, is therefore unchanged, while an l is stretched or compressed only above y = 1, leaving its body the same size as its neighbours'.

Why there is a floor of 0.35 A writer whose measured ascender is close to 1.0, which happens when the measurement failed or when most of their ascender letters were rejected, would otherwise give a ratio near zero and flatten every ascender to the height of the band. An l indistinguishable from an e is worse than a slightly wrong l, so the floor keeps at least a third of the designed reach whatever the measurement says.

How the advance is derived The advance is the shape's own horizontal extent plus 0.12 x-heights of spacing, computed from the points rather than stored. Deriving it means the two alphabets need no advance table to be maintained alongside them, and the margin is added so that consecutive fallback letters do not touch.

7.8 UNIFORM_INK

```
UNIFORM_INK = True
```

Output. true, meaning every writer is drawn at the same pen width.

Why the ink is uniform, and why that is not a limitation The profile records the writer's own stroke width, so the renderer could vary the pen width per writer. By default it does not, and the reason is that the machine cannot. The plotter has one pen fitted in one holder; every stroke it draws is the same width. Rendering a writer with a thick pen when the machine will draw them with the standard one would produce an image unlike the thing being built.

That matters because the rendered image is not only for display: it is what the two judging networks score during the best-of-N search. A search that optimised against images the machine cannot produce would be optimising the wrong quantity. So the renderer is made to match the hardware, and the search then measures what will actually be drawn.

> *The non-uniform path is retained and is reachable by passing the flag, because it is the right choice when rendering a comparison against a real photographed page, where the real page does have the writer's own pen width. The default matches the machine; the option matches the camera.*

7.9 UNIFORM_PEN_WIDTH_XH

```
UNIFORM_PEN_WIDTH_XH = 0.085
```

Output. 0.085, the uniform pen width as a fraction of the x-height.

Why the width is relative to the x-height A width in pixels or millimetres would be right at one writing size only. Expressed as a fraction of the x-height it scales with the writing, so text drawn large and small both look like the same pen, which is what a real pen does.

Why it is thin The code comment records that this value was reduced from 0.11, and the reason is specific: a heavy stroke fills small loops. The enclosed white space inside an a, e or o is only a fraction of an x-height across, and a pen drawn at 0.11 closes it, turning an e into a c with a filled top and an a into a blob. Those are precisely the confusions that cost character accuracy, so the thinner pen is chosen to keep the counters open.

The trade-off is that a thinner stroke carries less ink, so a rendered line is lighter than a photographed one. That is addressed separately, by the soft edge below and by the ink density matching, rather than by thickening the pen.

7.10 UNIFORM_SOFT_FRAC

```
UNIFORM_SOFT_FRAC = 0.3
```

Output. 0.30, the blur radius as a fraction of the pen width.

Why a drawn stroke is blurred A rasterised stroke has a hard edge: a pixel is ink or paper with nothing in between. A photographed pen stroke does not. Ink spreads slightly into the paper, the camera's optics are not perfect, and the sensor averages over each pixel's area, so a real stroke edge is a gradient a pixel or two wide.

This is the domain-shift problem described under RenderRefStats. The networks judging the output were trained on photographs, so they have learned what a real stroke edge looks like. A hard-edged synthetic stroke is unlike anything in their training data, and their response to it is unreliable, which would corrupt the very scores the search is optimising. Blurring makes the rendered stroke statistically closer to a photographed one.

Why the radius is a fraction of the pen width Because the physical cause scales that way. The spread of ink and the softness of a stroke edge are roughly proportional to how much ink was laid down, so a thick stroke has a wider soft edge than a thin one. A fixed blur radius would over-soften a thin stroke, which at this pen width would close the small loops the thin pen was chosen to keep open, and under-soften a thick one.

7.11 JOIN_PROB_CAP

```
JOIN_PROB_CAP = 1.0
```

Output. 1.0, the greatest probability with which two letters may be joined.

What it would do Joining is decided per pair of letters by a random draw against the writer's connectedness, so a hand measured at 0.8 joins about four pairs in five. This constant caps that probability, so a value below 1 would force even a fully joined hand to leave some pairs unjoined.

Why it is set to 1, which disables it At 1.0 the cap never binds and the writer's measured connectedness is used as it stands. It is kept as a named constant rather than removed because it is the natural place to intervene if fully joined output proves unreadable, and because the alternative of editing the join condition itself is a change in behaviour hidden inside a long function. A constant at its neutral value documents an available adjustment.

> *Deciding each pair by a random draw, rather than by comparing connectedness against a threshold, is the substantive choice here. A threshold would make a hand either fully joined or fully separated, with nothing in between, whereas real writing of intermediate connectedness joins some pairs and not others. The draw reproduces that.*

7.12 CORE_SLANT_CAP_DEG

```
CORE_SLANT_CAP_DEG = 22.0
```

Output. 22.0, the greatest lean in degrees applied to a printed anchor letterform.

What it is for Every glyph is sheared to the writer's measured slant, so the page leans as they do. An anchor letterform is sheared too, because a perfectly upright substitute in a slanted hand is conspicuous. But it is sheared less, and this constant is the limit.

Why the anchor is treated differently The anchor exists to be readable; that is the whole reason it was substituted for the writer's own shape. Shearing degrades legibility, because a letterform drawn upright and then slanted is not the same as a letterform designed at that slant: the curves distort, loops narrow, and at a steep angle an o becomes a thin diagonal ellipse. Applying a 42-degree slant to the anchor would undo the substitution's purpose.

So the anchor keeps the writer's lean up to a point and then stops. Twenty-two degrees is a visible lean, enough that the substituted letter does not stand upright among slanted neighbours, while being well short of the angle at which the shear starts closing loops. A cursive anchor is allowed 30 degrees, since those forms were drawn to be joined and already lean.

> *This is one of several places where the anchor is deliberately held back from the writer's full style: it also receives less jitter and only a fraction of the baseline drift. The pattern is consistent, and the reasoning is always the same: style is applied to the anchor only as far as it does not cost the legibility that justified using it.*

7.13 CORE_JITTER_FRAC

```
CORE_JITTER_FRAC = 0.15
```

Output. 0.15, the fraction of the usual random variation applied to an anchor letterform.

What it is for Each glyph is nudged, rotated and scaled slightly at random so that repeated letters are not identical; see _Jitter. An anchor receives only 15 percent of that, so it barely wobbles.

Why The same reasoning as the slant cap. Jitter exists to make the output look handwritten rather than printed, which is a cosmetic goal, and it costs a little legibility, which is why the amount is small to begin with. An anchor was substituted precisely because legibility was at risk for that character, so it is the glyph that can least afford the cost. Keeping a small fraction rather than none at all means an anchor among its neighbours is not conspicuously rigid.

7.14 COLLISION_MIN_GAP_XH

```
COLLISION_MIN_GAP_XH = 0.05
```

Output. 0.05, the clear space forced between one glyph's ink and the next, in x-heights.

The problem it solves Placing glyphs by their advances assumes each glyph's ink fits inside the space its advance allots. That assumption fails regularly. A variant may be wider than the advance recorded for its character; the random scaling may enlarge it; the shear may push the top of a tall letter well to the right of its base. When it fails, the next glyph is drawn on top of the previous one, and two overlapping letters are usually unreadable as either.

How it is detected and fixed After a glyph is placed, its leftmost inked point is compared with the rightmost inked point of the previous glyph's body. If the gap is less than this constant, the pen is pushed right by the shortfall and the glyph is placed again at the new position.

$$\text{need} \;=\; x_{\max}^{\text{prev}} \;+\; 0.05\,\text{xh} \;-\; x_{\min}^{\text{cur}} \tag{7.2}$$

where

need
: how far the current glyph must be pushed right; nothing is done when it is not positive

$x_{\max}^{\text{prev}}$
: the rightmost inked point of the previous glyph's body stroke

$x_{\min}^{\text{cur}}$
: the leftmost inked point of the glyph just placed

xh
: the x-height in millimetres, converting the constant into the working units

Measuring actual ink positions rather than advances is what makes this work: it tests the thing that matters, where the ink really landed, after every transformation has been applied. A check based on advances could not see a collision caused by the shear, since the shear does not change the advance.

Why the correction only ever pushes right Writing proceeds left to right and the previous glyph has already been committed, so the only available remedy is to move the current glyph on. Pulling it left would reintroduce the collision, and moving the previous one would require reconsidering everything before it. The cost of pushing right is a slightly wider than intended gap, which is a far smaller defect than an overlap.

7.15 _ELIGIBLE_VARIANTS

```
_ELIGIBLE_VARIANTS = 5
```

Output. 5, how many of a character's stored variants may be chosen from.

The problem it solves Up to twelve variants of a letter are stored, and it might seem that drawing from all of them would give the most natural variety. It does the opposite, and the reason is worth stating carefully. A person forms a given letter fairly consistently: their a's differ in detail but share a shape. The stored variants are ranked by how close each sits to what the letter should look like, so the first few are that consistent shape and the tail is the unusual ones, including the mis-cuts that survived filtering.

Sampling evenly over all twelve therefore draws from the tail as often as from the characteristic shape. The output varies more, but what it varies over is no longer the writer's hand: it is a mixture of their hand and the extraction's errors. The result reads as scrambled rather than natural, and the writer identification drops, because the signature being averaged away is exactly what the namer recognises.

Why five Enough variants that repeated letters in a line are visibly different, which one or two would not achieve, and few enough that all of them are drawn from the well-formed part of the ranking. The selection among those five is itself weighted toward the first, so even within the eligible set the most typical shape is drawn most often.

This cap and the storage cap of twelve interact in a way worth knowing: variants ranked sixth and below are stored but can never be drawn. For this project's profiles that is a substantial fraction of everything stored. The storage is not wasted, since the whole set is what the medoid and the prototype are computed from, but raising the storage cap alone would not produce more variety in the output.

7.16 CORE_CURSIVE_CONN

```
CORE_CURSIVE_CONN = 0.5
```

Output. 0.5, the connectedness at or above which a hand is treated as cursive.

What it decides Which of the two built-in alphabets a writer's anchor letters come from, and whether an anchor may be joined to its neighbours at all. A hand at or above 0.5 gets the cursive forms and may join them; below, the printed forms, which never join.

Why the midpoint Connectedness is the fraction of adjacent in-word letter pairs that were genuinely joined, so 0.5 means half. That is the natural place to divide, and it is not a tuned value: a hand that joins more pairs than it separates is cursive in the ordinary sense of the word, and one that separates more is not.

Why a threshold is acceptable here, unlike for joining A hard threshold was rejected for deciding whether to join a given pair, because real writing of intermediate connectedness joins some pairs and not others, which a threshold cannot express. Here a threshold is right, because the thing being chosen is which alphabet to draw from, and there are only two alphabets. There is no meaningful way to be half-way between the printed form of an a and the cursive one; a choice must be made, and the measured majority behaviour is the sound basis for making it.

7.17 JOIN_LEGIBILITY_COUPLING

```
JOIN_LEGIBILITY_COUPLING = 0.0
```

Output. 0.0, how much the legibility setting reduces joining.

What it would do Joins are a major cause of illegibility in synthetic cursive: a ligature drawn to the wrong point, or several in succession, turns a word into a tangle. This constant would let the legibility setting thin the joins out, so that a writer whose library is poor would also be drawn with fewer joins.

$$\mathrm{conn}_{\mathrm{eff}} \;=\; \mathrm{conn} \cdot \big(1 - \kappa \cdot \mathrm{lam}\big) \tag{7.3}$$

where

$\mathrm{conn}_{\mathrm{eff}}$
: the connectedness actually used when deciding whether to join

conn
: the writer's measured connectedness

κ
: this constant, the coupling strength

lam
: the legibility setting, from 0 to 1

Why it is zero At zero the expression reduces to the measured connectedness and the coupling has no effect, which is the current behaviour. The code comment marks this as deliberate. The reasoning is that connectedness is the single most recognisable property of a hand, and the namer relies on it heavily: thinning the joins of a cursive writer makes the output more readable and markedly less like them. Since that writer's identification is already the weaker of the two measures, trading it away for text accuracy is the wrong direction.

> *Like JOIN_PROB_CAP, this is a mechanism left in place at its neutral setting. Both are the result of trying the adjustment and finding it harmful, and leaving the constant named at zero records that the possibility was considered, which removing it would not.*

7.18 IDEAL_ANCHOR_MAX_PRIORD

```
IDEAL_ANCHOR_MAX_PRIORD = 0.3
```

Output. 0.30, how far from typical the writer's own best letterform may sit and still serve as the anchor.

The decision it governs When a glyph needs replacing, the first choice of replacement is the writer's *own* most typical version of that letter, not a built-in form. That keeps the substitution inside their hand. But their most typical version is only useful if it is itself a decent letter, and for a heavily cursive writer it may not be: if most of their a's were cut badly, the most typical of them is a typical bad cut. This constant is where that judgement is made, against the prior distance described in style.py.

Why 0.30 The prior distance runs from 0 for a shape matching the cross-writer reference exactly upward. The same scale appears in the legibility thresholds, where 0.12 counts as clean and 0.55 as unusable. At 0.30 the limit sits between those: a shape noticeably unlike the general idea of the letter is still accepted, because it is the writer's own and the alternative is a generic form, but one approaching the unusable range is not. It is deliberately permissive, since preferring the writer's own shape is the whole point of trying them first.

7.19 IDEAL_SOURCE

```
IDEAL_SOURCE = 'medoid'
```

Output. 'medoid', selecting which of the two stored fallback letterforms is used as the anchor.

The two candidates style.py stores two single letterforms per character. The **medoid** is the one real variant most typical of the writer's own set. The **prototype**, stored as idealGlyphs, is the per-point median of several aligned variants, which is cleaner than any real variant but is an average and so is not something the writer ever wrote.

Why the medoid is chosen It is real writing. An averaged shape has had the irregularity removed along with the noise, and that irregularity is part of what identifies the writer, so a page of prototypes is smoother and less recognisable. The medoid keeps the writer's own character while still being the best-formed example they produced.

There is a second reason, visible in this project's measurements. Prototype-based anchors improved text accuracy slightly and cost more in writer identification than they gained, which is the wrong trade for a system whose purpose is to reproduce an individual hand. The constant is a string rather than a boolean so that switching between them is a one-word change, which is how that comparison was made.

7.20 _PRIOR_REJECT

```
_PRIOR_REJECT = 0.25
```

Output. 0.25, a prior distance beyond which a stored variant would not be offered.

What it was for A hard cut-off: a stored shape further than this from the cross-writer reference would simply not be considered for drawing, however the legibility setting stood.

Why the approach it represents was superseded A hard rejection is the wrong instrument for this decision, and comparing it with what replaced it is instructive. It admits no middle ground: a variant at 0.249 is used freely and one at 0.251 is never used, although nothing distinguishes them. For a writer whose every variant of some letter sits above the line it rejects all of them, leaving no shape at all. And it cannot take account of how trustworthy the library is as a whole, so it treats a stray bad variant in a good library the same as a uniformly poor one.

The mechanism that replaced it, the probabilistic swap in _SwapProb, addresses all three. It is graded rather than binary, so borderline shapes are used sometimes; it always leaves a shape available, because swapping requires a replacement to exist; and it is driven by a per-writer legibility value, so the same variant is treated differently depending on the company it keeps.

> *This constant is declared and never read. It is recorded here as the earlier design rather than as live behaviour, because a reader finding it in the settings block would reasonably assume variants beyond 0.25 are excluded, and they are not.*

7.21 Trajectory

```
class Trajectory
```

Output. an object holding the pen path: a list of pen-down strokes in millimetres, plus metadata describing how it was produced.

What it represents The output of synthesis, and the thing the rest of the project consumes. It is a list of **pen-down paths**: each entry is a sequence of points the pen visits with its tip on the paper, and the gaps between consecutive entries are where the pen lifts. That representation carries exactly the information a plotter needs and nothing more.

Why the pen-up moves are implicit Nothing records the lifts explicitly. They are implied by the boundaries between strokes, because by definition the pen must rise at the end of one and fall at the start of the next. Storing them would duplicate information already present and create the possibility of the two disagreeing, which is a class of bug that cannot arise if the state is stored once.

The coordinate convention, and the one place it changes Coordinates are millimetres with y increasing upward, which is the natural orientation for describing writing and the convention the stored glyphs use. Images have y increasing downward, and some machines expect one or the other. Rather than taking a position on that, the trajectory states its own convention and each consumer converts: the rasteriser flips y as it draws, and the G-code writer flips it if the machine needs it. The flip therefore happens at the boundary, once per consumer, rather than being half-applied inside the trajectory.

What the two methods report

- **Bounds** returns the smallest rectangle containing every point, which the rasteriser needs to size its canvas and the G-code writer needs to check the drawing fits the machine's travel.
- **PenTravelMm** returns the distance drawn with the pen down and the distance moved with it up. The first is how much ink the page needs; the second is wasted motion, and it is the quantity a stroke-ordering optimisation would try to reduce.

> *This is one of only two classes in the project, and it earns the form: it bundles data with two derived measurements of that same data, which is what a class is for. The rest of the pipeline is plain functions, because elsewhere there is no state that several operations need to share.*

7.22 _GLYPH_CLEAN

```
_GLYPH_CLEAN = 0.12
```

Output. 0.12, the prior distance at or below which a stored shape counts as clean.

What it marks The lower of the two reference points on the prior-distance scale used by the swap rule. A shape this close to the cross-writer reference for its letter is a well-formed example of that letter and is kept almost always, whatever the legibility setting says.

> *Almost always, not always: even a clean shape carries a small swap probability proportional to lam. That is deliberate, because a library can be poor overall while containing some clean shapes, and a writer whose lam is high is one whose extraction is not trusted in general.*

7.23 _GLYPH_GARBAGE

```
_GLYPH_GARBAGE = 0.55
```

Output. 0.55, the prior distance at or above which a stored shape is always replaced.

What it marks The upper reference point. A shape this far from the reference is not a recognisable instance of its letter at all: a fragment, or a cut carrying most of a neighbour. It is replaced unconditionally, with no regard to the legibility setting.

Why this one case is unconditional Every other decision in the swap rule is probabilistic, so this exception needs justifying. The argument is that lam expresses a trade-off between fidelity and legibility, and the trade-off only exists where there is something to trade. Drawing a fragment is not more faithful to the writer; it reproduces the extraction's mistake rather than their hand. With no fidelity to be gained, there is nothing to weigh against legibility, and the shape goes.

7.24 _COMMIT_LAM

```
_COMMIT_LAM = 0.85
```

Output. 0.85, the legibility setting at or above which the library is abandoned entirely.

What it does Above this setting every stored shape is replaced by the anchor, so the writer's extracted library contributes nothing and the output is built from anchor letterforms throughout.

Why there is a hard commitment point at all Without it, a writer at lam 0.95 would still draw a few percent of glyphs from their library, and those few would be drawn from a library that the lam value itself declares untrustworthy. The result is the worst of both: a page that is mostly anchor letterforms, so it does not look much like the writer, with a scattering of fragments in it, so it is not clean either. Committing fully above 0.85 at least produces a consistent page.

This case genuinely arises. A heavily cursive hand has no honest place to cut between letters, so most of its variants are fragments, its median prior distance is high, and AutoLegibility returns a value near 1. For such a writer the output is drawn almost entirely from anchor forms, which is the correct response to a library that could not be extracted, and it is also the mechanism behind this project's measured result that heavily cursive hands reproduce less faithfully than clean ones.

7.25 _SwapProb

```
def _SwapProb(lam, priorD)
```

Inputs

`lam` the legibility setting, from 0 to 1

`priorD`
: this shape's distance from the cross-writer reference

Output. the probability, from 0 to 1, that this shape is replaced by the anchor.

What it is for This small function is where the project's central trade-off is actually resolved. For one glyph, given how far it sits from what the letter should look like and how much the library as a whole is trusted, it returns the chance that the writer's own shape is set aside in favour of a clean one.

The four cases, and the order they are tested in

$$P_{\text{swap}} \;=\; \begin{cases} 1 & d \geq 0.55 \\ 1 & \text{lam} \geq 0.85 \\ 0 & \text{lam} \leq 0.001 \\ 0.06\,\text{lam} & d \leq 0.12 \\ \text{clip}\big(\text{lam}\,(0.55 + 0.9\,b),\ 0,\ 1\big) & \text{otherwise} \end{cases} \tag{7.4}$$

where

P_{swap}
: the returned swap probability

d
: the shape's prior distance

lam
: the legibility setting

b
: how bad the shape is on a 0 to 1 scale, defined below

0.55
: the base swap rate for a shape of middling quality

0.9
: how much worse quality raises that rate

$\text{clip}(\cdot, 0, 1)$
: the result held to a valid probability

The order matters because the cases overlap. Garbage is tested first, so a hopeless shape is replaced even at lam 0. Full commitment is next, so above 0.85 everything goes regardless of quality. Only then is lam 0 tested, which keeps every shape that survived the first test: this is what makes lam 0 mean "faithful reproduction, defects and all" rather than "no filtering whatever".

The interpolation in the middle range

$$b \;=\; \text{clip}\left(\frac{d - 0.12}{0.55 - 0.12},\ 0,\ 1\right) \tag{7.5}$$

where

b
: the badness of the shape: 0 at the clean threshold, 1 at the garbage threshold

d
: the shape's prior distance

0.12, 0.55
: the clean and garbage reference points

$\text{clip}(\cdot, 0, 1)$
: held in range, although the earlier cases already guarantee d lies between the two

This maps the prior distance onto a 0 to 1 scale between the two reference points, which is what lets a single expression cover the whole middle range. The swap probability is then lam times a factor running from 0.55 at the clean end to 1.45 at the garbage end, clipped to 1. So at a given lam, a worse shape is swapped more often, and at a given quality, a less trusted library is swapped more often. The two inputs act multiplicatively, which is the behaviour wanted: neither alone should force a swap.

Why a probability and not a decision Returning a probability rather than a yes or no is what produces a mixed page, and a mixed page is the point. A threshold rule would draw every instance of a given character from the same source, so a letter whose shape sits near the line would be either always the writer's or always the anchor. With a probability, some instances are theirs and some are not, which both preserves more of the hand and spreads the risk: one bad instance of a letter in a line is recoverable from context, where every instance being bad is not.

> *Because the draw is made per glyph from a seeded generator, the same text and seed give the same page. That is what makes the best-of-N search meaningful, since each attempt differs only by its seed, and what makes a result reproducible rather than something that happened once.*

7.26 _AUTO_LAM_GOOD

```
_AUTO_LAM_GOOD = 0.24
```

Output. 0.24, the median prior distance at or below which a library is trusted completely.

What it marks The lower end of the scale AutoLegibility maps onto. A writer whose variants have a median prior distance at or below this gets lam 0, meaning their own shapes are used throughout.

Why the median across the whole library The quantity being judged is the trustworthiness of the extraction as a whole, not of any one glyph, which the swap rule already handles per glyph. The median over every stored variant of every character answers the right question: is this writer's library, in general, made of letters? Using the median rather than the mean keeps the answer from being set by the worst few.

7.27 _AUTO_LAM_BAD

```
_AUTO_LAM_BAD = 0.32
```

Output. 0.32, the median prior distance at or above which a library is not trusted at all.

What it marks The upper end of the same scale. At or above this, lam is 1, the library is abandoned, and the output is drawn from anchor letterforms.

Why the window is so narrow The two thresholds are only 0.08 apart, which makes the mapping steep: a small change in median prior distance moves lam a long way. That is deliberate, and it reflects what the measurements show. Across this project's writers the median prior distance clusters tightly, around 0.18 to 0.24 for the hands that extracted cleanly and around 0.31 for the two most cursive. The thresholds are placed on either side of that gap, so the mapping separates the populations that actually exist rather than spreading a gentle gradient over a range where no writer falls.

> *The consequence is that most writers receive a lam of exactly 0 or exactly 1, with few in between. That is appropriate for a measurement with two clear groups, but it does mean the*

intermediate behaviour of the swap rule is exercised mainly by the repair mechanism, which raises lam for individual characters rather than for the writer.

7.28 AutoLegibility

```
def AutoLegibility(profile)
```

Inputs

`profile`
: the writer's style profile

Output. the legibility setting for that writer, from 0 to 1.

What it is for Derives the legibility setting from the profile instead of requiring it to be chosen. The question it answers is: how far can this writer's extracted library be trusted?

$$\text{lam} \;=\; \text{clip}\left(\frac{\text{median}\{d_g\} - 0.24}{0.32 - 0.24},\; 0,\; 1\right) \tag{7.6}$$

where

lam the returned legibility setting

d_g the prior distance of stored variant g, over every variant of every character in the library

median
: the middle value of those distances

0.24, 0.32
: the good and bad reference points above

clip$(\cdot, 0, 1)$
: held within the valid range, which is what makes the mapping flat outside the two reference points

Why this works, and what it is really measuring The chain of reasoning behind it is the substantive part. The prior distance measures how far a stored shape sits from what its letter should look like. A library whose shapes sit far from that is a library of bad cuts. Bad cuts come from boundaries placed where no honest boundary exists. And that happens exactly when the writing is joined up, because in cursive the pen does not lift between letters, so there is no place to cut that does not damage something.

So the median prior distance is, indirectly, a measure of how cursive the hand is, arrived at by measuring the consequence rather than the cause. That is why it works as a trust signal: it responds to the thing that actually makes an extraction unreliable. And it is why heavily cursive writers end up drawn from anchor forms, which is the mechanism behind this project's finding that such hands reproduce less faithfully.

Why an empty library returns 1 With no variants there is nothing to measure and nothing to draw from, so the anchor is the only available source and lam 1 states that. Returning 0 instead would claim a library worth trusting, and the caller would then try to draw from one that does not exist.

> *A profile may carry an explicit legibilityLambda, which callers prefer over this function. That is how a value found by experiment for a particular writer can override the derived one, and it is why the stored lam column appears in this project's profile tables even though it is usually empty.*

7.29 _CoreGlyph

```
def _CoreGlyph(ch, profile)
```

Inputs

`ch` the character wanted

`profile`
: the writer's style profile

Output. the anchor letterform as strokes, advance, lead and whether it is a cursive form; or nothing if no anchor exists.

What the anchor is The clean letterform that a stored variant is swapped for. The function implements a preference order with two steps, and the order is the whole content of it.

First choice: the writer's own most typical letterform The medoid stored in the profile is tried first, accepted if its prior distance is within IDEAL_ANCHOR_MAX_PRIORD. This matters more than it may appear. An anchor is by definition drawn when the writer's chosen variant was not trusted, so anchors account for a large share of a cursive writer's page. If the anchor were always a built-in form, such a page would be generic handwriting with the writer's spacing, and the namer would not identify it. Preferring their own medoid keeps the substitution inside their hand: it is still a letter they actually wrote, merely the best-formed one rather than a random one.

This project's experiments bear that out directly. Extending the anchor-selection improvements to more writers raised writer identification substantially for the most cursive hand, and the gain came from better anchor letterforms rather than from any change to how variants were chosen.

Second choice: the built-in alphabet If the medoid is absent or too rough, _BaselineGlyph supplies a built-in form, cursive or printed according to the writer's connectedness, scaled to their reach. This is a real loss of fidelity and is accepted only because the alternative is drawing a shape that is not a letter.

Why the advance is taken as a maximum

$$\text{adv} \;=\; \max\left(\text{adv}_{\text{stored}},\;\; \text{lead} + \text{width} + 0.04\right) \tag{7.7}$$

where

adv the advance actually used

$\text{adv}_{\text{stored}}$
: the advance recorded with the letterform

lead the gap from the cell's left edge to where the ink starts

width
: the horizontal extent of the ink

0.04 a small margin so two letters do not touch

The stored advance came from the cell the character was originally cut from, and that cell can be narrower than the ink it contains, since a cut can leave ink spilling past its own boundary. Advancing by the stored figure alone would then place the next letter on top of this one. Taking whichever of the two is larger guarantees the advance clears the ink while preserving the writer's own spacing wherever it is sufficient.

7.30 _GlyphSource

```
def _GlyphSource(profile,ch,rng,prevExitY=None,lam=0.0,forceCore=False,
        forcePrint=False)
```

Inputs

`profile`
: the writer's style profile

`ch` the character to draw

`rng` the seeded random generator

`prevExitY`
: the height the pen left the previous letter at, or nothing if not joining

`lam` the legibility setting for this glyph

`forceCore`
: force the anchor regardless of the swap draw

`forcePrint`
: force the printed alphabet, the last resort

Output. four things: the strokes, the advance, the lead, and a label saying which source the shape came from.

What it is for Given one character, decide what to draw. This is where the legibility trade-off, the variant ranking and the fallback chain all meet, and the label it returns is what makes the outcome auditable.

The order of preference

- **forcePrint**, when set, returns the printed alphabet immediately. It is the last resort invoked by the repair loop for a character the reader still cannot read.

- **The writer's own variants** of this exact character, ranked and sampled as described below, then possibly swapped for the anchor. Labelled `own`, or `core` if swapped.
- **The opposite case** of the character. A capital drawn from the writer's lowercase is scaled up to their ascender height. Labelled `case`.
- **A similar character** from the `_SIMILAR` table, such as an l for an I. Labelled `similar`.
- **The anchor**, when the writer has nothing for this character at all. Labelled `core`.
- **The built-in alphabet** directly, labelled `fallback`.
- **Nothing**, labelled `blank`: no strokes, but the advance is still applied, so an unavailable character leaves a gap rather than closing up the line.

The order descends by how much of the writer's own hand each option retains, which is the right criterion for a system whose purpose is to reproduce an individual. Note that the swap check is applied in the first three branches, not only the first: a borrowed or case-converted shape is still a stored cut and can still be a bad one.

How a variant is chosen: weighted sampling The candidates are ranked by prior distance, cut to the eligible few, and one is drawn with probability proportional to a weight.

$$w_i \;=\; \frac{1}{1+1.4\,i} \;\cdot\; \exp\!\left(-\left(\frac{\Delta y_i}{0.45}\right)^2\right) \tag{7.8}$$

where

w_i the weight given to the i-th ranked candidate

i its position in the ranking, 0 for the most typical

1.4 how steeply the weight falls with rank

Δy_i the difference between this variant's entry height and the height the pen left the previous letter at

0.45 the tolerance on that difference, in x-heights

$\exp(-(\cdot)^2)$
a Gaussian falloff, applied only when joining; otherwise this factor is absent

The first factor implements the preference for typical shapes: the top-ranked variant is weighted 1, the second about 0.42, the fifth about 0.15. So the characteristic shape is drawn most often while the others still appear, which is what gives repeated letters their variation. A reciprocal falloff is gentler than an exponential one, which would make the top variant dominate so heavily that the others might as well not be eligible.

Why the entry height matters when joining The second factor is the more interesting one. When this letter will be joined to the previous one, a ligature must be drawn from where the pen left that letter to where this one begins. If the two heights differ a great deal, the ligature has to climb or dive, which is what produces the tangled joins that make synthetic cursive unreadable. Since several variants are available and they differ in where they begin, the one whose entry height is closest to the pen's current height can be preferred, and the join is then short and natural.

This is a small instance of a general principle worth naming: where a choice exists anyway, it costs nothing to make it with regard to context. The variant selection had to pick one of five; biasing that pick toward the one that joins well improves the output without any extra drawing or searching.

Why the variant's own advance is used Each variant carries its own advance, and that is used rather than the character's median. The code comment gives the reason: a wide variant advanced by a narrow median would collide with the next letter. The median describes the character across all its instances, but what is being drawn is one particular instance, and the spacing must suit the ink actually placed.

> *The label returned is counted in the trajectory's metadata, giving a breakdown of how many glyphs came from each source. That breakdown is how the fidelity-legibility trade-off is measured in this project: a page reported as mostly* `core` *is readable but is largely not the writer's own letterforms, and the counts say so plainly where looking at the output would not.*

7.31 _Jitter

```
def _Jitter(strokes, rng, amp, rotAmp, scaleAmp)
```

Inputs

`strokes`
: one glyph's strokes

`rng` the seeded random generator

`amp` the position shift amplitude, in x-heights

`rotAmp`
: the rotation amplitude, in degrees

`scaleAmp`
: the scale variation amplitude, as a fraction

Output. the strokes shifted, rotated and scaled by small random amounts.

Why randomness is added deliberately Without it, every instance of a letter on the page is identical to the pixel, because the same stored variant is drawn the same way. Identical repetition is the visual signature of print, and a page of it reads as a font however handwritten the individual letterforms are. Real writing never repeats a letter exactly, so a small random variation per glyph is not a flaw introduced into the output; it is a property of handwriting that must be put back.

The three transformations

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} s_x x \\ s_y y \end{pmatrix} + \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} \tag{7.9}$$

where

x, y the glyph's original coordinates, in x-heights

x', y'
: the jittered coordinates

θ a small random rotation, drawn from a normal distribution

s_x, s_y
: independent random scale factors, each near 1

δ_x, δ_y
: a random translation

$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$
: the standard 2-D rotation matrix, which turns a point anticlockwise about the origin by theta

The order is scale, then rotate, then translate, and it is not interchangeable: rotating before scaling would make the independent x and y scalings act along the rotated axes, shearing the glyph rather than stretching it. Scaling x and y independently is itself deliberate, since a writer's letters vary slightly in proportion as well as in size.

Background: why a normal distribution Each amount is drawn from a **normal distribution**, the bell-shaped curve, rather than uniformly from a range. The difference is that a normal distribution makes small deviations much commoner than large ones, with no hard limit, whereas a uniform draw makes a large deviation exactly as likely as a tiny one. Real variation behaves the first way: most letters are close to the writer's usual form and a few are noticeably off. A uniform draw produces output that wobbles unconvincingly, because every letter is about equally displaced.

Why the vertical shift is smaller The vertical amplitude is 0.6 of the horizontal. Vertical position is read more sharply than horizontal: the eye uses the baseline as a reference, so a letter sitting high stands out in a way that one sitting slightly right does not. The recogniser is similarly sensitive, since its features were learned relative to the baseline. Reducing the vertical component keeps the variation from reading as a fault.

> *The amplitudes reaching this function are already scaled by the caller's jitter setting and, for an anchor glyph, by CORE_JITTER_FRAC. So the same function serves for a fully varied extracted glyph and a nearly rigid anchor, with the policy decision kept in the caller rather than here.*

7.32 _Resample

```
def _Resample(poly, step)
```

Inputs

`poly`
: a path

`step`
: the desired spacing between points

Output. the path with points evenly spaced at about that interval.

What it is for Respaces a path so its points sit at roughly equal distances along it, by the arc-length method described under _ResampleN in style.py: cumulative distance along the path is computed and new points are read off at even distances by interpolation.

Why layout needs it Paths reaching this point come from several sources with very different point densities. A stored glyph was simplified, so a straight run is two points. A ligature is generated from three control points. A built-in letterform was laid out by hand at whatever spacing was convenient. Concatenating them gives a path whose density varies several-fold along its length, and two later stages are sensitive to that.

- **The rasteriser** draws by stepping along the path, so a sparse region is drawn from fewer samples; with a long enough gap the round pen's footprints no longer overlap and the stroke appears as a row of dots.
- **The G-code writer** emits a move per point, so the machine's motion is smoother where points are dense. Even spacing gives even pen speed, which is visible in the drawn line.

Why 0.35 millimetres The spacing used throughout layout. It is fine enough that successive pen positions overlap at any plotted size, and coarse enough that a line of text does not become tens of thousands of points, which would slow both the rasteriser and the machine's motion planning for no visible gain.

> *This duplicates _ResampleN in style.py, differing in that it takes a spacing rather than a count and returns tuples rather than an array. Both forms are needed: averaging requires a fixed count so that points correspond between strokes, while drawing requires a fixed spacing so that density is uniform. Neither can be expressed as the other.*

7.33 USE_STYLE_CALIBRATION

```
USE_STYLE_CALIBRATION = True
```

Output. true, enabling the measured correction of the output's style back toward the writer's real pages.

What it enables A feedback step: render a sample of synthetic text, measure its slant and reach the same way they were measured on the writer's real lines, compare, and correct. Turning it off uses the profile's measurements directly.

Why any correction is needed The profile records the writer's slant and reach as measured from their pages, and synthesis applies exactly those numbers. One might expect the output to measure the same, and it does not. The discrepancy accumulates through the pipeline: glyphs were de-slanted during extraction and are re-sheared during layout, but not every stroke shears the same way; the jitter rotates glyphs slightly; the anchor glyphs are capped at a shallower slant than the writer's own; and the baseline drift shifts letters vertically. Each is small and the measured result is systematically off.

Rather than trying to account for each effect analytically, the calibration measures the total discrepancy and corrects for it. That is the sound approach when the individual contributions are numerous, small, and interact, which is the case here.

7.34 _STYLE_CAL

```
_STYLE_CAL = {}
```

Output. a cache of the computed correction, keyed by writer and render scale.

Why it is cached Computing the correction requires rendering a long sample sentence three times and measuring each, which is far more work than drawing the text actually requested. Since the correction depends only on the profile and the render scale, it is the same every time for a given writer, so computing it once per program run is sufficient.

Why the key includes the scale The key is the writer together with the x-height in millimetres and the pixels per millimetre. Both affect the measurement: at a small rendered size the stroke width is a larger fraction of the x-height, which changes what the binarisation and the band detection find, so a correction computed at one scale does not apply at another. Keying on the scale as well as the writer prevents a cached value being reused where it does not hold.

> *This is a module-level cache that is never cleared, so a profile edited within a single run would keep its old correction. That is acceptable because profiles are loaded from disk and not modified in place, but it is the kind of assumption worth knowing when a cache is keyed by identity rather than content.*

7.35 StyleCalibration

```
def StyleCalibration(profile, mmPerXh=4.0, pxPerMm=18.0)
```

Inputs

`profile`
: the writer's style profile

`mmPerXh`
: the x-height in millimetres, default 4.0

`pxPerMm`
: the render resolution, default 18.0

Output. a dictionary of three corrections: a shear adjustment and two zone scale factors.

The method: measure the output, compare, correct, repeat A fixed sample sentence is rendered with the current correction applied, the rendered image is measured exactly as RenderRefStats measured the writer's real lines, and the correction is updated from the difference. That is repeated three times.

Why the comparison is valid The crucial property is that both sides are measured by the same procedure. The reference values stored in the profile as slantMeasRef, ascMeasRef and descMeasRef were produced by running the same binarisation, band detection and slant estimation on the writer's real crops, for exactly this purpose. Comparing a quantity measured one way with the same quantity measured another way would confound the measurement methods with the thing being measured; comparing like with like isolates the real discrepancy.

The three corrections

$$\begin{aligned} \Delta_{\text{shear}} &\mathrel{+}= \tan(\text{slant}_{\text{ref}}) - \tan(\text{slant}_{\text{got}}) \\ k_{\text{asc}} &\mathrel{*}= \frac{\text{asc}_{\text{ref}} - 1}{\text{asc}_{\text{got}} - 1} \\ k_{\text{desc}} &\mathrel{*}= \frac{\text{desc}_{\text{ref}}}{\text{desc}_{\text{got}}} \end{aligned} \tag{7.10}$$

where

Δ_{shear}
: the additive shear correction, applied on top of the profile's slant

k_{asc}
: the multiplicative correction to the ascender zone

k_{desc}
: the same for the descender zone

ref
: the value measured on the writer's real lines

got
: the value measured on this render

$+=$, $*=$
: the correction is accumulated across rounds rather than replaced

-1
: subtracted from the ascender figures because the ascender zone is measured from the top of the x-height band, not from the baseline

The shear correction is additive and is computed on the tangent rather than on the angle. That is because shear is applied as a tangent: a point is displaced horizontally in proportion to its height, with the tangent as the constant. Differences of tangents therefore compose by addition, while differences of angles do not, so correcting in degrees would be wrong by an amount growing with the angle.

The zone corrections are multiplicative because they scale a zone's extent, and the subtraction of 1 in the ascender case matters: the ascender is measured as a height above the baseline, of which the first 1.0 is the x-height band itself. The part that can be corrected is the reach beyond the band, so the ratio must be taken of the excess rather than of the total. Taking the ratio of the totals would under-correct, since the fixed band would dilute the difference.

Why it iterates, and only three times One round is not exact, because the corrections interact: stretching the ascender zone changes which pixels the band detection finds, which changes the measured slant. So each round measures the state the previous round produced and corrects the remaining error. Three rounds are used because the residual after the first is already small and the process is convergent, so further rounds would change little at considerable cost, each requiring a full render and measurement.

Why the corrections are clipped Each zone factor is held between 0.5 and 1.6. The ratios are quotients, and a quotient whose denominator is near zero is unbounded: a render in which the measured ascender came out barely above the band gives a denominator near zero and a correction of hundreds, which would stretch the next render's ascenders off the page. Clipping bounds the damage from a bad measurement, and the bounds are wide enough not to interfere with a genuine correction, which is a few tens of percent.

The cases where no correction is made If any reference value is missing or not a number, or if the render cannot be measured, the neutral correction is returned and cached. Those are the cases where there is nothing to compare against, and applying a correction derived from a failed measurement would be worse than applying none.

> *The sample sentence is a pangram-like text chosen to contain ascenders, descenders and a wide spread of letters, rendered with jitter switched off and on a line wide enough that no wrapping occurs. All three are necessary: without jitter the measurement is repeatable, and without wrapping the measured band belongs to one line rather than several stacked.*

7.36 _ApplyCal

```
def _ApplyCal(strokes, cal)
```

Inputs

`strokes`
: one glyph's strokes

`cal` the correction dictionary

Output. the strokes with the ascender and descender zones rescaled.

What it does Applies the two zone corrections, stretching or compressing only the parts of a glyph outside the x-height band. The shear correction is not applied here; it was folded into the slant before layout began, since shear depends on the final height and must be applied as the glyph is placed.

$$y' = \begin{cases} 1 + (y - 1)\, k_{\text{asc}} & y > 1 \\ y \cdot k_{\text{desc}} & y < 0 \\ y & 0 \le y \le 1 \end{cases} \tag{7.11}$$

where

y the glyph's height at this point, in x-heights

y' the corrected height

k_{asc} the ascender correction factor

k_{desc}
: the descender correction factor

This is the same piecewise form as _BaselineGlyph uses for adapting a built-in letterform, and for the same reason: the band from 0 to 1 is the reference and must not move, so only the reach beyond it is scaled. Writing the ascender case as 1 plus a scaled excess, rather than scaling y directly, is what keeps the band's top edge fixed at exactly 1.

The early exit When both factors are within a thousandth of 1, the strokes are returned untouched. That is the common case, since many writers need no zone correction, and the check avoids rebuilding every point of every glyph to multiply by one.

> *This is applied only to extracted glyphs, never to an anchor. The anchor's proportions were either designed to be legible or are the writer's own best letterform, and re-stretching them would reintroduce the distortion the anchor exists to avoid. The caller enforces that by testing the source label before calling.*

7.37 _Ligature

```
def _Ligature(p0, p1, xh, rng)
```

Inputs

`p0` where the pen left the previous letter, in millimetres

`p1` where it enters the next

`xh` the x-height in millimetres

`rng` the seeded random generator

Output. a path joining the two points, as a resampled curve.

What a ligature is here The connecting stroke drawn between two joined letters. Because the stored glyphs had their ligature tails trimmed off during extraction, as described under _TrimLigatureTails, synthesis must supply its own, drawn between whatever pair of letters it actually placed. That is the advantage of trimming: the join suits the pair at hand rather than the pair the glyph was cut beside.

The shape: a dip whose depth depends on the distance

$$\text{sag} \;=\; \min\big(0.11\,\text{xh},\; 0.16\,|\Delta x|\big) \;+\; \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,\, 0.015\,\text{xh}) \tag{7.12}$$

where

sag how far below the lower endpoint the curve's midpoint is placed

Δx the horizontal distance between the two endpoints

xh the x-height in millimetres

0.11 the ceiling on the dip, as a fraction of the x-height

0.16 the dip as a fraction of the distance travelled

ε a small random variation

$\mathcal{N}(0, \sigma)$
: a normal distribution with mean 0 and standard deviation sigma

The code comment names the defect this avoids: a fixed dip turns a short join into a loop, and tangled loops are what makes synthetic joined writing unreadable. Where two letters are close together, a dip of a fixed depth has to descend and return within a very short horizontal distance, which draws a narrow vertical loop below the baseline rather than a connecting stroke. Making the dip proportional to the distance travelled means a short join is nearly straight and only a long one sags.

The ceiling then caps the other end: over a long gap, proportionality alone would produce a dip deeper than a descender, which would collide with the line below. Taking the smaller of the two expressions gives a dip that grows with distance up to a limit, which is how a real pen behaves.

Why a single midpoint and interpolation, not a formal curve The curve is constructed by placing one point below the midpoint of the two endpoints and resampling the three-point path, which the even resampling turns into a smooth arc. A formal spline would give a more precisely controlled curve, and it is not needed: a ligature spans a fraction of an x-height and is drawn with a pen whose width is a tenth of that, so the difference between a resampled three-point path and an exact curve is well below what the pen can express.

The degenerate case When the horizontal distance is under 0.06 x-heights, a straight segment between the two points is returned. At that distance the two letters are effectively touching and any dip would be the vertical loop the whole design avoids. Returning the two points still produces a connection, which is what joining means, without drawing anything misleading.

7.38 _SplitWords

```
def _SplitWords(text)
```

Inputs

`text`
: the text to be written

Output. a list of words with newline markers between the lines.

What it is for Breaks the text into the units layout works with. Words are the unit because wrapping happens at word boundaries: a word that will not fit on the current line moves to the next one whole, since handwriting is not hyphenated.

Why newlines survive as markers The text is split on newlines first and a marker is inserted between the resulting chunks, so an explicit line break in the input is preserved rather than being treated as a space. The distinction matters because an explicit break is the writer's instruction and must be obeyed wherever the line happens to be, while a space is merely a place where a break is permitted. Keeping the two distinct lets layout handle them as what they are.

> *Empty strings between consecutive spaces are discarded, so repeated spaces collapse to one gap, and a trailing marker is removed so text ending in a newline does not produce an empty final line. Both are small conveniences that keep the layout loop from needing to test for them.*

7.39 _WordWidth

```
def _WordWidth(word, profile)
```

Inputs

`word`
: one word

`profile`
: the writer's style profile

Output. approximately how wide the word will be, in x-heights.

What it is for An estimate used only to decide whether a word fits on the current line. It sums the writer's median advance for each character, falling back to 0.9 x-heights for a character with no recorded advance.

Why an estimate is acceptable here The true width is not available in advance, because it depends on which variant of each letter is chosen, how the jitter scales them, and whether the collision guard pushes anything right. All of that happens while the word is being drawn, which is after the decision to put it on this line has been made. Determining the exact width would mean laying the word out, measuring it, and discarding the result if it did not fit, which is several times the work for a decision whose only consequence is where a line breaks.

The estimate is slightly low on average, since it ignores the collision guard, which only ever widens. So a word occasionally overruns the requested line width by a small margin. That is the right direction for the error to fall: a line a few percent too long is unremarkable, where a conservative estimate that broke lines early would leave visible ragged gaps.

7.40 SynthesizeText

```
def SynthesizeText(text,profile,mmPerXh=4.0,seed=None,lineWidthMm=180.0,
    jitter=0.15,
         legibility=None,perCharLam=None,_cal=None)
```

Inputs

`text`
: the text to write

`profile`
: the writer's style profile

`mmPerXh`
: the x-height in millimetres, default 4.0

`seed`
: the random seed, so a run is reproducible

`lineWidthMm`
: where to wrap, default 180

`jitter`
: how much random variation, default 0.15

`legibility`
: the lam setting, derived if not given

`perCharLam`
: per-position overrides used by the repair loop

`_cal`
: a precomputed calibration, to avoid recomputing it

Output. a Trajectory: the pen path for that text in that hand.

What it is for The main layout function. It walks the text character by character and builds the pen path, handling glyph selection, calibration, variation, placement, collision, joining and line breaking. Everything above exists to be called from here.

The per-character sequence

- **Choose a shape** with _GlyphSource, passing the height the pen left the previous letter at so that a joinable variant can be preferred.
- **Calibrate** it, if it is an extracted glyph, so its zones match the writer's measured reach.
- **Jitter** it, at full strength for an extracted glyph and a fraction of that for an anchor.
- **Place** it at the pen position, applying the shear for slant and the current baseline drift.
- **Check for collision** against the previous glyph's body and push right if needed.
- **Identify the body stroke** and decide whether to join to the previous letter.
- **Advance** the pen position.

Why the slant cap depends on legibility

$$\text{cap}_{\text{author}} \;=\; 45 \;-\; 19 \cdot \text{clip}(\text{lam},\, 0,\, 1) \tag{7.13}$$

where

$\text{cap}_{\text{author}}$
: the greatest lean in degrees applied to the writer's own glyphs

lam
: the legibility setting

45
: the cap at lam 0, which is permissive

19
: how much the cap tightens as lam rises, giving 26 degrees at lam 1

Steep slant and poor legibility tend to occur together, because a steeply slanted hand is usually a joined one and a joined one extracts badly. So a high lam is evidence that the writing is hard to read, and shearing a glyph steeply makes it harder still, for the reasons given under CORE_SLANT_CAP_DEG. Linking the cap to lam reduces the lean exactly for the writers whose output most needs the help, while leaving a clean upright-ish hand free to keep its full measured slant. The anchor glyphs are capped harder again, at 22 or 30 degrees.

The baseline drift

$$v_t \;=\; 0.85\, v_{t-1} \;+\; \eta_t, \qquad \text{drift}_t \;=\; \text{clip}\big(\text{drift}_{t-1} + v_t,\; -0.25,\; 0.25\big) \tag{7.14}$$

where

v_t the drift velocity after this character

0.85 how much of the previous velocity is retained, which is what makes the wander slow

η_t a small random impulse drawn from a normal distribution

drift_t
: the vertical offset applied to this character, in x-heights

$\text{clip}(\cdot, -0.25, 0.25)$
: the offset held within a quarter of an x-height either way

A real hand does not write along a perfectly straight line; the baseline wanders slowly, especially on unruled paper. Adding an independent random offset per character would not reproduce that, because it would produce letters jumping up and down independently, which looks like a fault rather than a wander. What distinguishes a wander is that it is **correlated**: if this letter is high, the next is probably also high.

The construction achieves that by making the random impulse act on a velocity rather than on the position, and by retaining 85 percent of the velocity each step. The offset is then the accumulated velocity, which changes direction only gradually. This is a discrete **autoregressive** process, and it is the standard way to generate a slowly varying random signal. The clip prevents the accumulation drifting arbitrarily far over a long line, which an unbounded random walk would.

Why the body stroke is identified A glyph may consist of several pen-down strokes: the body, plus a dot, a crossbar or an accent. Only one of them carries the connection to the next letter. The body is identified as the stroke with the greatest horizontal extent, and the join is made from and to it.

Getting this wrong has a specific and visible consequence. If the exit point were taken from the last stroke drawn rather than the body, then for an i the exit would be the dot, and the ligature to the next letter would be drawn from the dot, sloping down from above the x-height. The width test identifies the body reliably because a dot or an accent is by nature narrow while the body spans the letter.

The conditions for joining

- **Not the first letter of the word,** and a previous exit point exists.
- **Both characters are alphabetic.** Writing does not join across a digit or a comma.
- **Both glyphs come from compatible sources:** either both are the writer's own shapes, or both are cursive anchor forms. A printed anchor never joins, because it has no entry or exit designed for one.
- **A random draw** against the effective connectedness succeeds.

The source-compatibility condition is the one that needs explaining. A join between an extracted glyph and a printed anchor would be drawn to a point on the anchor that was never meant to

receive one, typically partway up its side, producing a stroke across the middle of the letter. Requiring both ends of a join to be of a joinable kind means an anchor substituted into cursive writing creates a clean break in the word rather than a malformed connection, which is the less damaging of the two outcomes.

How strokes accumulate A running chain holds the current pen-down path. When a glyph joins, its ligature and body are appended to the chain, so the pen stays down across the join and several letters become one continuous stroke, which is what cursive writing is. When it does not join, the chain is flushed as a completed stroke and a new one begins. Non-body strokes such as dots are always flushed separately, since the pen must lift to draw them.

> *The metadata returned records the glyph source counts, the writer, the slant, the connectedness and the line count. The source counts in particular are what make the fidelity-legibility trade-off measurable after the fact, and they are the reason _GlyphSource returns a label at all.*

7.41 _Draw

```
def _Draw(traj, pxPerMm, padMm, strokePx, paper=255, ink=0, softPx=0.0)
```

Inputs

`traj`
: the trajectory to draw

`pxPerMm`
: the resolution in pixels per millimetre

`padMm`
: the margin in millimetres

`strokePx`
: the pen width in pixels

`paper`
: the background value, default 255

`ink` the stroke value, default 0

`softPx`
: the blur radius in pixels, 0 for none

Output. a grey image of the trajectory, ink on paper.

Why the pen path is turned into an image at all The plotter draws from the trajectory and never needs an image. The image exists because the two judging networks score pictures, not paths. Every evaluation in this project, and the best-of-N search itself, works by rendering a candidate and asking the networks what they see. So the rasteriser is not a display convenience; it is part of the optimisation loop, and how faithfully it imitates a photograph determines whether the scores mean anything.

The coordinate flip

$$x_{\text{px}} \;=\; (x - x_0 + \text{pad}) \cdot \text{ppm}, \qquad y_{\text{px}} \;=\; (y_1 - y + \text{pad}) \cdot \text{ppm} \tag{7.15}$$

where

x_{px}, y_{px}
: the pixel position

x, y
: the trajectory position in millimetres, y upward

x_0
: the leftmost extent of the trajectory

y_1
: the topmost extent

pad
: the margin in millimetres

ppm
: pixels per millimetre

The x expression merely shifts the origin and scales. The y expression also reverses direction, by subtracting from the top extent rather than adding to the bottom one, because the trajectory has y increasing upward and an image has row 0 at the top. This is the single place that conversion happens on the rendering path, which is the discipline described under Trajectory: each consumer converts at its own boundary.

Why the round pen needs no end caps Each stroke is drawn by DrawPolyline, which sets a pixel according to its distance from the path rather than by stepping along and stamping a shape. A consequence worth drawing out is that round ends and round joins come out automatically: the set of points within a fixed distance of a line segment is a rectangle with a semicircle at each end, and the union of two such regions is already correctly rounded where they meet. A stamping rasteriser would need explicit end caps and join handling, and would leave notches at sharp corners.

Why the blur comes last All the strokes are drawn and then the whole image is blurred once. Blurring each stroke as it was drawn would be both slower and wrong: where two strokes overlap, each having been individually softened, the overlap would be darker than a single stroke's core, producing a visible dark patch at every crossing. Blurring the finished image treats a crossing as the single region of ink that it is.

> *The canvas is sized from the trajectory's own bounds plus the margin, so it is exactly as large as the writing requires. A fixed canvas size would either clip long text or surround short text with blank paper, and the second matters as much as the first, because the proportion of the crop that is ink is one of the statistics the networks respond to.*

7.42 RenderTrajectory

```
def RenderTrajectory(traj,pxPerMm=8.0,padMm=None,strokePx=None,profile=
    None,
         size=None,matchInk=True,uniformInk=None)
```

Inputs

`traj`
: the trajectory to render

`pxPerMm`
: the resolution, default 8.0

`padMm`
: the margin, derived from the writing size if not given

`strokePx`
: the pen width in pixels, derived if not given

`profile`
: the writer's profile, for the non-uniform path

`size`
: a final size to resample to, or nothing

`matchInk`
: whether to search for the pen width matching the writer's ink density

`uniformInk`
: whether to use one pen width for every writer

Output. a grey image of the trajectory, prepared to resemble the crops the networks were trained on.

What it is for The policy layer above _Draw. It decides the pen width, the margin, the ink and paper levels and the softness, so that the rendered image resembles a photographed crop closely enough for the networks' scores to be meaningful.

The two modes

- **Uniform,** the default: one pen width for every writer, black on white, matching what the plotter physically draws. This is the mode the best-of-N search uses, because the search must optimise the thing that will actually be produced.
- **Per-writer:** the pen width, ink level, paper level and softness taken from the writer's profile, optionally with a search for the width that matches their measured ink density. This is for comparison against their real photographed pages, where the real page does carry their own pen.

Why the margin is proportional to the writing The margin defaults to 0.18 of the x-height rather than a fixed number of millimetres. The code comment gives the reason: a fixed pad would inflate the crop height and skew the line's aspect ratio. A line of writing is long and thin, so a fixed margin adds a large fraction to its height and a small one to its width, changing the proportions. Aspect ratio is one of the statistics recorded in the profile as a reference, and the networks were trained on crops of a characteristic shape, so distorting it is a domain shift of exactly the kind the rendering is designed to avoid.

Why the ink core is painted darker than the profile's ink level The profile's inkLevel is the median of the already-dark pixels of a real crop, which includes the soft edges of every stroke as well as their cores. It is therefore an average over the whole stroke, not the colour of its centre. Painting the core at that value and then blurring would darken nothing and lighten the edges

further, giving a stroke lighter overall than the real one. So the core is painted dark and the blur produces the intermediate values, which is the order the physical process works in.

> *Where a final size is requested, the image is resampled with the project's own resizer rather than being drawn at that size directly. Drawing at the final size would be better in principle, since resampling softens the image slightly, but the pen width is quantised to whole pixels, so at small sizes drawing directly would force a one-pixel pen and lose control of the stroke weight entirely.*

7.43 _MatchInkDensity

```
def _MatchInkDensity(traj,pxPerMm,padMm,startPx,target,mm,paper=255,ink
    =0,soft=0.0)
```

Inputs

`traj`
: the trajectory

`pxPerMm`
: the resolution

`padMm`
: the margin

`startPx`
: the pen width to start from

`target`
: the ink fraction to match

`mm` the x-height in millimetres

`paper`
: the paper level

`ink` the ink level

`soft`
: the blur radius

Output. a pair of the rendered image and the pen width that produced it.

What it is for Finds the pen width whose rendered image has the same fraction of dark pixels as the writer's real pages. Ink fraction is one of the statistics a network responds to most directly, since it is visible at every scale, so matching it removes one axis of domain shift between synthetic and photographed crops.

Why a search rather than a formula The relationship between pen width and ink fraction cannot be written down. Doubling the width does not double the ink, because strokes overlap: where two strokes cross, the ink is counted once, and the amount of overlap depends on the particular text and letterforms. The blur and the threshold used to count dark pixels add further non-linearity. So the quantity is evaluated rather than predicted.

The search: hill climbing in one dimension The current width is probed, the direction that should help is determined from whether there is too much ink or too little, and steps of one pixel are taken in that direction for as long as the error decreases. It stops at the first step that does not improve.

$$\mathrm{err}(w) \;=\; \big|\, f(w) - f^{*} \,\big|, \qquad f(w) \;=\; \text{fraction of pixels below 128 at pen width } w \tag{7.16}$$

where

$\mathrm{err}(w)$
: the error at pen width w, which the search reduces

$f(w)$
: the ink fraction produced by that width

f^{*} the target ink fraction from the writer's real pages

$|\cdot|$ the absolute value, since overshooting and undershooting are equally wrong

This is **hill climbing**: move in whichever direction improves the objective and stop when no step does. It is only guaranteed to find a local best, but here that is enough, because ink fraction increases with pen width throughout, so the error has a single minimum and any local best is the global one.

Why the pen width is a whole number of pixels The width is quantised to whole pixels because that is what the rasteriser can express, which is what makes a one-pixel step the natural increment and makes the search finite. The consequence is that the target can rarely be hit exactly; the search returns the closest achievable, and at small rendered sizes the available widths are coarse enough that the residual error is noticeable.

Why results are cached Each probe requires a full render and a resize, which is the expensive part. The search revisits a width when it steps past the minimum and compares back, so caching by width removes those repeats. The upper bound on the width, a little over half an x-height, exists so that a target the geometry cannot reach makes the search terminate rather than thickening the pen until the text is a solid block.

7.44 _LineLogProbs

```
def _LineLogProbs(reader, img)
```

Inputs

`reader`
: a loaded reader

`img` a rendered line image

Output. the reader's log probabilities, one row per column and one entry per character.

What it is for Runs a rendered image through the reader and returns the raw per-column scores rather than decoded text. Those scores are what the alignment needs, and the alignment is what identifies which characters are weak.

Why the raw scores and not the text Decoded text says what the reader concluded; the scores say how confidently, and where. The repair loop needs the second, because its question is not whether the line reads correctly overall but which particular characters are responsible for it not doing so. That information exists only in the per-column scores, and decoding discards it.

> *The batch axis is indexed away immediately, since rendering and scoring are done one line at a time. The reader retains the axis because the training code uses it.*

7.45 _WeakChars

```
def _WeakChars(reader, img, visible)
```

Inputs

`reader`
: a loaded reader

`img` a rendered line image

`visible`
: the text that was supposed to be written

Output. the positions of the characters the reader scored worst, worst first.

What it is for Identifies which characters of the rendered line the reader is least sure about. The repair loop uses the answer to redraw only those characters, instead of redrawing the whole line and hoping for a better outcome.

Why forced alignment rather than decoding The text is known, so there is no recognition to be done. What is wanted is per-character evidence, and forced alignment provides it: it finds where each known character sits in the rendered line, after which the reader's own score for that character over exactly those columns is a direct measure of how legible that character came out.

$$s_i \;=\; \frac{1}{t_1 - t_0} \sum_{t=t_0}^{t_1-1} \log p_t(c_i) \tag{7.17}$$

where

s_i the score for the i-th character: higher is more legible

c_i that character

t_0, t_1
: the first and last columns the alignment assigned to it

$\log p_t(c_i)$
: the reader's log probability for that character at column t

$\frac{1}{t_1 - t_0}$ the mean over its columns, so a wide letter is not favoured over a narrow one merely for occupying more columns

Averaging rather than summing is the point of that factor. A summed score would rank an m below an i automatically, since it spans more columns and so accumulates more negative terms, and the repair loop would spend its attention on wide letters regardless of how legible they were.

The unaligned case A character the alignment could not place at all is given a very large negative score, so it sorts to the front. That is the correct ranking: a character with no columns of its own was so poorly drawn that the aligner had to pass through it in a single step shared with its neighbours, which is a more complete failure than a character that was placed but scored badly.

> *Whitespace is skipped, since a space is not a drawn shape and cannot be repaired by substituting a letterform. Positions are returned as indices into the original text, not into the filtered character list, because that is what the per-character override table is keyed by.*

7.46 REPAIR_ROUNDS

```
REPAIR_ROUNDS = 3
```

Output. 3, how many targeted redraw passes follow the search.

What a repair round is After the best-of-N search has chosen its best attempt, the weakest characters in it are identified and pinned to the anchor letterform, and the line is drawn again. If the result scores better it is kept. That is one round, and three are performed.

Why repair is more effective than more attempts This is the substantive point. Drawing the whole line again with a different seed changes every character at once, so a line failing because of one bad letter is as likely to acquire a new bad letter elsewhere as to fix the original. Repair changes only the characters identified as weak and leaves the rest exactly as they were, so it is a directed improvement rather than another sample. It is the difference between searching randomly and using the gradient.

Why three Each round costs a full render and two network evaluations, and the returns fall away quickly: the first round addresses the worst characters, which are where most of the loss is, and by the third the remaining weak characters are usually ones the anchor cannot help, because the difficulty lies in the neighbouring letters or the join rather than in the letterform. Three rounds reach that point without spending on further ones.

7.47 REPAIR_WORST_K

```
REPAIR_WORST_K = 3
```

Output. 3, how many characters are pinned to the anchor per round.

Why not just the worst one Pinning one character per round would need many more rounds to fix a line with several bad letters, and each round costs a render and two network evaluations. Pinning three advances faster at the same cost per round.

Why not all of them Because the effects interact and the acceptance test is all-or-nothing. A round's result is kept only if the overall score improves, so a round that fixes four characters and breaks a fifth may be rejected entirely, losing the four improvements with it. Changing a few at a time keeps each round's effect small enough to be judged usefully. Pinning everything at once would simply be the lam-1 case, which abandons the writer's hand altogether.

> *Characters already pinned are not reconsidered in later rounds, whether or not the round that pinned them was accepted. That prevents the loop cycling on the same characters and guarantees it terminates, at the cost of occasionally passing over a character that a different combination would have fixed.*

7.48 DEFAULT_LEGIBILITY

```
DEFAULT_LEGIBILITY = 0.65
```

Output. 0.65, the legibility setting used when none is supplied.

Where it applies Only in SynthesizeLegible, the earlier search that optimises text accuracy alone. The joint search derives its setting from AutoLegibility instead, which is the better approach because it responds to the writer's own library rather than applying one figure to everyone.

Why a middling value 0.65 leans toward legibility without abandoning the writer's shapes, which is the reasonable default for a search whose only objective is text accuracy: with nothing measuring fidelity, a setting that preserved more of the hand would not be rewarded for it. The value is a compromise standing in for a measurement that the function does not make, which is precisely what AutoLegibility supplies and why the joint search prefers it.

7.49 SynthesizeLegible

```
def SynthesizeLegible(text,profile,nTries=6,mmPerXh=4.0,lineWidthMm
   =180.0,
         jitter=0.5,seed=0,reader=None,pxPerMm=18.0,legibility=None,repair
            =True)
```

Inputs

`text`
: the text to write

`profile`
: the writer's style profile

`nTries`
: how many attempts, default 6

`mmPerXh`
: the x-height in millimetres

`lineWidthMm`
: where to wrap

`jitter`
: the variation amount, default 0.5

`seed`
: the base random seed

`reader`
: a loaded reader, constructed if not given

`pxPerMm`
: the render resolution

`legibility`
: the lam setting

`repair`
: whether to run the repair rounds, default true

Output. the best trajectory found, with its scores recorded in the metadata.

What it does Draws the text several times with different seeds, renders and reads each, keeps the attempt the reader reads most accurately, and then runs repair rounds on it. It optimises text accuracy alone.

Why a search is needed at all Synthesis is random: which variant is chosen for each character, whether each is swapped for the anchor, the jitter and the drift all depend on the seed. So the same text in the same hand has many possible renderings of differing quality, and nothing in the layout process knows which it has produced. Drawing several and measuring them is a direct way to find a good one, and it requires no model of what makes a rendering good: the reader supplies that.

The escalation ladder in the repair loop

- **A fresh weak character** is pinned to the anchor, which is preferably the writer's own most typical letterform.
- **A character still weak after being pinned** escalates to the printed alphabet, which is the last resort and abandons the writer's hand for that character entirely. Only the single worst such character escalates per round.

The ladder exists because the anchor sometimes does not help. If the writer's own best form of a letter is itself the problem, pinning to it changes nothing, and the only remaining option is a designed letterform. Escalating one character per round, and only one that has already failed at the lower rung, keeps the loss of fidelity to the minimum that produces a readable line.

> *The function falls back to plain layout if a reader cannot be constructed, returning text drawn without any search. That keeps synthesis usable without the network weights present, which matters for testing the layout alone.*

7.50 SynthesizeJointBestOf

```
def SynthesizeJointBestOf(author,text,profile,nTries=6,mmPerXh=4.0,
        lineWidthMm=180.0,jitter=0.5,seed=0,reader=None,authorModel=None,
        authorMapping=None,pxPerMm=18.0,legibility=None,repair=True,
           repairRounds=3,
        repairWorstK=2)
```

Inputs

`author`
: the writer's name, for looking up their output slot

`text`
: the text to write

`profile`
: their style profile

`nTries`
: how many attempts, default 6

`mmPerXh`
: the x-height in millimetres

`lineWidthMm`
: where to wrap

`jitter`
: the variation amount, default 0.5

`seed`
: the base random seed

`reader`
: a loaded reader

`authorModel`
: a loaded namer

`authorMapping`
: the mapping from writer name to output slot

`pxPerMm`
: the render resolution

`legibility`
: the lam setting, derived if not given

`repair`
: whether to run repair rounds

`repairRounds`
: how many, default 3

`repairWorstK`
: how many characters per round, default 2

Output. the best trajectory found, judged by text accuracy and writer identification together.

What it is for The search this project's reported results were produced with. It differs from the one above in judging each attempt by both networks at once: the reader, for whether the output says the right words, and the namer, for whether it still looks like this writer.

Why both judges are needed Optimising text accuracy alone has a predictable failure. The most readable rendering of any text is the one drawn entirely from clean anchor letterforms, so a search rewarded only for readability will drive the output toward generic handwriting. It would score well and would have abandoned the purpose of the project. Adding the namer puts a cost on that, so the search cannot buy readability with fidelity.

The harmonic mean, and why not an average

$$S = \frac{2\,a\,w}{a + w} \tag{7.18}$$

where

S the combined score the search maximises

a the text accuracy: the fraction of characters the reader got right

w the namer's confidence that this is the right writer

$2aw/(a + w)$
: the harmonic mean of the two, which is zero if either is zero

The **harmonic mean** is used rather than the arithmetic mean, and the difference is the whole point of the combinator. An arithmetic mean lets one measure compensate for the other: an attempt scoring 0.95 on text and 0.15 on writer averages 0.55, the same as a balanced 0.55 and 0.55, so the search would be indifferent between a perfectly readable page in nobody's hand and a moderate page in the right one. The harmonic mean of the first pair is 0.26 and of the second 0.55, so the balanced candidate wins clearly.

The mechanism is that the harmonic mean is dominated by the smaller value: as either term approaches zero the product in the numerator does too, while the denominator does not. So it rewards being adequate on both axes rather than excellent on one, which is exactly the requirement here. It is the same quantity as the F-measure used to combine precision and recall, and for the same reason.

Why the repair loop is more cautious here Two characters are pinned per round rather than three, and an accepted round's pins are retained while a rejected round's are discarded. The caution follows from what is being optimised. Pinning a character to the anchor reliably helps text accuracy and reliably costs writer identification, so under the joint score a repair is no longer close to free, and taking smaller steps keeps each one's net effect measurable.

Why the search stops early at 0.99 Both loops break once the combined score reaches 0.99, since further attempts cannot improve meaningfully and each costs a render and two network evaluations. With a search of six to fourteen attempts per line and a page of many lines, that saving is substantial on the writers whose first attempt is already good.

> *The writer mapping defaults to an attribute read from the namer under the name* `author_mapping`*, while the namer stores it as* `authorMapping`*. The default therefore raises an attribute error if reached, and every working caller passes the mapping explicitly. It is recorded here because a reader would otherwise assume the argument is optional.*

8 reproduce.py

This file draws. It takes a list of strokes in millimetres and turns them into motion, and nothing in it knows anything about handwriting: the strokes may have come from synthesis or from tracing a real page, and the file behaves the same either way. That separation is deliberate, and it is what allows the drawing half to be tested without any of the recognition half being involved.

The two halves, and why they are kept apart

- **The top half** converts a pen path into a G-code file. It is pure arithmetic, runs on any computer, and is the half this project's measured geometry results were taken on.
- **The bottom half** drives the real gantry: stepper motors, limit switches, homing and the pen lift. It needs the single-board computer and the wiring, so its imports are guarded and the top half still works without them.

The guard matters more than it appears. Without it, importing this module on a laptop would fail at the hardware import, and since every test and every evaluation script imports it, none of them could run anywhere but on the machine itself.

Background: what G-code is **G-code** is the standard language for telling a machine tool where to move. A line is a command letter and number, such as G0 or G1, followed by parameters: `G1 X12.5 Y40.0` means move in a straight line to that position. It dates from numerically controlled milling and is what nearly every plotter, 3-D printer and mill accepts, which is why it is used here rather than a private format.

- **G0** is a rapid move, used with the pen up, where only the destination matters.
- **G1** is a controlled move at a feed rate, used with the pen down, where the path matters.
- **G2 and G3** are circular arcs, clockwise and anticlockwise.
- **G4** is a dwell: wait for a stated time.
- **M3 and M5** are auxiliary commands, here mapped to the pen actuator.
- **G21 and G90** declare millimetres and absolute coordinates.

Units The top half works in millimetres throughout. The bottom half works in motor steps, converting at the boundary using a steps-per-millimetre figure that calibration measures rather than assumes.

8.1 GantryConfig

```
class GantryConfig
```

Output. a record of every physical number describing the machine, with derived step scaling available as properties.

What it is for Collects everything physical about the machine into one place: the gearing, the work area, the feed rates and the pen actuator's timing. A physical change to the machine, a different pulley or a larger bed, is then one edit here rather than a search for the number wherever it was used.

Why steps per millimetre is derived and not typed in This is the one piece of real reasoning in the record. Steps per millimetre is the number that converts a distance into motor steps, and it could simply be stated. It is instead computed from the parts the machine is actually built from.

$$\text{stepsPerMm} = \frac{\text{fullStepsPerRev} \times \text{microstepping}}{\text{mmPerRev}} \tag{8.1}$$

where

stepsPerMm
: how many motor steps move the axis one millimetre

fullStepsPerRev
: the motor's own step count per revolution, 200 for the NEMA 17 used

microstepping
: how many electrical sub-steps the driver divides each full step into, 16 here

mmPerRev
: how far the axis travels per motor revolution, set by the belt and pulley

With the values given this is 200 times 16 divided by 40, or 80 steps per millimetre. The reason for deriving it is that each of the three inputs is a fact someone can check against the hardware: the motor's label says 1.8 degrees per step, the driver's jumpers say 1/16, and the belt is 2 mm pitch with a 20-tooth pulley, giving 40 mm per revolution. A single stated 80 could not be checked that way, and if a pulley were changed it would be silently wrong, with the only symptom being a drawing of the wrong size.

Background: what microstepping is A stepper motor moves in fixed increments, 1.8 degrees for a 200-step motor. **Microstepping** is the driver's ability to hold the rotor between two full steps by driving the coils at intermediate currents, dividing each step into smaller ones. At 1/16 the resolution becomes 0.1125 degrees, which for this gearing is 12.5 micrometres of travel. Finer resolution is not the main benefit: full stepping produces an abrupt jump between positions that makes the motor vibrate audibly and visibly, and microstepping smooths that into nearly continuous motion, which matters when the thing being drawn is a curve a few millimetres across.

Why the pen actuator has timing in the config The pen is raised and lowered by a geared motor that turns 90 degrees per pulse, so the time one pulse takes is a physical property that the G-code must wait for. The pulse and settling times are recorded here and used to emit a dwell of the right length. Getting that wrong does not fail cleanly: too short a dwell means drawing begins while the pen is still descending, which produces a stroke that starts in the air.

> *The acceleration limit is declared in this record and is never read: no acceleration limiting is implemented. Motion steps at a constant rate computed from the feed, so a move begins at full speed. For the light gantry and modest feeds used that is tolerable, but a reader should not take the field's presence as evidence that a trapezoidal profile is applied.*

8.2 ToMachine

```
def ToMachine(traj, cfg, fitToBounds=True, optimizeOrder=True)
```

Inputs

`traj`
: the trajectory to place

`cfg` the machine configuration

`fitToBounds`
: whether to shrink the drawing to fit the work area, default true

`optimizeOrder`
: whether to reorder the strokes, default true

Output. a pair: the strokes in bed coordinates, and the scale factor applied.

What it is for A trajectory is positioned wherever synthesis happened to put it, with its origin at the start of the first line. The bed has its own coordinates, its own extent, and a chosen place where text should begin. This function moves the drawing from one to the other.

The transformation

$$x_{\text{bed}} \;=\; \text{originX} + (x - x_0)\,k, \qquad y_{\text{bed}} \;=\; \text{originY} - (y_1 - y)\,k \tag{8.2}$$

where

x_{bed}, y_{bed}
: the position on the bed, in millimetres

x, y the position in the trajectory

x_0 the trajectory's leftmost extent

y_1 its topmost extent

originX, originY
: where text is to begin on the bed

k the scale factor

Both frames have y increasing upward, so no flip is needed, unlike the rasteriser's conversion. What the y expression does instead is measure downward *from the top* of the drawing: the topmost row of text lands exactly at the chosen origin and later rows descend from there. Anchoring at the top rather than the bottom is what makes a one-line and a ten-line page both start in the same place, which is how a person writes on a sheet.

Why it never enlarges

$$k \;=\; \min\!\left(1,\; \frac{\text{availW}}{w},\; \frac{\text{availH}}{h}\right) \tag{8.3}$$

where

k the scale factor actually applied

availW
: the bed width available to the right of the origin

availH
: the height available below it

w, h
: the drawing's own width and height

1 the cap, which is what prevents enlargement

The 1 in the minimum means the drawing is shrunk when it would not fit and is otherwise left at the size asked for. That is the right behaviour because the size was chosen deliberately: the caller specified an x-height in millimetres, so the writing is meant to be that size. Scaling up to fill the bed would override that choice, and a request for small writing would produce large writing. Scaling down is different in kind: it is not overriding the request but avoiding an impossibility, since a drawing larger than the bed cannot be drawn at all.

Why both axes get the same factor A single factor is taken as the smaller of the two ratios, rather than each axis being scaled to fit independently. Independent scaling would stretch the writing, changing the proportions of every letter, which is both visibly wrong and destroys the property the whole pipeline maintains, that the stored shapes are reproduced faithfully.

8.3 _Decimate

```
def _Decimate(poly, minSeg)
```

Inputs

`poly`
: a path in millimetres

`minSeg`
: the shortest segment to keep, in millimetres

Output. the path with points closer together than that merged away.

What it is for A path may carry far more points than the machine can act on. A traced stroke has a point per pixel, and resampling during layout placed points every 0.35 mm. This removes the points that are closer together than the machine can resolve.

The method, and why it is not Douglas-Peucker Each point is kept only if it is at least the minimum distance from the last point kept. This is a different rule from the path simplification in style.py, which keeps points according to how far they stray from a straight line, and the difference is appropriate to the different purpose. Here the question is not whether a point carries shape information but whether the machine can act on it at all: a commanded move shorter than the minimum segment is below the resolution of the motion, so it costs a line in the file and produces nothing.

Why the comparison is against the last kept point Comparing against the last kept point rather than the previous input point is what makes the rule accumulate correctly. Against the previous input point, a long run of points each slightly under the threshold would all be dropped, however far the path travelled in total. Against the last kept point, the distance accumulates until the threshold is reached, so a point is emitted every minimum segment along the path regardless of how finely the input was sampled.

> *The first and last points are always kept, whatever the spacing. The endpoints define where a stroke begins and ends, so dropping one would shorten the stroke, and the final point in particular is where the pen lifts and where the next stroke's travel is measured from.*

8.4 OptimizeStrokeOrder

```
def OptimizeStrokeOrder(strokes, rowTolMm=None)
```

Inputs

`strokes`
: the strokes in bed coordinates

`rowTolMm`
: how far apart two strokes may sit vertically and still count as the same row; derived if not given

Output. the same strokes in an order that reduces pen-up travel.

What it is for The strokes may be drawn in any order, since each is independent. The order determines how far the pen travels raised between them, and that travel is pure waste: it takes time and draws nothing. For a page of handwriting with hundreds of strokes the difference between a good order and a poor one is substantial.

Background: this is a travelling salesman problem Finding the order that minimises total travel is the **travelling salesman problem**: given a set of places, find the shortest route visiting all of them. It is one of the classic hard problems in computing, with no known method that finds the exact best route in reasonable time as the number of places grows. With several hundred strokes an exact answer is out of reach, so a **heuristic** is used: a rule that gives a good answer quickly with no guarantee of being best.

The heuristic: bands, then nearest neighbour Two ideas are combined. The strokes are first grouped into horizontal bands by their vertical centres, and the bands are processed from top to bottom. Within a band, the nearest unvisited stroke is taken next.

- **Banding** exploits the structure of the problem. Text is arranged in lines, and almost all the strokes of one line of writing are near each other, so a route that finishes one line before starting the next is close to optimal. A pure nearest-neighbour search without banding can wander between lines, since a stroke on the line below may happen to be nearer than the next stroke along.

- **Nearest neighbour** within a band is the standard simple heuristic: from where the pen is, go to the closest place not yet visited. It is greedy, so it can paint itself into a corner and leave a long move to the last stroke, but within one line of text the strokes are nearly collinear and the greedy route is close to the best.

Why the band tolerance is derived from the strokes

$$\text{rowTol} \;=\; \max\big(1,\; 1.2 \cdot \text{median}\{\, h_s \,\}\big) \tag{8.4}$$

where

rowTol
: how far apart two strokes' vertical centres may be and still share a band, in millimetres

h_s the height of stroke s, from its lowest point to its highest

1.2 a margin above the typical stroke height

1 a floor in millimetres, for a drawing of very small strokes

A fixed tolerance would be wrong at every size but one. Deriving it from the median stroke height ties it to the writing: a band should be about as tall as a line of text, and a typical stroke is about the height of a letter. The median again rather than the mean, so that one tall stroke spanning several lines, which a long descender or an underline can produce, does not widen every band.

Reversing a stroke to enter from the nearer end A stroke can be drawn from either end, and the result is identical ink. So when the far end is closer to the pen than the near end, the stroke is reversed. The code comment notes that this is free for a plotter, and it is worth saying why: unlike a milling cut or an extrusion, a pen stroke has no direction-dependent behaviour, so nothing is lost. It can nearly halve the travel to a long stroke.

> *Fewer than three strokes are returned unchanged. With two strokes there is only one meaningful order, and the banding statistics would be computed from a sample too small to describe anything.*

8.5 _Clamp

```
def _Clamp(strokes, cfg)
```

Inputs

`strokes`
: the strokes in bed coordinates

`cfg` the machine configuration

Output. a pair: the strokes with every point forced inside the work area, and how many points were moved.

What it is for A final safety net. Every point is forced inside the machine's declared bounds, and the number of points that had to be moved is counted and reported.

Why it exists when ToMachine already fits the drawing The fitting scales the drawing to the available area, so in principle nothing can lie outside it. This clamp exists for the cases where that reasoning does not hold: the fitting can be switched off by the caller, the origin can be set such that the available area is computed wrongly, and a stroke can be reversed or reordered by code that has no knowledge of the bounds. A point outside the work area is not a cosmetic problem; it commands the machine to drive into its own frame.

Why the count is returned Clamping silently would hide a real fault. A drawing that needed clamping was mispositioned, and the clamped version is not what was asked for: the affected strokes are flattened against the boundary, which is visible as a straight edge where writing should be. Returning the count lets the writer emit a warning into the file, so the condition is recorded where someone will see it rather than inferred from the drawing.

> *Clamping each coordinate independently is not geometrically principled: a stroke crossing the boundary diagonally is bent rather than cut, so its shape changes. Properly clipping the path to the rectangle would preserve the shape of the part inside. That is not done because the condition is a fault to be reported rather than a case to be handled well, and the simple form cannot itself fail.*

8.6 NewPenState

```
def NewPenState(cfg)
```

Inputs

`cfg` the machine configuration

Output. a small mutable record holding whether the pen is up and how many pulses have been emitted.

Why the pen's state must be tracked at all This is the central peculiarity of the machine and it shapes the whole of the G-code writing. The pen is raised and lowered by a geared motor that turns 90 degrees per pulse. It is a **one-way actuator**: it accepts "move" and nothing else. There is no command meaning "be up", and nothing in the G-code can ask where the pen currently is.

The consequence is that the writer must know the pen's state by remembering it. The state begins at a known value, assumed at power-on, and every pulse emitted flips it. Emitting a toggle when the pen is already in the wanted state would move it to the wrong one, and from then on every pen command in the file would be inverted: the machine would travel with the pen down and draw with it up.

Why it is a list rather than a plain value The record is a two-element list so that the function updating it can modify it in place and the caller sees the change. A tuple or an integer would be copied, and the caller would keep the old state. This is the smallest possible piece of shared mutable state and it exists because the pen genuinely has a state that persists across many lines of output.

The starting state is an assumption, not a measurement. If the machine is powered on with the pen down, every pen command in the file is inverted. The hardware half resolves this properly, with a switch that reports the pen's actual position, but a written file cannot: it has no way to ask.

8.7 SetPenLines

```
def SetPenLines(state, cfg, wantUp)
```

Inputs

`state`
: the pen state record, updated in place

`cfg` the machine configuration

`wantUp`
: the state wanted

Output. the G-code lines needed to reach that state, which is nothing at all when it is already current.

What it does Compares the wanted state with the tracked one. If they agree, no lines are emitted. If they differ, the state is flipped, the pulse count incremented, and two lines are produced: the toggle command and a dwell.

Why emitting nothing is the important case The empty return is what makes the mechanism correct. Because the actuator toggles, a redundant command is not merely wasteful, it is wrong. The writer calls this function before every travel move and before every stroke, which means it is called many times with a state that is already current, and every one of those calls must produce nothing. Placing that check inside the function rather than at each call site is what guarantees it, since a single forgotten check at one call site would invert the rest of the file.

Why a dwell follows every toggle

$$\text{dwell} \;=\; \frac{\text{penPulseMs} + \text{penSettleMs}}{1000} \tag{8.5}$$

where

dwell
: the wait emitted after a toggle, in seconds

penPulseMs
: how long one 90-degree rotation takes

penSettleMs
: extra time for the mechanism to come to rest

1000
: the conversion from milliseconds to the seconds a G-code dwell expects

G-code is executed line by line, and the machine would begin the next move as soon as the toggle command was issued, while the pen was still travelling. Drawing would then start before the pen reached the paper, losing the beginning of the stroke, or a travel move would begin before the pen was clear, dragging a line across the page. The dwell blocks until the motion is complete. The settling time is added on top of the rotation time because the mechanism has backlash and the pen oscillates slightly as it lands.

> *The pulse count is accumulated and reported by the writer. It is a useful figure because each pulse is a mechanical operation taking about a third of a second: a file with twice the necessary pulses takes noticeably longer to draw and wears the actuator twice as fast. It is also how a fault in the stroke ordering shows up, since a poor order lifts the pen more often than necessary.*

8.8 WriteGcode

```
def WriteGcode(traj, cfg, path, title='')
```

Inputs

`traj`
: the trajectory to write

`cfg` the machine configuration

`path`
: where to write the file

`title`
: an optional title for the header comment

Output. a dictionary summarising what was written: line and stroke counts, pen pulses, the scale applied, and how many points were clamped.

What it is for The entry point of the top half. It places and orders the strokes, clamps them, and writes the file.

The pattern for one stroke

- **Pen up,** if it is not already, so the travel does not draw.
- **G0 to the stroke's first point,** at travel feed. A rapid move, since the path taken does not matter with the pen raised.
- **Pen down,** with its dwell.
- **G1 at drawing feed,** then one G1 per remaining point.

The two feed rates are set once rather than repeated on every line, because G-code feed is modal: it persists until changed. Repeating it would lengthen the file by a third for no effect.

Why the header carries the machine's own numbers The header comments record the writer, the stroke count, the scale applied, the steps per millimetre for both axes, and how the pen codes are to be interpreted. None of it affects the machine, since comments are ignored. It is there because a G-code file outlives the program that wrote it, and a file whose scale and gearing are unknown cannot be checked against the machine it is about to be run on. Recording the steps per millimetre in particular makes it possible to tell afterwards whether a drawing came out the wrong size because the file was wrong or because the machine was configured differently.

The warning comment Where points had to be clamped, a warning comment is written into the file. That places the diagnosis where the evidence is: someone examining a drawing with a flattened edge looks at the file, and the explanation is in it. A warning printed to the console at the time of writing would have scrolled away long before the drawing was made.

Parking at the end The file ends by raising the pen, moving back to the origin, and issuing the program-end command. Parking matters practically: it leaves the machine in a known position, so the next file starts from the same place, and it takes the pen off the drawing, where a pen left resting bleeds ink into the paper.

8.9 ParseGcode

```
def ParseGcode(path)
```

Inputs

`path`
: a G-code file to read

Output. the strokes recovered from it, as lists of points in millimetres.

What it is for, and why it is written separately Reads a G-code file back into strokes, by following the pen state and the coordinates through the file and collecting the points visited with the pen down. Its purpose is verification: comparing what comes back with what went in measures whether the file faithfully represents the intended geometry.

The code comment makes the design decision explicit: it shares no code with the writer, deliberately. That is the whole value of the test. If both used a common routine to format and interpret coordinates, a fault in that routine would cancel between them and the comparison would pass while the file was wrong. Two independent implementations can only agree if the file itself is right, which is the property being tested.

What the test can and cannot detect

- **It detects** rounding error from writing coordinates to three decimal places, a stroke reordering or reversal that lost a point, a clamp that moved geometry, and any error in the pen-state tracking, since a mistracked pen makes the reader collect the wrong points entirely.
- **It cannot detect** anything about the machine. The feed rates, the dwells, the acceleration and every mechanical effect are invisible to it. The measured agreement of better than 0.999 by area overlap is a statement about the file, not about the drawing.

Being clear about that boundary is the point of describing the test at all. A round-trip agreement this close is easy to misread as a statement that the robot draws accurately, and it is not: it says the arithmetic between a pen path and a G-code file loses essentially nothing, which locates any remaining error in the mechanics rather than in the software.

How the reader tracks state It maintains a current position and a pen flag. Comments are stripped, the command letter is taken from the first word, and coordinates are extracted by pattern. A pen-down command starts a new stroke at the current position; a pen-up command ends it; a move updates the position and, if the pen is down, appends the point.

Starting the stroke at the current position on pen-down, rather than at the first move after it, is what makes the recovered stroke begin where the writer's travel move left the pen. Without it every stroke would be missing its first point, and the comparison would show a systematic error that was the reader's and not the file's.

> *Strokes of fewer than two points are discarded, which matches the writer, since it skips such strokes too. A stroke still open at the end of the file is closed and kept, so a file that ends without a final pen-up does not lose its last stroke.*

8.10 GantryNotConnectedError

```
class GantryNotConnectedError
```

Output. an error type raised when something needs the hardware and the hardware is not available.

What it is for Raised when the GPIO library is missing or the chip device cannot be opened, which is the ordinary situation on any computer that is not the machine's own controller.

Why a named error rather than letting the import fail An unguarded import of the hardware library would make this module unimportable off the machine, and since the test and evaluation scripts all import it, none of them could run anywhere else. A distinct error type lets the hardware functions fail cleanly when called, while the arithmetic half of the file remains usable.

Being a distinct type rather than a generic error is what lets a caller distinguish "there is no machine here" from "the machine is there and something went wrong". The first is expected and should be reported as a plain message; the second is a fault.

8.11 NotCalibratedError

```
class NotCalibratedError
```

Output. an error type raised when an absolute move is attempted before homing.

Why calibration is a precondition and not a convenience A stepper motor has no idea where it is. It moves a commanded number of steps from wherever it happens to be, and nothing in the motor or the driver reports a position. So a command to move to a particular place on the bed is meaningless until the machine has established where that place is relative to its current position, which is what homing does.

Executing such a command anyway is not a silent error. The machine drives toward a position computed from an unknown origin, which is as likely as not to be outside the bed, and it will drive into its own frame until the limit switches catch it. A distinct error type refusing the command is the appropriate response to a request that cannot be carried out safely.

> *This is one of the few places in the project where a function refuses rather than falling back. The usual pattern is to return a degraded result, but there is no degraded version of a physical move: it either goes to the right place or it drives the machine somewhere arbitrary.*

8.12 CHIP_PATH

```
CHIP_PATH = '/dev/gpiochip0'
```

Output. the device path of the GPIO controller.

Background: what a GPIO chip is **GPIO** stands for general-purpose input/output: pins on a single-board computer that software can drive high or low, or read as high or low. They are how a program controls hardware that has no other interface, which here means the stepper drivers, the limit switches and the pen motor.

On Linux the pins are reached through a device file representing the controller. Opening it yields a handle through which individual lines are requested and then set or read. The path names which controller, and a board with several exposes several.

> *This is a Linux path, which is part of why the hardware half must be guarded: on any other system the file does not exist, and the library that opens it is not installed.*

8.13 PEN_SWITCH_PIN

```
PEN_SWITCH_PIN = 70
```

Output. the GPIO line number of the switch that reports the pen's position.

Why the pen has a sensor when the G-code path only tracks it The written file has no choice but to remember the pen's state, because a file cannot ask a question. The machine can. This switch closes when the pen is raised, so the hardware half knows the pen's actual position rather than its assumed one.

The difference is what allows the hardware to be correct where the file can only be consistent. SetPen compares the wanted state with the measured one and does nothing if they already agree, so a pen that starts in an unexpected position is handled rather than inverting everything that follows. It also allows a failure to be detected: if the motor runs and the switch never changes, something is mechanically wrong, and that can be reported instead of the machine continuing to draw with a pen that never came down.

The switch reports only one of the two states directly: it is closed when up and open otherwise. That is sufficient, since there are only two states, but it means a fault in which the pen sticks halfway reads as "down". The overrun time below is what guards against the related case of stopping the motor the instant the switch first trips.

8.14 PEN_MOTOR_PIN

```
PEN_MOTOR_PIN = 71
```

Output. the GPIO line number that drives the pen actuator.

How the actuator is driven Setting this line active runs the geared motor; setting it inactive stops it. There is no position control and no direction: the motor turns one way only, and a quarter turn of its cam moves the pen between up and down.

Why the software must decide when to stop it Because the motor has no limit of its own, running it too long carries the cam past the position wanted and begins the next quarter turn, which returns the pen to where it started. So the software runs the motor while watching the switch and stops it when the wanted state is reached. That is a closed loop, in contrast with the written file's open-loop timed dwell, and it is why the hardware path is more robust than the file path for the same operation.

8.15 ENDSTOP_PINS

```
ENDSTOP_PINS = {'X_MIN': X_MIN_PIN,'X_MAX': X_MAX_PIN,'Y_MIN': Y_MIN_PIN,
        'Y_MAX': Y_MAX_PIN}
```

Output. a mapping from each limit switch's name to its GPIO line number.

What a limit switch is for An **endstop** or limit switch is a mechanical switch at the end of an axis's travel, closed when the carriage reaches it. Four are fitted, at the minimum and maximum of each axis, and they serve two distinct purposes.

- **Homing.** Since the motors cannot report their position, the only way to establish one is to drive until a switch is reached, which is a known place. Calibration drives to both ends of each axis, which also measures the travel between them.
- **Protection.** During any motion, a switch closing means the carriage has reached a limit it was not supposed to reach, which stops the machine. Without that, a mistaken coordinate drives the carriage into the frame and the motor stalls, skipping steps and losing the position entirely.

Why they are named rather than numbered The switches are held in a mapping keyed by name, so the code refers to X_MIN rather than to a pin number. The names carry the information the logic needs: which axis a switch belongs to and which direction to retreat when it trips. The retreat direction table is keyed by the same names, so the two cannot drift out of correspondence, which a pair of parallel numeric lists could.

8.16 INVERT_X

```
INVERT_X = True
```

Output. true, meaning the X axis's direction signal is reversed.

What it corrects A stepper driver has a direction pin: one level turns the motor one way and the other the opposite. Which physical direction each level produces depends on how the motor's coils happen to be wired to the driver and on how the motor is mounted relative to the axis. There is no convention that settles it, and the wiring is a four-conductor connector that can be fitted either way round.

So the mapping between the signal and the physical direction is determined by trying it, and the result is recorded here. The alternative, rewiring the motor until the default convention happens to be right, achieves nothing a single flag cannot.

How the flag is applied AxisDirValue combines the wanted direction with this flag using an exclusive-or, so a wanted positive direction with inversion set gives the inactive level. Keeping the correction in one function means every motion path inherits it, and a new motion routine cannot forget it as long as it asks for a direction by meaning rather than by signal level.

8.17 INVERT_Y

```
INVERT_Y = True
```

Output. true, meaning the Y axis's direction signal is reversed as well.

Why each axis has its own flag The two axes are wired and mounted independently, so the correction each needs is independent. Both happen to require inversion on this machine, which is coincidence rather than something that could be assumed: a single shared flag would work here and would be wrong on a machine where only one axis needed it.

8.18 MICROSTEPS

```
MICROSTEPS = 16
```

Output. 16, the microstepping division set on the drivers.

What it records The driver divides each of the motor's full steps into this many sub-steps, which is set by jumpers on the driver board. The value here must match those jumpers: the software cannot read them, so a mismatch produces motion scaled by a factor of two or more with no error of any kind, only a drawing of the wrong size.

Why sixteen It is the finest division the A4988 driver used offers, and the reason to take the finest is smoothness rather than resolution. At 1/16 the motor advances in increments small

enough that the motion is effectively continuous, which matters for drawing curves a few millimetres across: coarser stepping is audible as a buzz and visible as faceting on a curve. The cost is that sixteen times as many step pulses must be issued per millimetre, which for a software-timed stepper sets the maximum speed.

> *This constant duplicates the microstepping field in GantryConfig, because the two halves of the file were written to be independent and each carries its own copy of the machine's numbers. The duplication is a real hazard: the two must be changed together, and nothing enforces it.*

8.19 STEPS_PER_REV

```
STEPS_PER_REV = 200 * MICROSTEPS
```

Output. 3200, the microsteps per motor revolution.

How it is obtained The motor's 200 full steps per revolution multiplied by the microstepping. Computing it rather than stating 3200 keeps the two factors visible, each of which is a checkable fact about the hardware, and means a change to the microstepping jumpers is one edit.

> *200 appears here as a literal rather than as a named constant, which is the one piece of the derivation not spelled out. It is the motor's step angle of 1.8 degrees divided into 360.*

8.20 STEPS_PER_MM

```
STEPS_PER_MM = 80.0
```

Output. 80.0, a provisional steps-per-millimetre figure.

What it is for, and why it is provisional A placeholder used only before calibration has run. Once the machine has homed, it has *measured* its own steps per millimetre, by counting the steps between the two limit switches and dividing by the known distance between them, and the measured figure replaces this one.

Why a measurement beats the calculation The calculated 80 assumes the gearing is exactly as specified, and it is not quite. A GT2 belt's pitch varies slightly with tension and temperature, a pulley's effective radius depends on where the belt sits in its teeth, and both are manufactured to a tolerance. Measuring the actual travel captures all of that at once, without needing to know which part contributed what. The measured value is typically within a percent or two of the calculated one, which over a 200 mm page is a few millimetres.

> *The calculated value remains in use in one place: RunGcodeFile reads the starting position using this constant rather than the calibrated figure, so the position it begins from is slightly off even after calibration. The subsequent moves use the calibrated value, so the error does not accumulate, but the first move of a job is affected.*

8.21 CIRCLE_STEP_DELAY_S

```
CIRCLE_STEP_DELAY_S = 2e-06
```

Output. 0.000002, the shortest delay between step pulses, in seconds.

What it bounds A step pulse is produced by setting a pin high, waiting, and setting it low. This is the wait, and it therefore sets the fastest the motor can be driven. Two microseconds between edges is near the practical floor for a driver's minimum pulse width and well beyond what the motor can follow.

Why it is used as a floor rather than a speed Motion is normally timed from the requested feed rate: the delay is computed so that the move takes the time the feed implies. This constant is the lower bound on that computation, so a feed rate too high to be achieved is silently reduced to the fastest the machine can step instead of producing a negative or zero delay.

> *The delay is produced by asking the operating system to sleep, which on a general-purpose Linux system is accurate to perhaps tens of microseconds rather than to two. So at short delays the actual rate is set by the operating system's scheduling rather than by this value, and the real maximum speed is lower and less regular than the constant suggests. Step timing of this kind properly belongs in dedicated firmware or a hardware timer.*

8.22 DEFAULT_CIRCLE_RADIUS_MM

```
DEFAULT_CIRCLE_RADIUS_MM = 8.0
```

Output. 8.0, the radius of the test circle in millimetres.

What it is for The machine has a test mode that drives a continuous circle, and this is its default radius. A circle is a useful test pattern because it exercises both axes continuously and in varying proportion, so it reveals faults that a straight line does not: a mismatch between the two axes' steps per millimetre shows as an ellipse, and backlash in a belt shows as a flat spot where an axis reverses.

8.23 X_SWITCH_TRAVEL_MM

```
X_SWITCH_TRAVEL_MM = 204.0
```

Output. 204.0, the measured distance between the two X limit switches.

Why this number is the key to calibration Calibration counts the steps between the two switches, which gives a travel in steps. Converting that into steps per millimetre requires knowing the travel in millimetres, and that is this constant, measured once from the physical machine with a rule.

$$\text{stepsPerMm}_X = \frac{\text{travelSteps}_X}{\text{X_SWITCH_TRAVEL_MM}} \tag{8.6}$$

where

stepsPerMm$_X$
: the calibrated scaling for the X axis

travelSteps$_X$
: the steps counted between the two switches during homing

X_SWITCH_TRAVEL_MM
: this constant, the same distance in millimetres

So the accuracy of every subsequent move rests on this one hand-measured figure. An error of one percent in it scales the whole drawing by one percent, and nothing in the system can detect that, because the machine has no other reference for absolute distance. It is the single value in the project that must be got right by measurement rather than by reasoning.

> *The distance is between the switches'* trip points*, not between the switch bodies or the ends of the rail. A switch trips a little before the carriage reaches it, by however much its lever travels, so the figure has to be taken from where motion actually stops.*

8.24 Y_SWITCH_TRAVEL_MM

```
Y_SWITCH_TRAVEL_MM = 262.0
```

Output. 262.0, the same distance for the Y axis.

Why the two axes differ The axes are physically different lengths, 204 and 262 millimetres, which is simply how the machine is built. Each is calibrated against its own figure, giving each its own steps per millimetre. Keeping them separate also allows the two axes to come out with slightly different scaling, which they will even with identical gearing, because belt tension and pulley tolerance differ between them. Forcing a single shared value would make circles come out as ellipses.

8.25 EDGE_TOLERANCE_MM

```
EDGE_TOLERANCE_MM = 5.0
```

Output. 5.0, the margin kept clear of every limit switch, in millimetres.

What it is for After homing, the usable area is defined as the measured travel less this margin at each end of each axis, and the coordinate origin is placed this far in from the minimum switches. So the machine never deliberately drives within five millimetres of any switch.

Why a margin is necessary Driving to a position where a switch is about to trip is hazardous in a specific way. A limit switch tripping during normal motion is treated as an emergency: the drive stops, the pen lifts and the job aborts. If the usable area reached the switches exactly, then ordinary rounding, a step lost to a stiff belt, or the thermal expansion of the frame would

occasionally trip a switch during a legitimate move, and a job would abort partway through a drawing for no real fault.

Five millimetres is comfortably more than any of those effects and costs ten millimetres of travel on each axis, under five percent of the bed. That is a cheap price for the difference between a boundary that is occasionally crossed and one that is crossed only when something is genuinely wrong.

$$\text{usableWidth} \;=\; \text{switchTravel} \;-\; 2 \cdot \text{EDGE_TOLERANCE_MM} \tag{8.7}$$

where

usableWidth
: the width the machine will address

switchTravel
: the measured distance between the switches

2
: the margin applies at both ends

> *Because the origin is offset by this margin, the conversion between a coordinate and a step count adds it back: a commanded 0 corresponds to the margin's worth of steps from the minimum switch, not to zero steps. Both _GcodeLinearMove and the position readback have to apply that offset, and a mismatch between them would put every move out by five millimetres.*

8.26 HOMING_STEP_DELAY_S

```
HOMING_STEP_DELAY_S = 0.0008
```

Output. 0.0008, the per-step delay used while homing, in seconds.

Why homing is slow This delay is four hundred times the minimum, making homing much slower than ordinary motion. Three reasons require that.

- **The switch must be detected before the carriage reaches the frame.** At high speed many steps elapse between consecutive switch readings, and the carriage can travel several millimetres past the trip point, which on an axis whose switch sits close to the frame means driving into it.
- **The stop must not lose steps.** Stopping a fast-moving axis abruptly can make the motor skip, and a skipped step during homing corrupts the measurement that calibration exists to produce.
- **The direction is unknown at the start.** Homing drives until something stops it, with no knowledge of how far away that is, so the motion is deliberately cautious.

Since homing happens once per session and takes a few seconds, the cost is negligible, which makes this an easy trade: all the speed in exchange for reliability on the one measurement everything else depends on.

8.27 HOMING_MAX_STEPS

```
HOMING_MAX_STEPS = 200000
```

Output. 200000, the most steps homing will take before giving up.

What it protects against Homing drives until a switch trips. If no switch ever trips, because one is disconnected, miswired or mechanically stuck, the loop would run forever while the motor drives the carriage into the frame and stalls there. This bound turns an endless fault into a reported one.

Why this many At 80 steps per millimetre, 200000 steps is 2500 millimetres, about ten times the longest axis. So a correctly functioning machine cannot approach the limit however far from the switch it starts, while a broken one reaches it in under a minute at the homing delay. The margin is deliberately generous because the consequence of setting it too low, a false failure on a machine that was simply far from its switch, is worse than taking longer to report a real one.

> *Exceeding it raises an error naming the wiring as the likely cause, which is the right message: by far the commonest reason for a switch never tripping is a connector that has worked loose, and the person reading the message is standing next to the machine.*

8.28 ENDSTOP_DEBOUNCE_S

```
ENDSTOP_DEBOUNCE_S = 0.006
```

Output. 0.006, the gap between confirmation samples of a switch, in seconds.

Background: contact bounce and electrical noise A mechanical switch does not close cleanly. Its contacts strike and separate several times over a few milliseconds before settling, so a single reading taken during that window may catch it either way. This is **contact bounce**. Separately, the wires running to the switches are long and run alongside the motor cables, which carry switched currents of an amp or more, and that induces voltage spikes on the switch lines.

Either effect can make a switch read as closed when it is not. Since a switch reading closed triggers an emergency stop, a spurious reading aborts a drawing that was proceeding correctly, which on a long page is a real cost.

The remedy: sample repeatedly, spaced out in time A switch is believed only if it reads closed on several consecutive samples with this delay between them. The spacing is what makes it work: bounce and induced noise are brief, lasting microseconds to a millisecond or two, so samples six milliseconds apart are effectively independent of one another and a transient cannot be present in all of them. Sampling rapidly instead would read the same disturbance several times and confirm it.

8.29 ENDSTOP_CONFIRM_SAMPLES

```
ENDSTOP_CONFIRM_SAMPLES = 3
```

Output. 3, how many consecutive readings are required before a switch is believed.

The trade-off More samples reject noise more reliably and delay a genuine detection. The delay matters because the axis keeps moving while the switch is being confirmed: with three samples six milliseconds apart the confirmation takes twelve milliseconds, during which a carriage travelling at the drawing feed covers about 0.2 millimetres. Three is enough that a transient surviving all three is very unlikely, while the overshoot stays well inside the five-millimetre margin kept clear of the switches.

Why it exits early on a disagreement The check stops as soon as any sample reads open, concluding that the switch is not pressed. This is the asymmetry that makes the cost acceptable: the expensive path, taking all three samples, is only followed when the first reading is closed, which is rare. In the normal case of a switch reading open the function returns after one reading, so the debounce costs nothing during ordinary motion, and it is called before every single step.

8.30 RETRACT_MM

```
RETRACT_MM = 5.0
```

Output. 5.0, how far to back off a tripped switch, in millimetres.

Why a retract is needed after a stop When a switch trips, the carriage is resting against it. Leaving it there is unworkable for two reasons. The switch continues to read closed, so every subsequent check sees a tripped endstop and refuses to move, including the move needed to get clear: the machine is stuck. And a switch held closed under pressure can be damaged over time.

So after an emergency stop the axis is stepped away from the switch until it releases, by this distance. Five millimetres matches the edge margin, which is sensible: it is the distance that puts the carriage back inside the usable area, so the machine ends up where it is allowed to be rather than just clear of the switch.

Why the retract watches the other switches The retract loop checks for any switch other than the one being retracted from, and stops if one trips. That case means something is genuinely wrong, a switch stuck closed or an axis shorter than believed, and continuing to drive would push against a second limit. Stopping and reporting is the correct response, since the situation cannot be resolved by moving.

8.31 PEN_UP_OVERRUN_S

```
PEN_UP_OVERRUN_S = 0.04
```

Output. 0.04, extra running time after the pen switch first trips, in seconds.

The problem it solves The switch trips the moment the pen reaches the up position, and stopping the motor there leaves the cam exactly at the point where the switch just closed. That is the worst place to stop. A small amount of backlash, a vibration from the next move, or the

pen's own weight can rotate the cam back past the trip point, and the pen drops onto the page during a travel move, drawing a line across the drawing.

Running on for a further 40 milliseconds carries the cam past the trip point into the middle of its up region, where neither backlash nor vibration can bring it back. The principle is general: when a detector marks the boundary of a safe region, stop inside the region rather than on its edge.

Why only on the way up The overrun is applied when raising the pen and not when lowering it. Down is the position gravity and the pen's spring favour, so it is stable without help, whereas up is held against them. Overrunning on the way down would also press the pen harder into the paper than intended, which broadens the stroke and can damage the nib.

8.32 GCODE_MIN_FEED_MM_S

```
GCODE_MIN_FEED_MM_S = 0.1
```

Output. 0.1, the slowest feed rate accepted, in millimetres per second.

What it protects against The step delay is computed by dividing the move's duration by its step count, and the duration is the distance divided by the feed. A feed of zero therefore divides by zero. A file containing `F0`, whether through a typing error or a generator's fault, would crash the job partway through a drawing.

Why a floor rather than a rejection Clamping to a very slow but finite rate means such a line produces a slow move instead of a failure. That is the better behaviour here because the alternative, aborting, loses the whole drawing, whereas one unusually slow move is a visible pause and nothing more. At a tenth of a millimetre per second the move is clearly wrong and clearly deliberate, so it is noticeable without being destructive.

8.33 _GCODE_WORD_RE

```
_GCODE_WORD_RE = re.compile('([A-Za-z])\\s*(-?[0-9]*\\.?[0-9]+)')
```

Output. the compiled pattern that splits a G-code line into letters and numbers.

What a G-code word is A G-code line is a sequence of **words**, each a letter followed by a number: `G1`, `X12.5`, `F900`. This pattern matches one such word, capturing the letter and the number separately, and finding every match in a line splits it into its parts.

Why the pattern allows space between them It permits whitespace between the letter and the number, so `X 12.5` parses as well as `X12.5`. Both forms occur, because different generators format differently and some files are hand-edited. A reader of a standard interchange format should accept the variations the standard permits rather than only the form it writes itself.

Why the number pattern is as it is It accepts an optional minus sign, any number of digits, an optional decimal point and more digits, so it matches `5`, `-5`, `5.25` and `.5` alike. The last of those is why the integer part is optional: a coordinate written as `.5` is valid and would otherwise be read as a word with no number at all.

> *The pattern is compiled once at module level rather than on each use. A compiled pattern is parsed into its matching machinery once instead of on every call, which matters here because this is applied to every line of a file that may run to tens of thousands.*

8.34 _RETREAT_SIGN

```
_RETREAT_SIGN = {'X_MIN': +1, 'X_MAX': -1, 'Y_MIN': +1, 'Y_MAX': -1}
```

Output. a mapping from each switch's name to the direction to move to get off it.

What it encodes Which way is away from each switch: positive for the minimum switches and negative for the maximum ones. A minimum switch sits at the low end of the axis, so escaping it means moving toward higher coordinates, and the reverse for a maximum.

Why it is a table and not computed from the name The sign could be derived by testing whether the name ends in MIN. A table states it instead, which is better for a value with physical consequences: driving the wrong way off a switch pushes the carriage harder into its limit. A table is read directly against the machine, whereas a derivation depends on a naming convention holding, and a switch named differently would silently produce the wrong direction rather than a missing-key error.

8.35 Calibration

```
Calibration = {'done': False,'steps_per_mm_x': STEPS_PER_MM,
        'steps_per_mm_y': STEPS_PER_MM,
        'usable_width_mm': X_SWITCH_TRAVEL_MM - 2 * EDGE_TOLERANCE_MM,
        'usable_height_mm': Y_SWITCH_TRAVEL_MM - 2 * EDGE_TOLERANCE_MM}
```

Output. a dictionary holding the measured machine geometry and whether homing has been done.

What it holds The measured steps per millimetre for each axis, the usable area in millimetres, and a flag saying whether calibration has run. Everything in it is a measurement rather than a configuration: the values are produced by driving the machine and counting, not by being stated.

Why the flag is checked rather than the values Code requiring calibration tests the flag, not whether the steps-per-millimetre values look plausible. That is the right test because the values have defaults that are perfectly plausible, being the calculated 80: there is no value of them that means "not yet measured". An explicit flag can express that where the data cannot.

> *This is module-level mutable state, shared by every function in the hardware half, which is the pattern the whole bottom half uses. It is defensible here in a way it would not be elsewhere:*

there is exactly one physical machine, so a single shared record of its state is an accurate model of the situation, and two independent copies would be able to disagree about a machine that can only be in one state.

8.36 req

```
req = None
```

Output. the live GPIO line request once a connection has been made, or nothing before that.

What it holds The handle through which every pin is set and read. It is obtained by requesting a set of lines from the GPIO chip, and until that has happened it is nothing, which is also how the module knows whether it is connected.

Why requesting lines is exclusive The Linux GPIO interface grants a requested line exclusively to the process that asked for it: no other program can drive the same pin while the request is held. That is essential here, since two programs both driving a stepper's step pin would produce motion neither intended. It also means the request must be released on shutdown, or the pins remain claimed by a process that has exited and the next run cannot acquire them.

Using the handle's presence as the connection flag means there is one piece of state rather than two, so the flag cannot disagree with reality. A separate boolean would be able to say connected while the handle was absent, which is the kind of inconsistency that produces an error far from its cause.

8.37 CurrentMode

```
CurrentMode = 'RPM'
```

Output. which of the manual control modes the machine is in.

What the modes are The hardware half includes a small interactive console for testing, and this says what it is currently doing: turning the motors at a set speed, drawing a test circle, running a G-code file, or homing. The main loop reads it each pass and acts accordingly.

Why a mode variable rather than separate loops A single loop that consults a mode can be interrupted and redirected from the keyboard thread at any moment, because the thread only has to change one variable. Separate loops per mode would each need their own mechanism for being told to stop, and switching between them would mean leaving one loop and entering another, during which the motors would be in an undefined state.

The whole manual console exists for bringing the machine up and diagnosing it, not for drawing. A drawing job is run by RunGcodeFile, which does not consult the mode at all.

8.38 targetRpm

```
targetRpm = 0.0
```

Output. 0.0, the wanted motor speed in revolutions per minute.

Why the initial value matters The code comment is explicit that this must start at zero, and the reason is a real hazard rather than tidiness. The main loop begins stepping as soon as it starts, at whatever speed this says. A non-zero initial value would move the gantry the instant the program ran, before the operator had asked for anything and quite possibly before homing, so the machine would drive from an unknown position toward an unknown one.

A device that moves on startup is a genuine safety problem, and the general principle is that a machine's initial state should be the one in which it does nothing. The same reasoning sets the step pins inactive and the pen motor off when the pins are first claimed.

8.39 stepDelayS

```
stepDelayS = 0.0
```

Output. 0.0, the pause between step pulses derived from the wanted speed.

What it is The delay the main loop waits between step edges, computed from the requested speed by CalculateDelay. A value of zero means stopped, which is consistent with the initial speed of zero.

> *A delay and a speed are inversely related, so this variable and targetRpm hold the same information in two forms. The derived form is cached because it is needed in the innermost loop, where recomputing a division per step would be wasteful. The hazard of a cached derivative is that the two can disagree, which is why the only function that sets the speed also sets this.*

8.40 running

```
running = True
```

Output. true while the main loop should continue.

How a thread is asked to stop The main loop tests this each pass and exits when it becomes false, which the keyboard thread sets on a quit command. This is a **cooperative shutdown**: the loop is asked to stop and does so at a point of its own choosing, rather than being terminated from outside.

Why that matters for a machine Terminating a thread part way through a step pulse would leave a step pin high, so the driver would hold the coil energised indefinitely, heating the motor. Letting the loop finish its current iteration guarantees every pulse completes and the shutdown path can then set the pins to a safe state and release them. For software a forced stop is usually recoverable; for a machine holding a physical state it is not.

8.41 CurrentXSteps

```
CurrentXSteps = 0
```

Output. 0, the X axis position in steps from the homing reference.

Why the position is tracked in software This is the machine's only knowledge of where it is. A stepper motor has no position feedback: it moves when pulsed and reports nothing. So the position is maintained by counting, incremented or decremented on every single step issued by every motion routine.

What that implies, and why it is accepted The count is the position only as long as every pulse produced a real step. A motor that stalls, from an obstruction or an over-fast acceleration, skips steps that were counted, and the recorded position silently becomes wrong by that amount with no way to detect it. This is **open-loop** control, and it is universal in low-cost motion systems because closing the loop requires an encoder on every axis.

What makes it workable is that the error does not accumulate unnoticed indefinitely: homing re-establishes the position from a physical reference, and the limit switches catch the case where the error is large enough to drive into a limit. Within one drawing, the loads are light and the feeds modest, so stalling is unlikely.

> *A step count is an integer, so counting cannot accumulate rounding error the way tracking a position in millimetres would. That is the reason the count is kept in steps and converted to millimetres only when a coordinate is needed.*

8.42 CurrentYSteps

```
CurrentYSteps = 0
```

Output. 0, the Y axis position in steps.

Why the axes are counted separately Each axis has its own motor, its own switches and its own calibrated scaling, so each needs its own count. Keeping them separate also means the two can be converted with different steps-per-millimetre figures, which they must be: the calibration measures each axis independently and the two differ slightly even with identical gearing.

8.43 theta

```
theta = 0.0
```

Output. 0.0, the current angle of the test circle in radians.

What it is for The circle test mode advances this angle a little each pass and moves to the corresponding point on the circle, so the carriage traces a continuous circle. It persists between loop passes because the position must continue from where it reached, not restart.

This is a convenience for mechanical testing rather than part of drawing. A circle run continuously is the most useful single test pattern for a gantry, since it exercises both axes in constantly varying proportion and makes axis mismatch and belt backlash visible as a distorted shape.

8.44 LastCircleTime

```
LastCircleTime = time.perf_counter()
```

Output. the time the circle test last advanced.

Why the time is recorded rather than the loop simply sleeping The circle must advance at a steady rate, and the main loop does other work whose duration varies. Recording when the last advance happened and comparing against the current time lets the loop advance when enough time has elapsed, independently of how long the rest of the pass took. Sleeping for a fixed interval instead would make the real period the interval plus the work, so the circle's speed would depend on what else the loop was doing.

A high-resolution performance counter is used rather than wall-clock time, because it is monotonic: it cannot jump backwards if the system clock is adjusted, which would make an elapsed-time comparison give a negative result.

8.45 CircleIntervalS

```
CircleIntervalS = 0.0015
```

Output. 0.0015, how often the test circle advances, in seconds.

What it sets The interval between successive points of the circle, which together with the angular step sets how fast the circle is traced. At 1.5 milliseconds the motion appears continuous.

8.46 CircleRadiusMm

```
CircleRadiusMm = DEFAULT_CIRCLE_RADIUS_MM
```

Output. the current test circle radius in millimetres.

Why it is separate from the default It starts from the default constant and can be changed from the keyboard during testing, so the current value and the default are distinct: the default is what the machine starts with and this is what it is doing now. Varying the radius during a test is useful because the faults a circle reveals show up differently at different sizes, backlash being most visible on a small circle and axis mismatch on a large one.

8.47 CircleRadiusSteps

```
CircleRadiusSteps = CircleRadiusMm * STEPS_PER_MM
```

Output. the same radius converted into steps.

Why it is precomputed The circle loop needs the radius in steps on every pass, and converting once avoids repeating the multiplication inside the motion loop.

> *The conversion uses the uncalibrated constant rather than the measured scaling, so a circle drawn after calibration is slightly the wrong size. For a mechanical test pattern that does not matter, since what is being looked at is the shape rather than the dimensions, but it is the same duplication hazard noted under STEPS_PER_MM.*

8.48 estopped

```
estopped = False
```

Output. false, becoming true once an emergency stop has fired.

What it enforces Once an emergency stop has happened, this stays set until someone clears it from the console. Every motion path checks it, so nothing moves in the meantime.

Why the stop latches instead of clearing itself This is the important property. After the retract, the switch is no longer pressed, so a test that only looked at the switches would conclude all was well and allow motion to resume. But the reason the switch tripped has not been addressed: the coordinate was wrong, or the belt slipped, or the work is not where it was thought to be. Resuming would drive into the same limit again.

A **latching** fault requires a human to acknowledge it, which is the standard arrangement for a machine's emergency stop and for the same reason: the stop indicates that the machine's idea of the world is wrong, and only a person can establish whether it is right again.

8.49 TrippedSwitch

```
TrippedSwitch = None
```

Output. which switch caused the emergency stop, or nothing.

Why the identity is kept Knowing that a stop occurred is not enough to act on it. Which switch tripped says which axis and which end, which tells the retract which way to move and tells the operator where to look. The message printed on a stop names the switch for exactly that reason.

8.50 Connect

```
def Connect()
```

Output. nothing; it claims the GPIO lines or raises if it cannot.

What it does Requests every pin the machine uses from the GPIO chip in a single call, declaring each as an input or an output and giving each output its initial level.

Why all the pins are requested at once One request for the whole set is better than ten separate ones. It is atomic: either every pin is obtained or none is, so the module cannot end up holding half the pins it needs, with some motion possible and some not. And it establishes every initial level in the same operation, so there is no window during which some outputs are configured and others are floating.

The initial levels, and why they are what they are

- **Step pins inactive.** A step is a transition, so holding them low means no step is in progress and none is commanded.
- **Pen motor inactive.** The actuator is off, so the pen stays wherever it is rather than beginning to turn.
- **Direction pins active.** The level is arbitrary, since every motion sets the direction before stepping, and a direction pin by itself causes no movement.
- **Every switch an input.** Driving a switch line as an output would fight the switch and could damage the pin.

The common thread is that the initial state must be one in which nothing moves, which is the same principle that puts the initial speed at zero.

Why failure is translated Any failure to open the chip is caught and re-raised as the module's own not-connected error, with the original attached. The handle is set back to nothing first, so a failed connection does not leave the module believing it is connected. Translating the error lets a caller distinguish an absent machine, which is expected on a laptop, from a fault on a machine that is present, without having to know what the GPIO library raises.

> *Calling it twice returns immediately rather than requesting the lines again, which would fail since the lines are held exclusively by this same process. Making repeat calls harmless lets every entry point call it without tracking whether someone else already has.*

8.51 Disconnect

```
def Disconnect()
```

Output. nothing; it sets the outputs safe and releases the lines.

The order of operations The step pins and the pen motor are set inactive first, and only then are the lines released. The order is necessary: once a line is released, its level is no longer controlled, and a line released while active may stay active. Releasing a step pin while high would leave the driver holding a coil energised, which heats the motor with nothing running to notice.

Why the release is in a finally clause The setting of safe levels is wrapped so that the release happens whether or not it succeeded. If a pin cannot be set, because the chip has gone away or the device was unplugged, the lines must still be released, or they stay claimed by a process that has exited and the next run cannot acquire them. Releasing a resource must not depend on the success of the tidying done before it.

8.52 IsConnected

```
def IsConnected()
```

Output. whether the GPIO lines are currently claimed.

What it is for Tests whether the handle exists. Callers use it to report the machine's availability without attempting an operation that would raise, which is what a user interface needs: it must say whether the machine is there, not find out by trying to move it.

8.53 CalculateDelay

```
def CalculateDelay(rpm)
```

Inputs

`rpm` the wanted motor speed in revolutions per minute

Output. nothing; it sets the module's step delay.

The conversion

$$t_{\text{delay}} = \frac{1}{2} \cdot \frac{60}{|\text{rpm}| \cdot \text{stepsPerRev}} \tag{8.8}$$

where

t_{delay}
: the pause between step pin edges, in seconds

rpm
: the wanted speed in revolutions per minute

stepsPerRev
: microsteps per revolution, 3200 here

60
: seconds per minute, converting the speed into revolutions per second

$\frac{1}{2}$
: the halving, explained below

Read it from the inside out. The speed in revolutions per minute divided by 60 is revolutions per second; multiplied by the steps per revolution it is steps per second; its reciprocal is seconds per step. That is the time one whole step must occupy.

Why the result is halved A step is produced by two transitions: the pin goes high, waits, goes low, waits. So the delay is used twice per step, and each wait must be half the step period for the step to take the time intended. Omitting the halving would run the motor at half the requested speed, which is the kind of error that looks like a mechanical problem.

The dead band at low speeds A requested speed below 0.1 revolutions per minute sets the delay to zero, meaning stopped. Without it, a speed approaching zero gives a delay approaching infinity, and the loop would wait essentially forever between steps with no way to interrupt it. Treating a very small speed as stopped is both what the operator means and the only behaviour that remains controllable.

8.54 TriggeredEndstop

```
def TriggeredEndstop(ignore=frozenset())
```

Inputs

`ignore`
: switch names to disregard

Output. the name of a switch confirmed to be pressed, or nothing.

What it is for The safety check, called before essentially every step of every motion. It reads each switch and returns the name of the first confirmed to be pressed.

How the debounce is arranged A switch reading open is dismissed immediately. One reading closed is then sampled twice more, spaced by the debounce delay, and is believed only if all three agree. The reasoning behind the spacing and the count is given under ENDSTOP_DEBOUNCE_S and ENDSTOP_CONFIRM_SAMPLES.

Why the asymmetry is what makes this affordable This function runs before every step pulse, so its cost is multiplied by the tens of thousands of steps in a drawing. The early dismissal is what keeps that cost down: in the normal case every switch reads open and the function performs four pin reads and returns, with no delays at all. The twelve milliseconds of confirmation are spent only when a switch actually reads closed, which happens once per job at most.

Why some switches can be ignored The ignore set exists because there are legitimate situations in which a switch is pressed and should not stop the machine. Retracting off a switch necessarily begins with that switch pressed, and homing one axis may hold a switch while the other axis is moved. Without the ability to disregard a named switch, the only remedy would be to omit the safety check entirely during those operations, which would remove it exactly where the machine is deliberately being driven against its limits.

> *The check returns at the first switch it confirms rather than examining all four. For stopping that is sufficient, since one tripped switch is reason enough. The separate _AllTriggeredEndstops exists for the emergency stop, which does need the full list so that it can retract off every one of them.*

8.55 _AllTriggeredEndstops

```
def _AllTriggeredEndstops()
```

Output. a list of every switch currently confirmed as pressed.

Why a complete list is needed An emergency stop must retract off every pressed switch, not just the one that was noticed first. Two can be pressed at once: a carriage driven into a corner reaches an X and a Y limit together, and retracting off only one leaves the machine still held against the other, so the next motion check still finds a tripped switch and nothing can move.

How it is built from the single check Each switch is tested by calling the single check with every *other* switch in the ignore set, so the answer concerns that switch alone. It is a roundabout construction, but it reuses the debounce logic rather than duplicating it, which matters for a safety check: two implementations of a debounce could diverge, and the one in the rarely exercised path is the one that would not be noticed.

8.56 AxisDirValue

```
def AxisDirValue(invert, positive)
```

Inputs

`invert`
: whether this axis's direction signal is reversed

`positive`
: whether the wanted motion is in the positive direction

Output. the level to set on the direction pin.

What it resolves Turns a wanted direction into a pin level, accounting for the axis's wiring inversion. It is the single place the inversion is applied, so every motion routine inherits it by asking for a direction by meaning rather than by level.

$$\text{forward} \;=\; \text{positive} \oplus \text{invert} \tag{8.9}$$

where

forward
: whether the direction pin should be driven active

positive
: whether the wanted motion is toward increasing coordinates

invert
: this axis's wiring inversion flag

$\oplus$ **exclusive or**: true when exactly one of the two is true

Exclusive or is exactly the right operation, and it is worth checking the four cases. Wanting positive motion on an uninverted axis drives the pin active; wanting negative on the same axis drives it inactive. On an inverted axis both answers swap. So the output is active when the two inputs disagree, which is what exclusive or computes. Written as nested conditions it would be four branches that must each be right; as an exclusive or it is one expression that cannot be half correct.

8.57 _RetractOffSwitch

```
def _RetractOffSwitch(hit, ignore=frozenset())
```

Inputs

`hit` the name of the pressed switch

`ignore`
: other switches to disregard while retracting

Output. nothing; it steps the axis clear and updates the position count.

What it does Steps the affected axis away from the switch by the retract distance, at homing speed, updating the step count as it goes. The direction comes from the retreat table and the axis and pins from the switch's name.

Why it watches the other switches Each step checks for any switch other than the one being escaped and the ones explicitly ignored, and stops if one trips. The code comment calls that a real fault, and it is: moving away from one limit should not approach another, so if it does, the machine's geometry is not what is believed, perhaps because a switch is stuck closed or an axis is shorter than its recorded travel. Continuing to drive would push against a second limit.

Why the position count is updated The retract moves the carriage, so the recorded position must follow, or every subsequent absolute move is out by the retract distance. The count is accumulated during the loop and applied to the right axis afterwards, which also means a retract cut short by another switch updates by the distance actually travelled rather than the distance intended.

> *Retracting at homing speed rather than drawing speed is deliberate. The machine is at a limit, in a state it was not supposed to reach, and the geometry is not trusted. Moving slowly means the other-switch check has time to act before much further travel occurs.*

8.58 _TriggerEmergencyStop

```
def _TriggerEmergencyStop(hit)
```

Inputs

`hit` the switch that caused the stop

Output. nothing; it stops the drive, retracts and latches the fault.

The sequence, and why it is in this order

- **Latch the fault first,** before anything else. Setting the flag and recording the switch immediately means that any other code checking it from this point on sees the stop, including the retract that follows.
- **Set the step pins inactive.** This ends any pulse in progress so no coil is left energised mid-step.
- **Find every pressed switch,** since more than one may be.
- **Retract off each of them,** so the machine is clear of its limits and able to move again once the fault is cleared.
- **Report,** naming the switches and the retract distance.

Why re-entry is refused The function returns immediately if the fault is already latched. Without that, a second switch tripping during the retract would trigger another stop, which would retract again, which could trip again, and the recursion would continue until the program ran out of stack. The guard makes the stop happen once and the retract complete, which is the behaviour wanted: the first stop has already established that the machine is to halt, and a second detection adds nothing.

Why the pen is raised Raising the pen is part of stopping, not a courtesy. A pen left down during an abort rests on the paper at the point of failure, where the ink bleeds into a visible blot, and if the machine is subsequently moved by hand it drags a line across the drawing. The drawing is already spoiled by the abort; the object is to avoid spoiling it further.

8.59 BresenhamMove

```
def BresenhamMove(dx, dy, stepDelayS=CIRCLE_STEP_DELAY_S)
```

Inputs

`dx` the X displacement in steps

`dy` the Y displacement in steps

`stepDelayS`
: the pause between step edges

Output. nothing; it steps both motors so the carriage travels in a straight line.

The problem Two motors must move together so that the carriage travels along a straight line from one point to another. Each can only take whole steps, so the path is necessarily a staircase; the task is to arrange the staircase so that it stays as close to the true line as whole steps permit. A move of 300 steps in X and 100 in Y must take one Y step for every three X steps, spread evenly, not all 100 at the start.

The naive approach, and why it is avoided The obvious method computes, for each X step, the exact Y position on the line and rounds it. That requires a floating-point multiplication and a rounding per step, and this loop runs for every one of tens of thousands of steps in a drawing. It also accumulates rounding error, since the exact position is recomputed from a division that is not exact.

Bresenham's method The algorithm [23] keeps a running error term instead, representing how far the minor axis has fallen behind the ideal line, and uses only integer addition and comparison.

$$\begin{aligned} & & e &\leftarrow \lfloor a_x/2 \rfloor \\ \text{for each major step:} & & e &\leftarrow e - a_y \\ & \text{if } e < 0: & &\text{step the minor axis, } e \leftarrow e + a_x \end{aligned} \tag{8.10}$$

Source: [23]

where

e the error term: how far the minor axis is behind

a_x the major axis displacement in steps, taken as positive

a_y the minor axis displacement, positive

$\lfloor a_x/2 \rfloor$
: the starting value, which centres the error so the first minor step falls half-way through the first run rather than at its start

The reasoning is a scaled comparison. The true slope is the minor displacement over the major, and testing whether the accumulated fraction has passed a half-step would need division. Multiplying the whole test through by the major displacement removes the division: subtracting the minor displacement each major step accumulates the same quantity scaled up, and comparing against zero is the same test as comparing the unscaled fraction against its threshold. Adding the major displacement back when the minor axis steps is the scaled equivalent of subtracting one whole step.

So the loop contains one subtraction, one comparison and occasionally one addition, all on integers. There is no division, no floating point and no accumulated rounding error: the error term is exact at every step, so the path is the closest staircase to the true line by construction rather than by approximation.

Why the major axis is chosen first The loop runs over whichever axis has the greater displacement, with the other as the minor. That is necessary rather than cosmetic: the minor axis steps at most once per major step, so if the shorter axis were driven as the major the longer one could not keep up and the line would come out at the wrong angle. The two branches of the code are mirror images, differing only in which pin is pulsed and which count is updated.

The safety check inside the loop Every iteration checks the endstops and returns immediately if one has tripped. Returning part way leaves the move incomplete and the carriage somewhere along the line, which is why the step counts are updated as the loop proceeds rather than at the end: the recorded position is correct for where the carriage actually stopped, not for where it was told to go.

Bresenham's algorithm was published for drawing lines on raster displays, where the problem is identical: choose which pixels best approximate a straight line between two points. A stepper gantry is the same problem with the pixels replaced by step positions, which is why an algorithm from computer graphics is the standard solution for motion control.

8.60 PenIsUp

```
def PenIsUp()
```

Output. whether the pen switch currently reads as up.

What it is for Reads the pen position switch. This is the capability the written G-code file lacks: the machine can ask where the pen is, so it never has to assume.

The read is not debounced, unlike the endstop check. That is acceptable because of how it is used: SetPen polls it repeatedly in a loop while the motor runs, so a single spurious reading advances the loop by one iteration and the next reading corrects it. A spurious endstop reading, by contrast, aborts the whole job, which is why that one is confirmed and this one is not.

8.61 SetPen

```
def SetPen(targetUp, timeoutS=3.0)
```

Inputs

`targetUp`
: the pen position wanted

`timeoutS`
: how long to wait for confirmation, default 3 seconds

Output. true if the pen reached the wanted position, false otherwise.

The closed loop The motor is started, the switch is polled until it reports the wanted position or the timeout expires, and the motor is stopped. This is **closed-loop** control: the action continues until the measurement confirms the result, in contrast with the written file's timed dwell, which commands a pulse and assumes it worked.

Why it checks the position first If the pen already reads as being where it is wanted, the function returns immediately without running the motor. The code comment is emphatic, and the reason is the one-way actuator: running it when the pen is already correct turns the cam a further quarter turn and moves the pen to the wrong position. For a toggle, a redundant command is not merely wasteful but actively wrong.

Why it refuses while an endstop is tripped A pen move is declined outright if any limit switch is pressed. The machine is in a fault state, the pen motor draws current from the same supply as the steppers, and moving the pen does nothing to resolve the fault. Refusing keeps the machine still until the fault has been dealt with, which is what a latched stop is for.

Why a timeout, and what its expiry means If the switch never reports the wanted position within three seconds, the motor is stopped and the function reports failure. A toggle takes about a third of a second, so three seconds is ample and its expiry means something is mechanically wrong: the motor is not turning, the switch is disconnected, or the linkage has broken. Without a timeout the motor would run indefinitely, turning the cam round and round and overheating. Reporting the failure lets the caller abort rather than continuing to draw with a pen in an unknown position.

The overrun On a successful raise the motor runs a further 40 milliseconds past the switch trip, for the reason given under PEN_UP_OVERRUN_S: stopping exactly on the trip point leaves the cam where backlash or vibration can drop the pen onto the page during a travel move.

8.62 _StepAxis

```
def _StepAxis(stepPin, dirPin, dirValue, stopWhen, ignore=frozenset())
```

Inputs

`stepPin`
: the step pin for this axis

`dirPin`
: its direction pin

`dirValue`
: the level to drive the direction pin at

`stopWhen`
: a test saying which switch is the expected one

`ignore`
: switches to disregard

Output. the switch that stopped the motion and how many steps were taken.

What it is for Drives one axis in one direction until a switch stops it, counting the steps. It is the primitive the homing and calibration are built from, and the step count it returns is the measurement that calibration converts into steps per millimetre.

Why the stopping condition is a test rather than a name The caller supplies a function saying which switch is the expected one, rather than naming it. That is necessary because homing begins without knowing which end of the axis it will reach: the carriage may be anywhere, so driving in one direction may bring it to either switch. The first call accepts either, and the second accepts only the opposite one.

Why an unexpected switch is an error rather than a stop A switch that trips which is not the expected one raises an error naming the wiring, rather than being treated as a normal stop. The distinction matters because the step count is about to be used as a measurement: stopping on the wrong switch means the count does not measure what it is believed to, and a calibration built on it would scale every subsequent move wrongly. A measurement known to be invalid should fail rather than be used.

Why the step limit is also an error Exceeding the step bound raises an error too. A correctly functioning machine reaches a switch in a fraction of that distance, so the bound being reached means no switch is going to trip, which is the disconnected-wiring case. Again the appropriate response is to stop and say so, since there is no sensible way to proceed without a reference position.

8.63 _CalibrateAxis

```
def _CalibrateAxis(stepPin, dirPin, minName, maxName, OtherAxisNames=
    frozenset())
```

Inputs

`stepPin`
: the step pin

`dirPin`
: the direction pin

`minName`
: the minimum switch's name

`maxName`
: the maximum switch's name

`OtherAxisNames`
: the other axis's switches, to disregard

Output. the first switch reached, the second, and the steps measured between them.

The measurement The axis is driven in one direction until either switch trips, then reversed until the opposite one does, counting the steps of the second move. That count is the full travel of the axis in steps, which divided by the known travel in millimetres gives the steps per millimetre.

Why the count comes from the second move The first move starts from wherever the carriage happened to be and ends at a switch, so its length is arbitrary and carries no information. The second starts at a known position, one switch, and ends at another known position, the other switch, so its length is the distance between two physical references. Only a move between two known points measures anything.

Why the first switch is ignored during the reverse The reverse move begins with the first switch still pressed, since the carriage is resting against it, and it stays pressed for the first few steps until the lever releases. The code comment calls this release lag and notes it is not a fault. Without adding that switch to the ignore set, the reverse move would immediately detect it, decide it was unexpected, and raise an error on every calibration.

Why the other axis is ignored throughout The other axis may be resting against one of its own limits while this one is being calibrated, which is a perfectly normal state and nothing to do with this axis's motion. Ignoring its switches lets each axis be calibrated regardless of where the other happens to be parked.

8.64 Calibrate

```
def Calibrate()
```

Output. a copy of the calibration record: the measured scaling for both axes and the usable area.

What it establishes Two things the machine cannot otherwise know: where it is, and how far a step moves it. Both come from driving to physical references, since the motors report neither.

The sequence

- **Refuse if a switch is already pressed.** Calibration drives until a switch trips, so starting against one makes the first move meaningless.
- **Measure the X axis** between its two switches, and set its steps per millimetre from the count.
- **Retract off the switch** it finished against, so it is clear of its limit.
- **Measure the Y axis** the same way, ignoring the X switches.
- **Retract** again, record the usable area, and park at the safe corner.

Where the position count comes from After each axis, the step count is set to the full travel if the axis finished at its maximum switch, and to zero if at its minimum. That is the whole point of homing: the carriage is at a switch, the switch is at a known place, so the count can be set rather than guessed. Every subsequent absolute move is measured from it.

The usable area and the switch-travel allowance

$$\text{usable} = \text{switchTravel} - 2 \cdot \text{edgeTolerance} \tag{8.11}$$

where

usable
: the area the machine will address, per axis

switchTravel
: the hand-measured distance between the two switches

edgeTolerance
: the margin kept clear at each end

The usable area is deliberately smaller than the measured travel, for the reasons given under EDGE_TOLERANCE_MM. The coordinate origin is placed at the margin's distance in from the minimum switches, so a commanded zero is a position the machine is allowed to occupy rather than the switch itself.

Why it parks at the end Calibration finishes with a move to the origin of the usable area. That leaves the machine at a known, legal position, clear of every switch, which is what a drawing job expects to start from. Finishing while resting against a limit would mean the first motion check of the next job found a tripped switch.

8.65 ParseGcodeLine

```
def ParseGcodeLine(raw)
```

Inputs

`raw` one line of a G-code file

Output. the command and its parameters, or nothing for a line with no command.

What it does Strips comments, splits the line into words, takes the first G or M word as the command, and collects the rest as parameters keyed by their letters.

The two comment conventions G-code has two: a semicolon makes the rest of the line a comment, and text in parentheses is a comment wherever it appears, including in the middle of a line. Both are handled, the first by truncating at the semicolon and the second by removing every parenthesised span. Handling only the first would leave comment text to be parsed as words, and a comment containing a letter followed by a digit would be read as a parameter.

Why the first G or M word becomes the command A line has one command and any number of parameters, and the command is identified by being a G or M word. Taking the first such word rather than the first word generally is what lets a line with a leading line number be parsed, and the test that the command slot is still empty is what stops a later G word being mistaken for a second command.

> *The command's number is converted to an integer while parameters stay as floating point. That is correct for each: G1 and G01 denote the same command, so an integer normalises them, while a coordinate of 12.5 must keep its fraction.*

8.66 _ClampToUsableArea

```
def _ClampToUsableArea(xMm, yMm)
```

Inputs

`xMm` the target X coordinate

`yMm` the target Y coordinate

Output. the coordinates forced inside the homed usable area.

Why the machine clamps as well as the writer The G-code writer already clamps to the configured bounds. This clamps again, against the *measured* usable area, and the two are not the same check. The writer's bounds are configuration, stated in advance; this one is what calibration actually found, which may be smaller, and it applies to any file whatever its origin, including a hand-written one or one from another program.

The principle is that the machine must protect itself rather than trusting its input. A clamp at the point of motion is the last line of defence and the only one that cannot be bypassed by using a different generator.

Why it prints when it acts A clamp that fires means the file asked for something outside the machine, so the drawing will be wrong: strokes are flattened against the boundary. Printing the original and clamped coordinates makes that visible while the job runs, so the operator can stop it rather than discovering afterwards that part of the drawing is a straight line along one edge.

8.67 _GcodeLinearMove

```
def _GcodeLinearMove(targetXMm, targetYMm, feedMmMin)
```

Inputs

`targetXMm`
: the destination X coordinate

`targetYMm`
: the destination Y

`feedMmMin`
: the feed rate in millimetres per minute

Output. nothing; it moves the carriage there at that rate.

What it does Converts a destination in millimetres into a step displacement, works out the step timing from the feed rate, and hands the move to the Bresenham stepper.

The coordinate conversion

$$\text{targetSteps}_X \;=\; \text{round}\big((x_{\text{mm}} + \text{edgeTol}) \cdot \text{stepsPerMm}_X\big) \tag{8.12}$$

where

targetSteps_X
: the absolute step position wanted

x_{mm}
: the commanded coordinate

edgeTol
: the margin, added because a commanded zero sits that far in from the minimum switch

stepsPerMm_X
: the calibrated scaling for this axis

The margin must be added here and subtracted when the current position is read back, and the two must agree. If one applied the offset and the other did not, every move would be out by the margin, and because both appear in the same function the error would be a constant five-millimetre shift of the whole drawing.

The step timing

$$t_{\text{step}} \;=\; \max\!\left(\frac{d_{\text{mm}}/v}{\text{majorSteps}},\; t_{\min}\right), \qquad v = \max\!\left(\frac{\text{feed}}{60},\; v_{\min}\right) \tag{8.13}$$

where

t_{step} the delay between step edges

d_{mm} the straight-line distance of the move

v the feed rate in millimetres per second

majorSteps
: the step count on the axis that moves furthest, which is how many loop iterations the move takes

$t_{\min}$ the minimum step delay

$v_{\min}$ the minimum feed, guarding against a zero feed rate

60 the conversion from per-minute to per-second

Dividing by the major axis step count rather than the total is correct because the Bresenham loop iterates once per major-axis step, taking a minor step inside some of those iterations. So the major count is the number of delays the move will actually contain, and dividing the intended duration by it gives the delay each must be.

> *No acceleration is applied: the move begins and ends at full speed. For a light gantry at these feeds that is tolerable, though it is the likeliest source of lost steps on a fast travel move, and it is why the configuration's acceleration field, which is never read, would be the first thing to implement.*

8.68 _GcodeArcMove

```
def _GcodeArcMove(x0Mm, y0Mm, x1Mm, y1Mm, iMm, jMm, clockwise, feedMmMin)
```

Inputs

`x0Mm`
: the arc's start X

`y0Mm`
: its start Y

`x1Mm`
: its end X

`y1Mm`
: its end Y

`iMm` the centre's X offset from the start

`jMm` the centre's Y offset from the start

`clockwise`
: the direction of travel

`feedMmMin`
: the feed rate

Output. nothing; it draws the arc as a sequence of short straight moves.

Why arcs must be subdivided The machine can only move in straight lines, because that is what the Bresenham stepper does. An arc is therefore approximated by a sequence of short chords. This is what every machine controller does; the only question is how short.

How G-code specifies an arc The command gives the end point and the **centre offset**, the vector from the start point to the centre, conventionally written as I and J. The radius is then the length of that offset, and the start and end angles follow from the two endpoints relative to the centre. Specifying the centre by offset rather than absolutely is the convention because it makes the command independent of where the arc sits.

Forcing the sweep to go the right way The two angles alone do not determine the arc: the same pair of endpoints can be joined by a short arc one way or a long one the other. The direction flag settles it, and the code enforces it by adjusting the end angle by whole turns until the sweep runs strictly the right way. For a clockwise arc the end angle is decreased until it is below the start; for anticlockwise, increased until above. Without that, an arc crossing the point where the angle measurement wraps would be traced the long way round, sweeping nearly a full circle instead of a small arc.

How many chords

$$n \;=\; \max\!\left(4,\; \left\lfloor \frac{|\Delta\theta| \cdot r}{0.4} \right\rfloor\right) \tag{8.14}$$

where

n the number of straight segments the arc becomes

$|\Delta\theta|$ the angle swept, in radians

r the arc's radius in millimetres

0.4 the target chord length in millimetres

$\max(4, \cdot)$
: a floor, so even a tiny arc is drawn as a curve rather than a straight line

The numerator is the arc's length, since arc length is radius times angle in radians, so the expression is simply the arc length divided by the wanted chord length. Fixing the chord length rather than the number of segments is what makes the approximation uniformly good: a large arc gets proportionally more segments, so the deviation from the true curve is the same whatever the size.

Why 0.4 millimetres The relevant quantity is the **sagitta**, how far the chord departs from the arc at its midpoint, which for a short chord is approximately the chord length squared divided by eight times the radius. For a 0.4 mm chord on a 5 mm radius that is about 4 micrometres, far below the pen's line width and well below what the machine can position to. So the approximation is invisible, and making the chords shorter would add moves without changing the drawing.

> *Arcs are included for completeness of the G-code reader rather than because this project emits them: handwriting is written out as straight segments throughout. They matter because the reader is meant to run any ordinary G-code file, and arcs are among the commonest commands in files from other sources.*

8.69 RunGcodeFile

```
def RunGcodeFile(path)
```

Inputs

`path`
: the G-code file to execute

Output. nothing; it draws the file or aborts.

What it does Reads the file and executes it line by line, maintaining the current position, the feed rate and whether coordinates are absolute or relative, and carrying out each move.

Why calibration is required first The function refuses with a dedicated error if homing has not run. A file's coordinates are absolute positions on the bed, and an absolute position is meaningless until the machine knows where it is. Executing one anyway would drive from an unknown origin toward a position computed as though it were known.

The safety check before every line Each line begins by checking the endstops and the latched fault flag, and aborts on either. Checking per line as well as per step gives two different guarantees: the per-step check inside the stepper stops a move in progress, and this one stops the job rather than merely the move. Without it, a move cut short by a switch would be followed by the next line of the file being executed from the wrong position.

The abort reports the line number. That is what makes a failure diagnosable: it says how far the drawing got and which command was being executed, so the file can be examined at that point rather than searched.

Modal state G-code is **modal**, meaning several settings persist until changed: the feed rate, absolute or relative coordinates, and the units. The interpreter must therefore carry that state across lines, which is why the feed and the absolute-mode flag are variables of the loop rather than parameters of each move. A line consisting only of coordinates inherits the feed set many lines earlier, which is how G-code files are normally written.

> *The starting position is read using the uncalibrated steps-per-millimetre constant rather than the measured value, so the position the job believes it starts from is slightly wrong even after calibration. Subsequent moves use the calibrated figure, so the error does not accumulate, but the first move of a job is displaced by the difference between the calculated and measured scaling.*

8.70 WriteGcodeFile

```
def WriteGcodeFile(path)
```

Inputs

`path`
: where to write

Output. nothing; it writes a small test file.

What it is for Emits a short G-code file for exercising the machine during bring-up, so the execution path can be tested without the synthesis half being involved. It is a diagnostic convenience, not part of drawing handwriting: a real drawing is written by WriteGcode from a trajectory.

8.71 InputListener

```
def InputListener()
```

Output. nothing; it runs the keyboard control loop until told to stop.

What it is for A console for operating the machine by hand: home it, run a file, set a speed, draw a test circle, clear a latched fault, quit. It is what the machine is brought up and diagnosed with.

Why it runs in its own thread Reading a line from the keyboard blocks until something is typed. If that happened in the main loop, the machine would stop stepping while waiting for input, so motion would proceed only while the operator was typing. Running the console in a separate thread lets it wait on the keyboard while the main loop continues to step the motors.

How the two threads communicate Entirely through the module-level state: the console sets the mode, the speed, the fault flag, and the loop reads them. There are no locks. That is sound here for a specific reason rather than by luck: every shared value is a single machine word, whose assignment cannot be observed half-complete, and each is written by only one of the two threads. Nothing requires two values to change together, which is the situation locks exist to protect.

Why the thread is a daemon A **daemon** thread does not keep the program alive: when the main thread finishes, the process exits whether or not the daemon is still running. That is wanted here because the console is blocked reading the keyboard most of the time, so if it had to finish before the program could exit, a quit command issued from anywhere else would hang until a further key was pressed.

> *The clear command for a latched fault is deliberately a manual action. It is the acknowledgement described under the latching flag: the machine stopped because its idea of the world was wrong, and only a person standing at it can establish that it is right again.*

9 train.py

This is where the networks are trained. recognise.py runs them forward; everything that works out how the weights should change is here, derived by hand and written in numpy. No automatic differentiation is used, which is the main reason this file is the largest in the project.

Background: what training actually is A network starts with random weights and gives nonsense answers. Training is a loop: run some examples through, measure how wrong the output is with a **loss function**, work out how each weight would have to change to make the loss smaller, and change it a little in that direction. Repeat for days.

The quantity that says how a weight should change is the **gradient**: the rate at which the loss changes as that weight changes. If the gradient with respect to some weight is positive, increasing that weight increases the loss, so the weight should decrease. Moving every weight a small step against its gradient is **gradient descent**, and the size of that step is the **learning rate**.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \tag{9.1}$$

where

- w one weight of the network
- η the learning rate: how big a step to take
- L the loss, a single number saying how wrong the output was
- $\frac{\partial L}{\partial w}$ the **partial derivative** of the loss with respect to that weight: how much the loss changes per unit change in the weight, holding every other weight fixed
- $\leftarrow$ assignment: the weight is replaced by the value on the right

Background: the chain rule and backpropagation A network has millions of weights and the loss depends on all of them through many layers, so computing each gradient separately is out of the question. What makes it feasible is the **chain rule** of calculus: if a quantity depends on another through an intermediate step, the derivatives multiply.

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x} \tag{9.2}$$

where

- L the loss
- y a layer's output
- x that layer's input
- $\frac{\partial L}{\partial y}$ the gradient arriving from the layer above: how the loss changes with this layer's output
- $\frac{\partial y}{\partial x}$ the layer's own derivative: how its output changes with its input
- $\frac{\partial L}{\partial x}$ the gradient to pass down to the layer below

So each layer needs to know only two things: the gradient of the loss with respect to its own output, which the layer above hands it, and how its own output depends on its input and weights, which is local knowledge. It multiplies them, keeps the weight gradients, and passes the input gradient down. Starting at the loss and working backwards layer by layer computes every gradient in the network in one sweep, at about the cost of one forward pass. That is **backpropagation**.

Why every function here comes in pairs A layer's backward pass needs values from its forward pass. The gradient through a ReLU depends on which inputs were positive; the gradient through a convolution depends on what the input patches were. So each forward function returns its output together with a **cache** of exactly what its backward partner will need, and the backward function takes that cache. The pairing is the whole structure of the first part of this file.

The naming convention follows from it: a function named Forward here differs from the same-named function in recognise.py only in returning that cache, and sometimes in behaving differently during training, as batch normalisation and dropout do.

A note on notation Throughout, a quantity written with a leading d, such as `dout` or `dx`, means the gradient of the loss with respect to that quantity, not a small change in it. So `dx` has exactly the same shape as x, because there is one gradient per element. This is the usual convention in hand-written backpropagation code and it is worth fixing in mind before reading any of what follows.

9.1 Conv2dForward

```
def Conv2dForward(x, weight, bias, padding=1)
```

Inputs

`x` the input, shaped (batch, channels, height, width)

`weight`
: the filters

`bias`
: one number per output channel, or nothing

`padding`
: zeros added round the input, default 1

Output. a pair: the filtered result, and the cache the backward pass needs.

What it adds to the inference version The arithmetic is identical to Conv2d in recognise.py, including the im2col rearrangement and the strided view, both explained in that entry. The only difference is that this one keeps a cache.

What is cached, and why each item is needed

- **The input's shape,** not the input itself. The backward pass builds a gradient array of that shape and never needs the values.

- **The weights,** because the input gradient is formed by multiplying the output gradient by them.
- **The flattened columns,** which is the significant one. The weight gradient is the output gradient multiplied by exactly these patches, so keeping them avoids rebuilding them, which would mean redoing the strided view and the reshape.
- **The padding and the filter and output sizes,** so the backward pass can fold the column gradient back into image shape without recomputing the geometry.

The cost of caching, and why it is accepted The columns are the memory cost of training. A 3 by 3 filter over three channels makes each patch 27 numbers, and there is one patch per output position, so the cached columns are roughly nine times the size of the input feature map. Every convolution in the network holds its own for the whole forward pass, since none can be released until the backward pass has used it. That, rather than the weights, is what sets the largest batch size that fits in memory, and it is why training needs far more memory than inference.

> *No bias-related value is cached. The bias gradient is just the output gradient summed over every position, which needs nothing from the forward pass, because adding a constant has a derivative of one.*

9.2 Conv2dBackward

```
def Conv2dBackward(dout, cache)
```

Inputs

`dout`
: the gradient of the loss with respect to this layer's output

`cache`
: what the forward pass saved

Output. three gradients: with respect to the input, the weights and the bias.

Where the three formulas come from The forward pass, in matrix form, is the flattened weights multiplied by the columns. Everything below follows from differentiating a matrix product, so it is worth stating that rule first: if Y = W X, then the gradient with respect to W is the output gradient times X transposed, and the gradient with respect to X is W transposed times the output gradient. Both the shapes and the transposes follow from requiring the result to have the same shape as the thing differentiated.

The weight gradient

$$\frac{\partial L}{\partial W} = \sum_{b} \frac{\partial L}{\partial Y_b} X_b^{\mathsf{T}} \tag{9.3}$$

where

$\frac{\partial L}{\partial W}$ the gradient for the filters, same shape as the filters

$\frac{\partial L}{\partial Y_b}$ the output gradient for batch item b, as a matrix of output channels by positions

X_b the cached columns for that item

$^\mathsf{T}$ matrix transpose

$\sum_b$ summed over the batch

The sum over the batch is not an approximation or an average applied afterwards; it is what the derivative is. The same filter was applied to every item in the batch and at every position, so the loss depends on each weight through all of them, and the chain rule adds the contributions. This is the general rule for a shared parameter: wherever one value is used in several places, its gradient is the sum over those places.

The input gradient, and the fold back into image shape

$$\frac{\partial L}{\partial X} \;=\; W^\mathsf{T}\,\frac{\partial L}{\partial Y} \tag{9.4}$$

where

$\frac{\partial L}{\partial X}$ the gradient with respect to the columns

W the flattened filters

$\frac{\partial L}{\partial Y}$ the output gradient

That gives a gradient with respect to the *columns*, which is not what is wanted: the layer below needs a gradient with respect to the image. Converting one to the other is the only part of this function that is not a matrix product, and it is the subtle part.

The reason it is subtle is that the forward view was overlapping. One input pixel appears in up to nine patches, so it influenced up to nine output positions, and by the chain rule its gradient is the sum of the gradients from all of them. The loop over the nine filter offsets does exactly that: for each offset, the corresponding slice of the column gradient is *added* into the padded image gradient at the shifted position. Nine additions over whole arrays accumulate every contribution.

The accumulation is the whole point, and it is why the forward trick cannot be run in reverse. The forward pass could use a strided view because reading the same memory from several places is harmless; writing to it from several places is not, since the writes would overwrite rather than accumulate. So the backward direction must be an explicit addition, and the operation is known as **col2im**, the counterpart of im2col.

Finally, removing the padding The gradient was accumulated into an array the size of the padded input, and the layer below knows nothing of the padding, so the border is discarded. That is correct rather than merely convenient: the padding was constant zeros, not data, so the gradient with respect to it has nowhere to go.

The bias gradient

$$\frac{\partial L}{\partial b_o} \;=\; \sum_{b,p,q} \frac{\partial L}{\partial Y[b,o,p,q]} \tag{9.5}$$

where

$\frac{\partial L}{\partial b_o}$ the gradient for the bias of output channel o

$\sum_{b,p,q}$
: summed over the batch and both spatial axes

One bias is added to every position of its channel, so by the shared-parameter rule its gradient is the sum of the output gradient over all those positions. No cache is needed because the derivative of adding a constant is one.

9.3 BatchNorm2dForwardTrain

```
def BatchNorm2dForwardTrain(x,gamma,beta,runningMean,runningVar,momentum
    =0.1,
         eps=1e-05)
```

Inputs

`x` the input

`gamma`
: the learned scale per channel

`beta`
: the learned shift per channel

`runningMean`
: the stored mean, updated in place

`runningVar`
: the stored variance, updated in place

`momentum`
: how fast the stored statistics follow the batches, default 0.1

`eps` the guard on the division, default 1e-5

Output. a pair: the normalised and rescaled output, and the cache.

How this differs from the inference version The inference version, in recognise.py, standardises using statistics stored in the checkpoint. This one standardises using the current batch's own mean and variance, and updates the stored ones as a side effect.

$$\mu_c \;=\; \frac{1}{N}\sum_{b,p,q} x[b,c,p,q], \qquad \sigma_c^2 \;=\; \frac{1}{N}\sum_{b,p,q}\big(x[b,c,p,q]-\mu_c\big)^2 \tag{9.6}$$

where

μ_c the mean of channel c over this batch

σ_c^2 its variance

N how many values that is: the batch size times the height times the width

$\sum_{b,p,q}$
: summed over the batch and both spatial axes, but not over channels

The statistics are per channel and pooled over everything else, for the reason given in the inference entry: a channel is the map of where one filter responded, and that filter's typical response size is one property however many positions and examples it is measured over.

Why using the batch is what makes it work Standardising by the batch is not a convenience; it is the mechanism. The purpose of the layer is to fix the scale a layer presents to the next one, and during training that scale is changing constantly because the layers below are being updated. Measuring it fresh from the current batch tracks those changes immediately, where stored statistics would always describe an earlier state of the network.

The running statistics, and why they are an exponential average

$$\hat{\mu} \leftarrow (1 - m)\,\hat{\mu} + m\,\mu_c \tag{9.7}$$

where

$\hat{\mu}$ the stored mean, kept for inference

m the momentum, 0.1

μ_c this batch's mean

This is an **exponential moving average**: each update keeps 90 percent of the old value and takes 10 percent from the new observation, so the stored figure is a weighted average over recent batches with older ones fading away. It is used rather than a plain average over all batches for two reasons. It needs no count and no accumulator, only the current value. And the fading is correct here: early batches were produced by a network whose weights were nearly random, so their statistics do not describe the trained network and should be forgotten.

Why the stored variance is the unbiased estimate

$$\sigma^2_{\text{unbiased}} = \sigma^2_{\text{biased}} \cdot \frac{N}{N-1} \tag{9.8}$$

where

σ^2_{biased} the variance computed with the mean measured from the same data

$\sigma^2_{\text{unbiased}}$ the corrected estimate of the population variance

N the number of values

Measuring the variance about a mean estimated from the same sample systematically underestimates it, because the sample mean is by construction the point about which the squared deviations are smallest. **Bessel's correction**, dividing by N−1 rather than N, compensates. The biased form is used to normalise this batch, since that is the quantity that actually standardises these values, while the corrected form is stored, since that is an estimate of the variance of all data. The code comment notes this matches the reference framework's behaviour, which matters because checkpoints must be interchangeable.

What is cached The standardised values, the standard deviation, the scale and the element count. Notably the input itself is not cached: the backward derivation below is written entirely in terms of the standardised values, which is both less memory and fewer operations than recovering the input.

9.4 BatchNorm2dBackward

```
def BatchNorm2dBackward(dout, cache)
```

Inputs

`dout`
: the gradient with respect to this layer's output

`cache`
: what the forward pass saved

Output. three gradients: with respect to the input, the scale and the shift.

Why this one is genuinely hard Every other layer in this file has a local derivative: the output at one position depends on the input at one position, or a few. Batch normalisation does not. The mean and variance are computed from *every* value in the channel, so changing one input changes the mean, which changes the normalisation of every other value in the channel as well. Each input therefore affects its own output directly and every other output indirectly, and the gradient must account for all three routes.

The three paths

- **Directly,** through its own standardised value.
- **Through the mean,** since the input contributed to the mean, and the mean is subtracted from every value in the channel.
- **Through the variance,** since the input contributed to the variance, and the variance divides every value in the channel.

Differentiating each path and collecting the terms gives an expression that simplifies considerably, which is the form the code uses.

$$\frac{\partial L}{\partial x} = \frac{1}{N\sigma}\left(N\frac{\partial L}{\partial \hat{x}} - \sum \frac{\partial L}{\partial \hat{x}} - \hat{x}\sum\left(\frac{\partial L}{\partial \hat{x}}\,\hat{x}\right)\right) \tag{9.9}$$

where

$\frac{\partial L}{\partial x}$ the gradient to pass down

$\frac{\partial L}{\partial \hat{x}}$ the gradient with respect to the standardised values, which is the incoming gradient multiplied by the learned scale

$\hat{x}$ the cached standardised values

σ the cached standard deviation

N the number of values per channel

$\sum$ summed over the batch and both spatial axes, per channel

The three terms correspond to the three paths, and each has a readable meaning. The first is the direct effect. The second subtracts the average gradient over the channel, which is the correction for the mean: it ensures the gradients sum to zero, so the layer below cannot be pushed in a direction that would merely shift the whole channel, since such a shift would be normalised away anyway. The third subtracts the component aligned with the standardised values themselves, which is the correction for the variance: it removes any push that would merely scale the channel, for the same reason.

So the layer passes down only the part of the gradient that is not a pure shift or a pure rescaling of the channel. That is the deeper statement of what batch normalisation does: it makes the network's behaviour independent of the scale and offset of each channel, and the backward pass correspondingly refuses to pass down gradients that would only change those.

The scale and shift gradients

$$\frac{\partial L}{\partial \gamma} = \sum \frac{\partial L}{\partial y}\,\hat{x}, \qquad \frac{\partial L}{\partial \beta} = \sum \frac{\partial L}{\partial y} \tag{9.10}$$

where

γ, β the learned scale and shift

$\frac{\partial L}{\partial y}$ the incoming gradient

$\hat{x}$ the standardised values

$\sum$ summed over everything but the channel axis

These two are straightforward by comparison. The output is the scale times the standardised value plus the shift, so the derivative with respect to the scale is the standardised value and with respect to the shift is one, each summed over every position the parameter was used at, by the shared-parameter rule.

> *The code comment attributes the simplified form to the standard derivation taught in the Stanford CS231n course notes, which is where this arrangement of the terms is usually given. The project's numerical gradient check exists largely to confirm that this function in particular is right, since it is the one most easily got wrong and an error in it produces slow or unstable training rather than an obvious failure.*

9.5 ReluForward

```
def ReluForward(x)
```

Inputs

`x` any array

Output. a pair: the rectified array, and a mask of which entries were positive.

What is cached and why it is cheap The forward arithmetic is the same clamping as in recognise.py. What it adds is a boolean mask recording which inputs were positive, which is all the backward pass needs. A boolean array is one byte per element against eight for a double, so this is the cheapest cache of any layer, which matters because ReLU appears after every convolution.

> *The mask could be derived from the output instead, since the output is positive exactly where the input was. Caching it explicitly costs a little memory and keeps the backward pass independent of what the caller did with the output, which would otherwise be a hidden requirement that the output not be modified in place.*

9.6 ReluBackward

```
def ReluBackward(dout, cache)
```

Inputs

`dout`
: the gradient with respect to the output

`cache`
: the positivity mask

Output. the gradient with respect to the input.

The derivative

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases} \tag{9.11}$$

where

$\frac{\partial L}{\partial x}$ the gradient to pass down

$\frac{\partial L}{\partial y}$ the gradient arriving

x the original input, consulted only through the cached mask

ReLU is the identity where the input was positive and constant zero where it was not, so its derivative is one or zero accordingly. The backward pass is therefore the incoming gradient with the inactive positions zeroed: the gradient passes unchanged through the units that fired and stops dead at the ones that did not.

Why this is the property that makes deep networks trainable A derivative of exactly 1 on the active path is what distinguishes ReLU from the smooth activations it replaced. The sigmoid's derivative is at most 0.25 and much smaller when saturated, so a gradient crossing twelve sigmoid layers is multiplied by twelve such factors and arrives at the first layer effectively zero. That is the vanishing gradient problem, and it is why networks of this depth were impractical before ReLU. With a factor of exactly 1 the gradient reaches the first layer at full strength.

> *The derivative at exactly zero does not exist, since the function has a corner there. Taking it as zero is conventional and the choice has no practical consequence: an input of exactly zero is a measure-zero event in floating point, and either choice gives a valid subgradient.*

9.7 MaxPool2dForward

```
def MaxPool2dForward(x, kernel=2, stride=2)
```

Inputs

`x` the input

`kernel`
: the window size, default 2

`stride`
: the step between windows, default 2

Output. a pair: the pooled output, and the cache recording which element won each window.

What must be cached, and why The forward arithmetic is as in recognise.py. What training requires in addition is the **position** of the winner in each window, not merely its value. The backward pass must send the gradient to the element that was passed through, and only the position identifies it.

The position is stored as an index into the flattened window, a number from 0 to 3 for a 2 by 2 window, rather than as a pair of coordinates. A single index is one array instead of two and can be used directly by the indexed-write operation the backward pass uses to place the gradient.

> *This is why the forward pass flattens the window axes and takes the maximum by finding the winning index and then reading it, instead of simply taking a maximum over two axes. The inference version does the latter, which is simpler; here the index is needed anyway, so the value is obtained by reading it.*

9.8 MaxPool2dBackward

```
def MaxPool2dBackward(dout, cache)
```

Inputs

`dout`
: the gradient with respect to the pooled output

`cache`
: the winning indices and the geometry

Output. the gradient with respect to the input.

The routing rule Max pooling passes one element of each window through unchanged, so its derivative with respect to that element is one and with respect to every other element of the window is zero. The gradient is therefore routed entirely to the winner and the rest of the window receives nothing.

$$\frac{\partial L}{\partial x[b,c,i,j]} = \begin{cases} \dfrac{\partial L}{\partial y[b,c,p,q]} & (i,j) \text{ was the maximum of window } (p,q) \\ 0 & \text{otherwise} \end{cases} \tag{9.12}$$

where

$x[b, c, i, j]$
: an input element

$y[b, c, p, q]$
: the pooled output of the window containing it

(p, q)
: which window

An input that lost its window has no influence on the output at all, so changing it slightly changes nothing and its gradient is genuinely zero. That is a real consequence of pooling rather than an approximation: only a quarter of the inputs to a 2 by 2 pooling receive any gradient on a given example, which is part of why the layers below a pooling train more slowly.

Why no accumulation is needed here The code comment notes that the stride equals the kernel, so the windows do not overlap and each input belongs to exactly one window. The gradient blocks can therefore be written straight into position rather than added. With overlapping windows an input could win several, and the contributions would have to be accumulated as they are in the convolution's backward pass.

> *The transpose before the final reshape interleaves the window axes back with the output axes, so that the blocks land in the right places. Getting this wrong produces a gradient that is correct in magnitude and scrambled in position, which trains to something, slowly and wrongly, without any error being raised. It is exactly the kind of fault the numerical gradient check exists to catch.*

9.9 AdaptiveMaxPoolHeightForward

```
def AdaptiveMaxPoolHeightForward(x)
```

Inputs

`x` the input

Output. a pair: the height collapsed to one row by taking the maximum, and the winning row indices.

What is cached Which row won in each column of each channel. The reasoning is the same as for ordinary max pooling: the gradient goes entirely to the element that was passed through, so its position must be known.

This is the layer that converts the image into a sequence, as described in the inference entry, so it is also the point where the gradient converts back: the sequence gradient coming from the recurrent layers becomes an image gradient here, placed into the rows that won.

9.10 AdaptiveMaxPoolHeightBackward

```
def AdaptiveMaxPoolHeightBackward(dout, cache)
```

Inputs

`dout`
: the gradient with respect to the collapsed output

`cache`
: the input shape and the winning rows

Output. the gradient with respect to the full-height input.

What it does Creates a zero array of the original shape and writes the incoming gradient into the winning row of each column. Every other row gets nothing.

What that means for training Of the sixteen rows the feature map has at this point, exactly one per column receives any gradient on a given example. So each column of each channel trains from one row at a time, though which row varies between examples and over the course of training as the weights change. The effect is that the layer below learns to make its strongest response correct, rather than learning anything about the rows that lose, which is the behaviour a maximum should produce: the network is being asked whether a feature appears anywhere in the column, so only the best evidence matters.

9.11 AdaptiveAvgPoolWidthForward

```
def AdaptiveAvgPoolWidthForward(x)
```

Inputs

`x`
: the input

Output. a pair: the average along axis 2, and the length averaged over.

Why the cache is a single number An average needs nothing from its input to be differentiated, only how many values it was taken over, which is one integer. That is the cheapest cache in the file, and the contrast with max pooling is instructive: a maximum depends on *which* value won and so must remember it, whereas an average treats every value identically and so has nothing to remember.

9.12 AdaptiveAvgPoolWidthBackward

```
def AdaptiveAvgPoolWidthBackward(dout, cache)
```

Inputs

`dout`
: the gradient with respect to the pooled output

`cache`
: how many values were averaged

Output. the gradient spread evenly back over those values.

The derivative

$$y = \frac{1}{N}\sum_{k} x_k \quad \Longrightarrow \quad \frac{\partial y}{\partial x_k} = \frac{1}{N} \quad \text{for every } k \tag{9.13}$$

where

y the pooled output

x_k the k-th input averaged over

N how many there were

$\Longrightarrow$ implies

Each input contributes one N-th of the average, so each receives one N-th of the gradient. The backward pass is therefore the incoming gradient repeated across the axis and divided by the length.

The contrast with maximum pooling, and what it means for training Where max pooling routes the whole gradient to one element and nothing to the rest, average pooling spreads it thinly over all of them. Every position therefore receives gradient on every example, which is why the writer namer's backbone trains more evenly than the reader's. It is also the right behaviour for what that layer means: the namer asks how prevalent a feature is across the line, so every position genuinely contributes to the answer and every position should be told how it contributed.

9.13 LinearForward

```
def LinearForward(x, weight, bias)
```

Inputs

`x` the input, with features along the last axis

`weight`
: the weight matrix

`bias`
: the bias, or nothing

Output. a pair: the result, and the cache holding the input and the weights.

What is cached and why both items are needed Both the input and the weights, because the backward pass needs each for a different gradient: the weight gradient is formed from the input and the input gradient is formed from the weights. This is the only layer here that caches its full input values, which is affordable because a linear layer sits at the end of the network where the data is small, a vector of a few hundred numbers rather than a feature map.

9.14 LinearBackward

```
def LinearBackward(dout, cache)
```

Inputs

`dout`
: the gradient with respect to the output

`cache`
: the input and the weights

Output. three gradients: with respect to the input, the weights and the bias.

The three gradients

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Y}^{\mathsf{T}} X, \qquad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W, \qquad \frac{\partial L}{\partial b} = \sum_{\text{rows}} \frac{\partial L}{\partial Y} \tag{9.14}$$

where

$\frac{\partial L}{\partial Y}$ the incoming gradient, one row per example

X the cached input, one row per example

W the weight matrix, one row per output

T transpose

$\sum_{\texttt{rows}}$
: summed over the examples

All three follow from differentiating a matrix product, as in the convolution. The transposes are determined by the shapes: there is only one arrangement of each product that yields an array the same shape as the quantity being differentiated, which is a useful check when deriving such expressions by hand.

Why the leading axes are flattened The code comment explains the one wrinkle. The input may carry more than a batch axis before its feature axis: the reader's text head applies this layer to a sequence, shaped time by batch by features, so there are two leading axes. The weight gradient must sum over every one of those positions, since the same weights were used at each.

Flattening every leading axis into one reduces any such case to the two-dimensional form, after which the single matrix product performs the sum automatically. The input gradient needs no flattening, because it has the same leading axes as the output and the product acts only on the last; it is reshaped back to the input's shape at the end. This is what lets one implementation serve both the namer's single vector and the reader's sequence.

9.15 DropoutForward

```
def DropoutForward(x, p, training, rng)
```

Inputs

`x` the input

`p` the fraction to drop

`training`
whether this is a training pass

`rng` the seeded random generator

Output. a pair: the input with some entries zeroed and the rest scaled up, and the mask; or the input unchanged outside training.

What dropout is for **Overfitting** is when a network learns the training examples rather than the pattern in them, so it performs well on what it has seen and poorly on anything new. One way it happens is that the network comes to rely on a particular combination of features that happens to be reliable in the training data. **Dropout** [21] prevents that by randomly removing a fraction of the values at each training step, so no feature can be depended on: it may not be there. The network is forced to spread its evidence across several features, which generalises better.

Why the survivors are scaled up

$$\text{mask} \;=\; \frac{[\,u \ge p\,]}{1-p}, \qquad u \sim \text{Uniform}(0,1) \tag{9.15}$$

where

mask
what the input is multiplied by, elementwise

u a random number drawn uniformly between 0 and 1, one per element

p the drop probability

$[\,u \ge p\,]$
1 when the element survives and 0 when it is dropped

$\frac{1}{1-p}$ the compensating scale applied to the survivors

Dropping a fraction p of the values reduces the sum reaching the next layer by that fraction, so without compensation the layer below would see a smaller signal during training than at inference, and everything calibrated for one would be wrong for the other. Dividing the survivors by 1−p restores the expected total: a fraction 1−p of the values survive and each is multiplied by 1/(1−p), so the expectation is unchanged.

Why the scaling is done here rather than at inference The compensation could instead be applied by scaling everything down by 1−p at inference, which is how dropout was originally described. Putting it in the training path instead is called **inverted dropout**, and it is now standard for a practical reason: inference then requires no dropout-related code at all. The layer simply vanishes outside training, which is exactly what recognise.py does, where the gap in the parameter numbering is the only trace of it.

> *Outside training the input is returned unchanged with no mask. The backward pass accepts that absent mask and passes the gradient through untouched, which keeps the two halves consistent without either needing to know whether the other is in training mode.*

9.16 DropoutBackward

```
def DropoutBackward(dout, cache)
```

Inputs

`dout`
: the gradient with respect to the output

`cache`
: the mask, or nothing

Output. the gradient with respect to the input.

The derivative Dropout multiplies elementwise by a mask, and the derivative of multiplying by a constant is that constant, so the backward pass multiplies by the same mask. The mask includes the scaling factor, so the gradient is scaled by it too, which is necessary: the forward pass amplified the survivors, so their influence on the loss is amplified by the same factor.

A dropped element receives exactly zero gradient, which is correct rather than approximate. It genuinely had no effect on the output for this example, so the loss does not depend on it at all this step. It will receive gradient on steps where it survives, which is how a unit that is dropped half the time still trains, from half the examples.

9.17 ConvBlockForwardTrain

```
def ConvBlockForwardTrain(x, params, prefix, nLayers)
```

Inputs

`x`
: the input feature maps

`params`
: the parameter dictionary

`prefix`
: the block's name prefix

`nLayers`
: how many groups the block contains

Output. a pair: the output, and a list of per-group caches.

What it adds to the inference version The same convolution, normalise, activate repetition, with the batch-statistics version of the normalisation and with every group's three caches collected into a list.

Why the caches are kept in order The list is built front to back as the groups are applied, and the backward pass walks it in reverse. That correspondence is the whole structure of back-propagation through a sequence of layers: the gradient must pass through them in the opposite order to the data, so the last cache created is the first one needed.

> *The running statistics are updated in place inside the parameter dictionary rather than returned. They are not parameters in the usual sense, since no gradient is computed for them and no optimiser touches them, but they must be saved in the checkpoint, so they live alongside the parameters and are excluded from the trainable set by name.*

9.18 ConvBlockBackwardTrain

```
def ConvBlockBackwardTrain(dout, caches, prefix, nLayers)
```

Inputs

`dout`
: the gradient with respect to the block's output

`caches`
: the per-group caches from the forward pass

`prefix`
: the block's name prefix

`nLayers`
: how many groups

Output. a pair: the gradient with respect to the block's input, and a dictionary of weight gradients keyed as the parameters are.

The reversal The groups are walked in reverse, and within each group the three layers are walked in reverse too: activation, then normalisation, then convolution. Each step takes the gradient from the step before it, so the gradient flows back through the block in exactly the opposite order to the way the data flowed forward.

Why the gradients are keyed by parameter name The weight gradients are collected into a dictionary using the same names the parameters themselves use. That is what lets the optimiser be written without any knowledge of the network's structure: it iterates over the parameters and looks up each one's gradient by name. The names are therefore a contract between three separate things, the initialiser, the backward pass and the checkpoint format, and a mismatch in any of them fails with a missing key rather than silently training the wrong array.

> *The variable holding the gradient is reassigned at each step rather than being given a new name per layer. That reads oddly but is exactly right: there is only ever one gradient travelling backwards at a time, and naming its successive values separately would suggest they coexist.*

9.19 LstmForwardTrain

```
def LstmForwardTrain(x, weightIh, weightHh, biasIh, biasHh, hiddenSize)
```

Inputs

`x` the input sequence, shaped (time, batch, features)

`weightIh`
: the input weights, all four gates stacked

`weightHh`
: the recurrent weights

`biasIh`
: the input bias

`biasHh`
: the recurrent bias

`hiddenSize`
: the width of the hidden and cell state

Output. a pair: the output at every step, and a cache holding every intermediate value at every step.

What it adds to the inference version The same gate arithmetic as LstmForward in recognise.py, with one addition that dominates its memory use: at every time step it records the input, the two incoming states, all four gate activations and the squashed cell value.

Why so much must be kept The backward pass needs each of them, and the reason is that the LSTM's updates are products of the gates with the states. Differentiating a product requires both factors, so the gradient with respect to the forget gate needs the previous cell value and the gradient with respect to the previous cell needs the forget gate. None of it can be recomputed cheaply either, because recomputing the state at step t requires replaying every step before it.

$$\text{memory} \;\approx\; T \times B \times \big(\,\text{inputSize} + 6H\,\big) \tag{9.16}$$

where

memory
: the number of values cached, per LSTM direction per layer

T the sequence length, 160 columns here

B the batch size

inputSize
: the width of the input at that layer

H the hidden size, 256

$6H$ the six vectors kept per step: two incoming states, four gates, and the squashed cell, less the one that is recomputed

With two layers and two directions that is eight such caches. At a hidden size of 256 and 160 steps it comes to a few million values per batch item, which is why the text model trains with small batches. This is the characteristic cost of **backpropagation through time**: the whole history of the sequence must be held in memory until the backward pass has consumed it.

> *The step caches are kept in a plain list indexed by time, built forwards and consumed backwards, which is the same arrangement as the convolution block's cache list and for the same reason.*

9.20 LstmBackward

```
def LstmBackward(dOutputs, cache)
```

Inputs

`dOutputs`
: the gradient with respect to the output at every step

`cache`
: the per-step values and the weights

Output. five gradients: with respect to the input sequence, both weight matrices and both biases.

Backpropagation through time An LSTM applies the same weights at every step, so the loss depends on each weight through all 160 steps. By the shared-parameter rule the weight gradient is the sum over the steps, and the state gradients must be carried backwards through them, because the state at step t influenced the output at every later step. The procedure is **backpropagation through time**: walk the steps in reverse, accumulating weight gradients and carrying state gradients.

The two gradients carried backwards At each step two quantities arrive from the step after it: the gradient with respect to the hidden state and the gradient with respect to the cell state. The hidden gradient is combined with the gradient arriving directly from the layer above, because the hidden state at step t is used twice, as that step's output and as the next step's input.

$$\mathrm{d}h_t \;=\; \frac{\partial L}{\partial y_t} \;+\; \mathrm{d}h_{t+1}^{\text{carried}} \tag{9.17}$$

where

$\mathrm{d}h_t$ the total gradient with respect to the hidden state at step t

$\frac{\partial L}{\partial y_t}$ the gradient arriving from the layer above for this step's output

$\mathrm{d}h_{t+1}^{\text{carried}}$
: the gradient the following step computed with respect to its incoming hidden state

Adding the two is the single most important line of the function. Omitting the carried term computes the gradient as though each step's output were independent, which trains something that works tolerably on short sequences and never learns any long-range dependency, which is the entire reason an LSTM was chosen.

Differentiating the two state updates The updates are h = o · tanh(c) and c = f · c_prev + i · g, and everything else follows from differentiating them. Each gate's gradient is the gradient of whatever it multiplied, and then each is passed back through its own squashing function.

$$\begin{aligned} \mathrm{d}o &= \mathrm{d}h \odot \tanh(c_t) \\ \mathrm{d}c &= \mathrm{d}c^{\text{carried}} + \mathrm{d}h \odot o \odot \left(1 - \tanh^2(c_t)\right) \\ \mathrm{d}f &= \mathrm{d}c \odot c_{t-1} \\ \mathrm{d}i &= \mathrm{d}c \odot g \\ \mathrm{d}g &= \mathrm{d}c \odot i \\ \mathrm{d}c_{t-1} &= \mathrm{d}c \odot f \end{aligned} \tag{9.18}$$

where

$\mathrm{d}o$, $\mathrm{d}f$, $\mathrm{d}i$, $\mathrm{d}g$
: the gradients with respect to the four gate outputs

$\mathrm{d}c$
: the gradient with respect to this step's cell state

$\mathrm{d}c_{t-1}$
: the gradient carried back to the previous cell state

c_t, c_{t-1}
: the cell states at this and the previous step

$\odot$
: elementwise multiplication

$1 - \tanh^2$
: the derivative of the hyperbolic tangent

Every line is the product rule: the gradient with respect to one factor is the incoming gradient times the other factor. The cell gradient has two contributions because the cell is used twice, revealed through the hidden state at this step and carried forward to the next, which is the same double-use argument as for the hidden state.

The gradient path that makes long memory possible The last line is worth isolating. The gradient carried back to the previous cell state is the current cell gradient multiplied by the forget gate, and by nothing else: no weight matrix is applied along this path. So if the forget gate is near 1, the gradient passes backwards through that step essentially unchanged, and it can do so for many steps in succession without shrinking.

That is the precise sense in which the LSTM solves the vanishing gradient problem. A plain recurrent layer multiplies the carried gradient by its recurrent weight matrix at every step, so after a hundred steps it has been multiplied by that matrix a hundred times and is either negligible or enormous. The LSTM's cell path multiplies only by gate values the network itself controls, and it can learn to set them near 1 where information must survive.

Through the squashing functions Each gate gradient is then passed back through its own activation to reach the pre-activation value, which is what the weights actually produced. For the three sigmoid gates the derivative is the output times one minus the output, and for the candidate it is one minus the output squared. Both are written in terms of the output rather than the input, which is why the cache holds the gate activations and not the pre-activations.

The weight gradients The four pre-activation gradients are concatenated back into the stacked layout the weights use, and the weight gradients are then matrix products of that stack with the step's input and previous hidden state. They are accumulated across steps with addition rather than assignment, which is the shared-parameter sum. Both biases receive the same gradient, since both were added to the same quantity, which is the backward counterpart of the inference note that only their sum has any effect.

> *The input gradient is written per step into a preallocated array rather than appended, because the steps are visited in reverse and the result must come back in forward order. Appending and reversing would also work; writing by index makes the ordering explicit at the point it matters.*

9.21 BidirectionalLstmForwardTrain

```
def BidirectionalLstmForwardTrain(x, paramsFwd, paramsBwd, hiddenSize)
```

Inputs

`x` the input sequence

`paramsFwd`
: the forward direction's parameters

`paramsBwd`
: the backward direction's parameters

`hiddenSize`
: the width per direction

Output. a pair: the two directions concatenated, and a cache holding both directions' caches.

What it does Runs both directions as in the inference version and keeps both caches. Each direction is a complete independent LSTM with its own weights, so each has its own full set of per-step values, and the memory cost is doubled.

> *The backward direction is run on a reversed view of the input, which is free, and its output is reversed again to align positions. The reversals must be undone consistently in the backward pass too, which is the one real opportunity for error in an otherwise mechanical function.*

9.22 BidirectionalLstmBackward

```
def BidirectionalLstmBackward(dout, cache)
```

Inputs

`dout`
: the gradient with respect to the concatenated output

`cache`
: both directions' caches and the hidden size

Output. the gradient with respect to the input, and both directions' weight gradients.

Splitting the gradient The forward pass concatenated the two directions along the feature axis, so the incoming gradient is split at the same point: the first half belongs to the forward direction and the second to the backward one. Concatenation and splitting are exact inverses, so no arithmetic is involved.

Why the two input gradients are added Both directions read the *same* input sequence, so the input is a shared quantity and by the shared-parameter rule its gradient is the sum of the two directions' contributions. This is easy to get wrong by treating them as separate inputs and returning only one, which halves the gradient reaching the convolutional backbone and makes it train at half rate with no visible symptom.

The reversal, undone The backward direction's gradient must be reversed before being handed to its LSTM backward pass and reversed again afterwards, mirroring the two reversals of the forward pass. The invariant to hold on to is that every array crossing the boundary of this function is in forward time order, and the reversed order exists only inside it.

9.23 MultilayerBidirectionalLstmForwardTrain

```
def MultilayerBidirectionalLstmForwardTrain(x, layerParams, hiddenSize)
```

Inputs

`x` the input sequence

`layerParams`
: a list of per-layer parameters

`hiddenSize`
: the width per direction

Output. a pair: the final output, and the list of per-layer caches.

What it does Applies each bidirectional layer in turn, collecting the caches. The same build-forwards, consume-backwards arrangement as the convolution block, one level up: a list of layer caches, each of which holds two direction caches, each of which holds a list of step caches.

9.24 MultilayerBidirectionalLstmBackward

```
def MultilayerBidirectionalLstmBackward(dout, caches)
```

Inputs

`dout`
: the gradient with respect to the final output

`caches`
: the per-layer caches

Output. the gradient with respect to the input, and every layer's weight gradients.

What it does Walks the layers in reverse, passing each layer's input gradient to the layer below and collecting the weight gradients. There is no arithmetic of its own; it is the sequencing that makes the chain rule hold across the stack.

> *This completes the recurrent half. The gradient arriving here came from the text head, and the gradient leaving goes to the height-collapsing pool, which converts it back from a sequence gradient into an image gradient. Those two conversions, forward and backward, are what join the convolutional and recurrent halves of the reader.*

9.25 LogSumExp

```
def LogSumExp(a, b)
```

Inputs

`a` one array of log-space values

`b` another of the same shape

Output. the logarithm of the sum of their exponentials, computed without overflow.

What it is for CTC adds probabilities together, and the probabilities are held as logarithms to avoid underflow, as explained under LogSoftmax. But a logarithm turns a *product* into a sum, not a *sum* into anything simple: adding two probabilities held as logarithms requires this function.

$$\log(e^a + e^b) \;=\; m + \log\bigl(e^{a-m} + e^{b-m}\bigr), \qquad m = \max(a, b) \tag{9.19}$$

where

a, b the two log-space values being combined

m the larger of the two, factored out

e^{a-m}
at most 1, since a is at most m, so it cannot overflow

The identity follows from factoring the larger exponential out of the sum and taking the logarithm. Its value is numerical: the exponentials in the rearranged form are at most 1, so nothing can overflow, while a direct computation would exponentiate a log probability of -800 and get zero, or of a large positive number and get infinity.

The infinity case, and why it needs handling A forbidden transition is represented by negative infinity, meaning zero probability. When both arguments are that, the maximum is negative infinity and the rearranged expression computes infinity minus infinity, which is not a number. The answer should be negative infinity: the sum of two impossibilities is still impossible.

So the result is selected explicitly in that case. The code comment notes that numpy evaluates both branches of the selection eagerly, so the invalid arithmetic still happens and still warns even though its result is discarded, and the warning is suppressed for that reason. Suppressing a warning is usually a bad sign; here it is suppressing a warning about a value that has already been correctly replaced, and the suppression is scoped narrowly to this one expression.

Preserving negative infinity exactly, rather than using a large negative number, matters for correctness: an impossible path must contribute exactly nothing, and a finite stand-in contributes a tiny amount that accumulates over 160 steps and can make a forbidden alignment the winner.

9.26 CtcLoss

```
def CtcLoss(rawLogits, targets)
```

Inputs

`rawLogits`
: the network's raw output, shaped (time, batch, classes)

`targets`
: the label sequences, one per batch item

Output. a pair: the mean loss over the batch, and the gradient with respect to the raw output.

What is being computed This is the heart of training the reader. The label says what was written and not where, so the loss cannot compare the output to a target column by column. Instead CTC [18] defines the probability of the target text as the total probability of *every* alignment of it to the columns, and the loss is the negative logarithm of that.

$$L \;=\; -\log P(\mathbf{y} \mid \mathbf{x}) \;=\; -\log \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} p_t(\pi_t) \tag{9.20}$$

Source: [18]

where

L the loss for one example

$\mathbf{y}$ the target text

$\mathbf{x}$ the input image

π one alignment: a choice of label for each column

$\mathcal{B}^{-1}(\mathbf{y})$
: the set of every alignment that collapses to the target text, by the merge-then-remove-blanks rule described under DecodeCtc

$p_t(\pi_t)$
: the network's probability for the label this alignment chose at column t

T the number of columns

The negative logarithm is the standard form for a loss built from a probability: maximising the probability is minimising its negative logarithm, and the logarithm turns the product over columns into a sum, which is both numerically stable and what makes the gradient manageable.

Why the sum cannot be computed directly The number of alignments is enormous. A 40-character label across 160 columns has on the order of 10 to the power 40 valid alignments, so enumerating them is as impossible as enumerating the seam paths in segment.py, and for the same reason. The remedy is also the same: dynamic programming over a quantity that can be built up column by column.

The forward variable

$$\alpha_t(s) \;=\; \Big(\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \alpha_{t-1}(s-2)\Big) \cdot p_t\big(\mathrm{ext}[s]\big) \tag{9.21}$$

Source: [18]

where

$\alpha_t(s)$
: the total probability of every alignment that reaches state s of the extended label sequence by column t

$\mathrm{ext}[s]$
: the label at position s of the extended sequence, built as blank, label, blank, label, ... as described under CtcForcedAlign

$p_t(\cdot)$ the network's probability for that label at column t

$\alpha_{t-1}(s-2)$
: the skip term, excluded when the skip is forbidden

The recurrence is the same topology as the forced alignment: the same three transitions and the same restriction that a skip may not land on a blank or repeat the label two states back. The one crucial difference is that alignment took the maximum of the three arrivals and this takes the **sum**.

Why sum here and maximum there This is the distinction worth being clear about. Alignment wants one particular layout, the most probable, so it takes the best arrival. Training wants the total probability of the text however it is laid out, so it adds the arrivals: every alignment that reaches this state contributes, and their probabilities add because they are mutually exclusive ways of the same thing happening.

Summing is also what makes CTC trainable at all. If training used the maximum it would be committing to one alignment and pushing the network toward it, which would be arbitrary early on when the network has no idea where anything is. Summing lets the network be rewarded for any layout that spells the right text, and the alignment emerges from training rather than being imposed.

In log space the sum is performed by LogSumExp, applied twice to combine the three terms, which is why that function exists.

The backward variable

$$\beta_t(s) \;=\; \sum_{s' \in \{s,\, s+1,\, s+2\}} \beta_{t+1}(s') \cdot p_{t+1}\big(\mathrm{ext}[s']\big) \tag{9.22}$$

Source: [18]

where

$\beta_t(s)$
: the total probability of completing the label from state s at column t onwards

s' : the states reachable from s, the mirror of the forward transitions

$p_{t+1}(\cdot)$
: the probability of the label emitted on arrival

The forward variable measures the probability of everything *before* a point and the backward variable the probability of everything *after* it. Both are needed because the gradient requires knowing, for each column and each label, the total probability of all the complete alignments passing through that point, and that is the product of the two.

The total probability, and the loss The total probability of the label is the forward variable at the final column, in either of the two states a valid alignment may end in, the last character or the trailing blank, summed. The loss is its negative logarithm.

The gradient, and why it takes this remarkably simple form

$$\frac{\partial L}{\partial z_t(k)} \;=\; p_t(k) \;-\; \frac{1}{P(\mathbf{y} \mid \mathbf{x})} \sum_{s\,:\,\mathrm{ext}[s]=k} \alpha_t(s)\,\beta_t(s) \tag{9.23}$$

Source: [18]

where

$\frac{\partial L}{\partial z_t(k)}$
: the gradient with respect to the raw score for class k at column t

$p_t(k)$
: the network's current probability for that class, from the softmax

$\alpha_t(s)\beta_t(s)$
: the total probability of all complete alignments passing through state s at column t

$\sum_{s:\mathrm{ext}[s]=k}$
: summed over every state of the extended sequence that emits class k, since a character may appear several times

$P(\mathbf{y} \mid \mathbf{x})$
: the total probability of the label, which normalises the product into a proportion

The second term is called the **occupation**: among all the alignments that spell the target, the proportion that have class k at column t. So the gradient is the difference between what the network currently predicts for that class and what it *should* predict given the target. Where the network already agrees with the occupation the gradient is zero; where it predicts too much of a class the gradient is positive and that score is pushed down.

That this comes out as a simple difference is the same simplification that makes the cross-entropy gradient a difference, and it arises for the same reason: the softmax and the logarithm in the loss are inverse operations, so differentiating their composition cancels the awkward terms. It is why the gradient is computed with respect to the raw logits rather than with respect to the log probabilities, even though the loss was defined on the latter: taking the whole softmax and loss together is far simpler than either separately.

Why there is a loop over the batch Each example has its own label and so its own length of extended sequence, so the dynamic programming tables are different shapes and cannot be computed as one array. The loop over the batch is therefore unavoidable, and inside it the loops over time and over states remain in Python too, which makes this the slowest function in the file. The small loop over states checking the skip restriction could be vectorised by precomputing the mask once per example instead of once per column, which is the obvious remaining optimisation.

> *Both the loss and the gradient are divided by the batch size, so the loss is a mean over examples rather than a total. That makes it comparable between batch sizes, which matters because the learning rate is tuned against the gradient's magnitude: without the division, doubling the batch size would double the gradient and effectively double the learning rate.*

9.27 CrossEntropyLoss

```
def CrossEntropyLoss(logits, targetIdx)
```

Inputs

`logits`
: the raw scores, one row per example

`targetIdx`
: the correct class index for each example

Output. a pair: the mean loss, and the gradient with respect to the logits.

What it is for The loss for the writer namer, which is an ordinary classification: one correct answer per example, and its position is known, so none of the CTC machinery is needed.

$$L \;=\; -\frac{1}{B}\sum_{b=1}^{B}\log p_b(c_b) \tag{9.24}$$

where

L the mean loss over the batch

B the batch size

c_b the correct class for example b

$p_b(c_b)$
: the probability the network assigned to that class, from the softmax

Why the negative logarithm of the correct class **Cross-entropy** for a single correct answer reduces to the negative logarithm of the probability assigned to it. Its shape is what makes it a good loss: a probability of 1 gives a loss of 0, a probability of 0.5 gives about 0.69, and as the probability approaches zero the loss grows without bound. So being confidently wrong is punished far more than being uncertain, which is the behaviour wanted from a classifier that reports confidences.

The gradient

$$\frac{\partial L}{\partial z_b(k)} \;=\; \frac{1}{B}\Big(p_b(k) - [\,k = c_b\,]\Big) \tag{9.25}$$

where

$\frac{\partial L}{\partial z_b(k)}$
: the gradient with respect to the raw score for class k of example b

$p_b(k)$
: the predicted probability for that class

$[\,k = c_b\,]$
: 1 for the correct class and 0 for every other

$\frac{1}{B}$
: the batch mean, matching the loss

The gradient is the predicted distribution minus the target distribution, the latter being 1 on the correct class and 0 elsewhere. The code expresses it exactly that way: copy the probabilities and subtract one from the correct class. Every wrong class is pushed down in proportion to how much it was predicted, and the right one is pushed up in proportion to how much it was missed.

The same cancellation as in CTC produces this. Differentiating the softmax alone gives a full matrix of terms, and differentiating the logarithm gives a reciprocal, and composing them cancels almost everything. This is the standard reason softmax and cross-entropy are implemented as one unit rather than two layers.

> *The log-sum-exp stabilisation is applied here too, the maximum being subtracted before exponentiating, for the reason given under LogSoftmax. It costs one pass and removes any possibility of overflow on a large logit.*

9.28 RMSprop

```
class RMSprop
```

Output. an optimiser object holding the parameters and a running average of squared gradients.

The problem plain gradient descent has A single learning rate must suit every weight, and it cannot. Different weights have gradients of wildly different magnitudes, by orders of magnitude between layers and between a bias and a convolution weight. A rate small enough to be stable for the largest gradients is far too small for the smallest, so some weights barely move while others oscillate.

The idea: divide by the typical gradient size **RMSprop** [24] keeps, for each weight, a running average of its squared gradient, and divides the step by the square root of that. A weight whose gradients are habitually large is divided by a large number and takes a small step; one with habitually small gradients takes a comparatively large one. The effect is that every weight moves at a comparable rate regardless of its gradient's scale.

$$v \;\leftarrow\; \alpha\, v + (1-\alpha)\, g^2, \qquad w \;\leftarrow\; w - \eta\, \frac{g}{\sqrt{v} + \epsilon} \tag{9.26}$$

where

v the running average of the squared gradient, per weight

α how much of the old average to keep, 0.99 here

g the current gradient for this weight

w the weight

η the learning rate

ϵ a small constant, 1e-8, so a weight with no gradient history does not divide by zero

The square root is what makes the division dimensionally sensible: the running average is of squared gradients, so its square root is in the same units as a gradient, and the ratio is dimensionless. The name is from that root-mean-square. At 0.99 the average covers roughly the last hundred steps, so it describes the gradient's recent typical size rather than this step's value, which is what makes the scaling stable.

> *Weight decay here is added to the gradient, which is the classical formulation. The entry for AdamW explains why that interacts badly with the per-weight scaling and what the alternative is.*

9.29 AdamW

```
class AdamW
```

Output. an optimiser object holding the parameters and two running averages per weight.

What it adds to RMSprop **Adam** [25] keeps a running average of the gradient itself as well as of its square, and uses the smoothed gradient for the step direction. The first is **momentum**: averaging successive gradients cancels the part that fluctuates between batches and keeps the part that is consistent, so the step follows the persistent downhill direction rather than the noise of whichever examples happened to be in this batch.

$$\begin{aligned} m &\leftarrow \beta_1 m + (1-\beta_1)\, g \\ v &\leftarrow \beta_2 v + (1-\beta_2)\, g^2 \\ \hat{m} &= \frac{m}{1-\beta_1^{\,t}}, \qquad \hat{v} = \frac{v}{1-\beta_2^{\,t}} \\ w &\leftarrow w - \eta \left(\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} + \lambda w \right) \end{aligned} \tag{9.27}$$

Source: [25]

where

m the running average of the gradient, the first moment

v the running average of its square, the second moment

β_1 the decay for the first, 0.9: an average over roughly ten steps

β_2 the decay for the second, 0.999: roughly a thousand steps

t the step number, counted from 1

$\hat{m}$, $\hat{v}$
: the bias-corrected averages

λ : the weight decay coefficient

η : the learning rate

Why the averages must be bias-corrected Both averages start at zero, which is not a neutral starting point but a strong claim that the gradient is zero. After one step the first moment is only one tenth of the observed gradient, and the second is one thousandth of its square, so early steps would be far too small and, worse, badly misproportioned between the two, since they are biased by different amounts.

The correction divides each by one minus its decay raised to the step number, which is exactly the total weight the average has accumulated so far. At step 1 the first moment is divided by 0.1, recovering the full gradient; by step 100 the divisor is nearly 1 and the correction fades away. So the optimiser behaves sensibly from its very first step, which matters because the first steps are taken from random weights where large corrections are most needed.

Why the W, and what it fixes **Weight decay** shrinks every weight slightly at each step, which discourages large weights and reduces overfitting. Classically it is implemented by adding a multiple of the weight to its gradient, which for plain gradient descent is exactly equivalent.

With Adam it is not equivalent, and that is the point of AdamW [26]. A decay term added to the gradient passes through the division by the square-root average like any other gradient, so a weight with large gradients has its decay divided down and is barely regularised, while one with small gradients is decayed heavily. The strength of the regularisation therefore varies per weight in a way nobody intended, and it depends on the gradient history rather than on the weight.

The fix is to apply the decay to the weight directly, outside the scaled step, which is the separate term in the update above. Every weight is then shrunk by the same proportion, which is what weight decay is supposed to mean. This is called **decoupled** weight decay, and the W in the name stands for it.

> *Writing the decay inside the same parenthesis as the scaled step means it is multiplied by the learning rate, so the effective decay changes as the learning rate is annealed. The original formulation decouples it from the learning rate as well. The difference matters mainly late in training, when the rate has fallen and the decay falls with it.*

9.30 ReduceLrOnPlateau

```
class ReduceLrOnPlateau
```

Output. a scheduler object that lowers the learning rate when progress stalls.

Why the learning rate should change during training A large rate is right early on, when the weights are far from any good configuration and large steps make rapid progress. It is wrong later, when the weights are close and a large step jumps past the minimum and bounces around it. So the rate should start large and end small, and a **scheduler** is what changes it.

This scheduler's rule It watches a metric, usually the validation loss, and remembers the best value seen. Each epoch that fails to beat the best counts as a bad epoch; once the bad epochs exceed the patience, the learning rate is multiplied by the factor and the count resets. A new best resets it too.

Why it reacts rather than following a schedule The advantage over a fixed schedule is that it needs no estimate of how long training will take. It lowers the rate exactly when progress has stopped, which is the evidence that the current rate is now too large, whatever epoch that happens to be. The disadvantage is that it only reacts after the patience has elapsed, so several epochs are spent at a rate already known to be too large.

Why the patience is needed Validation loss is noisy, because it is measured on a finite set and the weights change between epochs. A single epoch failing to improve means little. Requiring five consecutive failures distinguishes a genuine plateau from the ordinary fluctuation of a metric that is still improving overall. Too little patience lowers the rate on noise and ends training early at a rate too small to make progress.

> *The floor on the learning rate stops the repeated halving from reaching zero, at which point training would continue indefinitely without changing anything. It is a way of bounding wasted time rather than a property of the method.*

9.31 CosineAnnealingLr

```
class CosineAnnealingLr
```

Output. a scheduler object that eases the learning rate down a cosine curve.

The schedule

$$\eta_t \;=\; \frac{\eta_0}{2}\left(1+\cos\!\left(\frac{\pi t}{T}\right)\right) \tag{9.28}$$

where

η_t	the learning rate at epoch t
η_0	the initial learning rate
t	the current epoch
T	the total number of epochs planned
cos	the cosine, which runs from 1 at the start to -1 at the end over half a period

At epoch 0 the cosine is 1 and the rate is the full initial value; at the final epoch it is -1 and the rate is zero, with a smooth decrease between.

Why a cosine rather than a straight line The shape is what recommends it. The cosine is flat at both ends and steepest in the middle, so training spends a while at nearly the full rate, then drops briskly, then spends a while at a very small rate. Both plateaux are useful: the first makes rapid early progress, and the second lets the weights settle precisely into a minimum rather than continuing to step past it. A linear decay spends equal time at every rate, including the middling ones that are neither fast nor precise.

The trade-off against the plateau scheduler This one requires knowing the total number of epochs in advance, since the schedule is defined against it. In exchange it never wastes epochs at a rate that has become too large, because it lowers the rate continuously rather than waiting for evidence. Having both available lets the choice be made per training run according to whether the length is known.

9.32 ClipGradNorm

```
def ClipGradNorm(grads, maxNorm=5.0)
```

Inputs

`grads`
: the gradient dictionary, modified in place

`maxNorm`
: the greatest allowed total magnitude, default 5.0

Output. the total gradient magnitude before any clipping.

The problem it addresses Recurrent networks occasionally produce an enormous gradient. A particular batch, often one with an unusually long or badly labelled line, can give gradients orders of magnitude larger than usual, and a single step with such a gradient moves the weights so far that everything learned so far is destroyed. Training then has to recover from near-random weights, and hours of work are lost to one batch. This is the **exploding gradient** problem.

The remedy

$$\|g\| \;=\; \sqrt{\sum_{k}\sum_{i} g_k[i]^2}, \qquad g \;\leftarrow\; g \cdot \min\!\left(1,\; \frac{\tau}{\|g\| + \delta}\right) \tag{9.29}$$

Source: [27]

where

$\|g\|$ the total gradient norm: every parameter's gradient treated as one long vector

$g_k[i]$ element i of the gradient for parameter k

τ the maximum norm allowed, 5.0

δ a tiny constant preventing a division by zero

$\min(1, \cdot)$
: so the scaling only ever shrinks

If the total magnitude exceeds the limit, every gradient is multiplied by the same factor so that the total comes to exactly the limit. The published remedy for exploding gradients is precisely this [27].

Why the whole network is scaled together This is the part worth understanding. The norm is computed over every parameter at once, and one factor is applied to all of them, rather than each being clipped separately. The reason is that the gradient's *direction* is the useful information: it says which way the loss decreases. Scaling everything by one factor preserves that direction exactly and changes only the distance travelled. Clipping each parameter independently would shorten some components and not others, which rotates the direction into one that is not a descent direction at all, so the step might increase the loss.

Why the norm is returned The function returns the magnitude it measured, before any clipping, so the training loop can log it. That number is one of the most informative diagnostics available during training: a norm that is steady and comfortably below the limit means training is healthy, one that is clipped on most batches means the learning rate or the limit is wrong, and a sudden spike identifies the batch that caused it. Without the return value, clipping would silently conceal the very instability it is protecting against.

9.33 EncodeCrop

```
def EncodeCrop(arr)
```

Inputs

`arr` a line crop as an array

Output. the crop packed into bytes for storage.

Why crops are cached at all Extracting line crops from a page is slow: the page must be read, binarised, analysed for line positions and cut. Training visits every line once per epoch for dozens of epochs, so repeating that work would dominate the run. The crops are therefore extracted once and stored.

What encoding means here The array is flattened into a byte sequence with its shape recorded alongside, so it can be written into a single file and read back by offset. Storing each crop as a separate image file would mean tens of thousands of small files, which is slow to open and wasteful of disk, since a filesystem allocates a whole block to a file however small.

9.34 DecodeCrop

```
def DecodeCrop(raw)
```

Inputs

`raw` the stored bytes

Output. the crop as an array again.

What it does Reverses the encoding: reads the shape and reinterprets the bytes as an array of that shape. Reinterpreting rather than converting means no arithmetic is performed and no copy is made beyond the read itself, which matters because this runs for every crop of every batch of every epoch.

9.35 ReadLabelLines

```
def ReadLabelLines(imgPath, labelPath=None)
```

Inputs

`imgPath`
: the page image's path

`labelPath`
: the transcript's path, derived from the image's if not given

Output. the transcript as a list of lines.

What it is for Reads the text that was written on a page. Supervised training needs the correct answer for every input, and for a page of handwriting that is the transcript, typed when the page was collected.

Why the transcript is per line rather than per page The reader works on one line at a time, so each crop needs its own label. Keeping the transcript as one line of text per line of writing means the correspondence is positional: the n-th transcript line labels the n-th crop. That is what makes it possible to check the extraction, since a page whose crop count does not match its transcript line count has been segmented wrongly and can be rejected rather than silently mislabelled.

> *Deriving the transcript's path from the image's by convention, rather than being told it, is what lets a directory of pages be processed by listing the images alone. The cost is that the convention is implicit: a transcript named differently is simply not found, and the page is skipped.*

9.36 ClusterRows

```
def ClusterRows(rows, maxGap=5)
```

Inputs

`rows`
: row indices

`maxGap`
: the largest gap that still counts as the same cluster, default 5

Output. the indices grouped into runs of nearby rows.

What it is for Given a set of interesting rows, such as those containing a printed rule, groups the ones that are close together. A single printed line a few pixels thick registers as several adjacent rows, and what is wanted is one entry per rule rather than one per row.

The rule The indices are taken in order and a new cluster is started whenever the gap to the previous one exceeds the limit. This is single-link clustering in one dimension, and it is the natural formulation when the data is ordered: two rows belong together if they are close, and closeness chains, so a run of rows each near the last forms one cluster however long it is.

> *A gap of five pixels suits the printed rules on ruled paper, which are a pixel or two thick with occasional breaks where writing crosses them. The chaining property is what bridges those breaks, and it is also the method's weakness: a dense enough sequence of separate features merges into one cluster.*

9.37 FindRuleLineClusters

```
def FindRuleLineClusters(binaryImg, rawGrayscaleArray=None)
```

Inputs

`binaryImg`
: the binarised page

`rawGrayscaleArray`
: the original grey page, if available

Output. the row clusters that look like printed rules.

What it is for Finds the printed ruling on lined paper, so that it can be removed before the handwriting is analysed and so that its positions can hint at where the lines of writing sit.

How a printed rule is recognised By the same property segment.py uses: a printed rule crosses the whole page in one straight unbroken run, which handwriting never does. Rows containing an unusually long horizontal run of ink are therefore candidates, and clustering them groups each rule into one entry.

Why the rules are useful as well as unwanted They are removed because they corrupt the analysis, joining every letter on a line into one blob. But their positions are also the best possible evidence about where lines of writing are, because the writer wrote on them. So they are returned rather than merely deleted, and the line splitter takes them as hints.

> *This is a separate implementation from the equivalent in segment.py, because this file processes database pages and that one processes photographs, which differ enough that one routine serving both would be a compromise for each. The duplication is real but the inputs genuinely differ: a database scan has even lighting and no perspective.*

9.38 RemoveStraightFormLines

```
def RemoveStraightFormLines(hwRegion)
```

Inputs

`hwRegion`
: the region of the page containing handwriting

Output. the region with the printed ruling deleted.

What it does Deletes the long straight runs identified as printing, leaving the handwriting. The reasoning is set out at length under RemovePrintedStructure in segment.py: length, not darkness or thickness, distinguishes printing from writing, and the threshold sits in a wide gap between the two.

Why the removal must be of whole runs Removing the rule at the places where writing crosses it would cut the writing too, since at a crossing the same pixels belong to both. The remedy used here and in segment.py is to restrict the deletion to pixels that form part of a long run, which a letter's stroke does not, so a crossing loses the rule and keeps the letter.

9.39 DetectPageBounds

```
def DetectPageBounds(binaryImg, rawGrayscaleArray=None)
```

Inputs

`binaryImg`
: the binarised page

`rawGrayscaleArray`
: the grey page, if available

Output. the rectangle of the page that holds handwriting.

What it is for A database page is a scanned form, with a printed header, a typed prompt and sometimes a footer, of which only the handwritten region is wanted. This finds that region so the rest can be ignored.

Why the printed parts must be excluded rather than tolerated Printed text is text, so the reader would read it, and it has no corresponding transcript line, so every crop after it would be labelled with the wrong line. A single extra crop at the top of a page misaligns the whole page's labels, which is far worse than losing the page: a mislabelled example teaches the network something false, where a missing one teaches nothing.

> *This is why the crop count is checked against the transcript line count and the page rejected if they disagree. The check cannot tell which crop is wrong, only that something is, and rejecting the page is the right response: the database has thousands of pages and losing a few is cheaper than training on bad labels.*

9.40 MergeUndersizedFragments

```
def MergeUndersizedFragments(lines, minRelativeHeight=0.4)
```

Inputs

`lines`
: the candidate line regions

`minRelativeHeight`
: the least height relative to the typical line, default 0.4

Output. the regions with the too-short ones merged into their neighbours.

The problem it solves Line splitting can divide one line of writing into two, typically when a row of low ink inside the line is mistaken for the gap between lines. The symptom is a region much shorter than the others, since one real line has been halved.

Why the test is relative The height is compared with the typical line height on the same page rather than with a fixed figure, because the right height depends on the writing and the scan resolution. A region under 40 percent of the typical height cannot be a line, since no writer varies their line height that much within a page, so it must be a fragment of one.

Why merging rather than discarding A fragment contains real writing, which the transcript expects to find. Discarding it would lose that text and, worse, would change the crop count and so misalign every label after it. Merging it back into its neighbour restores both the writing and the count.

9.41 MergeOrSplitToExpectedLineCount

```
def MergeOrSplitToExpectedLineCount(lines, rowInkCount, expectedLineCount
    )
```

Inputs

`lines`
: the candidate line regions

`rowInkCount`
: the ink per row profile

`expectedLineCount`
: how many lines the transcript has

Output. the regions adjusted to match that count.

The idea: use the answer to fix the method The transcript says how many lines the page has. That is information the splitter did not use, and it is reliable, because it was typed by a person reading the page. So if the splitter produced the wrong number, the discrepancy is known exactly, and the regions can be adjusted until the count agrees: merging the most doubtful boundary when there are too many, splitting the most promising region when there are too few.

Why this is legitimate and where its limit is Using the label to guide the extraction would be improper if the label were being predicted, and it is not: the task here is to prepare training data, and the transcript is given for exactly that purpose. It is the same reasoning that justifies forced alignment in style.py, where the known text is used to locate characters rather than to recognise them.

The limit is that it cannot verify correctness, only count. A page split in the wrong places but into the right number of pieces passes, and its labels are then confidently wrong. The count is a necessary condition for a correct extraction, not a sufficient one.

9.42 EstimateLinePitch

```
def EstimateLinePitch(smoothedRowInk, minPitch=40, maxPitch=400)
```

Inputs

`smoothedRowInk`
: the smoothed ink per row profile

`minPitch`
: the smallest pitch considered, default 40

`maxPitch`
: the largest, default 400

Output. the estimated spacing between lines of writing, in pixels.

What it is for The same quantity LinePitch finds in segment.py, estimated from a smoothed row profile, and used as the scale for the line splitting below. The reasoning for using a projection profile, and for smoothing it first, is given in that entry.

Why the search is bounded The bounds keep the estimate within the range a line of writing on a scanned page can plausibly occupy. Without a lower bound the method can lock onto the spacing between the strokes within a word, which is a genuine periodicity in the profile and a much shorter one. Without an upper bound it can lock onto the spacing between paragraphs. Both are real patterns; the bounds say which scale is being asked about.

9.43 SplitLinesByPeriodicity

```
def SplitLinesByPeriodicity(binaryRegion,ruleLineHints=None,smoothWindow
   =9,
        minLineHeight=30)
```

Inputs

`binaryRegion`
: the handwriting region

`ruleLineHints`
: the printed rule positions, if found

`smoothWindow`
: the profile smoothing width, default 9

`minLineHeight`
: the shortest acceptable line, default 30

Output. the row ranges of the individual lines of writing.

The approach, and why periodicity is the right tool Lines of writing on a page are close to evenly spaced, because the writer followed the ruling or kept a steady hand. That regularity is strong evidence and it is what the splitter uses: rather than looking for each gap independently, it establishes the pitch and then expects boundaries at roughly that spacing.

The advantage over independent gap-finding is robustness to a single bad gap. A line whose descenders reach into the next line leaves almost no gap at all, and a method looking only for gaps misses that boundary entirely, merging two lines. A method that knows the expected spacing looks for a boundary where one ought to be and places it at the best available row, even where that row is not a clear gap.

Why the printed rules are used as hints Where the ruling was found, it says where the writer intended each line to sit, which is better evidence than the ink profile: it is unaffected by descenders, by a line of unusual height, or by a blank line. The hints are used to guide the boundaries rather than to set them, because writers do not follow ruling exactly.

> *This whole routine is a separate, earlier solution to the problem segment.py solves with seam carving. The seam approach is better, since it cuts around descenders instead of choosing a row to cut through. This one remains because it is what the cached training data was extracted with, and re-extracting it would change the data the reported checkpoints were trained on.*

9.44 ExtractLinePatches

```
def ExtractLinePatches(page,targetHeight=32,maxWidth=1024,
    expectedLineCount=None,
         labelLines=None,isDataset=True)
```

Inputs

`page`
: the page image

`targetHeight`
: the height to scale crops to, default 32

`maxWidth`
: the greatest width kept, default 1024

`expectedLineCount`
: how many lines the transcript has

`labelLines`
: the transcript lines

Output. the line crops, with their labels where the count matched.

What it is for The whole page-to-crops pipeline for database pages, in one call: find the page bounds, remove the printed ruling, split into lines, merge the fragments, reconcile against the expected count, and cut and scale each crop.

Why the crops are scaled here Each crop is scaled to a fixed height, preserving its aspect ratio. Doing it at extraction rather than at training time means the cache holds crops of a known height, so loading a batch needs no resizing, and the cache is smaller, since a scanned line is several times taller than the network needs.

Why width is capped rather than scaled An unusually wide crop is truncated rather than squeezed. Squeezing would change the aspect ratio and so the letter proportions, for the reasons given under ResizeLineImageFixed. An over-wide crop is also usually a fault rather than a long line, two lines having been merged, so truncating loses little that was right.

> *Where the crop count does not match the transcript, the crops are returned without labels and the page cannot be used for training the reader, though it remains usable for the writer namer, which needs only to know who wrote the page. That is why the two networks are trained from different views of the same cache.*

9.45 EncodeText

```
def EncodeText(text)
```

Inputs

`text`
: a line of text

Output. the sequence of class indices for its characters.

What it does Converts a label into the numbers CTC works with, by looking each character up in the charset mapping from recognise.py and dropping anything not in it.

Why unknown characters are dropped rather than substituted A character outside the charset has no output slot, so the network cannot be asked to produce it. Substituting a placeholder would teach it to emit that placeholder whenever something unfamiliar appears, which is worse than omitting it: the network would learn a response to a situation it can never handle correctly.

Dropping the character does introduce a small inconsistency, since the image contains a mark the label does not mention. CTC tolerates that, because it allows the network to emit blank there, and the alternative would be discarding every line containing any unusual character, which over a large database is a substantial loss.

9.46 FindPeaks

```
def FindPeaks(sm, floorFrac=0.0, minGap=0, prominence=None, distance=None
    )
```

Inputs

`sm` a smoothed profile

`floorFrac`
: the least height as a fraction of the maximum

`minGap`
: the least separation between peaks

`prominence`
: the least prominence required

`distance`
: an alternative name for the separation

Output. the indices of the accepted peaks, in order.

What it is for A general peak finder, used by the line splitting. It applies three filters in turn, each addressing a different way a spurious peak arises.

Finding candidates A candidate is at least as large as its left neighbour and strictly larger than its right. The asymmetry is deliberate, as the code comment notes: if both comparisons were inclusive, a flat-topped peak several samples wide would report every one of those samples. With one strict comparison it reports exactly one.

The height floor, and a trap in it A candidate must exceed a fraction of the profile's maximum. The comment records a real pitfall: the floor must not default to zero, because the profile can be negative. A profile with its mean removed has negative valleys, and a peak in a negative region is still a peak. Defaulting the floor to negative infinity rather than zero is what makes the filter optional rather than silently active.

Prominence, which is the interesting filter **Prominence** measures how much a peak stands out from its surroundings, as opposed to how tall it is. It is defined as the peak's height above the higher of the two lowest points reached before the profile rises above the peak again on either side.

$$\mathrm{prom}(i) \;=\; \mathrm{sm}[i] \;-\; \max\Big(\min_{j\in\mathrm{left}} \mathrm{sm}[j],\; \min_{j\in\mathrm{right}} \mathrm{sm}[j]\Big) \tag{9.30}$$

where

$\mathrm{prom}(i)$
: the prominence of the peak at index i

$\mathrm{sm}[i]$
: the profile value at the peak

left, right
: the stretches on each side, walked outward until the profile exceeds the peak

min the lowest value reached in each stretch, called the key saddle

max the higher of the two, which is what the peak must rise above to be independent

The code comment names what this rejects: a ripple riding on a big peak's flank. Such a ripple can be tall in absolute terms, taller than a genuine small peak elsewhere, so a height filter keeps it. But it does not descend far on either side before the profile rises again, so its prominence is small and this filter removes it. Height asks how large a peak is; prominence asks whether it is a peak at all rather than a bump on something larger. The term is borrowed from topography, where it distinguishes an independent summit from a shoulder of a bigger mountain.

The separation filter Finally, peaks too close together are thinned by the same greedy strongest-first suppression used for the line centres in segment.py: take the tallest, exclude everything within the minimum separation, repeat. Processing tallest first is what makes it correct rather than arbitrary, for the reason given there.

> *The three filters are each optional and disabled by default, so the function is a plain local-maximum finder unless asked for more. That keeps one implementation usable for both the careful line-splitting case and the simple ones.*

9.47 MyAffine

```
def MyAffine(img, angleDeg=0.0, shearDeg=0.0, fillValue=255)
```

Inputs

`img` the image

`angleDeg`
: the rotation in degrees

`shearDeg`
: the shear in degrees

`fillValue`
: what to fill uncovered pixels with, default 255

Output. the image rotated and sheared about its centre.

Background: what an affine transformation is An **affine** transformation is any combination of rotation, scaling, shearing and translation: it maps straight lines to straight lines and keeps parallel lines parallel. Every such transformation can be written as a matrix multiplication once the coordinates are expressed in **homogeneous** form, which means appending a 1 to each point so that a translation becomes part of the matrix rather than a separate addition.

Why the transformations are composed as matrices

$$M \;=\; T_{+c}\, R\, S\, T_{-c} \tag{9.31}$$

where

M the combined transformation

T_{-c} a translation moving the image's centre to the origin

S the shear

R the rotation

T_{+c} a translation moving the centre back

Rotation and shear are both defined about the origin, which for an image is a corner. Applying them directly would swing the image about that corner and out of frame. Translating the centre to the origin first, transforming, and translating back makes both act about the centre instead. Matrix composition is what allows the four steps to be combined into one, so the image is resampled once rather than four times, which matters because each resampling blurs it slightly.

The order reads right to left, since the rightmost matrix acts on the point first. That is the usual convention for column-vector coordinates and it is the opposite of the order the operations are listed in prose.

Why the matrix is inverted The combined matrix is inverted before being used. The reason is the direction resampling works in: for each *output* pixel, the warp asks which input position it came from, which is the inverse map. Working forward instead, pushing each input pixel to its output position, leaves gaps where no input pixel happens to land and collisions where two do. Working backwards fills every output pixel exactly once, which is why it is universal in image warping. The same argument appears under Warp in segment.py.

9.48 AugmentShear

```
def AugmentShear(img)
```

Inputs

`img` a line crop

Output. the crop with a random shear applied.

Background: what augmentation is for **Data augmentation** creates extra training examples by altering existing ones in ways that do not change their label. A sheared line of handwriting still says the same words, so it is a valid extra example. The purpose is to teach the network which variations are irrelevant: having seen the same line at many slants, it learns to read the letters rather than to recognise one particular slant.

Why shear in particular Slant is the largest single source of variation between hands, and between one writer's pages. Augmenting it is directly useful because the network must read every slant, and it is cheap because shearing is exactly the transformation that changes slant and nothing else.

The range, and why it is expressed as a tangent The shear factor is drawn uniformly from -0.6 to 0.6 and then converted to an angle. Working in the factor rather than the angle is deliberate: the factor is what the shear matrix contains, so a uniform draw on it gives a uniform transformation strength, where a uniform draw on the angle would concentrate the effect near the extremes. A factor of 0.6 is about 31 degrees, which spans the range of real handwriting, from upright to the steepest cursive in this project's writers.

9.49 AugmentRotate

```
def AugmentRotate(img)
```

Inputs

`img` a line crop

Output. the crop rotated by a small random angle.

What it simulates A line that is not quite level, which happens when a page is photographed slightly askew or when a writer's line drifts. The range is only 2.5 degrees either way, far smaller than the shear range.

Why the range is small Because the real variation is small. The segmenter deskews every page before cutting crops, so by the time a crop reaches the network its residual tilt is a fraction of a degree. Augmenting much further than the real variation teaches the network to handle a situation that never occurs, at the cost of capacity it could spend on ones that do. A rotation also changes the measured slant and the apparent x-height, so a large one would conflict with the shear augmentation rather than complementing it.

9.50 FindPerspectiveCoeffs

```
def FindPerspectiveCoeffs(startpoints, endpoints)
```

Inputs

`startpoints`
: the four source corners

`endpoints`
: where they should land

Output. the eight coefficients of the perspective transformation.

Background: what a perspective transformation is An affine transformation keeps parallel lines parallel, which a photograph taken at an angle does not: the far edge of a page appears shorter than the near one, so its sides converge. A **perspective** or **projective** transformation is the more general map that allows that convergence, and it is what actually happens when a flat page is photographed from an angle.

$$x' = \frac{ax + by + c}{gx + hy + 1}, \qquad y' = \frac{dx + ey + f}{gx + hy + 1} \tag{9.32}$$

where

x, y a point in the source image

x', y'
: where it lands

$a \ldots f$
: the six coefficients an affine map would also have

g, h the two extra coefficients that produce the perspective effect

1 the fixed ninth coefficient; the map is unchanged by scaling all nine, so one can be fixed

The division by a function of position is what distinguishes it from an affine map and what makes parallel lines converge: points further away in the implied depth direction are divided by a larger number and so are compressed toward one another.

How the eight coefficients are found Four corner correspondences are given, each fixing two equations, giving eight equations for the eight unknowns. The equations above are not linear in the coefficients because of the division, but multiplying each through by its denominator makes them linear, and that rearrangement is what the rows of the matrix in the code are: each row is one such multiplied-out equation, with the cross terms carrying the negative signs the rearrangement produces.

The resulting system is then solved by standard linear algebra. The reason four corners are exactly right is that eight unknowns need eight equations: three corners would leave the system underdetermined and five would overdetermine it, requiring a least-squares fit rather than a solution.

9.51 AugmentPerspective

```
def AugmentPerspective(img)
```

Inputs

`img` a line crop

Output. the crop with a random perspective distortion applied.

What it simulates, and why it matters for this project A page photographed from an angle rather than square on. This augmentation matters more here than it would for a database-only system, because this project's own pages are phone photographs: a page on a desk photographed by hand always carries some perspective. A network trained only on flat scans would be seeing a distortion it had never met.

Why it is applied to the line rather than the page Strictly the perspective applies to the whole page, and a line crop is a narrow strip of it, so the distortion within one strip is a small part of the page's. Applying a crop-sized perspective is therefore an approximation, and a slightly stronger one than reality. That is acceptable for augmentation, whose purpose is to cover the variation rather than to model it exactly, and erring toward stronger distortion is the safer direction.

9.52 PerspectiveJitterPoints

```
def PerspectiveJitterPoints(width, height, DistortionScale)
```

Inputs

`width`
: the crop width

`height`
: its height

`DistortionScale`
: how far the corners may move, as a fraction

Output. a pair of corner lists: the originals and the jittered destinations.

What it does Moves each of the four corners by a random amount within a fraction of the crop's size, producing the correspondences the coefficient solver needs.

Why the corners are moved independently Independent corner displacements generate a general perspective distortion rather than a parameterised one. The alternative, choosing a viewing angle and deriving the corners from it, would be more physically faithful and would require a camera model, a focal length and an assumed distance, none of which are known. Moving the corners directly covers the same space of distortions with no model at all, which is what augmentation needs.

> *The displacement is a fraction of the crop's dimensions rather than a number of pixels, so the strength of the distortion is the same for a crop of any size. This is the same scale-relative reasoning that governs the thresholds throughout the project.*

9.53 AugmentElastic

```
def AugmentElastic(img)
```

Inputs

`img` a line crop

Output. the crop with a smooth random warp applied.

What it simulates **Elastic distortion** [8] displaces every pixel by a small smoothly varying random amount, so the image is stretched here and compressed there. For handwriting it imitates the natural variation of a hand: the same person writing the same word twice produces shapes that differ in just this way, locally stretched and squeezed rather than globally transformed.

How the smoothness is obtained, and why it is essential Two random displacement fields are generated, one per axis, and each is blurred with a Gaussian before being applied. The blurring is what makes the method work. An unblurred random displacement moves each pixel independently, which is simply noise: it destroys the strokes rather than deforming them. After blurring, nearby pixels move together, so a stroke stays a connected stroke and is merely bent.

The blur width therefore controls the character of the distortion. A wide blur gives a nearly uniform displacement, which is just a translation; a narrow one approaches noise. The useful range is a width comparable to the features being deformed, which for handwriting is around a letter's size. The amplitude scales the displacement after blurring, since blurring reduces the magnitude as well as smoothing it.

> *The implementation calls the Elastic function in segment.py, which is the same operation the glyph pipeline uses. Sharing it means the augmentation applied during training and any distortion applied elsewhere are identical, which matters if the two are ever compared.*

9.54 AugmentLineImage

```
def AugmentLineImage(pilImg)
```

Inputs

`pilImg`
: a line crop

Output. the crop with a randomly chosen augmentation applied, or unchanged.

What it does Picks one augmentation at random from the available list and applies it, or sometimes leaves the crop alone.

Why one at a time rather than all of them Composing several distortions compounds their severity, and the result quickly stops resembling handwriting: a crop sheared, rotated, warped in perspective and elastically distorted is unreadable, and training on unreadable examples teaches nothing. Applying one at a time keeps every augmented example plausible while still covering each kind of variation across the training set as a whole.

Why unaugmented crops are kept too Some proportion of crops pass through untouched, so the network sees the real distribution as well as the augmented one. If every example were distorted, the network would be trained exclusively on distorted data and would be slightly mismatched to the undistorted crops it meets at inference, which is the domain-shift problem applied to augmentation itself.

9.55 AUGMENTATIONS

```
AUGMENTATIONS = [AugmentShear, AugmentRotate, AugmentElastic,
    AugmentPerspective]
```

Output. the list of augmentation functions to choose from.

Why the augmentations are a list Collecting them in a list means the chooser needs no knowledge of what they are: it picks an entry and calls it. Adding or removing one is an edit to the list alone, and more importantly, running an experiment on which augmentations help means editing one line, which is how such questions are answered in practice.

> *All four take a crop and return a crop, which is what makes them interchangeable. That uniform signature is the real content of the design: a function needing extra parameters could not be in the list, so each augmentation draws its own random amounts internally rather than being given them.*

9.56 LineImageCache

```
class LineImageCache
```

Output. an object holding extracted crops on disk and serving them by index.

What it is for Stores every extracted crop in a single file with an index of where each one starts, so a crop can be read by number without reading the rest. It is what makes the extraction cost payable once rather than once per epoch.

Why one file with an index Tens of thousands of separate image files would be slow, because each open is a separate filesystem operation, and wasteful, because a filesystem allocates a whole block per file. One file read at an offset is a single operation, and the index is small enough to hold in memory.

Why the whole cache is not simply loaded into memory The extracted crops for the full database run to several gigabytes, more than is comfortable to hold while also holding the activations a training step needs. Reading each crop as the batch needs it keeps the memory use to the batch, and the operating system's own file cache keeps the recently used parts in memory anyway, which recovers most of the speed without the program managing it.

9.57 IAMLineDatasetRaw

```
class IAMLineDatasetRaw
```

Output. an object presenting the database's pages as a sequence of labelled line crops.

What it is for Wraps the cache and the labels so that training sees a simple sequence: ask for item n and get a crop and its text. Where the pages came from, how they were cut and how they are stored are all hidden behind that.

Why this separation matters The training loop should not know whether its data came from the public database, from this project's own photographs, or from a third source. Presenting every source through the same interface is what allows TrainTextModel to accept a source argument and otherwise be unchanged, and what allows the two sources to be combined by AddPersonalSamples without the loop being aware of it.

> *The dataset is the public IAM database of handwritten English [1], whose value for this project is its transcripts: without a known text per line there is nothing to train against and no way to align characters.*

9.58 TrainEvalView

```
class TrainEvalView
```

Output. a view of a dataset that applies augmentation or does not.

Why the same data needs two views Augmentation must be applied when training and must not be applied when validating. The reason is what validation is for: it estimates how the network will perform on real data, so it must use real data. Measuring on augmented crops would measure performance on distorted handwriting, which nobody is going to present.

A **view** wraps the same underlying dataset and changes how items are served, rather than copying the data. So the training and validation views share one cache and one set of labels, and there is no possibility of them drifting apart, which two copies would allow.

> *This also makes the training and validation split a property of which indices each view holds rather than of the data itself. A line can therefore appear in only one of them, which is the whole requirement of a held-out set, and it is enforced by construction rather than by care.*

9.59 TextCollateFn

```
def TextCollateFn(batch)
```

Inputs

`batch`
: a list of crop and label pairs

Output. the batch assembled into arrays.

What collation is A dataset serves one item at a time; a network wants a batch as one array. **Collation** is the step between: taking a list of items and stacking them, which for images means padding them to a common size.

Why the labels stay as a list The crops are stacked into one array, but the labels cannot be: they are sequences of different lengths, and CTC requires each one's true length, since padding a label would add characters the image does not contain. So the labels are kept as a list of varying-length sequences, which is exactly what CtcLoss expects and why that function loops over the batch.

9.60 AuthorLabeledView

```
class AuthorLabeledView
```

Output. a view of the same crops labelled by who wrote them.

What it is for Presents the same cached crops with a different label: the writer's identity instead of the text. The namer is trained from this view and the reader from the text view, over one shared cache.

Why one cache can serve both Because the input is the same object and only the question differs. Extracting the crops twice would double the slowest part of the pipeline and, worse, would allow the two extractions to differ, so that a finding about one network might not transfer to the other.

There is also a practical gain: a page whose crop count did not match its transcript is unusable for the reader but perfectly usable here, since the writer's identity is a property of the page and does not depend on the lines being correctly divided. So this view can draw on pages the text view must reject.

9.61 AuthorCollateFn

```
def AuthorCollateFn(batch)
```

Inputs

`batch`
: a list of crop and writer-index pairs

Output. the batch assembled into arrays.

How it differs from the text version Simpler, because the labels are single integers rather than sequences, so they stack into one array like the crops. There is no variable length to preserve and no padding question for the labels.

> *That simplicity is a direct consequence of the task: classification has one answer per example, where transcription has a sequence whose length varies. It is the same distinction that makes the namer need no CTC and no recurrence.*

9.62 AddPersonalSamples

```
def AddPersonalSamples(base)
```

Inputs

`base`
: the database-derived dataset

Output. the dataset with this project's own photographed pages added.

What it is for Mixes the five photographed writers' pages into the database training set. The result is the joint checkpoint, as opposed to the scratch one trained on the database alone.

Why mixing rather than training separately Two things are wanted at once. The database supplies volume: thousands of writers, which is what teaches the network the general shape of handwriting. This project's pages supply the specific domain: phone photographs of these particular hands, which is what the system will actually be given.

Training on the database alone leaves a domain gap, since the network has never seen a photograph. Training on the personal pages alone gives far too few examples, a few hundred lines against tens of thousands, and the network would overfit them immediately. Mixing gets the volume and the domain together, which is why the joint checkpoint measures so much better on this project's crops than the scratch one: 96 to 99 percent character accuracy against 22 to 26.

> *That difference is the single largest measured effect in the project, and it is a statement about which data a network was trained on rather than about any architectural choice. It is also why the checkpoint actually loaded at inference matters so much, and why the candidate ordering in recognise.py deserves attention.*

9.63 TekliaLineDataset

```
class TekliaLineDataset
```

Output. a dataset presenting an alternative public corpus of line images.

What it is for A second public source of labelled line images, already segmented into lines, selectable by the source argument of the training function.

Why a second source is useful It arrives already cut into lines, so it exercises the training path without the page-segmentation path, which separates two possible causes of poor results: a network that trains well on pre-cut lines and badly on self-cut ones has a segmentation problem rather than a training one. It is also simply more data, from a different collection with different scanning conventions, which broadens what the network has seen.

9.64 MAX_SAFE_LABEL_CHARS

```
MAX_SAFE_LABEL_CHARS = int(INPUT_WIDTH // 4 * 0.75)
```

Output. the longest label that can safely be trained on, derived from the input width.

The constraint it expresses CTC needs at least one column per character, and in practice more, since a character usually occupies several columns and blanks need somewhere to go. The network produces one column per four input pixels, after two halvings, so the number of columns is fixed by the input width and the longest trainable label follows from it.

$$\text{maxChars} \;=\; \left\lfloor 0.75 \cdot \frac{\text{INPUT_WIDTH}}{4} \right\rfloor \tag{9.33}$$

where

maxChars
: the longest label accepted

INPUT_WIDTH
: the network's input width, 640

4
: the downsampling factor from the two poolings

0.75
: a safety margin, leaving a quarter of the columns spare for blanks and for characters occupying more than one

What happens without the limit A label longer than the available columns has no valid alignment at all, so its total probability is zero and the loss is infinite. Infinity propagates through the gradient and destroys every weight in the network on that single step. So this is not a quality filter but a correctness one: a single over-long label in a training set ruins the run, and the symptom is a loss that becomes not-a-number and never recovers.

9.65 HIDDEN_SIZE

```
HIDDEN_SIZE = 256
```

Output. 256, the LSTM width per direction.

What it sets How many numbers the recurrent layer carries from one column to the next, per direction. With two directions concatenated, each position is described by 512 numbers after the first layer.

The trade-off A wider state can remember more about the line, which the task needs: the reader must carry enough about preceding letters to resolve an ambiguous one. But the recurrent weight matrices are four times the hidden size by the hidden size, so the parameter count grows with the square of this number, as does the work per step, and the cached activations grow with it linearly. At 256 the two layers hold the large majority of the network's parameters.

> *256 is the value the architecture this network follows uses [17]. Taking a published figure rather than searching for one is reasonable for a parameter whose effect is gradual and whose evaluation would cost a full training run each time.*

9.66 WEIGHTS_DIR

```
WEIGHTS_DIR = paths.Weights
```

Output. the directory checkpoints are written to and read from.

What it is for One named location for trained weights, taken from the central paths module, so that the training code and the inference code agree on where checkpoints live without either hard-coding a path.

9.67 DEFAULT_TEXT_CACHE_DIR

```
DEFAULT_TEXT_CACHE_DIR = paths.Cache / 'line_image_cache'
```

Output. where the reader's extracted line crops are cached.

Why the two networks have separate caches The reader and the namer are trained from different page sets: the reader from as much labelled text as possible, the namer from the specific enrolled writers. The extractions differ in which pages they cover and in whether a page lacking a usable transcript is kept, so they are stored separately rather than one being a subset of the other.

9.68 DEFAULT_AUTHOR_CACHE_DIR

```
DEFAULT_AUTHOR_CACHE_DIR = paths.Cache / 'line_cache_authors10'
```

Output. where the namer's extracted crops are cached.

What it holds Crops from the ten writers the namer is trained to distinguish, labelled by writer rather than by text. The directory name records the writer count, which is useful because the namer's output layer size is fixed by it: a cache built for a different number of writers is incompatible with a checkpoint trained on this one, and a differently named directory makes that visible.

9.69 DEFAULT_AUTHOR_DATA_DIR

```
DEFAULT_AUTHOR_DATA_DIR = paths.Data.parents[1] / 'Data' / 'Datasets' / '
    IAMpages10'
```

Output. where the source pages for the namer live.

What it points at The page images and transcripts for the enrolled writers, built from the central paths module. The distinction from the cache directory is worth keeping clear: this holds the original scanned or photographed pages, which are the input, and the cache holds what was extracted from them, which can be regenerated and is excluded from version control for that reason.

9.70 DEFAULT_TEXT_DATA_DIR

```
DEFAULT_TEXT_DATA_DIR = DEFAULT_AUTHOR_DATA_DIR
```

Output. where the source pages for the reader live, the same directory as the namer's.

Why they are the same Both networks are trained from the same collection of pages, differing in what is taken from them rather than in where they come from. Defining one as the other rather than repeating the path means the two cannot drift apart, and it records that the sharing is intentional rather than coincidental.

9.71 InitConv

```
def InitConv(rng, outCh, inCh, kh, kw)
```

Inputs

`rng` the seeded random generator

`outCh`
: the number of filters

`inCh`
: the input channel count

`kh` the filter height

`kw` its width

Output. a pair: the randomly initialised filters and a zero bias.

Why initialisation matters at all The starting weights are random, but not arbitrarily so. If they are too small, the signal shrinks as it passes through each layer and arrives at the output as nearly nothing, so there is no gradient to learn from. If too large, it grows and saturates. With twelve convolutional layers either effect compounds twelve times, so the scale of the initial weights determines whether the network trains at all.

He initialisation

$$w \sim \mathcal{N}\left(0,\ \sqrt{\frac{2}{\text{fanIn}}}\right), \qquad \text{fanIn} = \text{inCh} \times k_h \times k_w \tag{9.34}$$

Source: [28]

where

w each weight, drawn independently

$\mathcal{N}(0, s)$
: a normal distribution with mean 0 and standard deviation s

fanIn
: how many input values each output is computed from

2 the factor that accounts for ReLU, explained below

The reasoning runs as follows. An output is a sum of fanIn products of a weight and an input. For independent terms, variances add, so the output's variance is fanIn times the product of the weight and input variances. Keeping the output's variance equal to the input's therefore requires the weight variance to be one over fanIn. That is the Glorot argument [29].

The factor of 2 is the correction for ReLU [28]. ReLU sets every negative value to zero, discarding about half the signal's variance, so without compensation the variance would halve at every layer and after twelve layers be reduced by a factor of four thousand. Doubling the weight variance exactly offsets it. The difference between the two initialisations is only that factor, and for a network this deep it is the difference between training and not.

Why the bias starts at zero There is nothing to break symmetry in a bias, because the weights are already random, so each unit already computes something different. Zero is the neutral starting point, and it is also harmless here because the batch normalisation that follows removes any constant offset anyway, as noted under ConvBlockForward.

9.72 InitBatchNorm

```
def InitBatchNorm(ch)
```

Inputs

`ch` the channel count

Output. four arrays: the scale at one, the shift at zero, and the running statistics at zero and one.

Why the scale starts at one and the shift at zero Those values make the learned part of the layer the identity: the output is the standardised input unchanged. So the layer begins by doing exactly the normalisation it exists for and nothing else, and any departure from that is something training chose. Starting the scale at a random value would add a per-channel distortion the network would have to learn to undo before making progress.

Why the running statistics start at zero and one A mean of zero and a variance of one describe already-standardised data, so a network evaluated before any training behaves as though normalisation had been applied correctly. They are also the values the exponential average converges away from fastest: being the neutral assumption, any real data moves them in a meaningful direction immediately.

> *These two pairs are treated differently everywhere else in the file. The scale and shift are parameters with gradients and are updated by the optimiser; the running statistics have no gradients and are updated by observation. TrainableParams separates them by name, which is why the naming convention for them is load-bearing.*

9.73 InitLinear

```
def InitLinear(rng, outF, inF)
```

Inputs

`rng` the random generator

`outF`
: the output size

`inF` the input size

Output. a pair: the randomly initialised weight matrix and bias.

The distribution used

$$w,\, b \;\sim\; \mathrm{Uniform}\!\left(-\frac{1}{\sqrt{\mathrm{inF}}},\; +\frac{1}{\sqrt{\mathrm{inF}}}\right) \tag{9.35}$$

where

w, b each weight and each bias, drawn independently

$\mathrm{Uniform}(a, b)$
: a uniform distribution between the two bounds

inF the number of inputs to each output

The same one-over-fan-in scaling as the convolution, expressed as a uniform range rather than a normal distribution and without the ReLU factor of two. The reasoning for the scale is identical; the differences are matters of convention, and they are followed here because the reference framework's linear layer uses exactly this, which keeps checkpoints interchangeable.

Why the bias is random here but zero for a convolution Because no normalisation follows a linear layer. A convolution's bias is removed by the batch normalisation after it, so its initial value is immaterial; a linear layer's bias survives into the output and a small random value is as good a starting point as zero.

9.74 InitLstmDirection

```
def InitLstmDirection(rng, inputSize, hiddenSize)
```

Inputs

`rng` the random generator

`inputSize`
: the input width

`hiddenSize`
: the hidden width

Output. four arrays: both weight matrices and both biases for one direction.

The distribution Uniform between plus and minus one over the square root of the hidden size, for every weight and both biases. The code comment records that this matches the reference framework's own initialisation, which is the reason for the specific form.

Why the hidden size rather than the input size sets the scale The input weights are scaled by the hidden size too, although their fan-in is the input size. That is the framework's convention rather than what the variance argument would give. It is defensible because the two are comparable in this network, the input to the second layer being twice the hidden size, and because the gates' sigmoids are forgiving: they saturate gracefully rather than destroying the signal, so the initialisation scale is less critical than for a deep convolutional stack.

> *All four gates are initialised identically here. A common refinement is to initialise the forget gate's bias to a positive value, so the gate starts near 1 and the cell retains information from the first step, which helps a network learn long dependencies sooner. It is not done here, which matches the reference framework's default.*

9.75 InitConvBlock

```
def InitConvBlock(rng, params, prefix, inCh, outCh, nLayers)
```

Inputs

`rng` the random generator

`params`
: the parameter dictionary, filled in place

`prefix`
: the block's name prefix

`inCh`
: the input channels

`outCh`
: the output channels

`nLayers`
: how many groups

Output. nothing; the parameters are added to the dictionary.

What it does Creates one block's worth of parameters under the names the forward and backward passes expect, with the channel count switching from the block's input to its output after the first group.

Why the naming convention is load-bearing These names are a contract between four separate pieces of code: this initialiser, the forward pass, the backward pass and the checkpoint format. They must agree exactly, and a disagreement fails with a missing key rather than silently training the wrong array, which is the reason for building them from the same arithmetic in every one of those places rather than listing them.

> *The first group takes the block's input channel count and every later group takes its output count, which is why the channel variable is reassigned inside the loop. Getting that wrong produces a shape mismatch at the first forward pass rather than a subtle fault.*

9.76 InitBackboneParams

```
def InitBackboneParams(rng, params)
```

Inputs

`rng` the random generator

`params`
: the parameter dictionary, filled in place

Output. nothing; the backbone's parameters are added.

The channel progression One channel in, then 32, then 64, then 128, with two, four and six groups in the three stages. The channel count doubles at each stage while the spatial size halves, which is the standard arrangement in a convolutional backbone.

Why that arrangement Two reasons, one about capacity and one about cost. Deeper layers represent larger and more varied structures, strokes giving way to junctions and then to letter parts, and there are more distinguishable large structures than small ones, so more channels are needed to hold them. And since each pooling quarters the spatial size, doubling the channels leaves the work per layer roughly halved rather than growing, so the deeper stages are affordable despite being wider and having more groups.

Why the backbone is shared between the two networks Both the reader and the namer use exactly this backbone, initialised by this one function. The features useful for recognising a letter and for recognising a hand are largely the same features, as argued under AuthorClassifierCNNNumpy, so there is no reason for the two to differ, and sharing the initialiser means a change to the architecture applies to both rather than to one.

9.77 TrainableParams

```
def TrainableParams(params)
```

Inputs

`params`
: the full parameter dictionary

Output. the subset the optimiser may update.

What it excludes and why The running mean and variance of every batch normalisation. They are stored alongside the parameters, because they must be saved in the checkpoint, but they are not parameters: no gradient is computed for them, and they are updated by observing the data rather than by descending a loss.

What would happen if they were included The optimiser would look up a gradient for them, find none, and fail with a missing key, which is the benign outcome. Had a zero gradient been supplied instead, the optimiser would apply weight decay to them and shrink them steadily toward zero, so the stored variances would fall and inference would divide by a smaller and smaller number, amplifying every activation. That is a silent corruption which would appear only at inference, as a network whose validation accuracy during training was fine and whose saved checkpoint performs badly.

> *The separation is by name suffix, which works but ties correctness to a naming convention. A parameter accidentally named with one of those suffixes would be silently frozen, and one of these accidentally named otherwise would be silently decayed.*

9.78 SaveCheckpoint

```
def SaveCheckpoint(path, params, extra=None)
```

Inputs

`path`
: where to write

`params`
: the parameters

`extra`
: additional values such as the epoch number

Output. nothing; the checkpoint file is written.

What goes into a checkpoint Every parameter, including the running statistics, plus metadata: which epoch it came from and the best validation loss seen so far. The metadata is what makes resuming possible.

Why the metadata must be in the file Resuming a run requires knowing which epoch to continue from and what the best loss was, so that the scheduler and the best-checkpoint logic behave as though the run had never stopped. Keeping those in the checkpoint rather than in a separate file means they cannot be separated from the weights they describe, which is the failure a parallel log file invites: resuming from the wrong epoch number silently misschedules the learning rate.

> *The metadata keys begin with a double underscore, which is the same convention LoadStateDictAny uses to tell metadata from weights. The two functions are in different files and must agree on it, which is worth noting as another implicit contract.*

9.79 LoadCheckpointMeta

```
def LoadCheckpointMeta(path)
```

Inputs

`path`
: a checkpoint file

Output. the epoch it was saved at and the best loss recorded, where present.

What it is for Reads only the metadata, without the weights, so that a resume can establish where it is continuing from. Reading the metadata alone is cheap, where loading several hundred megabytes of weights to read two numbers is not.

> *Both values are optional, since a checkpoint written before the metadata convention existed has neither. Returning nothing rather than a default lets the caller distinguish "no record" from "epoch zero", and it starts the run at epoch 1 rather than claiming to resume.*

9.80 LoadParamsInto

```
def LoadParamsInto(params, checkpointPath)
```

Inputs

`params`
: the parameter dictionary, filled in place

`checkpointPath`
: the checkpoint to read

Output. how many arrays were loaded.

Why it loads into an existing dictionary The parameters are created first by the initialiser, with the right names and shapes, and the checkpoint's values are then copied into them. The alternative, building the dictionary from the checkpoint, would take the network's structure from the file, so a checkpoint from a different architecture would load without complaint and fail later with a shape mismatch deep in a forward pass.

Why partial loading is allowed, and why the count is returned Names present in both are copied and the rest are left as initialised. That is what makes a **warm start** possible: loading the backbone from one checkpoint while leaving the head freshly initialised, which is how the namer is started from the reader's learned features.

But partial loading is also how a wrong checkpoint fails quietly: a file matching nothing loads nothing and training proceeds from random weights, which looks like training that is simply going badly. Returning the count, which the callers print as a fraction of the total, is what makes that visible. A warm start reporting a small fraction is a warning that the two checkpoints do not correspond.

9.81 AppendLog

```
def AppendLog(path, line)
```

Inputs

`path`
: the log file

`line`
: the line to add

Output. nothing; the line is appended.

Why a log file as well as printed output Training runs for days, so the console scrollback is lost or truncated long before the run ends. A log file persists, and because each epoch writes one line of keyed values, the whole history of a run can be read or plotted afterwards.

Why appending rather than rewriting Appending means a resumed run continues the same log rather than replacing it, so the record spans the interruptions. Appending is also safe against a crash: each line is complete when written, so an interrupted run leaves a readable log ending at the last finished epoch, where a rewritten file could be truncated mid-write.

9.82 BackboneForwardTrain

```
def BackboneForwardTrain(x, params)
```

Inputs

`x`
: the input crops

`params`
: the parameters

Output. a pair: the collapsed feature sequence, and the cache.

What it is for The shared convolutional backbone with caching: three blocks, two poolings, and the collapse of the height. Both networks call it, so the part they have in common is written once.

Why sharing the implementation matters more than sharing the weights The two networks do not share weights; each trains its own copy. What they share is the code, and that is what guarantees the architectures are identical, which in turn is what makes a warm start from one to the other possible: the parameter names and shapes agree because they were produced by the same function.

9.83 BackboneBackwardTrain

```
def BackboneBackwardTrain(dx, cache)
```

Inputs

`dx` the gradient with respect to the backbone's output

`cache`
: what the forward pass saved

Output. the gradient with respect to the input and every backbone weight gradient.

What it does Walks the backbone in reverse: uncollapse the height, then the third block, then the second pooling, the second block, the first pooling and the first block, collecting weight gradients. It is the mirror of the forward sequence and contains no arithmetic beyond the sequencing.

> *The input gradient is computed and returned although nothing uses it, since the crops are data rather than parameters. It is produced because each layer's backward pass produces it as a matter of course, and discarding it earlier would mean a special case in the first convolution. It is also what the numerical gradient check compares against when verifying the backbone.*

9.84 EvalBackboneForward

```
def EvalBackboneForward(params, x)
```

Inputs

`params`
: the parameters

`x` the input crops

Output. the backbone's output, with no cache.

Why a third version of the backbone exists Validation needs a forward pass with the inference behaviour, using the stored batch statistics rather than the batch's own, and with no caching, since no backward pass follows. Running the training version and discarding the caches would use the wrong statistics and would allocate the caches anyway.

Why using the stored statistics for validation is the point Validation estimates how the saved checkpoint will perform, and the saved checkpoint will be used with the stored statistics. Validating with batch statistics would measure something slightly different and, worse, something that depends on how the validation set happened to be divided into batches, so the same weights would score differently with a different batch size.

9.85 IterateBatches

```
def IterateBatches(dataset, batchSize, rng, shuffle)
```

Inputs

`dataset`
: the dataset to iterate

`batchSize`
: how many items per batch

`rng` the random generator

`shuffle`
: whether to shuffle the order

Output. batches of items, one at a time.

Why the order is shuffled for training The examples arrive grouped by page and therefore by writer. Without shuffling, consecutive batches would each contain one writer's hand, so the network would be pulled toward that hand and then away from it, which both slows convergence and makes the loss oscillate with the data order rather than with learning. Shuffling makes each batch an approximately representative sample, which is the assumption the gradient estimate rests on.

Why validation is not shuffled Validation computes a mean over the whole set, and a mean does not depend on the order. Keeping the order fixed makes the measurement exactly reproducible between runs, which matters when comparing two checkpoints: any difference is then attributable to the weights rather than to the batching.

> *Batches are produced one at a time rather than built as a list, so only the current batch is in memory. For a dataset of tens of thousands of crops that is the difference between a few hundred megabytes and several gigabytes.*

9.86 InitTextModelParams

```
def InitTextModelParams(seed, numClasses)
```

Inputs

`seed`
: the random seed

`numClasses`
: the output size, the charset plus the blank

Output. the reader's complete parameter dictionary.

What it builds The shared backbone, two bidirectional LSTM layers, and a linear text head producing one score per class.

Why the seed is explicit Every random draw comes from a generator created from this seed, so the same seed gives the same initial weights. That is what makes a training run reproducible, which matters for comparing two configurations: without it, a difference in the result could be a difference in the starting point rather than in the change being tested.

Why the class count is one more than the charset The extra class is the CTC blank, which is not a character but needs an output slot of its own, as described under DecodeCtc. Deriving the count from the charset rather than stating it means the two cannot disagree, and a disagreement would be severe: the network would be trained with every character's index shifted by one relative to what inference assumes.

9.87 BuildLstmLayerParams

```
def BuildLstmLayerParams(sd)
```

Inputs

`sd` the parameter dictionary

Output. the LSTM parameters gathered into the nested structure the forward pass expects.

What it is for The parameters are stored flat, keyed by name, because that is what a checkpoint is and what the optimiser iterates over. The recurrent forward pass wants them nested: a list of layers, each with a forward and a backward direction, each with its four arrays. This converts one view to the other.

Why the flat storage is the primary form Because three separate things need it flat: the optimiser, which iterates over every parameter without knowing the structure; the checkpoint, which is a flat collection of named arrays; and the gradient dictionary, which must be keyed the same way. The nested form is needed by one function, so it is the derived one.

> *The nested structure holds references to the same arrays, not copies, so an update applied by the optimiser to the flat dictionary is visible through the nested view. That is essential and would break silently if any step copied.*

9.88 TextModelForward

```
def TextModelForward(x, params)
```

Inputs

`x` the input crops

`params`
the parameters

Output. a pair: the raw per-column class scores, and the cache.

What it does The reader's whole forward pass with caching: the backbone, the transpose that turns the image into a sequence, the two recurrent layers, and the text head.

Why it returns raw scores rather than log probabilities The inference version ends with a log softmax; this one stops at the raw scores, because CtcLoss performs the softmax itself. The reason is the cancellation described in that entry: taking the softmax and the loss together gives a gradient that is a simple difference, whereas computing the softmax here would require differentiating back through it as a separate step, which is both more work and numerically worse.

> *This is why the gradient the loss returns is named for the raw logits: it is the gradient with respect to the output of this function, so the backward pass can start here with no intervening layer.*

9.89 TextModelBackward

```
def TextModelBackward(dRawLogits, cache)
```

Inputs

`dRawLogits`
: the gradient from the loss

`cache`
: what the forward pass saved

Output. the gradient with respect to the input and every weight gradient.

What it does Reverses the forward sequence: the text head, the two recurrent layers, the transpose, and the backbone, collecting weight gradients keyed by parameter name.

The transpose, reversed The forward pass moved the width axis to the front to make a sequence; the backward pass moves it back, so the gradient leaving for the backbone is an image gradient again. A transpose is its own inverse given the right axis order, so nothing is computed, but the axes must be put back in exactly the order the backbone expects, and an error there produces a gradient that is correct in magnitude and scrambled in arrangement.

9.90 EvalTextForward

```
def EvalTextForward(params, x)
```

Inputs

`params`
: the parameters

`x`
: the input crops

Output. the log probabilities per column, with no cache.

What it is for The reader's forward pass in inference mode, for validation: stored batch statistics, no caching, and the log softmax applied, since validation decodes the output rather than passing it to the loss.

9.91 TextNumpyBatch

```
def TextNumpyBatch(batch)
```

Inputs

`batch`
: a collated batch

Output. the images, the encoded labels and the original texts.

What it does Prepares a batch for the network: the images converted to the input convention and the labels encoded into class indices. The original texts are carried through as well so that validation can compare the decoded output against them.

> *The image conversion must match TensorFromResized in recognise.py exactly, including the inversion and the standardisation. A difference between the two would produce a network that trains successfully and reads nothing at inference, which is the hardest class of fault to find because both halves appear to work.*

9.92 ValidateTextModel

```
def ValidateTextModel(valSet, params, batchSize, rng)
```

Inputs

`valSet`
: the held-out set

`params`
: the parameters

`batchSize`
: the batch size

`rng` the random generator

Output. a pair: the mean validation loss and the character accuracy.

Why a held-out set is necessary Training loss falls whether or not the network is learning anything general, because a network with enough capacity can memorise its training examples. The only way to tell memorisation from learning is to measure on data the network has never been trained on. That is the **held-out** or validation set, and the gap between training and validation loss is the measure of overfitting.

Why two numbers are reported The loss and the character accuracy answer different questions, and both are needed. The loss is what training minimises, so it is the quantity the scheduler and the best-checkpoint logic must use, and it is sensitive to small changes in confidence. Character accuracy is what anyone actually cares about, but it is coarse: a network can improve its loss substantially with no change in accuracy, because it has become more confident about answers it already had right.

Augmentation is off here, through the evaluation view, and batch normalisation uses its stored statistics, through the evaluation forward pass. Both are necessary for the measurement to describe the checkpoint that will be saved rather than some variant of it.

9.93 BuildPagesSplit

```
def BuildPagesSplit(maxPages, dataDir)
```

Inputs

`maxPages`
: how many pages to use

`dataDir`
: where the pages are

Output. a pair of training and validation datasets.

Why the split is by page and not by line This is the important decision. Lines from one page are written by one person, on one sheet, with one pen, in one session. Splitting by line would put some of a page's lines in training and others in validation, and the network could then score well on validation by recognising that particular page's characteristics rather than by reading. The validation figure would be optimistic, and the amount by which it was optimistic would be unknowable.

Splitting by page makes the validation lines come from sheets the network has never seen, which is the situation it will actually face. This is **leakage** avoidance, and it is the commonest way a held-out measurement is quietly invalidated: the split has to be made at the level at which examples are genuinely correlated, which here is the page.

The page limit exists to allow short runs during development. It takes the first n pages rather than a random sample, which means a short run always uses the same pages, so two such runs are comparable with each other even though neither represents the whole database.

9.94 BuildTekliaSplit

```
def BuildTekliaSplit(dataDir)
```

Inputs

`dataDir`
: where the corpus is

Output. a pair of training and validation datasets from the alternative corpus.

What it is for The same split for the second corpus, which arrives already cut into lines. Having both behind the same interface is what lets the training function take a source argument and otherwise be identical, so a change to the training procedure applies to both without being made twice.

9.95 TrainTextModel

```
def TrainTextModel(numEpochs,batchSize,lr,seed,maxPages,dataDir,source='
    pages',
         resume=False,warmStart=None)
```

Inputs

`numEpochs`
: how many passes over the data

`batchSize`
: examples per step

`lr`
: the initial learning rate

`seed`
: the random seed

`maxPages`
: how many pages to use

`dataDir`
: where the data is

`source`
: which corpus, default pages

`resume`
: whether to continue a previous run

`warmStart`
: a checkpoint to initialise from

Output. the best validation loss reached.

The training loop

- **Build the split** and initialise the parameters.
- **Create the optimiser and scheduler:** RMSprop with a small weight decay, and the plateau scheduler watching validation loss.
- **Optionally warm start or resume.**
- **For each epoch:** iterate shuffled batches, and for each run the forward pass, compute the CTC loss and its gradient, run the backward pass, select the trainable gradients and take an optimiser step.
- **After each epoch:** validate, step the scheduler, save the checkpoint and append to the log.

Why two checkpoints are kept A best checkpoint, written only when validation loss improves, and a rolling checkpoint written every epoch. They serve different purposes and neither substitutes for the other. The best one is the model to use, since later epochs may overfit and be worse. The rolling one is for resuming, since resuming must continue from where the run actually stopped, not from the best epoch, or the epochs between would be repeated.

Why the best is chosen on loss rather than accuracy Loss is the finer measure. Character accuracy on a validation set of a few thousand lines moves in visible steps, so several epochs can tie, and choosing between them would be arbitrary. Loss distinguishes them, because it responds to confidence as well as to correctness.

What warm starting is for Initialising from an existing checkpoint rather than from random weights. It is how the joint checkpoint is produced: start from the database-trained weights and continue on the mixed data, so the network keeps what it learned from the large corpus while adapting to the photographs. Training that from scratch would need the whole corpus again and would reach a similar place far more slowly.

> *The progress line is printed with a carriage return and no newline, so it overwrites itself rather than filling the console with thousands of lines. The per-epoch summary is printed normally and also written to the log, so the transient detail stays transient and the durable record is kept.*

9.96 InitAuthorModelParams

```
def InitAuthorModelParams(seed, numAuthors)
```

Inputs

`seed`
: the random seed

`numAuthors`
: how many writers to distinguish

Output. the namer's complete parameter dictionary.

What it builds The same shared backbone, then a two-layer head: a linear layer, a ReLU, a dropout, and a second linear layer producing one score per writer. No recurrent layers, for the reasons given under AuthorClassifierCNNNumpy.

Why the writer count is a parameter It sets the final layer's output size, so a checkpoint is tied to the number of writers it was trained on. Adding a writer means a new final layer and retraining it, and a checkpoint from a different count cannot be loaded. That is also why the writer names are stored inside the checkpoint, as described under LoadStateDictAny: the count alone is not enough, since the same ten writers in a different order would load without complaint and attribute every page to the wrong person.

9.97 AuthorModelForward

```
def AuthorModelForward(x, params, training, rng)
```

Inputs

`x` the input crops

`params`
: the parameters

`training`
: whether dropout should be active

`rng` the random generator for dropout

Output. a pair: the raw per-writer scores, and the cache.

What it does The backbone, the average pooling over position, and the two-layer head with dropout between the layers.

Why the training flag is an argument Because dropout must be active during training and inactive during validation, and the same function serves both. Passing it explicitly rather than holding it as state makes it impossible to validate with dropout accidentally left on, which would add noise to the measurement, or to train with it off, which would silently remove the regularisation.

> *The random generator is passed in rather than created here, so that the dropout masks come from the run's own seeded generator and a training run is reproducible. A generator created inside the function would reseed itself on every call and the masks would be unrepeatable.*

9.98 AuthorModelBackward

```
def AuthorModelBackward(dLogits, cache)
```

Inputs

`dLogits`
: the gradient from the loss

`cache`
: what the forward pass saved

Output. the gradient with respect to the input and every weight gradient.

What it does Reverses the namer's forward sequence: the second linear layer, the dropout, the ReLU, the first linear layer, the average pooling, and the backbone. The average pooling's backward pass spreads the gradient evenly back over every position, which is why the namer's backbone receives gradient everywhere rather than only at the winning rows as the reader's does.

9.99 EvalAuthorForward

```
def EvalAuthorForward(params, x)
```

Inputs

`params`
: the parameters

`x`
: the input crops

Output. the raw per-writer scores, with no cache and no dropout.

What it is for The namer's inference-mode forward pass for validation: stored batch statistics, dropout off, no caching. The same three differences as the reader's evaluation pass, and for the same reasons.

9.100 AuthorNumpyBatch

```
def AuthorNumpyBatch(batch)
```

Inputs

`batch`
: a collated batch

Output. the images and the writer indices.

What it does Prepares a batch for the namer. Simpler than the text version, because the labels are single integers needing no encoding, though the image conversion is identical and must stay so.

9.101 ValidateAuthorModel

```
def ValidateAuthorModel(valSet, params, batchSize, rng)
```

Inputs

`valSet`
: the held-out set

`params`
: the parameters

`batchSize`
: the batch size

`rng` the random generator

Output. a pair: the mean validation loss and the accuracy.

What it measures The cross-entropy loss and the fraction of crops attributed to the right writer, over the held-out set with dropout off and stored statistics in use.

Why accuracy means more here than for the reader For a classifier, accuracy is the quantity of interest directly: the crop is attributed correctly or it is not. For the reader, character accuracy is an aggregate over a sequence and a line can be mostly right, so the measure is coarser relative to what matters. The namer's accuracy is therefore the headline figure and the loss is the finer measure used for choosing the checkpoint.

> *The split for the namer must also be by page, for the leakage reason given under BuildPagesSplit, and it matters more here. The namer's task is to recognise a writer, and lines from the same sheet share far more than handwriting: the same paper, pen, lighting and scan. A line-level split would let it score nearly perfectly by recognising the sheet.*

9.102 TrainAuthorModel

```
def TrainAuthorModel(numEpochs, batchSize, lr, seed, resume=False,
    warmStart=None)
```

Inputs

`numEpochs`
: how many passes

`batchSize`
: examples per step

`lr` the initial learning rate

`seed`
: the random seed

`resume`
: whether to continue a previous run

`warmStart`
: a checkpoint to initialise from

Output. the best validation loss reached.

What differs from the reader's loop The same structure: batches, forward, loss, backward, step, validate, save, log. The differences are the loss, which is cross-entropy rather than CTC, the dropout, which must be enabled for training and disabled for validation, and the data, which is labelled by writer.

Why warm starting matters more here The namer has far less data: ten writers' pages against a database of thousands. Training its backbone from scratch on that would overfit badly, because a deep convolutional stack has ample capacity to memorise a few thousand crops. Starting from the reader's backbone gives it features already learned from a large corpus, so the small dataset is spent on learning to tell writers apart rather than on learning what a stroke is.

This is **transfer learning**, and the reason it works here is the argument made earlier: the features that distinguish letters and the features that distinguish hands are largely the same features. The reader learned them from data the namer could never have had.

The two networks are trained separately rather than together with a shared backbone and two heads, which would be the obvious alternative. Separate training keeps each network's accuracy independent of the other's, which matters for this project because both are used as judges of the synthesised output: a shared backbone would make the two scores correlated, and the harmonic mean that combines them assumes they are measuring different things.

Bibliography

[1] U.-V. Marti and H. Bunke, "The iam-database: An english sentence database for offline handwriting recognition," *International Journal on Document Analysis and Recognition*, vol. 5, pp. 39–46, 2002.

[2] F. C. Crow, "Summed-area tables for texture mapping," *ACM SIGGRAPH Computer Graphics*, vol. 18, no. 3, pp. 207–212, 1984.

[3] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[4] L. He, Y. Chao, and K. Suzuki, "A run-based two-scan labeling algorithm," *IEEE Transactions on Image Processing*, vol. 17, no. 5, pp. 749–756, 2008.

[5] R. E. Tarjan, "Efficiency of a good but not linear set union algorithm," *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.

[6] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984.

[7] G. Wolberg, *Digital Image Warping*. IEEE Computer Society Press, 1990.

[8] P. Y. Simard, D. Steinkraus, and J. C. Platt, "Best practices for convolutional neural networks applied to visual document analysis," in *Proc. 7th International Conference on Document Analysis and Recognition*, pp. 958–963, 2003.

[9] A. Vinciarelli and J. Luettin, "A new normalization technique for cursive handwritten words," *Pattern Recognition Letters*, vol. 22, no. 9, pp. 1043–1050, 2001.

[10] W. Postl, "Detection of linear oblique structures and skew scan in digitized documents," in *Proc. 8th International Conference on Pattern Recognition*, pp. 687–689, 1986.

[11] S. Avidan and A. Shamir, "Seam carving for content-aware image resizing," *ACM Transactions on Graphics*, vol. 26, no. 3, 2007.

[12] N. Arvanitopoulos and S. Süsstrunk, "Seam carving for text line extraction on color and grayscale historical manuscripts," in *Proc. 14th International Conference on Frontiers in Handwriting Recognition*, pp. 726–731, 2014.

[13] R. Bellman, "The theory of dynamic programming," *Bulletin of the American Mathematical Society*, vol. 60, no. 6, pp. 503–515, 1954.

[14] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *Proc. 10th International Workshop on Frontiers in Handwriting Recognition*, 2006.

[15] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd International Conference on Machine Learning*, pp. 448–456, 2015.

[16] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[17] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[18] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. 23rd International Conference on Machine Learning*, pp. 369–376, 2006.

[19] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967.

[20] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[22] D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[23] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965.

[24] T. Tieleman and G. Hinton, "Lecture 6.5 — rmsprop: Divide the gradient by a running average of its recent magnitude," 2012. COURSERA: Neural Networks for Machine Learning.

[25] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd International Conference on Learning Representations*, 2015.

[26] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. 7th International Conference on Learning Representations*, 2019.

[27] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. 30th International Conference on Machine Learning*, pp. 1310–1318, 2013.

[28] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proc. IEEE International Conference on Computer Vision*, pp. 1026–1034, 2015.

[29] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th International Conference on Artificial Intelligence and Statistics*, pp. 249–256, 2010.